Chapter 0

How this course works

practice teaches; the book shows you what to practise

You can already write Python: loops, functions, classes, a few sorting algorithms. The wall you hit is the next layer: a module you've never seen (`sqlite3`, `hmac`, `asyncio`), documentation that assumes you already know it, and programs big enough to need structure. This course is about that layer.

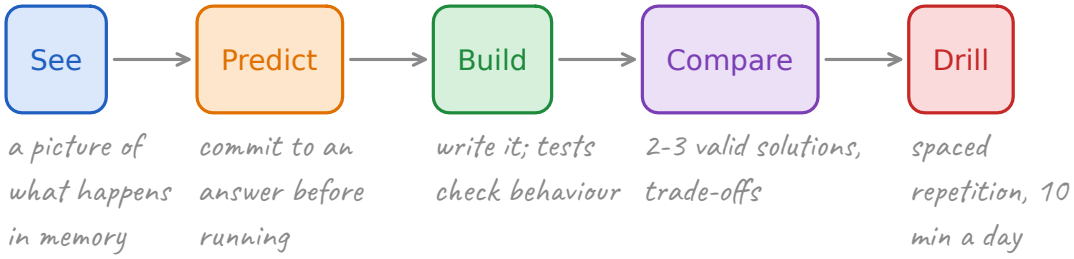
Text alone doesn't teach programming, any more than reading about press-ups builds muscle. So every page here is a **map of what to practise**, and the practice lives in a companion folder of exercises that check themselves. The book is short on purpose. The hours go into the exercises and into your own projects.

Three ideas run through everything:

- **The loop.** See it, predict it, build it, compare solutions, drill it.
- **The library card.** Six questions that crack open any library, in any language.
- **The language passport.** Twelve questions every language answers. Python fills them in now; C, Java and Rust fill in the same page later, so each new language is a diff, not a restart.

The first half of your Computing module (types, strings, lists, control flow, dictionaries, functions, classes) is squeezed into Chapter 1 as a fast recall sprint. NumPy gets a full chapter (18), because that's where the module gets interesting.

The loop: five steps, every topic



See. Every idea gets a picture: names pointing at objects, a resource opened and closed, bytes flowing through a pipeline, two threads on a timeline. The same pictures come back in every chapter, and later in every language.

Predict. Before running a snippet, write down what you think it prints. Being wrong is the useful part: it shows exactly where your mental model is off.

Build. The practice folder gives a task, an empty `solution.py` and a set of tests. The tests check what your code **does** (its contract), never how it looks, so any correct solution passes.

Compare. After you pass, you're shown two or three other solutions that also pass, with notes on speed, memory and readability. There's rarely one best answer; there's a best answer **for a situation**.

Drill. Ten minutes of quick questions a day ("what does this print?", "which module does X?"). Questions you get right come back less often; questions you miss come back tomorrow.

How to read a page

Code boxes. Grey boxes are code, with line numbers. Red handwriting points at the lines that matter. The dark box underneath is the **real output**: the book is built by running every snippet, so if a snippet printed something different, the book wouldn't build.

```
1 prices = {"tea": 1.20, "cake": 2.50}
2 order = ["tea", "cake", "tea"]
3 total = sum(prices[item] for item in order)
4 print(f"{len(order)} items, £{total:.2f}")
```

look each item up, add them

:.2f means 2 decimal places

```
3 items, £4.90
```

REPL boxes show an interactive session. Type the part after `>>>` yourself; Python echoes the value back. The REPL is your main tool for learning a library: poke it and see.

```
>>> word = "practice"
>>> word.upper()
'PRACTICE'
>>> len(word), word[::-1]
(8, 'ecitcarp')
```

Terminal boxes are commands typed in a shell (not run by the book):

```
$ cd Python-Visual-Guide
$ python -m practice next
```

KEY IDEA

Callout boxes like this one hold the one sentence to remember from a page. Red **TRAP** boxes mark the mistakes almost everyone makes once.

✎ YOUR GO (pencil, or the REPL, first)

Before reading on: what does print(sorted('banana')) print? Write it down, then try it.

Many right answers: tests check behaviour

Maths questions have one answer. Code usually has many: two sorting functions can look nothing alike and both be correct. So the practice tests never compare your code to mine. They check the **contract**: for a sort, "the output has the same items as the input, in order". They check it on hundreds of random inputs, including the awkward ones (empty, one item, all the same).

```python
import random

def is_sorted_version(out, original):
    same_items = sorted(out) == sorted(original)
    in_order = all(a <= b for a, b in zip(out, out[1:]))
    return same_items and in_order

def my_sort(xs):                # any correct sort would do
    xs = list(xs)
    for i in range(1, len(xs)):
        j = i
        while j and xs[j - 1] > xs[j]:
            xs[j - 1], xs[j] = xs[j], xs[j - 1]
            j -= 1
    return xs

rng = random.Random(0)
cases = [[], [7], [3, 3, 3]]
for _ in range(500):
    n = rng.randint(0, 12)
    cases.append([rng.randint(-9, 9) for _ in range(n)])
ok = all(is_sorted_version(my_sort(c), c) for c in cases)
print(ok, len(cases), "cases")
```

← same items...

← ...and in order

← the awkward cases first

```
True 503 cases
```

After you pass, the compare step shows other solutions side by side. Both of these pass the same tests; the bars are the trade-offs, not a ranking:

Insertion sort, by hand

```python
def sort_copy(xs):
    xs = list(xs)
    for i in range(1, len(xs)):
        j = i
        while j and xs[j-1] > xs[j]:
            xs[j-1], xs[j] = xs[j], xs[j-1]
            j -= 1
    return xs

print(sort_copy([5, 2, 9, 1]))
```

```
[1, 2, 5, 9]
```

Use the built-in

```python
def sort_copy(xs):
    return sorted(xs)


print(sort_copy([5, 2, 9, 1]))
```

```
[1, 2, 5, 9]
```

■□□□□	speed on big lists
■■■■■	teaches how sorting works
□□□■■	short and obviously right

RULE OF THUMB

Write it by hand to **learn** it; use the library to **ship** it. The exercises will often ask for both.

The library card: how to crack open any library

Getting stuck on `sqlite3` or `hmac` is rarely about syntax. It's about not knowing the **shape** of the library: which objects you create, which methods you call, what you must close, and what goes wrong. Every library, in every language, answers the same six questions. Chapter 7 teaches how to find the answers yourself; every later chapter fills in cards like this one:

LIBRARY CARD `json`

Job	Turn Python dicts/lists/str/numbers into JSON text and back.
Core objects	No objects to create: four functions. `dumps`/`loads` work on strings, `dump`/`load` on open files.
Lifecycle	data → `json.dumps(data)` → text → `json.loads(text)` → data.
Errors	`json.JSONDecodeError` for bad text; `TypeError` for things JSON can't hold (sets, dates).
Traps	Tuples come back as lists; dict keys always come back as strings.
Docs	docs.python.org/3/library/json.html, or `help(json)` in the REPL.

```
1 import json
2 data = {"id": 7, "tags": ("new", "hot"), 3: "three"}
3 text = json.dumps(data)
4 print(text)
5 print(json.loads(text))
```

← a tuple and an int key

← the traps, live

```
{"id": 7, "tags": ["new", "hot"], "3": "three"}
{'id': 7, 'tags': ['new', 'hot'], '3': 'three'}
```

The language passport

When you learn C, Java or Rust later, you won't start again. You'll answer the same twelve questions. Here is Python's passport in one page; each chapter ends with a small **stamp** filling in the questions it covered, and Appendix B collects them.

LANGUAGE PASSPORT: Python — same 12 questions for C, Java, Rust later

1. How does code run?
Source is compiled to bytecode, then run by the CPython virtual machine (Ch 2).

2. Memory model
Names point at objects on a heap; nothing is copied unless you ask (Ch 3).

3. Typing
Dynamic and strong: values have types, names don't; `'1' + 1` is an error. Optional hints (Ch 5, 17).

4. Control flow
`if`/`elif`, `for` over iterables, `while`, `match`; exceptions as non-local exits (Ch 1, 6).

5. Functions and scope
First-class functions, closures, LEGB scope, decorators, generators (Ch 4).

6. Data structures
list, tuple, dict, set built in; heapq, deque, bisect in the stdlib; know the costs (Ch 10, 20).

7. Abstraction
Classes with duck typing and protocols (dunder methods); dataclasses, ABCs (Ch 5).

8. Errors
Exceptions with `try`/`except`/`finally`; context managers for clean-up (Ch 6).

9. Modules and build
Modules and packages, `import`, venv + pip, `pyproject.toml` (Ch 2, 17, 19).

10. I/O
File objects, `pathlib`, text vs bytes with explicit encodings (Ch 8).

11. Concurrency
threads (GIL), processes, `asyncio`; free-threaded builds from 3.13 (Ch 16).

12. Stdlib and tooling
Huge stdlib ("batteries included"); `unittest`, `pdb`, `cProfile` built in (Appendix A).

Using the practice folder

The practice folder needs nothing but Python itself: no pip installs. From the course folder:

```
$ python -m practice list          # every exercise, and which you've passed
$ python -m practice next          # set up the next one and show the task
$ python -m practice check 12.3    # run the tests on your solution.py
$ python -m practice alt 12.3      # after a pass: other solutions + trade-offs
$ python -m practice drill         # today's spaced-repetition questions
```

Each exercise has a kind, so you know what's being asked:

kind	what you do
build from a spec	write it from nothing; the task says what it must do
fill the stub	the outline is there; fill in the missing parts
fix the bug	the code runs but is wrong; find and fix it
refactor	make it cleaner or faster without breaking the tests (sometimes with a rule, like "no loops")
learn an unseen module	use a module the book hasn't taught, with the library card method
measure and explain	time two versions with `timeit` and explain the difference
mini-project	a small real program that uses the whole chapter

A realistic day: 10 minutes of drill, 45 minutes of exercises, then the rest on your own project (your mini-uni infrastructure is perfect for this). The exercises give you the moves; the project is where they become habits.

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 0.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 0.1` shows other valid solutions and their trade-offs.

`0.1` build from a spec Warm-up: sort without sorted()

The map

part	chapters	what it unlocks
0 Recall	1 the module in one sprint	find your gaps fast
I Mental model	2 how Python runs · 3 names and memory · 4 functions · 5 the data model · 6 errors	stop being surprised by Python
II The unlock	7 how to learn any library	never stuck on a new module again
III Stdlib by job	8 files · 9 text · 10 algorithm toolkit · 11 numbers and time · 12 sqlite3 · 13 security · 14 OS and CLI · 15 networking · 16 concurrency · 17 quality	write most programs with no pip at all
IV Beyond	18 NumPy · 19 choosing pip packages	the outside world, safely
V Core CS	20 complexity and linked structures · 21 graphs and dynamic programming	the BSc algorithms module
Appendices	A stdlib atlas · B passport · C glossary	look things up

DEEP DIVE (your vault)

Your existing notes (the "Python and Computing Basics" vault) go far deeper on every topic. Each chapter points to the matching vault notes; read them when a page leaves you wanting the full story.

Chapter 1

Recall sprint

find your gaps in an hour, not a semester

This chapter is the first half of your Computing module (types, strings, numbers, collections, control flow, functions, classes) squeezed into three quizzes. You know most of it. The point is to find the **few things you only think you know**, because those are what make later chapters confusing.

How to use it:

- Do a quiz page with a pencil, no REPL. Writing the answer down matters: "I'd have got that" doesn't count.
- Turn the page, mark yourself, and read the why only for the ones you missed.
- Every miss points to the chapter that explains it properly. Note those chapters; they're your personal reading list.
- Then do the practice exercises: small programs that use all of it together.

quiz	covers	lecture
A	numbers, strings, booleans, comparison, formatting	1, 2
B	lists, tuples, dicts, sets, comprehensions, unpacking	3, 5
C	loops, match, functions, scope, classes, exceptions	4, 6, 7

> **KEY IDEA**
> Every answer on the answer pages was produced by running the snippet while building the book, so the answers are exactly what Python 3.14 prints.

Quiz A: values, strings, numbers

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
print(7 / 2, 7 // 2, -7 // 2, 7 % 3)
```

2

```
print(0.1 + 0.2 == 0.3)
print(round(0.1 + 0.2, 10) == 0.3)
```

3

```
print(round(2.5), round(3.5), round(-0.5))
```

4

```
s = "python"
print(s[1:4], s[-2:], s[::-1][:3])
```

5

```
print("ab" * 3, "-".join("abc"), "a,b,,c".split(","))
```

6

```
print(True + True, int("08"), bool("False"))
```

7

```
x = 1_000_000
print(f"{x:,} {x:.2e} {3.14159:.2f} {'hi':>5}|")
```

8

```
print(1 < 2 < 3, 3 > 2 > 2, "a" < "B")
```

9

```
a = [1, 2]
b = [1, 2]
print(a == b, a is b)
```

10

```
print("5" + str(5), int("5") + 5)
print("5" + 5)
```

11

```
print(type(3/1).__name__, 2**100 > 10**30, 10**20 + 1.0 == 10**20)
```

12

```
s = "  Hi There  "
print(s.strip().lower().replace(" ", "_"), len(s))
```

Answers A

#	prints	why
1	`3.5 3 -4 1`	`/` is true division (always float); `//` floors towards minus infinity, so `-7 // 2` is -4.
2	`False True`	floats are binary fractions; 0.1 can't be stored exactly. Compare with a tolerance or `math.isclose`.
3	`2 4 0`	`round` uses banker's rounding: ties go to the even neighbour.
4	`yth on noh`	slices are [start:stop) and never fail; negative indexes count from the end.
5	`ababab a-b-c ['a', 'b', '', 'c']`	`join` is a string method taking an iterable; `split` with a separator keeps empty fields.
6	`2 8 True`	bool is a subclass of int; any non-empty string is truthy, even "False".
7	`1,000,000 1.00e+06 3.14 hi|`	format spec mini-language: `,` thousands, `e` scientific, `.2f`, `>5` right-align.
8	`True False False`	comparisons chain (`a < b and b < c`); strings compare by code point, and uppercase letters come first.
9	`True False`	`==` compares values, `is` compares identity (same object). Use `is` only for `None`.
10	`55 10 TypeError!`	Python is strongly typed: no silent conversion between str and int.
11	`float True True`	ints are arbitrary precision; floats have 53 bits, so adding 1.0 to 1e20 is lost.
12	`hi_there 12`	string methods return new strings (str is immutable); they chain.

RULE OF THUMB

Missed 2, 3 or 11? Chapter 11 on numbers. Missed 9? Chapter 3. Everything else: re-read the why, then make up a variation and test it in the REPL.

▢ Quiz B: collections

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
a = [3, 1, 2]
b = a
b.append(0)
print(a)
```

2

```
a = [3, 1, 2]
print(a.sort(), sorted(a, reverse=True))
```

3

```
t = (1)
u = (1,)
print(type(t).__name__, type(u).__name__, len(u))
```

4

```
d = {"b": 1, "a": 2}
d["c"] = 3
print(list(d), d.get("z", 0))
```

5

```
d = {"x": 1}
print(d["y"])
```

6

```
a, b = {1, 2, 3}, {3, 4}
print(a | b, a & b, a - b)
```

7

```
print([x * x for x in range(6) if x % 2])
```

8

```
first, *rest = "hello"
print(first, rest)
```

9

```
for i, c in enumerate("ab", start=1):
    print(i, c, end="; ")
```

10

```
print(list(zip([1, 2, 3], "ab")))
```

11

```
nums = [0, 1, 2, 3, 4, 5]
del nums[1::2]
print(nums, nums[10:])
```

12

```
grid = [[0] * 2] * 2
grid[0][0] = 9
print(grid)
```

Answers B

#	prints	why
1	`[3, 1, 2, 0]`	`b = a` copies the reference, not the list (Chapter 3 draws it).
2	`None [3, 2, 1]`	`list.sort()` sorts in place and returns None; `sorted()` returns a new list.
3	`int tuple 1`	the comma makes the tuple, not the brackets.
4	`['b', 'a', 'c'] 0`	dicts keep insertion order (3.7+); `.get` gives a default instead of KeyError.
5	`KeyError!`	missing key with `[]` raises KeyError. Use `.get`, `in`, or `collections.defaultdict`.
6	`{1, 2, 3, 4} {3} {1, 2}`	set union, intersection, difference: the maths notation, as operators.
7	`[1, 9, 25]`	comprehension: expression, loop, optional filter.
8	`h ['e', 'l', 'l', 'o']`	star-unpacking collects the leftovers into a list.
9	`1 a; 2 b;`	enumerate gives (index, item) pairs; `end=` replaces the newline.
10	`[(1, 'a'), (2, 'b')]`	zip stops at the shortest input (`strict=True` makes it raise).
11	`[0, 2, 4] []`	slices can be deleted or assigned; slicing past the end gives an empty list, not an error.
12	`[[9, 0], [9, 0]]`	`* 2` repeats the **same** inner list twice. Use `[[0] * 2 for _ in range(2)]`.

RULE OF THUMB

Missed 1 or 12? Chapter 3 is for you: both are the same idea (two names, one object). Missed 4 or 5? Chapter 10's `collections` makes dicts easier.

Quiz C: control flow, functions, classes

Write down what each prints (an error counts: write its name). No running anything. Answers on the next page.

1

```
for n in [3, 5, 7]:
    if n % 2 == 0:
        break
else:
    print("no evens")
```

2

```
i = 0
while True:
    i += 3
    if i > 10:
        break
print(i)
```

3

```
def f(cmd):
    match cmd.split():
        case ["go", d]: return f"go {d}"
        case ["quit"]: return "bye"
        case _: return "?"
print(f("go north"), f("jump"))
```

4

```
def add(x, items=[]):
    items.append(x)
    return items
add(1)
print(add(2))
```

5

```
def f(a, b=2, *args, c, **kw):
    return a, b, args, c, kw
print(f(1, 5, 6, c=7, d=8))
```

6

```
x = 10
def f():
    print(x)
    x = 5
f()
```

7

```
def f():
    pass
print(f(), (lambda: 1)())
```

8

```
words = ["bb", "a", "ccc"]
print(sorted(words, key=len), max(words, key=len))
```

9

```
class Dog:
    tricks = []
    def __init__(self, n): self.n = n
a, b = Dog(1), Dog(2)
a.tricks.append("sit")
print(b.tricks)
```

10

```
class P:
    def __init__(self, x): self.x = x
    def __repr__(self): return f"P({self.x})"
print([P(1), P(2)])
```

11

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B+" + super().hi()
print(B().hi(), isinstance(B(), A))
```

12

```
try:
    print(int("x"))
except ValueError as e:
    print("bad:", e)
finally:
    print("done")
```

Answers C

#	prints	why
1	`no evens`	a loop's `else` runs only if the loop finished without `break`.
2	`12`	the loop runs until the first value over 10: 3, 6, 9, 12.
3	`go north ?`	structural pattern matching matches shapes and binds names (3.10+).
4	`[1, 2]`	defaults are evaluated **once**, when `def` runs; the same list is reused. Use `items=None`.
5	`(1, 5, (6,), 7, {'d': 8})`	positional, default, extra positionals, keyword-only, extra keywords.
6	`UnboundLocalError!`	assigning `x` anywhere in a function makes it local everywhere in that function: UnboundLocalError (Chapter 4).
7	`None 1`	a function without `return` returns None.
8	`['a', 'bb', 'ccc'] ccc`	`key=` is called on each item; the items themselves are returned.
9	`['sit']`	a class attribute is shared by every instance. Put per-object lists in `__init__`.
10	`[P(1), P(2)]`	containers show items with `repr`; define `__repr__` for debugging output.
11	`B+A True`	`super()` calls the parent's version; a B is an A.
12	`bad: invalid literal for int() with base 10: 'x' done`	`finally` always runs, error or not (Chapter 6).

RULE OF THUMB

Missed 4 or 6? Chapter 4 explains both with one picture. Missed 9, 10 or 11? Chapter 5 (the data model).

▢ The first half of the module, on one page

type	make one	most-used operations	cost to know
`str`	`'hi'`, `f'{x}'`, `str(x)`	`s[i]`, `s[a:b]`, `split`, `join`, `strip`, `replace`, `find`, `startswith`, `upper`	immutable; `+=` in a loop is O(n²), use `join`
`list`	`[1, 2]`, `list(it)`, `[x for x in it]`	`append`, `extend`, `pop()`, `insert`, `remove`, `sort`, `index`, slicing	`append`/`pop()` O(1); `insert(0)`/`pop(0)`/`in` O(n)
`tuple`	`(1, 2)`, `1,`, `tuple(it)`	indexing, unpacking `a, b = t`	immutable, hashable (dict key) if its items are
`dict`	`{'k': v}`, `dict(zip(ks, vs))`, `{k: v for ...}`	`d[k]`, `get`, `items`, `keys`, `values`, `setdefault`, `pop`, `|`	lookup/insert O(1) average; keys must be hashable
`set`	`{1, 2}`, `set()`, `{x for ...}`	`add`, `discard`, `|`, `&`, `-`, `^`, `in`	`in` is O(1); `{}` is an empty **dict**

control flow	shape
`if` / `elif` / `else`	conditional expression: `a if cond else b`
`for x in iterable:`	`range(a, b, step)`, `enumerate`, `zip`, `reversed`, `sorted`; `break`, `continue`, loop `else`
`while cond:`	for loops with no fixed count; watch the exit condition
`match value:`	`case 0:`, `case [x, y]:`, `case {"k": v}:`, `case Point(x=0):`, `case _:`
`try` / `except E as e` / `else` / `finally`	catch the narrowest error; `finally` for clean-up

functions and classes	remember
`def f(a, b=1, *args, c, **kw)`	defaults evaluated once; `*` makes keyword-only; return None by default
`lambda x: x + 1`	one expression; mostly for `key=` arguments
`class C(Base): def __init__(self, ...)`	`self.x` per object; class attributes shared; `super().__init__(...)`
`__repr__`, `__str__`, `__eq__`, `__len__`	dunder methods hook into built-ins (Chapter 5)
`import m`, `from m import f`, `import m as x`	`if __name__ == "__main__":` for script code

Scoring, and what to read next

missed	you're probably fuzzy on	read
A1–A3, A11	how numbers are stored	Ch 11 (floats, Decimal, Fraction)
A9, B1, B12	names vs objects (aliasing)	Ch 3: the most important chapter after this
B4, B5	dict behaviour	Ch 5, Ch 10 (`collections`)
C4, C6, C9	when things are evaluated and where names live	Ch 4 (functions, scope)
C10, C11	the data model (dunder methods)	Ch 5
C12	exceptions	Ch 6

30 or more out of 36: skim Part I and spend your time from Chapter 7 onwards. Under 25: read Chapters 2–6 properly; they're the foundation everything else stands on.

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 1.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 1.1` shows other valid solutions and their trade-offs.

`1.1` build from a spec Top words in a text
`1.2` build from a spec Run-length encoding
`1.3` build from a spec Group anagrams
`1.4` fix the bug The grid that changes itself
`1.5` build from a spec A command parser with match
`1.6` mini-project Mini-project: bank accounts with rules

PASSPORT STAMP ask the same question of every new language (Appendix B)

4. Control flow

`if`/`elif`/`else`, `for` over any iterable, `while`, `match` (3.10+), loop `else`, `break`/`continue`. Blocks are defined by indentation.

6. Data structures

Built in: `list`, `tuple`, `dict` (ordered), `set`, `str`, `bytes`; comprehensions for all of them.

3. Typing

Dynamic, strong typing: `"5" + 5` is a TypeError; bool is a subclass of int.

DEEP DIVE (your vault)

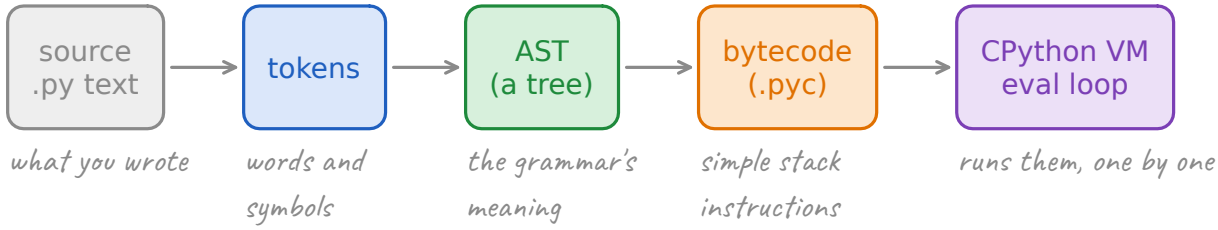
Vault notes: 5.9 Numeric objects · 5.11 Strings, Unicode, and textual value · 5.14 Truth values · 7.3 Lists as mutable sequences · 7.6 Dictionaries as mappings · 8.5 Defaults are evaluated at definition time · 91 Practice Questions.

Chapter 2

How Python runs your code

from the text you type to things happening

When you run `python3 grades.py`, Python doesn't read your file line by line and act on it. It **translates the whole file first**, then runs the translation on a virtual machine. Knowing the stages explains a lot of everyday puzzles: why a syntax error stops everything before line 1 runs, why a misspelt name only fails when that line is reached, and what those `__pycache__` folders are.



stage	done by	you can look at it with
tokenise	the tokenizer	`python3 -m tokenize file.py`
parse	the parser	`ast.parse(src)`, `python3 -m ast file.py`
compile	the compiler	`compile(src, name, 'exec')`, `dis.dis(func)`
execute	the evaluation loop (ceval.c)	`sys.settrace`, `pdb`, or just `print`

KEY IDEA

Compile time is when the whole file is checked and translated. **Run time** is when each instruction actually executes. Most errors you meet happen at run time; syntax errors are the exception.

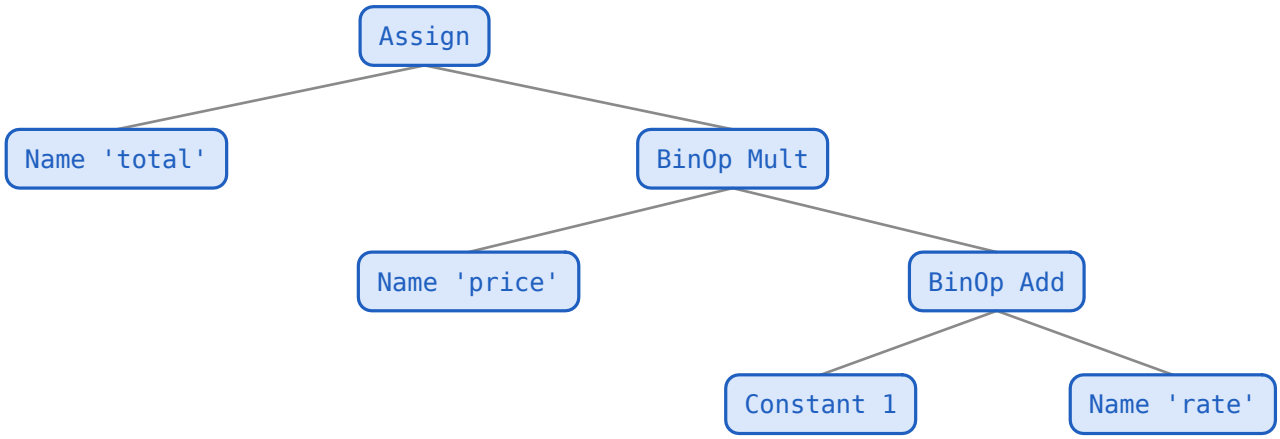
Stage 1 and 2: tokens, then a tree

The tokenizer chops text into tokens: names, operators, numbers, and indentation changes (Python's blocks are real tokens, `INDENT` and `DEDENT`).

```
import io, tokenize
src = "total = price * (1 + rate)"
for tok in tokenize.generate_tokens(io.StringIO(src).readline):
    if tok.string.strip():
        print(f"{tokenize.tok_name[tok.type]:<6} {tok.string!r}")
```

```
NAME   'total'
OP     '='
NAME   'price'
OP     '*'
OP     '('
NUMBER '1'
OP     '+'
NAME   'rate'
OP     ')'
```

The parser checks the tokens against Python's grammar and builds an **abstract syntax tree**. Brackets have gone: the tree's shape **is** the precedence. Multiplication sits above the addition, so the addition happens first:




```
>>> import ast
>>> print(ast.dump(ast.parse("x = a + b * 2").body[0].value))
BinOp(left=Name(id='a', ctx=Load()), op=Add(), right=BinOp(left=Name(id='b', ctx=Load…
```

> **WHY**
> This is where `SyntaxError` comes from: if the tokens can't form a tree, there's no program to run. That's why one missing bracket on line 200 stops line 1 from running.

Stage 3: bytecode, for a stack machine

The compiler turns the tree into **bytecode**: a list of tiny instructions for a virtual machine that works with a stack. Values are pushed on, operations pop their inputs and push the result. `dis` shows it:

```python
import dis

def f(a, b):
    return a * (b + 1)

for ins in dis.get_instructions(f):
    print(f"{ins.opname:<36} {ins.argrepr}")
```

```
RESUME
LOAD_FAST_BORROW_LOAD_FAST_BORROW    a, b
LOAD_SMALL_INT
BINARY_OP                            +
BINARY_OP                            *
RETURN_VALUE
```

Here's the stack as `f(3, 4)` runs (the long first name is two loads fused into one instruction, a 3.14 speed trick):

instruction	stack afterwards (top on the right)
`LOAD_FAST..._LOAD_FAST (a, b)`	`3  4`
`LOAD_SMALL_INT 1`	`3  4  1`
`BINARY_OP +`	`3  5`
`BINARY_OP *`	`15`
`RETURN_VALUE`	`(empty) → returns 15`

IN OTHER LANGUAGES

This is the same design as the Java Virtual Machine and .NET: compile to bytecode, run it on a VM. C and Rust compile straight to machine code instead, which is a big part of why they're faster for tight loops. PyPy is a different Python implementation that compiles hot bytecode to machine code while running (a JIT); CPython 3.13+ has an experimental JIT too.

Compile time vs run time

Two programs, each with one mistake on its last line. Watch **when** each mistake is noticed:

```
src = 'print("line 1 runs?")\nprint("oops"'
try:
    compile(src, "a.py", "exec")
except SyntaxError as e:
    print("SyntaxError before anything ran:", e.msg, "(line", e.lineno, ")")
```

```
SyntaxError before anything ran: '(' was never closed (line 2 )
```

```
src = 'print("line 1 runs?")\nprint(undefined_name)'
try:
    exec(compile(src, "b.py", "exec"))
except NameError as e:
    print("NameError, after line 1 ran:", e)
```

```
line 1 runs?
NameError, after line 1 ran: name 'undefined_name' is not defined
```

The first program never started. The second ran line 1 and only failed when it reached the bad name, because looking a name up is a run-time action. The same goes for functions: a typo inside a function body is only noticed when the function is **called**.

```
def report():
    return totl * 2          # typo, but no error yet

print("defined fine")
try:
    report()
except NameError as e:
    print("only now:", e)
```

← *compiles: it's just a name*

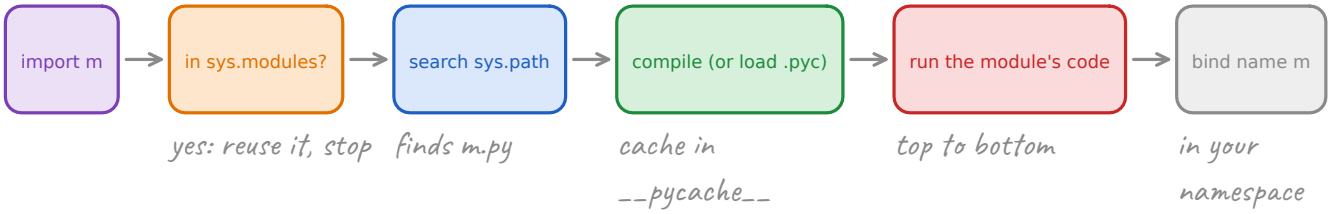
```
defined fine
only now: name 'totl' is not defined
```

TRAP

So "it ran without errors" means only the paths you exercised are free of these mistakes. Tests (Ch 17) and a linter or type checker (which read the code without running it) catch the rest.

What import really does

`import grades` isn't a text include. It's a run-time statement that **executes** the module once and caches the result:




```
1  import sys, pathlib, importlib
2
3  pathlib.Path("noisy.py").write_text('print("noisy is running!")\nX = 42\n')
4  sys.path.insert(0, ".")
5
6  import noisy
7  import noisy
8  print(noisy.X, "noisy" in sys.modules)
9  print(sorted(p.name for p in pathlib.Path("__pycache__").iterdir()))
10 importlib.reload(noisy)
```

```
noisy is running!
42 True
['noisy.cpython-314.pyc']
noisy is running!
```

line 6: runs the file

line 7: cached: prints nothing

line 9: the saved bytecode

line 10: reload forces a re-run

Because importing **runs** the file, any top-level code (prints, tests, a whole program) runs on import. The standard guard keeps script-only code from running when the file is imported:

```
def main():
    print("doing the real work")

if __name__ == "__main__":     # true only when run as a script
    main()
```

how the file is used	its `__name__` is	so the guarded code...
`python3 grades.py`	`"__main__"`	runs
`python3 -m grades`	`"__main__"`	runs
`import grades` (from another file)	`"grades"`	doesn't run

TRAP

Never name your file after a stdlib module (`random.py`, `json.py`, `secrets.py`). The current folder comes first on `sys.path`, so `import random` anywhere in your program imports **your** file instead.

Ways to start Python, and which Python

```
$ python3 grades.py              # run a file
$ python3 -m http.server 8000    # run a module found on sys.path (here: a stdlib web se
$ python3 -c 'print(2**100)'     # run a string
$ python3 -i grades.py           # run, then drop into the REPL with its variables
$ python3 -X importtime -c 'import json'   # how long each import took
```

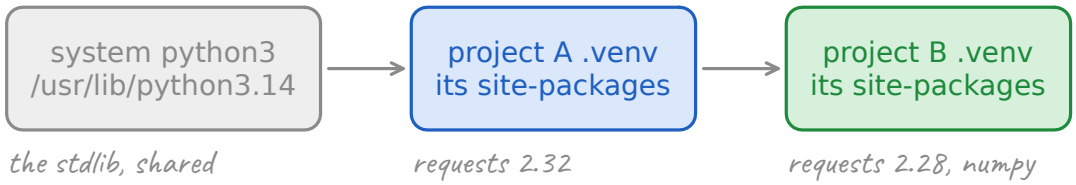
You may have several Pythons installed. Inside a program, `sys` tells you which one is running and where it looks for modules:

```python
import sys
print(sys.version_info[:3], sys.implementation.name)
print(sys.executable.rsplit("/", 2)[-2:])
print([p for p in sys.path if p.endswith("site-packages")][:1])
```

```
(3, 14, 6) cpython
['bin', 'python3']
['/home/Marau/.local/lib/python3.14/site-packages']
```

Virtual environments in one picture

A virtual environment (`python3 -m venv .venv`) is a folder with its own `python` link and its own `site-packages`, so each project's pip installs stay separate. Activating it just puts its `bin/` first on your `PATH`. Chapter 19 uses one.



Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 2.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 2.1` shows other valid solutions and their trade-offs.

`2.1` learn an unseen module Count the node types in a program (ast)
`2.2` build from a spec A safe calculator (no eval)
`2.3` learn an unseen module Which globals does a function use? (dis)
`2.4` refactor, tests stay green Make the script importable
`2.5` mini-project Mini-project: compile to your own stack machine

PASSPORT STAMP ask the same question of every new language (Appendix B)

1. How does code run?

CPython compiles each module to bytecode (cached as `.pyc` in `__pycache__`), then a stack-based VM interprets it. Syntax errors at compile time; name, type and value errors at run time.

9. Modules and build

`import` runs a module once and caches it in `sys.modules`; it's found on `sys.path`. `if __name__ == "__main__"` separates script code from library code.

DEEP DIVE (your vault)

Vault notes: 15.2 From source bytes to an abstract syntax tree · 15.4 Code objects · 15.5 Disassembly and bytecode · 15.8 The evaluation loop · 10.2 Import is execution with caching · 10.5 `__name__` and entry execution · 10.14 Import security and shadowing.

Chapter 3

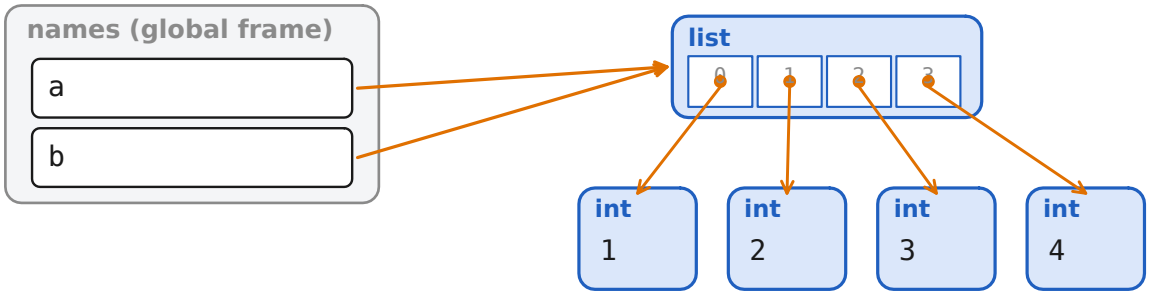
Names, objects and memory

names are labels on objects, not boxes holding values

Most "weird" Python behaviour comes from one wrong picture: thinking a variable is a box that **holds** a value. In Python a variable is a **name** that **refers to** an object living somewhere else in memory. Assignment never copies anything; it just points a name at an object.

```
1 a = [1, 2, 3]
2 b = a
3 b.append(4)
4 print(a, a is b, id(a) == id(b))
```

```
[1, 2, 3, 4] True True
```



There's **one** list and two names for it. `b.append(4)` changed the list, and `a` sees the change because `a` is the same list. Every picture in this chapter has this shape: names on the left, objects on the right, arrows for references.

> **KEY IDEA**
> `x = <expression>` means: evaluate the right side to an object, then make the name `x` refer to it. The name never contains the object.

Mutable vs immutable: the second half of the story

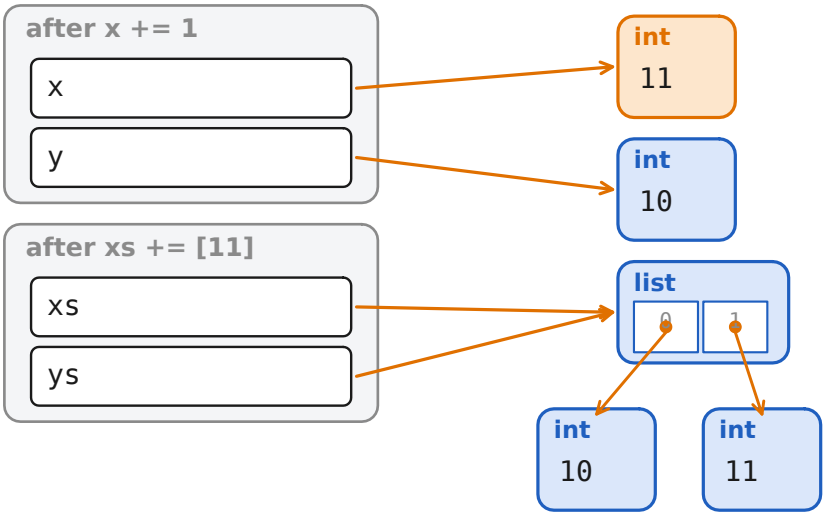
Some objects can change in place (**mutable**: list, dict, set, most class instances). Others can never change (**immutable**: int, float, str, tuple, frozenset, bytes). With an immutable object, "changing" a variable always means pointing the name at a **new** object:

```
1 x = 10
2 y = x
3 x += 1          # makes a NEW int 11 and rebinds x
4 print(x, y)
5
6 xs = [10]
7 ys = xs
8 xs += [11]      # list += mutates the SAME list in place
9 print(xs, ys)
```

← y still points at 10

← ys sees it

```
11 10
[10, 11] [10, 11]
```



immutable (safe to share)	mutable (sharing = aliasing)
`int`, `float`, `complex`, `bool`	`list`
`str`, `bytes`	`dict`, `set`, `bytearray`
`tuple` (but its items might be mutable)	instances of your classes (by default)
`frozenset`, `None`	`array.array`, NumPy arrays

TRAP

`x += 1` and `xs += [1]` look the same but do different things: for a list, `+=` is `xs.extend(...)` in place; for an int it builds a new object. `xs = xs + [1]` would build a new list and leave `ys` alone.

Passing arguments: the parameter is one more name

A function call doesn't copy arguments. The parameter becomes another name for the **same** object ("pass by assignment"). So a function can **mutate** what you pass in, but **rebinding** its parameter does nothing to the caller:

```python
def add_bonus(scores):
    scores.append(5)        # mutates the caller's list

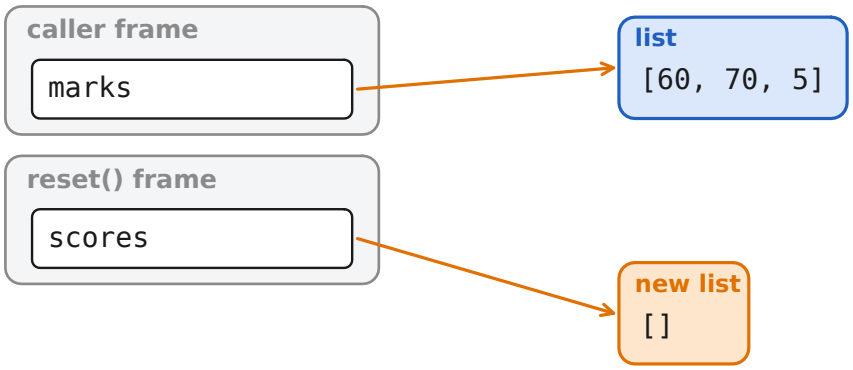
def reset(scores):
    scores = []             # rebinds the local name only

marks = [60, 70]
add_bonus(marks)
reset(marks)
print(marks)
```

← visible outside

← invisible outside

```
[60, 70, 5]
```



Inside `reset`, `scores = []` pointed the local name at a new empty list. The caller's list is untouched, and when `reset` returns, its frame and the empty list disappear.

> **RULE OF THUMB**
> A function should either **return a new value** or **mutate its argument and return None**, and its name should make clear which (`sorted(xs)` vs `xs.sort()`). Doing both confuses every caller.

Copies: shallow and deep

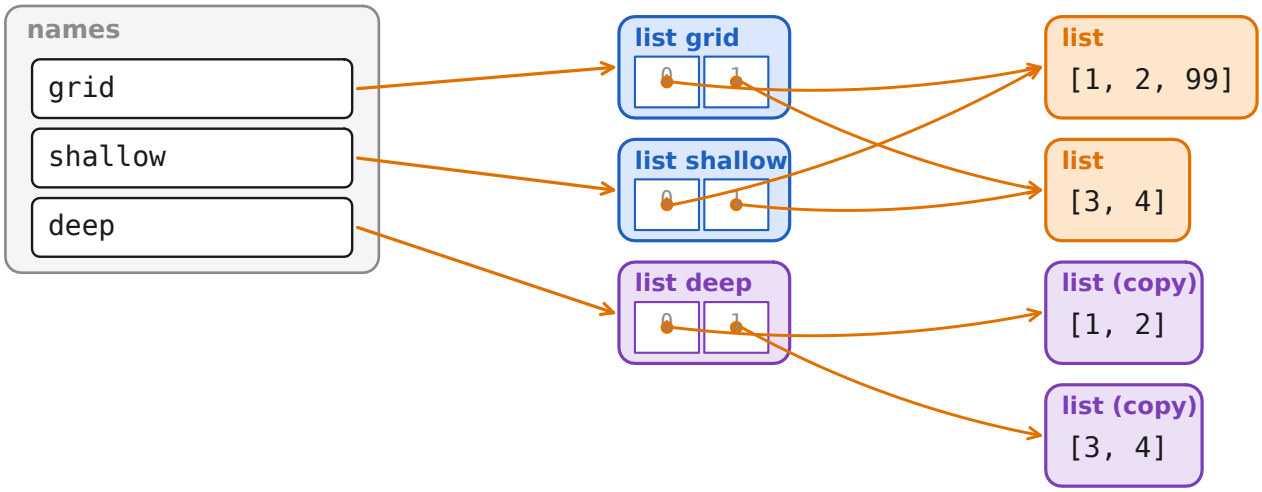
When you do want an independent object, copy it explicitly. A **shallow** copy makes a new outer container but reuses the items inside. For nested data that matters:

```python
import copy

grid = [[1, 2], [3, 4]]
shallow = grid.copy()           # same as list(grid) or grid[:]
deep = copy.deepcopy(grid)

grid[0].append(99)
print(shallow)                  # shares the inner lists
print(deep)                     # fully independent
```

```
[[1, 2, 99], [3, 4]]
[[1, 2], [3, 4]]
```



how	copies	inner objects
`b = a`	nothing (alias)	shared
`a.copy()`, `list(a)`, `a[:]`, `dict(d)`, `copy.copy(a)`	the outer container	shared
`copy.deepcopy(a)`	everything, recursively	new (sharing inside is preserved)

✎ YOUR GO (pencil, or the REPL, first)

Predict, then check: `m = [[0] * 3] * 3; m[1][1] = 5; print(m)`. Draw the boxes and arrows first.

Identity vs equality: is vs ==

`==` asks "same value?" by calling `__eq__`. `is` asks "same object?" by comparing identities (`id()`, in CPython the memory address). They differ more often than you'd think:

```
>>> a = [1, 2]; b = [1, 2]
>>> a == b, a is b
(True, False)
>>> x = 1000; y = int("1000")
>>> x == y, x is y
(True, False)
>>> s = None
>>> s is None
True
```

TRAP

Small ints (−5 to 256) and some strings are cached, so `is` sometimes "works" on them by accident. That's a CPython detail that changes between versions. Use `is` only for `None` (and other singletons like `True`/`False` or sentinel objects), and `==` for values.

Why lists can't be dictionary keys

A dict finds keys by their **hash**, computed once when the key is inserted. If a key could change after insertion, its hash would no longer match and the entry would be lost. So only immutable (hashable) objects can be keys or set members:

```
1 seen = {(1, 2): "tuple ok", "text": "str ok", frozenset({3}): "frozenset ok"}
2 print(len(seen))
3 seen[[1, 2]] = "list?"
```

```
3
TypeError: cannot use 'list' as a dict key (unhashable type: 'list')
```

```
seen[(1, [2])] = "tuple holding a list?"
```

```
TypeError: cannot use 'tuple' as a dict key (unhashable type: 'list')
```

How long objects live

CPython keeps a **reference count** on every object: how many names, list slots, attributes etc. point at it. When the count drops to zero the object is freed immediately. `del x` removes a **name**; the object only dies if that was the last reference:

```
import sys

data = [0] * 1000
alias = data
print(sys.getrefcount(data) - 1)   # minus the temporary ref from the call itself
del data
print(len(alias))                  # still alive: alias refers to it
```

```
2
1000
```

Reference counting can't free **cycles** (two objects pointing at each other), so a separate garbage collector (`gc`) finds and frees unreachable cycles from time to time:

```
import gc

class Node:
    def __init__(self):
        self.other = None

a, b = Node(), Node()
a.other, b.other = b, a       # a cycle
del a, b                      # unreachable, but refcounts are still 1
print(gc.collect() >= 2)      # the cycle collector frees them
```

```
True
```

size of...	on 64-bit CPython	why
`sys.getsizeof(0)`	28 bytes	every int is a full object (type pointer, refcount, digits)
`sys.getsizeof([])`	56 bytes	header plus room to grow
a list of 1,000 ints	~8 KB of pointers + 28 KB of int objects	the list holds references, not values
an `array.array` of 1,000 ints	~4 KB	raw 4-byte values packed together; NumPy does the same (Ch 18)

IN OTHER LANGUAGES

C has no references by default: `int x = y;` really copies, and you manage memory yourself. Java works like Python for objects (references) but copies primitives (`int`). Rust makes ownership explicit: assignment **moves** a value unless you borrow it. The passport question "what does assignment do?" has a different answer in each.

☐ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 3.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 3.1` shows other valid solutions and their trade-offs.

`3.1` fix the bug The function that changes your data
`3.2` build from a spec Write your own deep copy
`3.3` learn an unseen module Spot the aliasing (id and is)
`3.4` build from a spec Make it hashable
`3.5` mini-project Mini-project: undo for a document

PASSPORT STAMP ask the same question of every new language (Appendix B)

2. Memory model

Every value is an object on the heap; variables, list slots and attributes are references. Assignment and argument passing copy references, never objects. Freed by reference counting, plus a cycle collector.

3. Typing

Values have types; names don't. Mutable (list, dict, set, instances) vs immutable (int, str, tuple, frozenset); only hashable objects can be dict keys.

DEEP DIVE (your vault)

Vault notes: 5.3 Objects: identity, type, and value · 5.4 Names and bindings, not variable boxes · 5.6 References, reachability, and lifetime · 5.7 Mutability and shallow structure · 5.8 Equality, ordering, and hashing · 8.3 Python passes object references by assignment · 15.12 Reference counting · 15.13 Cyclic garbage collection.

Chapter 4

Functions, closures and generators

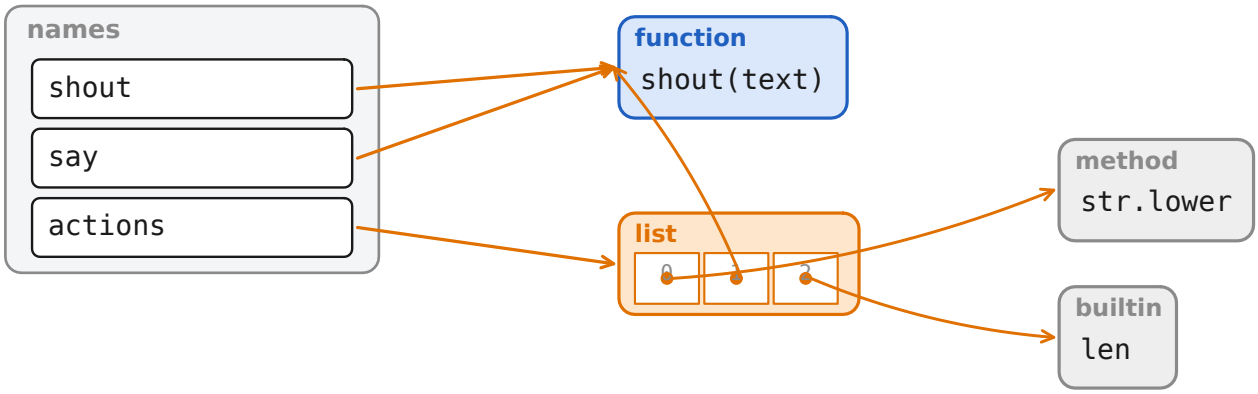
a function is an object you can pass, store, wrap and pause

You know how to define and call functions. This chapter is about what a function **is**: an object, created when `def` runs, bound to a name like any other value. Everything else follows: passing functions to functions, functions that remember things, functions that wrap other functions, and functions that pause halfway.

```
def shout(text):
    return text.upper() + "!"

say = shout                       # another name, same object
actions = [str.lower, shout, len]
print(say("hi"), [act("Hey") for act in actions])
print(type(shout).__name__, shout.__name__, shout.__code__.co_varnames)
```

```
HI! ['hey', 'HEY!', 3]
function shout ('text',)
```

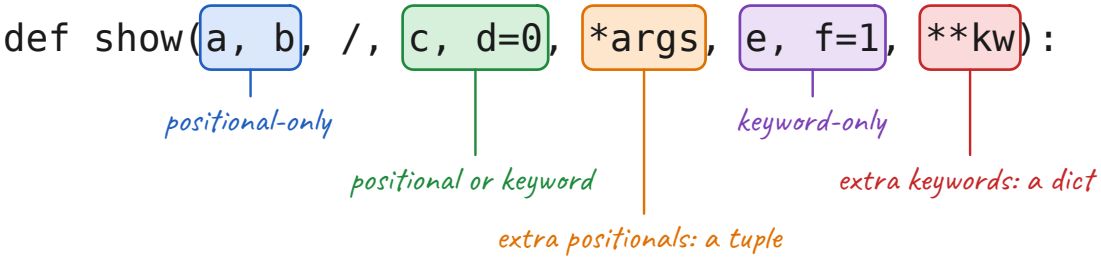


KEY IDEA

`def` is an assignment statement: it builds a function object (code, defaults, name, docstring) and binds it to a name. Calling is a separate step, written with `()`.

Parameter kinds: one signature, five zones

A signature can have up to five zones, always in this order. The `/` and `*` markers are walls: parameters before `/` can't be passed by name; parameters after `*` (or `*args`) must be:




```python
def show(a, b, /, c, d=0, *args, e, f=1, **kw):
    print(a, b, c, d, args, e, f, kw)

show(1, 2, 3, e=5)
show(1, 2, c=3, d=4, e=5, colour="red")
show(1, 2, 3, 4, 98, 99, e=5, f=6)
```

```
1 2 3 0 () 5 1 {}
1 2 3 4 () 5 1 {'colour': 'red'}
1 2 3 4 (98, 99) 5 6 {}
```

```python
show(1, b=2, c=3, e=5)
```

```
TypeError: show() missing 1 required positional argument: 'b'
```

you want...	write	why
a flag that must be named	`def save(path, *, overwrite=False)`	`save(p, True)` is unreadable; this forces `overwrite=True`
any number of values	`def total(*nums)`	`nums` is a tuple
to forward everything	`def wrapper(*args, **kwargs)`	decorators (later this chapter)
to spread a list into a call	`f(*items)`, `f(**options)`	the call-side mirror of `*args`
names you may rename later	`def area(w, h, /)`	callers can't depend on the names

The default-value trap

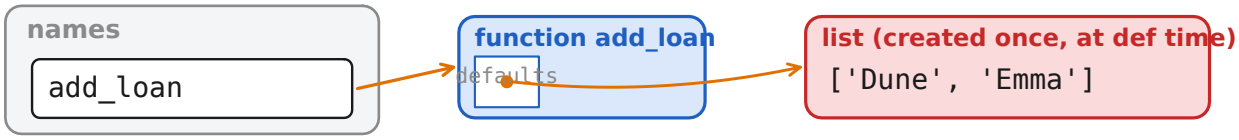
Defaults are evaluated **once**, when `def` runs, and stored on the function object. A mutable default is therefore shared by every call that doesn't pass that argument:

```python
def add_loan(title, loans=[]):
    loans.append(title)
    return loans

print(add_loan("Dune"))
print(add_loan("Emma"))           # surprise
print(add_loan.__defaults__)      # the one shared list
```

← *Dune is still there*

```
['Dune']
['Dune', 'Emma']
(['Dune', 'Emma'],)
```



The fix is the `None` sentinel: make the default immutable, create the list inside the call.

```python
def add_loan(title, loans=None):
    if loans is None:
        loans = []                # a new list on every call
    loans.append(title)
    return loans

print(add_loan("Dune"), add_loan("Emma"))
```

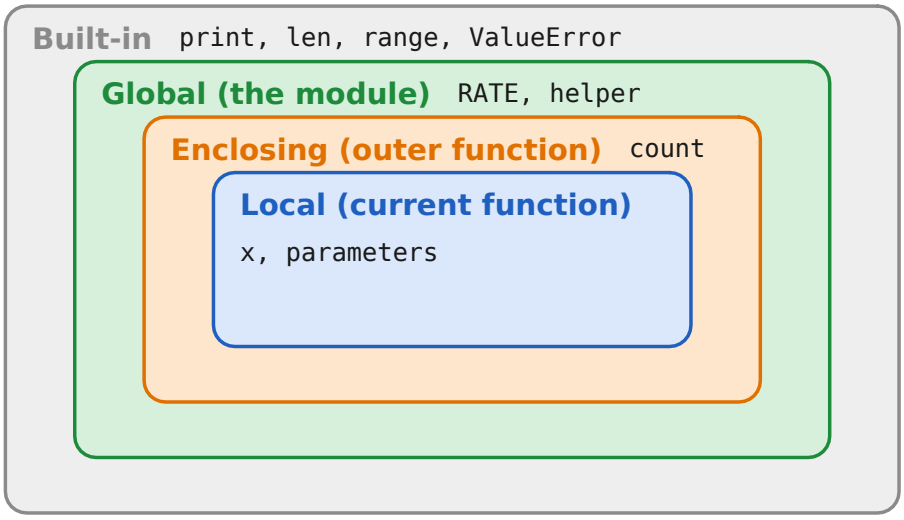
```
['Dune'] ['Emma']
```

WHY

This isn't a bug in Python; it falls out of "`def` runs once". The same rule makes `def stamp(t=time.time())` record the time the module loaded, not the time of the call.

Scope: where Python looks a name up

When code uses a name, Python searches four scopes in order, **L**ocal, **E**nclosing, **G**lobal, **B**uilt-in, and takes the first match:



Which scope a name belongs to is decided at **compile** time: if a function assigns to a name anywhere in its body, that name is local everywhere in that body. That produces this classic error:

```
count = 0

def bump():
    count += 1        # assignment makes count local, so it's read before being set

bump()
```

```
UnboundLocalError: cannot access local variable 'count' where it is not associated wi…
```

```
def make_counter():
    count = 0
    def bump():
        nonlocal count      # rebind the enclosing function's name
        count += 1
        return count
    return bump

tick = make_counter()
print(tick(), tick(), tick())
```

← *global count would mean the module's count*

```
1 2 3
```

RULE OF THUMB

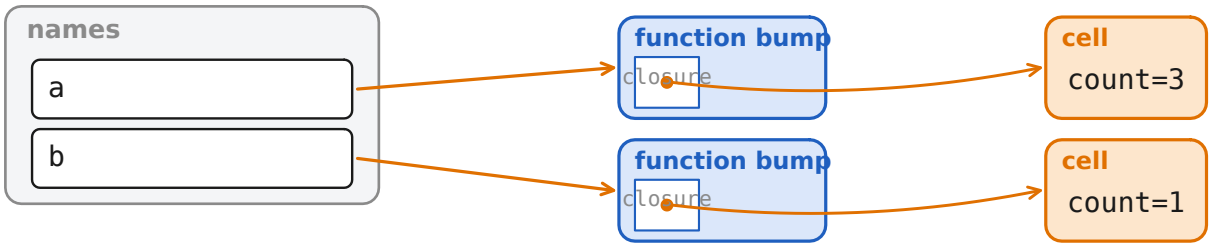
Read globals freely, but avoid `global` for writing; pass values in and return them out. Module-level constants (`RATE = 0.2`) are fine; module-level mutable state is a source of bugs.

Closures: functions that remember

`tick` above kept its own `count` after `make_counter` had returned. The inner function holds a reference to a **cell** that holds the variable. Each call of `make_counter` makes a new cell, so counters are independent:

```
a, b = make_counter(), make_counter()
a(); a()
print(a(), b())
print(a.__closure__[0].cell_contents, a.__code__.co_freevars)
```

```
3 1
3 ('count',)
```



The late-binding trap

A closure captures the **variable**, not its value at the time. Functions made in a loop all see the loop variable's **final** value:

captures the variable (bug)

```
fs = [lambda: i for i in range(3)]
print([g() for g in fs])
```

```
[2, 2, 2]
```

binds the value now (fix)

```
fs = [lambda i=i: i for i in range(3)]
print([g() for g in fs])
```

```
[0, 1, 2]
```

> **IN OTHER LANGUAGES**
> JavaScript closures work the same way (and had the same loop bug with `var`, fixed by `let`). Java lambdas may only capture variables that never change, which rules the bug out.

Functions that take functions

Because functions are values, a lot of the standard library takes a function as an argument ("higher-order" functions). The most useful by far is `key=`: tell `sorted`/`min`/`max` **what to compare**, not how to sort:

```python
from operator import itemgetter, attrgetter
from functools import partial

books = [("Dune", 1965, 4.3), ("Emma", 1815, 4.0), ("Ubik", 1969, 4.1)]
print(sorted(books, key=itemgetter(1))[0][0])          # oldest
print(max(books, key=lambda b: b[2])[0])               # best rated
print(sorted(books, key=lambda b: (-b[2], b[0]))[0])   # rating desc, then title

to_int = partial(int, base=2)                          # fix some arguments
print(to_int("1011"))
```

```
Emma
Dune
('Dune', 1965, 4.3)
11
```

map and filter

```python
nums = range(10)
out = list(map(lambda n: n * n,
               filter(lambda n: n % 2, nums)))
print(out)
```

```
[1, 9, 25, 49, 81]
```

a comprehension

```python
nums = range(10)
out = [n * n for n in nums if n % 2]

print(out)
```

```
[1, 9, 25, 49, 81]
```

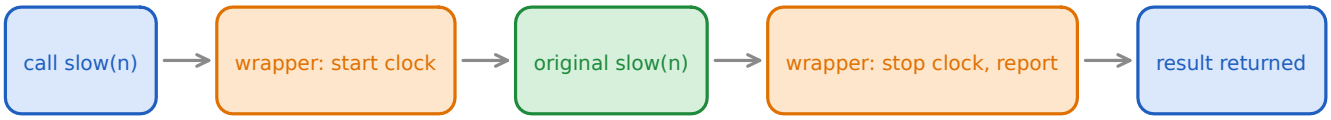


RULE OF THUMB

Prefer a comprehension when you'd need a `lambda`; `map(str, xs)` with an existing function is fine. `itemgetter`/`attrgetter` beat `lambda x: x[1]` for speed and clarity.

Decorators: wrapping a function

A decorator is a function that takes a function and returns a replacement, usually a wrapper that does something before and after calling the original. `@timed` above a `def` is just shorthand for `slow = timed(slow)`:




```
1  import functools, time
2
3  def timed(func):
4      @functools.wraps(func)                # copy name, docstring etc. onto the wrapper
5      def wrapper(*args, **kwargs):
6          start = time.perf_counter()
7          result = func(*args, **kwargs)
8          ms = (time.perf_counter() - start) * 1000
9          print(f"{func.__name__} took {ms:.3f} ms")
10         return result
11     return wrapper
12
13 @timed
14 def slow(n):
15     return sum(range(n))
16
17 print(slow(10_000), slow.__name__)
```

```
slow took 0.085 ms
49995000 slow
```

line 4: without it: __name__ == 'wrapper'

line 5: forwards any arguments

A decorator that takes options is one more layer: a function that **returns** a decorator.

```python
def retry(times):
    def decorate(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError:
                    if attempt == times:
                        raise
        return wrapper
    return decorate

calls = []
@retry(times=3)
def flaky():
    calls.append(1)
    if len(calls) < 3:
        raise ConnectionError("try again")
    return "ok"

print(flaky(), len(calls))
```

```
ok 3
```

The standard library's decorators

```python
import functools, time

@functools.cache                    # remember every result (lru_cache(maxsize=None))
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

t = time.perf_counter(); fib(300); ms = (time.perf_counter() - t) * 1000
print(fib(300) % 1000, ms < 50, fib.cache_info().hits)
```

```
600 True 299
```

decorator	does
`@functools.cache`, `@functools.lru_cache(maxsize=128)`	memoise results (arguments must be hashable)
`@property`, `@staticmethod`, `@classmethod`	change how a method is accessed (Ch 5)
`@dataclasses.dataclass`	write `__init__`, `__repr__`, `__eq__` for you (Ch 5)
`@contextlib.contextmanager`	turn a generator into a `with` block (Ch 6)
`@functools.singledispatch`	choose an implementation by argument type

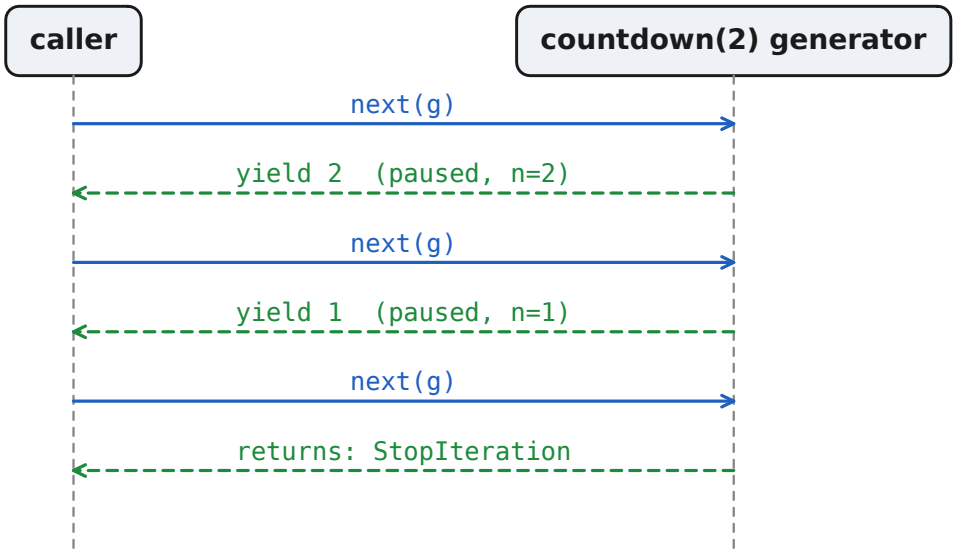
Generators: functions that pause

A function containing `yield` doesn't run when called. It returns a **generator**; each `next()` runs the body up to the next `yield`, hands out that value, and freezes there, with all its local variables kept, until the next `next()`:

```python
def countdown(n):
    print("  (starting)")
    while n > 0:
        yield n
        n -= 1
    print("  (done)")

g = countdown(2)
print("made it:", type(g).__name__)
print(next(g))
print(next(g))
print(list(g))           # runs to the end: StopIteration ends the list
```

```
made it: generator
  (starting)
2
1
  (done)
[]
```



Generators are lazy: they compute one item at a time, so a pipeline of them works on data far bigger than memory. Each stage pulls from the one before:

```python
import sys

def read_lines(n):                    # pretend this is a 10 GB log file
    for i in range(n):
        yield f"2026-09-{i % 30 + 1:02} GET /book/{i % 7}"

lines = read_lines(1_000_000)
hits = (ln for ln in lines if ln.endswith("/book/3"))     # generator expression
days = (ln[:10] for ln in hits)
print(next(days), sum(1 for _ in days) + 1)
print(sys.getsizeof(days), "bytes, vs",
      sys.getsizeof([0] * 1_000_000), "for a list of a million")
```

```
2026-09-04 142857
208 bytes, vs 8000056 for a list of a million
```

TRAP

A generator can only be consumed **once**. A second `for` over the same generator gets nothing, silently. If you need two passes, make a list, or call the generator function again.

Type hints: documentation a tool can check

Annotations say what a function expects and returns. Python itself **ignores** them at run time; they're read by humans, editors and checkers like mypy (Ch 17):

```python
from collections.abc import Iterable

def average(marks: Iterable[float], *, ndigits: int = 1) -> float | None:
    marks = list(marks)
    return round(sum(marks) / len(marks), ndigits) if marks else None

print(average([60, 71.5]), average([]), average.__annotations__["return"])
print(average("12"))     # the hint says numbers; Python only notices when sum() fails
```

```
65.8 None float | None
TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

HONEST NOTE

Type hints don't make Python statically typed. They catch a class of bugs early **if** you run a checker. For a passport answer: Python is dynamically typed, with optional, gradual static typing on top. TypeScript did the same for JavaScript.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 4.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 4.1` shows other valid solutions and their trade-offs.

`4.1` fix the bug Two bugs that every Python programmer writes once
`4.2` build from a spec Your own memoising decorator
`4.3` build from a spec Check types at run time from the hints
`4.4` build from a spec Lazy chunks and windows
`4.5` refactor, tests stay green Name your flags
`4.6` mini-project Mini-project: an event bus with decorators

PASSPORT STAMP ask the same question of every new language (Appendix B)

5. Functions and scope

Functions are first-class objects made at `def` time. Parameters: positional-only / normal / `*args` / keyword-only / `**kwargs`. Defaults evaluated once. LEGB scope fixed at compile time; closures capture variables (cells); decorators wrap.

4. Control flow

Generators: `yield` pauses a function; iteration pulls values lazily until `StopIteration`.

3. Typing

Dynamic typing with optional annotations, checked only by external tools.

DEEP DIVE (your vault)

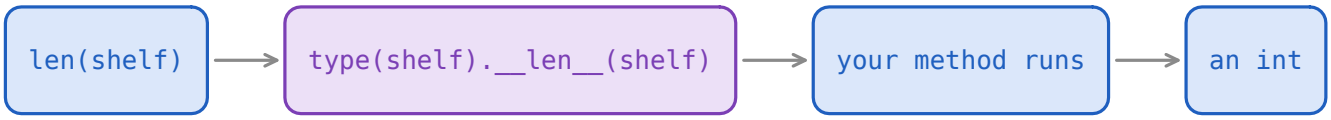
Vault notes: 8.2 Parameters and arguments · 8.5 Defaults are evaluated at definition time · 8.7 Namespaces and the LEGB rule · 8.8 First-class functions · 8.9 Closures · 8.13 Decorators derived from first-class functions · 8.14 Decorator factories and timing · 6.9 Generator functions and suspension · 6.10 Generator expressions · 11.13 functools · 12.18 Type hints.

Chapter 5

Classes and the data model

every piece of syntax is a method call you can define

You've written classes with `__init__` and methods. The step up is the **data model**: Python turns almost every piece of syntax into a call to a specially named ("dunder") method. Define the method, and your objects work with the syntax, the built-ins and the whole standard library.



you write	Python calls	you write	Python calls
`a + b`	`a.__add__(b)`	`len(x)`	`x.__len__()`
`a == b`	`a.__eq__(b)`	`x[k]`	`x.__getitem__(k)`
`a < b`	`a.__lt__(b)`	`k in x`	`x.__contains__(k)`
`for v in x`	`iter(x)` → `x.__iter__()`	`bool(x)`, `if x:`	`x.__bool__()` or `__len__`
`repr(x)`	`x.__repr__()`	`f(...)` on an object	`x.__call__(...)`
`hash(x)`	`x.__hash__()`	`with x:`	`__enter__`, `__exit__` (Ch 6)

KEY IDEA

This is why `len` is a function and not a method: `len(x)` works on any object whose class defines `__len__`. It's an interface, not inheritance. Ask "which protocol does this syntax use?" and you can make your own types feel built in.

Where attributes live

An instance has its own dict of attributes. Methods live on the **class**. `obj.name` looks in the instance first, then the class, then its base classes in order:

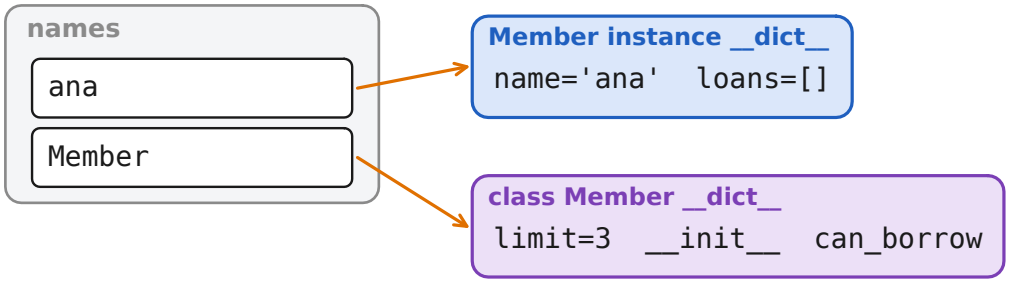
```
class Member:
    limit = 3                          # class attribute: shared by all members

    def __init__(self, name):
        self.name = name               # instance attribute
        self.loans = []

    def can_borrow(self):
        return len(self.loans) < self.limit

ana = Member("ana")
print(vars(ana), "limit" in vars(ana), ana.limit)
print(ana.can_borrow(), Member.can_borrow(ana))     # the same call
```

```
{'name': 'ana', 'loans': []} False 3
True True
```



`ana.can_borrow()` finds the function on the class and passes `ana` as `self`. That's all a method is: a function stored on the class, with the instance filled in as its first argument.

```
ana.limit = 10                  # creates an INSTANCE attribute that hides the class one
ben = Member("ben")
print(ana.limit, ben.limit, Member.limit)
```

```
10 3 3
```

TRAP

A mutable **class** attribute (`loans = []` at class level) is shared by every instance, the same bug as the mutable default argument. Create per-object state in `__init__`.

repr, equality and hashing

A class you write gets the default `repr` (`<Book object at 0x7f...>`) and identity equality (two books with the same title are not `==`). For a **value** type you want both fixed:

```python
class Book:
    def __init__(self, title, year):
        self.title, self.year = title, year

    def __repr__(self):
        return f"Book({self.title!r}, {self.year})"     # looks like the code to rebuild it

    def __eq__(self, other):
        if not isinstance(other, Book):
            return NotImplemented                       # let the other side try
        return (self.title, self.year) == (other.title, other.year)

    def __hash__(self):
        return hash((self.title, self.year))            # equal objects MUST hash equal

a, b = Book("Dune", 1965), Book("Dune", 1965)
print(a, a == b, a is b, len({a, b}))
```

```
Book('Dune', 1965) True False 1
```

TRAP

Defining `__eq__` without `__hash__` makes instances **unhashable** (Python sets `__hash__ = None` to protect you). And only hash fields that never change; mutate a key after putting it in a set and the set can't find it again.

dataclasses write all of that for you

by hand

```python
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __repr__(self):
        x, y = self.x, self.y
        return f"Point({x=}, {y=})"
    def __eq__(self, o):
        return (self.x, self.y) == (o.x, o.y)
    def __hash__(self):
        return hash((self.x, self.y))
print(Point(1, 2) == Point(1, 2), Point(1, 2))
```

```
True Point(x=1, y=2)
```

@dataclass(frozen=True)

```python
from dataclasses import dataclass

@dataclass(frozen=True)
class Point:
    x: int
    y: int



print(Point(1, 2) == Point(1, 2), Point(1, 2))
```

```
True Point(x=1, y=2)
```



dataclass option	gives you
`@dataclass`	`__init__`, `__repr__`, `__eq__` from the annotated fields
`frozen=True`	immutable fields (assignment raises), and `__hash__`
`order=True`	`<`, `<=`, `>`, `>=` comparing fields in order (so `sorted` works)
`field(default_factory=list)`	a fresh mutable default per instance (no shared-list bug)
`slots=True`	`__slots__`: smaller instances, no accidental new attributes

Container and iteration protocols

Implement a few dunders and your object works with `len`, `in`, indexing, slicing, `for`, `sorted`, `list()`, unpacking, `sum`... none of which knows your class exists:

```
1  class Shelf:
2      def __init__(self, titles=()):
3          self._titles = list(titles)
4
5      def __len__(self):
6          return len(self._titles)
7
8      def __getitem__(self, i):                    # handles ints AND slices
9          got = self._titles[i]
10         return Shelf(got) if isinstance(i, slice) else got
11
12     def __iter__(self):
13         yield from self._titles
14
15     def __contains__(self, title):
16         return title.lower() in (t.lower() for t in self._titles)
17
18     def __repr__(self):
19         return f"Shelf({self._titles})"
20
21 s = Shelf(["Dune", "Emma", "Ubik"])
22 print(len(s), s[0], s[1:], "emma" in s, sorted(s, reverse=True))
23 first, *rest = s
24 print(first, rest, bool(Shelf()))
```

```
3 Dune Shelf(['Emma', 'Ubik']) True ['Ubik', 'Emma', 'Dune']
Dune ['Emma', 'Ubik'] False
```

line 9: a slice returns a Shelf

line 15: case-insensitive

WHY

`bool(Shelf())` is `False` without a `__bool__` method: Python falls back to `__len__`, and zero length means false. Protocols have sensible fallbacks: with only `__getitem__`, `for` and `in` still work by trying indexes 0, 1, 2... until `IndexError`.

Operators: a money type

Arithmetic operators are two-sided. For `a + b` Python tries `a.__add__(b)`; if that returns `NotImplemented`, it tries `b.__radd__(a)`. That's what makes `sum()` (which starts from `0`) work:

```python
from dataclasses import dataclass
from decimal import Decimal
from functools import total_ordering

@total_ordering
@dataclass(frozen=True)
class Money:
    pence: int

    def __add__(self, other):
        if isinstance(other, Money):
            return Money(self.pence + other.pence)
        return NotImplemented

    def __radd__(self, other):          # 0 + Money, from sum()
        return self if other == 0 else NotImplemented

    def __mul__(self, k):
        if isinstance(k, (int, float, Decimal)):
            return Money(round(self.pence * k))
        return NotImplemented
    __rmul__ = __mul__

    def __lt__(self, other):
        return self.pence < other.pence

    def __str__(self):
        return f"£{self.pence / 100:.2f}"

fines = [Money(150), Money(275), Money(20)]
print(sum(fines), 3 * Money(150), max(fines), Money(1) >= Money(1))
```

← fills in <=, >, >= from < and ==

```
£4.45 £4.50 £2.75 True
```

```
Money(100) + 5
```

```
TypeError: unsupported operand type(s) for +: 'Money' and 'int'
```

RULE OF THUMB

Return `NotImplemented` (don't raise) when you don't recognise the other operand. Python then tries the reflected method and only raises `TypeError` if both sides give up, with a clear message. Money is stored in integer pence: never in a `float` (Ch 11).

Properties: attributes with rules

A `@property` looks like an attribute to the caller but runs code. Use it to **protect an invariant** (a rule that must always hold) or compute a value, without changing how callers write `obj.days`:

```
class Loan:
    MAX_DAYS = 28

    def __init__(self, title, days=14):
        self.title = title
        self.days = days                # goes through the setter below

    @property
    def days(self):
        return self._days

    @days.setter
    def days(self, value):
        if not 1 <= value <= self.MAX_DAYS:
            raise ValueError(f"days must be 1..{self.MAX_DAYS}, got {value}")
        self._days = value

    @property
    def weeks(self):                    # computed, read-only
        return self.days / 7

loan = Loan("Dune")
loan.days = 21
print(loan.days, loan.weeks)
loan.days = 90
```

```
21 3.0
ValueError: days must be 1..28, got 90
```

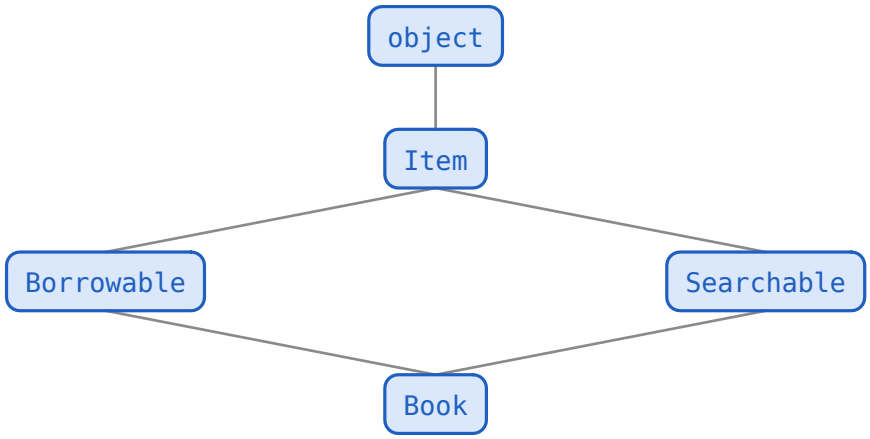
line 6: validated even in __init__

IN OTHER LANGUAGES

Java and C# write `getDays()`/`setDays()` up front, in case validation is needed later. Python starts with a plain attribute and swaps in a property only when needed, and callers don't change. The leading underscore in `_days` means "internal" by convention only.

Inheritance, composition and the MRO

Inheritance says "is a"; composition says "has a". Prefer composition unless the subclass truly can be used everywhere the base class can. When you do inherit, `super()` calls the next class in the **method resolution order** (MRO), a single line Python computes from the class tree:




```python
class Item:
    def describe(self):
        return ["item"]

class Borrowable(Item):
    def describe(self):
        return ["borrowable"] + super().describe()

class Searchable(Item):
    def describe(self):
        return ["searchable"] + super().describe()

class Book(Borrowable, Searchable):
    pass

print(" → ".join(c.__name__ for c in Book.__mro__))
print(Book().describe())
```

← next in the MRO, not 'my parent'

```
Book → Borrowable → Searchable → Item → object
['borrowable', 'searchable', 'item']
```

`super()` in `Borrowable` called `Searchable`, a class it has never heard of, because that's next in `Book`'s MRO. Every class runs exactly once, even in a diamond.

Interfaces: ABCs and duck typing

```python
from abc import ABC, abstractmethod
from typing import Protocol

class Notifier(ABC):                    # nominal: subclasses must implement send
    @abstractmethod
    def send(self, to, text): ...

class CanSend(Protocol):                # structural: anything with a send() matches
    def send(self, to, text): ...

class Email(Notifier):
    def send(self, to, text):
        return f"mail {to}: {text}"

print(Email().send("ana", "due"))
Notifier()
```

```
mail ana: due
TypeError: Can't instantiate abstract class Notifier without an implementation for abst…
```

Enums: a fixed set of named values

```python
from enum import Enum, auto

class Status(Enum):
    ON_SHELF = auto()
    ON_LOAN = auto()
    LOST = auto()

s = Status.ON_LOAN
print(s, s.name, s.value, s is Status["ON_LOAN"], [m.name for m in Status])
match s:
    case Status.ON_LOAN | Status.LOST:
        print("not available")
```

```
Status.ON_LOAN ON_LOAN 2 True ['ON_SHELF', 'ON_LOAN', 'LOST']
not available
```

RULE OF THUMB

Use an `Enum` instead of magic strings (`"on_loan"`) or numbers: typos become `AttributeError`s, editors autocomplete, and `match` reads clearly. `IntEnum`/`StrEnum` when the values must also behave like ints or strings (e.g. stored in SQLite).

need	reach for
a bag of fields with behaviour	a class; `@dataclass` if it's mostly data
an immutable record	`@dataclass(frozen=True)` or `typing.NamedTuple`
a fixed set of choices	`enum.Enum`
"anything with a `.send()`"	duck typing; `typing.Protocol` to document it

shared behaviour with enforced methods	`abc.ABC` with `@abstractmethod`

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 5.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 5.1` shows other valid solutions and their trade-offs.

`5.1` build from a spec A 2D vector that feels built in
`5.2` fix the bug Shared state and a vanishing set member
`5.3` build from a spec A container with slicing
`5.4` build from a spec Temperatures with rules
`5.5` learn an unseen module Who defines this method?
`5.6` mini-project Mini-project: a library catalogue

PASSPORT STAMP ask the same question of every new language (Appendix B)

7. Abstraction

Classes with single and multiple inheritance (C3 MRO, cooperative `super()`); duck typing via protocols of dunder methods; ABCs and `typing.Protocol` for explicit interfaces; properties for invariants; dataclasses for records; enums for choices.

6. Data structures

User types plug into built-in syntax by implementing protocols: `__len__`, `__getitem__`, `__iter__`, `__contains__`, `__eq__`/`__hash__`, arithmetic with `NotImplemented`.

DEEP DIVE (your vault)

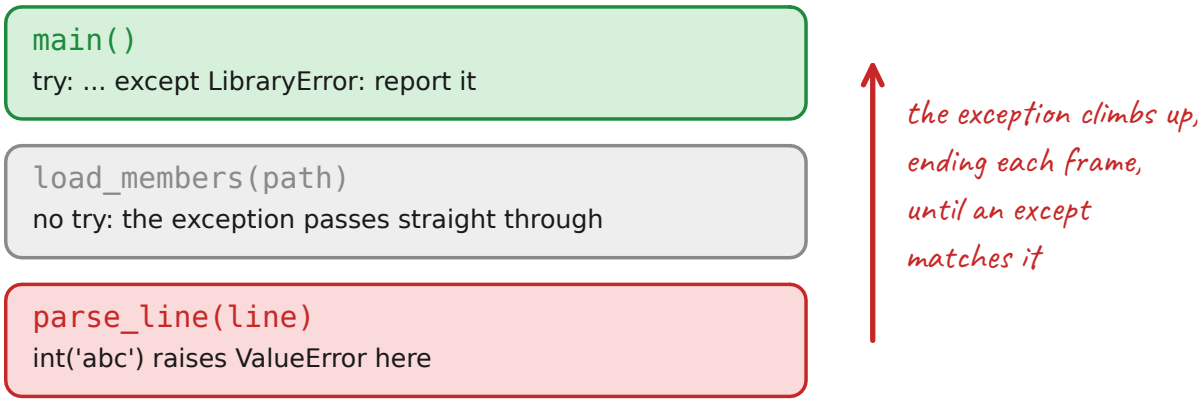
Vault notes: 9.3 Instance attributes, class attributes, and lookup · 9.4 Methods are functions transformed by descriptor access · 9.7 Representation methods · 9.8 Equality, hashing, and immutable value objects · 9.10 Composition and delegation · 9.11 Polymorphism and protocols · 9.12 Special methods and operator protocols · 9.13 Cooperative binary operations and NotImplemented · 9.14 Multiple inheritance and MRO · 9.15 Descriptors · 9.17 Slots.

Chapter 6

Errors and context managers

errors climb the call stack until someone catches them

When something goes wrong, Python **raises** an exception: an object describing the problem. The current function stops at once, and the exception travels **up** the call stack, ending each function it passes through, until it reaches a `try` block with a matching `except`. If none matches, the program stops and prints a traceback.



the call stack: each call adds a frame below its caller

```python
import traceback

def parse_line(line):
    name, age = line.split(",")
    return name, int(age)

def load_members(lines):
    return [parse_line(ln) for ln in lines]

try:
    load_members(["ana,20", "ben,abc"])
except ValueError:
    print(traceback.format_exc(limit=-2))
```

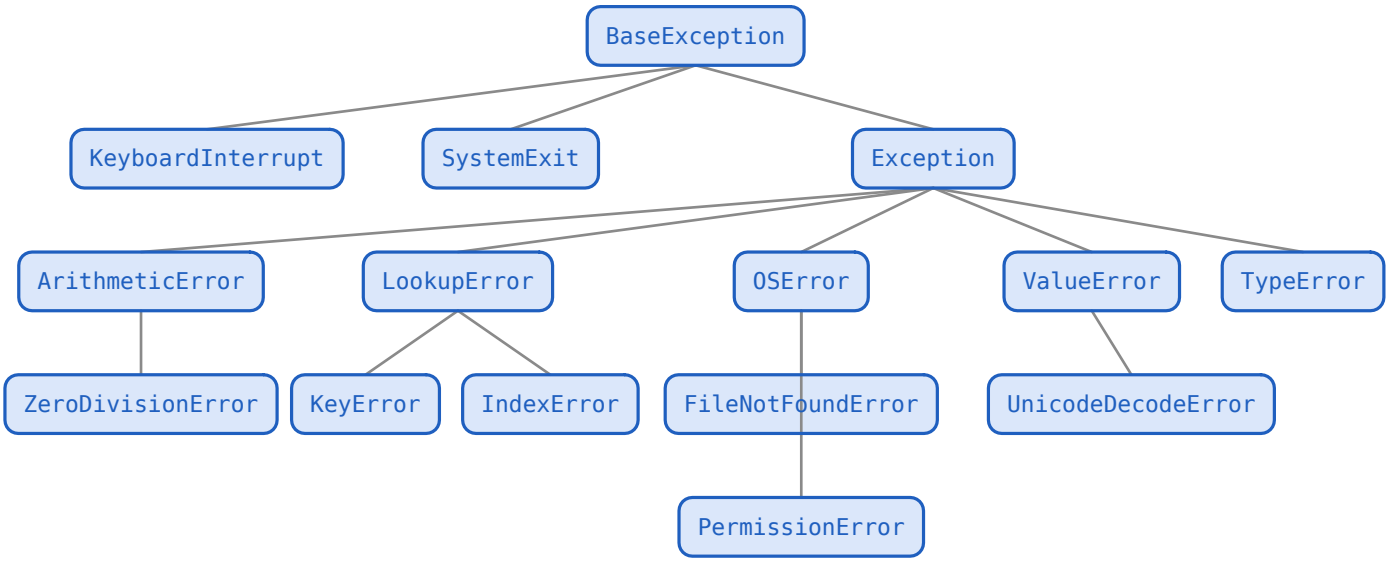
```
Traceback (most recent call last):
  File "<chapter 6>", line 8, in load_members
  File "<chapter 6>", line 5, in parse_line
ValueError: invalid literal for int() with base 10: 'abc'
```

RULE OF THUMB

Read a traceback from the **bottom**: the last line says what happened (`ValueError: invalid literal...`), the frame just above it says where. Then walk upwards to find the first line that's **your** code doing something wrong.

The exception family tree

Exceptions are classes, arranged in a hierarchy. `except X` catches `X` **and all its subclasses**, so where you catch in the tree decides how much you catch:




```python
def lookup(data, key):
    try:
        return data[key]
    except LookupError as e:                # catches KeyError AND IndexError
        return f"missing: {type(e).__name__}"

print(lookup({"a": 1}, "b"), lookup([10], 5), lookup([10], 0))
```

```
missing: KeyError missing: IndexError 10
```

TRAP

Never write a bare `except:` or `except BaseException:`. They also catch `KeyboardInterrupt` (Ctrl+C) and `SystemExit`, so your program can't be stopped. Even `except Exception:` hides bugs like typos (`NameError`). Catch the most specific class you can actually handle.

TRAP

Order matters: Python checks `except` clauses top to bottom and runs the first match. `except Exception` above `except ValueError` means the `ValueError` branch can never run.

try, except, else, finally

A `try` statement has up to four parts. Watch which ones run when the body succeeds and when it fails:

```python
def attempt(text):
    try:
        print("  try:", end=" ")
        n = int(text)
    except ValueError as e:
        print("except:", type(e).__name__, end=" | ")
    else:
        print("else: got", n, end=" | ")      # only if no exception
    finally:
        print("finally")                      # always, even on return or a new error

attempt("42")
attempt("4x2")
```

```
  try: else: got 42 | finally
  try: except: ValueError | finally
```

part	runs when	use it for
`try:`	always (until something raises)	only the lines that can fail
`except E as e:`	an `E` (or subclass) was raised in the `try`	recovering, or translating the error
`else:`	the `try` finished without an exception	the next steps, kept out of the `try`
`finally:`	always, whatever happened	cleanup: close, release, delete temp files

Ask forgiveness, or look before you leap?

LBYL: check first

```python
import os

path = "members.csv"
if os.path.exists(path):
    with open(path) as fh:
        print(fh.read())
else:
    print("no file")
```

```
no file
```

EAFP: try it

```python
path = "members.csv"
try:
    with open(path) as fh:
        print(fh.read())
except FileNotFoundError:
    print("no file")
```

```
no file
```

no race between check and use

clear when failure is common

WHY
The LBYL version has a race: the file can be deleted between `exists()` and `open()`. Python style generally prefers EAFP for things outside your control (files, network, dict keys from input), and plain `if` checks for your own program's logic.

Your own exceptions, and chaining

A library or app should raise exceptions that mean something in **its** terms. Make one base class, then specific subclasses; callers can catch broadly or narrowly:

```
class LibraryError(Exception):
    """Base class for everything this app raises on purpose."""

class LimitReached(LibraryError):
    def __init__(self, member, limit):
        super().__init__(f"{member} already has {limit} loans")
        self.member, self.limit = member, limit

class UnknownBook(LibraryError, KeyError):     # also a KeyError, for code that expects one
    pass

try:
    raise LimitReached("ana", 3)
except LibraryError as e:
    print(type(e).__name__, "-", e, "-", e.limit)
```

```
LimitReached - ana already has 3 loans - 3
```

When you catch a low-level error and raise your own, use `raise ... from err`. The original is kept as `__cause__` and printed too, so nobody loses the real reason:

```
import traceback

BOOKS = {"111": "Dune"}

def find(isbn):
    try:
        return BOOKS[isbn]
    except KeyError as err:
        raise UnknownBook(isbn) from err

try:
    find("999")
except LibraryError as e:
    print(repr(e), "caused by", repr(e.__cause__))
    print(traceback.format_exc().count("direct cause"), "'direct cause' line in the traceback")
```

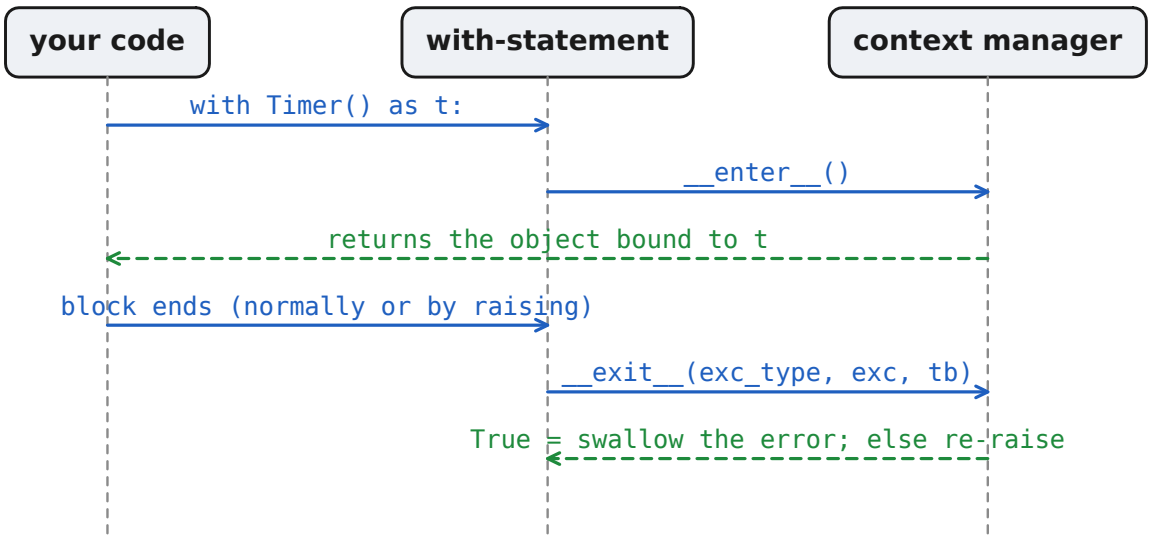
```
UnknownBook('999') caused by KeyError('999')
1 'direct cause' line in the traceback
```

KEY IDEA

Since 3.11 you can attach context without changing the exception: `e.add_note("while importing row 17")` adds a line to the traceback. Use it in loops over input data, where the row number is what you actually need to know.

with: cleanup that can't be forgotten

A `with` block guarantees that setup is paired with cleanup, even when the body raises. It calls `__enter__` at the start and `__exit__` at the end, **whatever happens**:




```python
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self                             # this is what 'as t' receives

    def __exit__(self, exc_type, exc, tb):
        self.ms = (time.perf_counter() - self.start) * 1000
        print(f"block took {self.ms:.2f} ms, error: {exc_type.__name__ if exc_type else None}")
        return False                            # don't swallow exceptions

with Timer() as t:
    sum(range(100_000))
try:
    with Timer():
        1 / 0
except ZeroDivisionError:
    print("the error still reached us")
```

```
block took 0.85 ms, error: None
block took 0.00 ms, error: ZeroDivisionError
the error still reached us
```

The quick way: a generator with @contextmanager

```
1  import contextlib, os, tempfile
2
3  @contextlib.contextmanager
4  def working_dir(path):
5      old = os.getcwd()
6      os.chdir(path)                  # setup: everything before yield
7      try:
8          yield path                  # the with-block runs here
9      finally:
10         os.chdir(old)               # cleanup: runs even if the block raised
11
12 with tempfile.TemporaryDirectory() as d, working_dir(d):
13     print(os.getcwd() == os.path.realpath(d))
14 print(os.getcwd() == os.path.realpath(d))
```

```
True
False
```

line 8: without try/finally, an error skips cleanup

stdlib context manager	guarantees
`open(...)`	the file is closed (and flushed)
`sqlite3` connection	commit on success, rollback on error (Ch 12; doesn't close!)
`threading.Lock()`	the lock is released (Ch 16)
`tempfile.TemporaryDirectory()`	the directory and its contents are deleted
`contextlib.suppress(E)`	errors of type `E` are ignored, nothing else
`contextlib.ExitStack()`	a variable number of resources, all cleaned up in reverse order

Several errors at once, and logging instead of print

When many independent tasks fail together (validating every row, running tasks concurrently), you want **all** the errors, not just the first. Python 3.11 added `ExceptionGroup` and `except*`:

```
def validate(rows):
    errors = []
    for i, (name, age) in enumerate(rows, 1):
        if not name:
            errors.append(ValueError(f"row {i}: empty name"))
        if not 0 < age < 130:
            errors.append(ValueError(f"row {i}: bad age {age}"))
    if errors:
        raise ExceptionGroup("invalid rows", errors)

try:
    validate([("ana", 20), ("", 200), ("cy", 30)])
except* ValueError as group:
    for e in group.exceptions:
        print(e)
```

```
row 2: empty name
row 2: bad age 200
```

`print` is for output the user asked for. Diagnostics belong in `logging`: it has levels, timestamps, one place to configure where messages go, and it can be turned down without editing code:

```
import logging, sys

logging.basicConfig(stream=sys.stdout, level=logging.INFO, force=True,
                    format="%(levelname)-7s %(name)s: %(message)s")
log = logging.getLogger("library.loans")

log.debug("not shown: below INFO")
log.info("lent %s to %s", "Dune", "ana")    # args, not an f-string: formatted only if shown
try:
    {}["999"]
except KeyError:
    log.exception("lookup failed")                  # ERROR level + the traceback
```

```
INFO    library.loans: lent Dune to ana
ERROR   library.loans: lookup failed
Traceback (most recent call last):
  File "<chapter 6>", line 10, in <module>
KeyError: '999'
```

level	for
`DEBUG`	details only useful while diagnosing
`INFO`	normal events worth a record (started, lent a book)
`WARNING`	unexpected but handled (retrying, using a default); the default threshold
`ERROR` / `CRITICAL`	something failed / the program can't continue

TRAP

`assert` is for **your own** assumptions while developing ("this list can't be empty here"), never for checking user input: `python -O` removes every assert. Validate input with `if ...: raise ValueError(...)`.

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 6.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 6.1` shows other valid solutions and their trade-offs.

`6.1` fix the bug The except that hides everything
`6.2` build from a spec All or nothing: a transaction context manager
`6.3` build from a spec Temporarily change environment variables
`6.4` build from a spec Exceptions that mean something
`6.5` build from a spec Retry, and keep the real cause
`6.6` mini-project Mini-project: report every bad row at once

PASSPORT STAMP ask the same question of every new language (Appendix B)

8. Errors

Exceptions are class instances that unwind the stack until a matching `except` (subclasses match). `try/except/else/finally`; `raise X from err` chains; `ExceptionGroup` + `except*` for many at once. Style: EAFP. Cleanup via `with` (`__enter__`/`__exit__`), never by hoping.

12. Stdlib and tooling

`logging` for diagnostics; `contextlib` (contextmanager, suppress, ExitStack) for resources.

IN OTHER LANGUAGES

Java has checked exceptions (the compiler forces you to handle them) and try-with-resources for cleanup. Go has no exceptions: functions return an `error` value you must check. Rust returns `Result<T, E>` and uses `?` to pass errors up. C++ uses destructors (RAII) where Python uses `with`.

DEEP DIVE (your vault)

Vault notes: 6.12 Exceptions as non-local control flow · 6.13 Exception propagation and stack unwinding · 6.14 Exceptions versus result values · 6.15 Iteration, exceptions, and generator cleanup · 11.1 Boundaries turn assumptions into failure modes · 11.4 Context managers and deterministic release · 12.11 Debugging as hypothesis testing · 12.12 Logging and diagnostics · 15.9 Exception machinery.

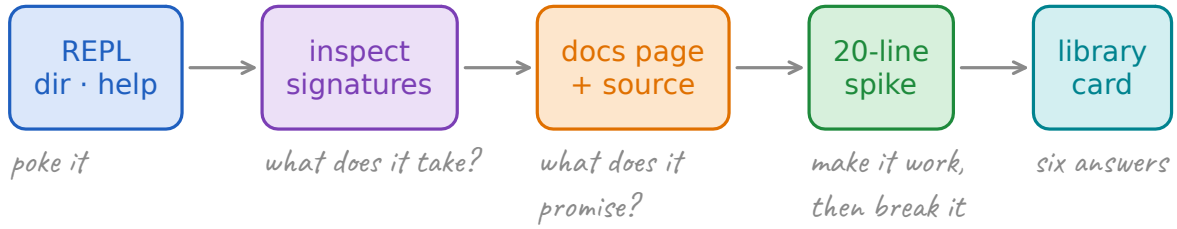
Chapter 7

How to learn any library

the method that makes every other chapter optional reading

When `sqlite3` or `hmac` feels impossible, the problem is almost never syntax. You can call a function and write a `with` block. What's missing is the **shape** of the library: what you create first, what you call on it, what you must close, and what goes wrong.

This chapter is the method for finding that shape on your own, for any module, in any language. It has four tools and one output:



Every later chapter is this method already applied to one module. Once it's a habit, you won't need a chapter (or a tutorial) to start using an unfamiliar library.

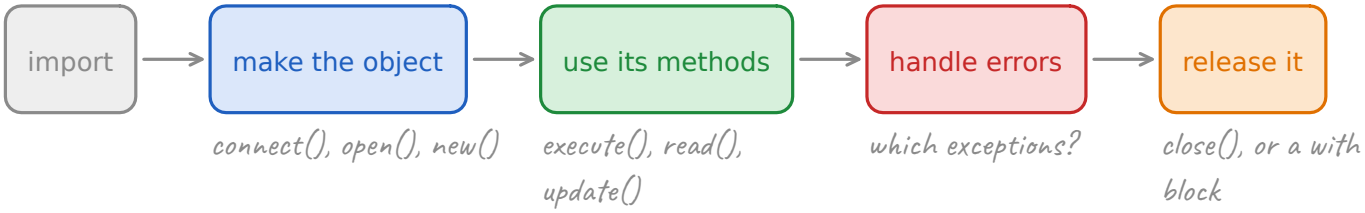
> **KEY IDEA**
> You don't learn a library by reading all of it. You learn its **shape** (the six card questions), then look up details only when you need them.

The four shapes a library comes in

Nearly every library is one of four shapes. Spotting which one tells you how to start:

shape	how you use it	examples
Bag of functions	import, call functions, get values back. Nothing to close.	`math`, `json`, `base64`, `statistics`, `difflib`
Object factory	a function makes an object; you call its methods; often you must close it.	`sqlite3.connect`, `open`, `hmac.new`, `zipfile.ZipFile`
Class to subclass	you write a subclass and fill in methods; the library calls them.	`unittest.TestCase`, `http.server`'s `BaseHTTPRequestHandler`
Framework / runtime	you hand over your functions; the library runs the loop.	`asyncio.run`, `tkinter` mainloop, `argparse` (lighter)

The object factory shape is where most people get stuck, because the important part is the **lifecycle**: the order of calls, and what must happen at the end.




```python
import zipfile, io

buf = io.BytesIO()                          # an in-memory 'file'
with zipfile.ZipFile(buf, "w") as z:        # make the object
    z.writestr("hello.txt", "hi there")     # use its methods
# leaving the with block released it (wrote the zip's index)

with zipfile.ZipFile(buf) as z:
    print(z.namelist(), z.read("hello.txt"))
```

```
['hello.txt'] b'hi there'
```

TRAP

Forgetting the release step causes the bugs that make no sense: a zip with no index, a database change that never saved, a file that's still empty. If a library hands you an object, your first question is **how do I close it?**

Tool 1: the REPL, with dir() and help()

Open `python3` and import the module. `dir()` lists every name on an object; filter out the underscore names and what's left is the **public surface**:

```
>>> import hmac
>>> [n for n in dir(hmac) if not n.startswith("_")]
['HMAC', 'compare_digest', 'digest', 'digest_size', 'new', 'trans_36', 'trans_5C']
>>> type(hmac.new), type(hmac.HMAC)
(<class 'function'>, <class 'type'>)
```

Now you know: one function (`new`), one class (`HMAC`), one helper (`compare_digest`). The `trans_` names are internal tables a stricter author would have hidden. `help(name)` shows the docstring; `__doc__` gives the same text as a string, handy for the first lines:

```
>>> print(hmac.new.__doc__[:200])
Create a new hashing object and return it.

key: bytes or buffer, The starting key for the hash.
msg: bytes or buffer, Initial input for the hash, or None.
digestmod: A hash name suitable for hashlib.
```

The factory makes an object. Make one, and `dir()` **that** too: the object's methods are the next layer of the shape.

```
>>> h = hmac.new(b"secret key", b"message", "sha256")
>>> print(*[n for n in dir(h) if not n.startswith("_")])
block_size blocksize copy digest digest_size hexdigest name update
>>> h.hexdigest()[:16]
'c431e4857a6b817f'
```

in the REPL	tells you
`dir(x)`	every attribute name (filter the underscores)
`help(x)` / `x.__doc__`	the docstring: what it does and what it takes
`type(x)`, `type(x).__mro__`	what kind of object, and which classes it inherits from
`x.__module__`, `module.__file__`	where it lives, so you can read the source
`callable(x)`, `isinstance(x, type)`	function or class? (filtering `dir()` output)

Tool 2: inspect, signatures and the source

`inspect.signature` answers "what does it take?" exactly. Learn to read the two markers, because the docs use them too:

```
>>> import inspect, json
>>> inspect.signature(sorted)
<Signature (iterable, /, *, key=None, reverse=False)>
>>> inspect.signature(hmac.new)
<Signature (key, msg=None, digestmod='')>
>>> inspect.signature(json.dumps)
<Signature (obj, *, skipkeys=False, ensure_ascii=True, check_circular=True, allow_nan…
```

in a signature	means	so you write
`a, b`	ordinary: by position or by name	`f(1, 2)` or `f(a=1, b=2)`
`x, /`	everything before `/` is **positional-only**	`sorted(xs)`, never `sorted(iterable=xs)`
`*, key=None`	everything after `*` is **keyword-only**	`sorted(xs, key=len)`, not `sorted(xs, len)`
`msg=None`	has a default, so optional	leave it out, or pass it
`*args`, `**kw`	any extra positional / keyword arguments	read the docs for what's accepted
`[, x]` in docs	older docs' notation for optional	same as a default

Then read the source. A surprising amount of the standard library is plain Python, and reading it is the fastest way to see what a function really does:

```
1 import inspect, statistics
2 src = inspect.getsource(statistics.median)
3 body = src.split('"""')[2]          # skip the docstring
4 print(body.strip())
```

← the real code

↖ drop the docstring

```
data = sorted(data)
    n = len(data)
    if n == 0:
        raise StatisticsError("no median for empty data")
    if n % 2 == 1:
        return data[n // 2]
    else:
        i = n // 2
        return (data[i - 1] + data[i]) / 2
```

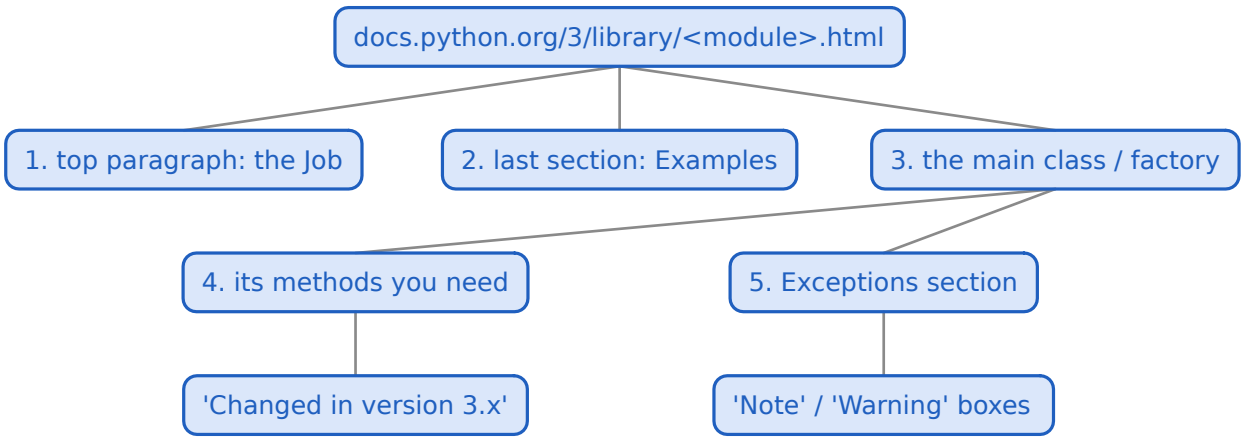
HONEST NOTE

Some modules are written in C (`math`, `_sqlite3`, parts of `hashlib`). For those `inspect.getsource` raises `TypeError`, and `inspect.signature(sqlite3.connect)` even raises `ValueError` on 3.14. Then fall back to `help()` and the docs page. `module.__file__` tells you which it is: a `.py` path, or a `.so` (compiled C).

```
>>> import math, sqlite3
>>> math.__file__.rsplit("/", 1)[-1]
'math.cpython-314-x86_64-linux-gnu.so'
>>> statistics.__file__.rsplit("/", 1)[-1]
'statistics.py'
```

▢ Tool 3: reading a docs page (without reading all of it)

A page on docs.python.org is a reference, not a story. You don't read it top to bottom. You read it in this order, and stop as soon as you can write your spike:



Skip ahead to the examples. Most module pages end with a worked example; it shows the lifecycle in ten lines. Warning boxes are where the traps live (for `hmac`: compare with `compare_digest`, not `==`). "Changed in version" lines tell you whether a blog post from 2016 still applies.

You can read the same docstrings offline in a terminal, which helps when you're on a train:

```
$ python3 -m pydoc hmac            # the module's docs, in a pager
$ python3 -m pydoc hmac.new        # one function
$ python3 -m pydoc -k zip          # search every module's summary line
```

RULE OF THUMB

Search in this order: the official docs page, then the source, then the PEP that introduced the feature (linked from the docs). Only then blogs and Q&A sites, and check the **date**: Python 2 answers still rank highly. AI answers are a fourth opinion: run what they give you in a spike before you trust it.

Tool 4: the 20-line spike

A **spike** is the smallest throwaway program that uses the library end to end. You write it in a scratch file before touching your real project. Then you **break it on purpose**, because seeing the errors now means you'll recognise them later.

Here's a spike for `difflib`, a module this book never teaches. Ten minutes with `dir()` and the docs page found `get_close_matches`:

```python
import difflib

commands = ["status", "commit", "checkout", "branch", "merge"]
for typo in ["comit", "chekout", "brnch", "xyz"]:
    print(typo, "->", difflib.get_close_matches(typo, commands, n=1))

# break it: what does the cutoff do?
print(difflib.get_close_matches("xyz", commands, cutoff=0.0))
```

← *make it work*

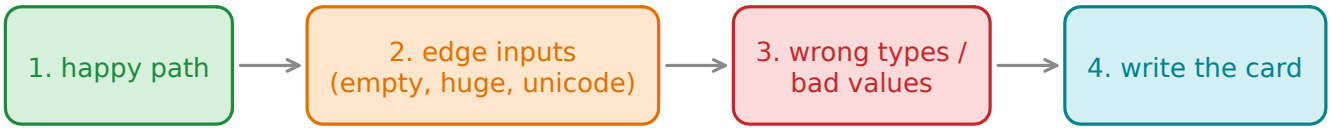
← *now poke it*

```
comit -> ['commit']
chekout -> ['checkout']
brnch -> ['branch']
xyz -> []
['status', 'merge', 'commit']
```

And break it with wrong input, to learn the error before it surprises you:

```python
difflib.get_close_matches("x", commands, n=0)
```

```
ValueError: n must be > 0: 0
```



✎ YOUR GO (pencil, or the REPL, first)

Spike the textwrap module: find (with dir and help, no searching) how to wrap a long sentence to 30 characters and how to indent every line by two spaces. Then pass it an empty string.

Worked example: cracking hmac from zero

Pretend you've never seen `hmac`. The project needs to check that a QR code's contents weren't edited. Five minutes of the method:

step	what you did	what you learned
Job	first line of `hmac.__doc__`, the top paragraph of the docs	makes a keyed hash: a tag only someone with the key could produce
Shape	`dir(hmac)`: `new`, `HMAC`, `compare_digest`, `digest`	object factory, plus a one-shot function `digest`
Signature	`inspect.signature(hmac.new)`	`(key, msg=None, digestmod='')`: key and message are bytes; you must choose a hash
Docs warning	the Warning box on the page	compare tags with `compare_digest`, never `==`
Spike	the 12 lines below	it works; wrong key gives a different tag; `str` key is a `TypeError`

```python
import hmac

KEY = b"server-only secret"
def tag(msg: bytes) -> str:
    return hmac.new(KEY, msg, "sha256").hexdigest()

t = tag(b"student=42;room=B12")
print(t[:20], len(t))
print(hmac.compare_digest(t, tag(b"student=42;room=B12")))
print(hmac.compare_digest(t, tag(b"student=43;room=B12")))
print(hmac.digest(KEY, b"student=42;room=B12", "sha256").hex() == t)
```

```
1b34cbb6d11eb6641380 64
True
False
True
```

← factory + method in one line

← unchanged: True

← edited: False

← the one-shot form agrees

```python
hmac.new("not bytes", b"m", "sha256")
```

```
TypeError: key: expected bytes or bytearray, but got 'str'
```

The output of the method is the card. Writing it down is what makes it stick, and it's what you'll look at in six months instead of the docs:

LIBRARY CARD	`hmac`
Job	A keyed hash (a **tag**): proves a message came from someone with the key and wasn't changed.
Core objects	`hmac.new(key, msg, digestmod)` → `HMAC` object with `.update(b)`, `.digest()`, `.hexdigest()`. One-shot: `hmac.digest(key, msg, 'sha256')`.
Lifecycle	new → update (any number of times) → digest. Nothing to close.
Errors	`TypeError` if key/msg are `str` (encode them); unknown digest name → `ValueError`.
Traps	Compare with `hmac.compare_digest` (**== leaks timing**). Not encryption: the message is still readable. Keep the key out of the source (Ch 13).
Docs	docs.python.org/3/library/hmac.html · RFC 2104

When you're stuck: a checklist

- **Which shape is it?** Bag of functions, factory, subclass, or framework. That decides what to look for first.
- **What's the core object?** Find the one or two classes everything else hangs off (`Connection` and `Cursor`, `ZipFile`, `Request` and `Response`).
- **Bytes or text?** Half of all stdlib confusion. Crypto, sockets and compression want `bytes`; you `.encode()` on the way in and `.decode()` on the way out (Ch 9).
- **Who closes it?** Look for `close()`, `__enter__` in `dir()` (it works with `with`), or a `commit()`.
- **What does it raise?** Look for names ending in `Error` in `dir(module)`; that's the Errors row of the card.
- **Is there a one-shot helper?** Many modules have a simple function for the common case (`hmac.digest`, `json.loads`, `subprocess.run`, `urllib.request.urlopen`). Start there.

```
>>> print(*[n for n in dir(sqlite3) if n.endswith("Error")])
DataError DatabaseError Error IntegrityError InterfaceError InternalError NotSupporte…
>>> "__enter__" in dir(sqlite3.Connection)
True
```

IN OTHER LANGUAGES

The method is language-neutral. In Java the REPL is `jshell` and the docs are Javadoc; in Rust, `cargo doc --open` and the docs.rs Examples section; in C, `man 3 qsort`. The questions (shape, core object, lifecycle, errors, traps) don't change.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 7.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 7.1` shows other valid solutions and their trade-offs.

`7.1` learn an unseen module Map a module's public surface
`7.2` learn an unseen module Read a signature with inspect
`7.3` learn an unseen module Did you mean...? (difflib)
`7.4` learn an unseen module Zip and unzip in memory (zipfile)
`7.5` learn an unseen module Wrap text for a terminal (textwrap)
`7.6` mini-project Library card generator

PASSPORT STAMP ask the same question of every new language (Appendix B)

9. Modules and build

A module is a `.py` file (or compiled C `.so`) found on `sys.path`; `module.__file__` says which. Packages are folders of modules.

12. Stdlib and tooling

Built-in introspection: `dir`, `help`, `type`, `inspect.signature`, `inspect.getsource`, `python -m pydoc`. Most of the stdlib is readable Python source.

DEEP DIVE (your vault)

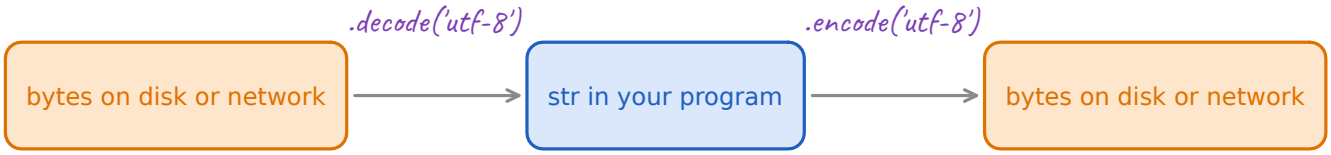
Vault notes: 11.10 Standard-library method: inspect by contract · 10.2 Import is execution with caching · 10.7 Public APIs and `__all__` · 8.15 Function attributes and introspection · 15.20 Reading CPython source responsibly.

Chapter 8

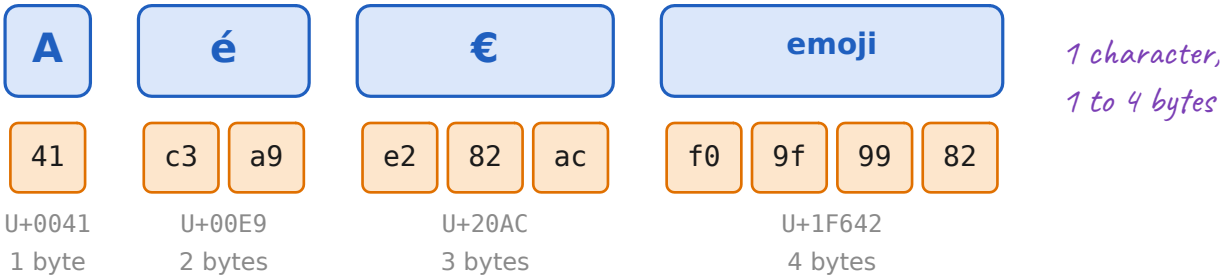
Files and data formats

a file is bytes; text is bytes plus an agreed encoding

Every file is a sequence of **bytes** (numbers 0–255). Text only exists once you agree how bytes map to characters: an **encoding**. Python keeps the two apart with two types: `bytes` for raw data, `str` for text, and you convert at the edges of your program:



UTF-8, the encoding of the web and of nearly every modern file, uses 1 byte for ASCII characters and up to 4 for everything else:




```
>>> s = "café €5"
>>> len(s), len(s.encode("utf-8"))
(7, 10)
>>> s.encode("utf-8")
b'caf\xc3\xa9 \xe2\x82\xac5'
>>> s.encode("utf-8").decode("latin-1")
'cafÃ© â\x82¬5'
>>> b"caf\xc3".decode("utf-8")
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc3 in position 3: unexpected en…
```

RULE OF THUMB

Decode as early as possible, work in `str`, encode as late as possible, and **always** pass `encoding="utf-8"` to `open()` for text. The default depends on the operating system (Windows has historically used cp1252), which is how "works on my machine" bugs start.

open(), modes and pathlib

mode	means	if the file exists	if it doesn't
`"r"`	read text (default)	read from the start	`FileNotFoundError`
`"w"`	write text	**emptied first**	created
`"a"`	append text	write at the end	created
`"x"`	exclusive create	`FileExistsError`	created
add `b`	bytes, not str (`"rb"`, `"wb"`)	no encoding, no newline translation	
add `+`	read and write (`"r+"`)	rarely what you want	

```python
from pathlib import Path

data = Path("data")
data.mkdir(exist_ok=True)
notes = data / "notes.txt"                          # / joins paths on any OS

notes.write_text("line one\nline two\n", encoding="utf-8")
with notes.open("a", encoding="utf-8") as fh:
    fh.write("line three\n")

with notes.open(encoding="utf-8") as fh:
    for n, line in enumerate(fh, 1):                # one line at a time: any size of file
        print(n, line.rstrip("\n"))

print(notes.name, notes.stem, notes.suffix, notes.parent, notes.stat().st_size, "bytes")
print(sorted(p.name for p in data.glob("*.txt")))
```

```
1 line one
2 line two
3 line three
notes.txt notes .txt data 29 bytes
['notes.txt']
```

pathlib	instead of
`Path.home() / "docs" / "a.txt"`	`os.path.join(os.path.expanduser("~"), ...)`
`p.exists()`, `p.is_file()`, `p.is_dir()`	`os.path.exists(p)` ...
`p.read_text()`, `p.write_bytes(b)`	open / read / close by hand
`p.glob("**/*.csv")`, `p.iterdir()`	`glob.glob`, `os.listdir`
`p.with_suffix(".bak")`, `p.resolve()`	string slicing on paths

TRAP

`for line in fh` keeps the `\n` on every line. Use `line.rstrip("\n")` (or `.strip()` if surrounding spaces don't matter). And `fh.read()` loads the whole file into memory: fine for kilobytes, not for a 4 GB log.

CSV: tables as text

CSV looks trivial until a field contains a comma, a quote or a newline. `line.split(",")` breaks on all three; the `csv` module handles the quoting rules:

```
import csv
from pathlib import Path

rows = [{"title": "Dune", "author": "Herbert, Frank", "note": 'says "spice"'},
        {"title": "Emma", "author": "Austen, Jane", "note": "two\nlines"}]

with open("books.csv", "w", newline="", encoding="utf-8") as fh:
    w = csv.DictWriter(fh, fieldnames=["title", "author", "note"])
    w.writeheader()
    w.writerows(rows)

print(Path("books.csv").read_text(encoding="utf-8"))

with open("books.csv", newline="", encoding="utf-8") as fh:
    back = list(csv.DictReader(fh))
print(back == rows, back[0]["author"])
```

```
title,author,note
Dune,"Herbert, Frank","says ""spice"""
Emma,"Austen, Jane","two
lines"

True Herbert, Frank
```

line 8: newline="": let csv handle line ends

split by hand (broken)

```
line = '"Herbert, Frank",Dune'
print(line.split(","))
```

```
['"Herbert', ' Frank"', 'Dune']
```

csv.reader

```
import csv
print(next(csv.reader(['"Herbert, Frank",Dune'])))
```

```
['Herbert, Frank', 'Dune']
```

TRAP

Everything from CSV comes back as a **string**: `"1965"`, not `1965`, and an empty field is `""`, not `None`. Convert and validate each column yourself (or use pandas, which guesses types for you, sometimes wrongly).

JSON: nested data as text

JSON stores nested lists and dicts, and it's what web APIs speak (Ch 15). It has fewer types than Python, so some things change on the round trip:

Python	JSON	comes back as
`dict` (str keys)	object `{...}`	`dict`
`dict` with int keys `{1: "a"}`	`{"1": "a"}`	**str keys!**
`list`, `tuple`	array `[...]`	`list` (tuples are lost)
`str`, `int`, `float`	string, number	same
`True`, `False`, `None`	`true`, `false`, `null`	same
`set`, `datetime`, `Decimal`, your classes	**not allowed**	`TypeError` unless you convert

```python
import json, datetime
from dataclasses import dataclass, asdict

@dataclass
class Loan:
    title: str
    due: datetime.date

def to_jsonable(obj):                              # called for anything json can't encode
    if isinstance(obj, datetime.date):
        return obj.isoformat()
    raise TypeError(f"can't encode {type(obj).__name__}")

data = {"member": "ana", "loans": [asdict(Loan("Dune", datetime.date(2026, 10, 3)))], 7: (1, 2)}
text = json.dumps(data, default=to_jsonable, indent=2)
print(text)
print(json.loads(text)["7"])
```

```
{
  "member": "ana",
  "loans": [
    {
      "title": "Dune",
      "due": "2026-10-03"
    }
  ],
  "7": [
    1,
    2
  ]
}
[1, 2]
```

KEY IDEA

For logs and big datasets use **JSON Lines**: one JSON object per line. You can append without rewriting the file, and read it line by line: `[json.loads(l) for l in fh]`.

TOML, pickle, and choosing a format

```python
import tomllib                    # read-only TOML parser, in the stdlib since 3.11

config = tomllib.loads('''
[library]
name = "Central"
max_loans = 3
open_days = ["Mon", "Wed", "Fri"]
''')
print(config["library"]["max_loans"] + 1, config["library"]["open_days"][-1])
```

```
4 Fri
```

`pickle` can save almost any Python object, but loading a pickle can run **arbitrary code**: the file itself says which functions to call while rebuilding objects. This harmless example runs `print`; a malicious one would run `os.system`:

```python
import pickle

class Evil:
    def __reduce__(self):                    # "to rebuild me, call print(...)"
        return (print, ("this ran just by loading the file!",))

payload = pickle.dumps(Evil())
pickle.loads(payload)
```

```
this ran just by loading the file!
```

TRAP

Never unpickle data from anyone you don't trust: not uploads, not downloads, not network messages. Use JSON for data exchange. Pickle is for your own caches, read only by your own program.

format	good for	watch out
CSV	flat tables, spreadsheets	everything is a string; quoting
JSON / JSON Lines	nested data, APIs, logs	no dates, sets or int keys
TOML	config files written by humans	stdlib can read but not write it
SQLite (Ch 12)	data you query and update	a schema to design
pickle	caching your own objects	**runs code when loaded**
binary (`struct`)	fixed file formats, protocols	byte order, versioning

Writing files safely

If the program crashes or the power fails half-way through `write_text`, the file is left truncated: the old data is gone and the new data is incomplete. The fix is to write a **temporary file** next to the target, then swap it in with `os.replace`, which is atomic (it either fully happens or not at all):




```python
import json, os, tempfile
from pathlib import Path

def write_atomic(path, text):
    path = Path(path)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=path.name, suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            fh.write(text)
            fh.flush()
            os.fsync(fh.fileno())          # make sure the bytes reached the disk
        os.replace(tmp, path)              # atomic swap on the same filesystem
    except BaseException:
        os.unlink(tmp)
        raise

write_atomic("members.json", json.dumps({"ana": 20}))
print(Path("members.json").read_text(), [p.name for p in Path(".").glob("*.tmp")])
```

```
{"ana": 20} []
```

Binary formats: struct

Many file formats start with a fixed binary header. `struct` packs numbers into exact bytes; the format string gives the byte order and the size of each field:

```python
import struct

header = struct.pack("<4sHI", b"LIBR", 2, 1_000_000)   # little-endian: 4 bytes, uint16, uint32
print(header, len(header))
magic, version, count = struct.unpack("<4sHI", header)
print(magic, version, count)
print(struct.pack(">I", 1).hex(), struct.pack("<I", 1).hex())    # big vs little endian
```

```
b'LIBR\x02\x00@B\x0f\x00' 10
b'LIBR' 2 1000000
00000001 01000000
```

IN OTHER LANGUAGES

Every language has the same pair: a text layer (encoding, newline handling) over a byte layer. Java has `Reader`/`InputStream`, C has `fopen("r")`/`fopen("rb")` (the distinction only matters on Windows), Rust has `String` vs `Vec<u8>` and refuses invalid UTF-8 in a `String`.

Compression, archives and copying

```python
import gzip, shutil, zipfile
from pathlib import Path

Path("log.txt").write_text("GET /book/3\n" * 10_000, encoding="utf-8")
with open("log.txt", "rb") as src, gzip.open("log.txt.gz", "wb") as dst:
    shutil.copyfileobj(src, dst)                    # streams in chunks: no full read
print(Path("log.txt").stat().st_size, "->", Path("log.txt.gz").stat().st_size, "bytes")

with zipfile.ZipFile("backup.zip", "w", zipfile.ZIP_DEFLATED) as z:
    z.write("log.txt")
    z.writestr("readme.txt", "made by Python")
print(zipfile.ZipFile("backup.zip").namelist())

shutil.make_archive("snapshot", "gztar", root_dir=".", base_dir="data")
print(Path("snapshot.tar.gz").exists())
```

```
120000 -> 295 bytes
['log.txt', 'readme.txt']
True
```

module	for
`shutil`	copy, move, delete trees (`rmtree`), archives, disk usage
`gzip`, `bz2`, `lzma`, `zlib`	compress one stream; `compression.zstd` is new in 3.14
`zipfile`, `tarfile`	archives of many files (beware `../` paths when extracting untrusted ones)
`tempfile`	temporary files and directories that clean themselves up

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 8.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 8.1` shows other valid solutions and their trade-offs.

8.1 build from a spec CSV that survives awkward names
8.2 fix the bug Counting characters, not bytes
8.3 build from a spec JSON that keeps its types
8.4 build from a spec Saving without ever corrupting the file
8.5 build from a spec A binary file format with struct
8.6 mini-project Mini-project: summarise a folder of logs

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Files are bytes; `str` is decoded text. `open(path, mode, encoding="utf-8")` in a `with`; `pathlib` for paths; `csv`, `json`, `tomllib`, `struct`, `gzip`/`zipfile` for formats. Atomic writes via temp file + `os.replace`.

12. Stdlib and tooling

Stdlib covers the common formats; never unpickle untrusted data.

DEEP DIVE (your vault)

Vault notes: 5.12 Bytes and the text boundary · 11.2 Binary and text I/O · 11.3 File objects, modes, and offsets · 11.5 Paths and filesystem operations · 11.6 Buffers, flushing, and durability · 11.7 JSON · 11.8 CSV · 11.9 Pickle and object reconstruction · 11.15 pathlib · 11.16 Temporary resources · 11.17 Integrated serialisation pipeline.

Chapter 9

Text and regular expressions

describe the shape of the text; the engine finds it

String methods (`split`, `strip`, `replace`, `startswith`, `in`) handle fixed text. When what you're looking for has a **shape** rather than a fixed value ("a date", "an email", "a word followed by a number"), you describe the shape with a **regular expression** and the `re` module finds every piece of text that fits:

```
re.finditer(r'\d{4}-\d{2}-\d{2}', text)                    3 matches
Loans: Dune due 2026-10-03, Emma due 2026-11-15 (renewed 2026-10-20).
```

Each box is one match. In this chapter every pattern is drawn like this, so you can **see** what it matches before trusting it. Capturing groups get coloured underlines:

```
re.finditer(r'(\w+) due (\d{4})-(\d{2})', text)            2 matches
Loans: Dune due 2026-10-03, Emma due 2026-11-15.
```

RULE OF THUMB

Always write patterns as raw strings: `r"\d+"`. Without the `r`, Python processes backslashes first (`"\b"` is a backspace character), and your pattern silently changes.

HONEST NOTE

Regular expressions are powerful and hard to read. If `str` methods can do the job, use them. Never parse HTML, JSON or a programming language with a regex: use a real parser (`html.parser`, `json`, `ast`).

Formatting: the mini-language after the colon

Inside an f-string, everything after `:` is a **format spec**: `{value:fill align width , .precision type}`. It's the same mini-language used by `format()` and `str.format`:

```python
title, price, count, ratio = "Dune", 7.5, 1234567, 0.4567

print(f"[{title:<10}] [{title:>10}] [{title:^10}] [{title:*^10}]")
print(f"[{price:8.2f}] [{count:,}] [{count:_d}] [{ratio:.1%}] [{count:.2e}]")
print(f"[{42:05d}] [{255:x}] [{255:#b}] [{-3:+d}] [{3:+d}]")
print(f"{title=}, {price * 2 = }")                    # self-documenting, for debugging
width = 12
print(f"[{title:>{width}}]")                           # nested: width from a variable
```

```
[Dune      ] [      Dune] [   Dune   ] [***Dune***]
[   7.50] [1,234,567] [1_234_567] [45.7%] [1.23e+06]
[00042] [ff] [0b11111111] [-3] [+3]
title='Dune', price * 2 = 15.0
[        Dune]
```

spec	means	example → result
`< > ^`	align left / right / centre in the width	`f"{'ab':>4}"` → `'  ab'`
`*^10`	fill character before the alignment	`f"{'ab':*^6}"` → `'**ab**'`
`.2f`, `.3g`, `.1%`, `.2e`	fixed decimals, significant figures, percent, exponent	`f"{0.5:.0%}"` → `'50%'`
`,` or `_`	thousands separator	`f"{10**6:,}"` → `'1,000,000'`
`05d`, `x`, `b`, `#x`	zero-pad, hex, binary, with prefix	`f"{10:#x}"` → `'0xa'`
`!r`	use `repr()` instead of `str()`	`f"{'a'!r}"` → `"'a'"`

Unicode: when equal-looking strings aren't equal

"é" can be stored as one code point (U+00E9) or as "e" plus a combining accent (U+0065 U+0301). They look identical and compare unequal. **Normalise** text from outside before comparing, and use `casefold()` (not `lower()`) for case-insensitive matching:

```
>>> import unicodedata
>>> a, b = "caf\u00e9", "cafe\u0301"
>>> a == b, len(a), len(b)
(False, 4, 5)
>>> unicodedata.normalize("NFC", a) == unicodedata.normalize("NFC", b)
True
>>> "Straße".lower() == "STRASSE".lower(), "Straße".casefold() == "STRASSE".casefold()
(False, True)
>>> "".join(c for c in unicodedata.normalize("NFKD", "Zoë Ångström") if not unicodeda…
'Zoe Angstrom'
```

Regex building blocks

A pattern is built from a small set of pieces. Learn these and you can read most regexes:

piece	matches	piece	matches
`abc`	exactly "abc"	`.`	any character except newline
`\d \w \s`	digit, word char (letter, digit, _), whitespace	`\D \W \S`	the opposite of each
`[aeiou]`	one of these	`[^0-9]`	one character NOT in the set
`x* x+ x?`	0 or more, 1 or more, 0 or 1	`x{3} x{2,4}`	exactly 3, between 2 and 4
`^ $`	start / end of the string (or line)	`\b`	word boundary (between \w and \W)
`a|b`	a or b	`(...)`	group: capture, and apply `*`, `|` to the whole
`(?P<name>...)`	named group	`(?:...)`	group without capturing

Build a pattern up one piece at a time, checking each step against real text:

`re.finditer(r'\d+', text)` — **9 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'\d{5} \d{6}', text)` — **1 match**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'(\+44 |0)7\d{3} \d{6}', text)` — **2 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

`re.finditer(r'(\+44 |0)(7\d{3}) (\d{6})', text)` — **2 matches**

Call 07700 900123 or +44 7700 900456; fax 0161-555-0199. Ref 12345.

WHY

Step 1 matched every run of digits, including the fax and the reference. Step 2 required the exact shape but missed the +44 form. Step 3 made the prefix an alternative and fixed the leading 7. Step 4 added groups to pull out the parts. That loop (match too much, tighten, check) is how regexes are written in practice.

Using re: search, findall, groups, sub

```python
import re

line = "2026-09-28 14:03:11 WARN loans: member=ana book=Dune days_over=3"
m = re.search(r"(?P<date>\S+) (?P<time>\S+) (?P<level>[A-Z]+) (?P<area>\w+):", line)
print(m["level"], m.group("area"), m.span(), m.groupdict()["date"])

pairs = dict(re.findall(r"(\w+)=(\w+)", line))           # with 2 groups: list of tuples
print(pairs)

print(re.fullmatch(r"[A-Z]{2}\d{2} ?\d[A-Z]{2}", "M13 9PL"))   # whole string must match
print(re.split(r"[;,]\s*", "Dune; Emma,Ubik ,  Kim"))
```

```
WARN loans (0, 31) 2026-09-28
{'member': 'ana', 'book': 'Dune', 'days_over': '3'}
None
['Dune', 'Emma', 'Ubik ', 'Kim']
```

function	returns	use when
`re.search(p, s)`	first `Match` anywhere, or `None`	"is it in there, and where?"
`re.match(p, s)`	`Match` only at the start, or `None`	rarely; prefer `fullmatch` or `^`
`re.fullmatch(p, s)`	`Match` if the **whole** string fits	validating input
`re.findall(p, s)`	list of strings (or tuples of groups)	extracting every occurrence
`re.finditer(p, s)`	iterator of `Match` objects	you need positions, or big text
`re.sub(p, repl, s)`	new string with matches replaced	cleaning, redacting, rewriting

sub with a function

`repl` can be a function: it receives each `Match` and returns the replacement. That turns `sub` into a general text transformer:

```python
import re

def mask(m):
    user, domain = m["user"], m["domain"]
    return f"{user[0]}{'*' * (len(user) - 1)}@{domain}"

text = "Contact ana.reader@uni.ac.uk or ben@library.org for keys."
print(re.sub(r"(?P<user>[\w.+-]+)@(?P<domain>[\w-]+(\.[\w-]+)+)", mask, text))
print(re.sub(r"(\d+) days", lambda m: f"{int(m[1]) * 7} days", "renew for 2 days, then 3 days"))
```

```
Contact a*********@uni.ac.uk or b**@library.org for keys.
renew for 14 days, then 21 days
```

Greedy, lazy, and readable patterns

`*` and `+` are **greedy**: they grab as much as possible and only give back what they must. Adding `?` makes them **lazy** (as little as possible). The classic surprise:

`re.finditer(r'<.*>', text)` — **1 match**

Dune and Emma

`re.finditer(r'<.*?>', text)` — **4 matches**

 Dune and Emma

`re.finditer(r'<[^>]*>', text)` — **4 matches**

 Dune and Emma

Greedy `.*` ran to the **last** `>`, swallowing the whole line. The lazy version stops at the first `>`. A negated class `[^>]*` ("anything but >") says exactly what you mean and is usually fastest.

Long patterns become readable with `re.VERBOSE`: whitespace is ignored and `#` starts a comment:

```python
import re

POSTCODE = re.compile(r"""
    ^ (?P<area> [A-Z]{1,2} )      # 1-2 letters: M, SW
      (?P<district> \d[A-Z\d]? ) # digit, optional digit/letter: 13, 1A
    \s?                           # optional space
      (?P<sector> \d )            # one digit
      (?P<unit> [A-Z]{2} ) $      # two letters
""", re.VERBOSE | re.IGNORECASE)

for pc in ["M13 9PL", "sw1a 1aa", "M13-9PL"]:
    m = POSTCODE.match(pc)
    print(pc, "->", m and m.group("area", "sector"))
```

```
M13 9PL -> ('M', '9')
sw1a 1aa -> ('sw', '1')
M13-9PL -> None
```

TRAP

Some patterns take exponential time on certain inputs (catastrophic backtracking), e.g. `(a+)+$` against `"aaaaaaaaaaaaaaaaaaaaaaaa!"`. If a regex runs on text from users, keep it simple, avoid nested quantifiers, and limit the input length. This has taken down real websites.

A scanner: turning text into tokens

Compilers, calculators and config parsers all start by splitting text into **tokens** (Ch 2). One regex with a named alternative per token type, scanned left to right, does it in a few lines:

NUMBER (orange) · NAME (purple) · OP (green) **11 matches**

total = price * (1 + rate_2) - 3.5

```python
import re

TOKEN = re.compile(r"""
    (?P<NUMBER> \d+(?:\.\d+)? ) | (?P<NAME> [A-Za-z_]\w* ) |
    (?P<OP> [-+*/=()] )         | (?P<SKIP> \s+ )           | (?P<BAD> . )
""", re.VERBOSE)

def tokens(text):
    for m in TOKEN.finditer(text):
        kind = m.lastgroup                    # the name of the group that matched
        if kind == "SKIP":
            continue
        if kind == "BAD":
            raise SyntaxError(f"unexpected {m.group()!r} at column {m.start()}")
        yield kind, m.group()

print(" ".join(f"{kind}:{text}" for kind, text in tokens("total = price * (1 + rate_2)")))
print(list(tokens("2 $ 3")))
```

```
NAME:total OP:= NAME:price OP:* OP:( NUMBER:1 OP:+ NAME:rate_2 OP:)
SyntaxError: unexpected '$' at column 2
```

Other text tools in the standard library

```python
import string, textwrap

t = string.Template("Dear $name, '$title' is due on ${day}.")
print(t.substitute(name="Ana", title="Dune", day="Friday"))
print(textwrap.fill("Regular expressions are a small language for describing text.", width=28))
print(textwrap.shorten("A very long book title that will not fit", width=24, placeholder="..."))
```

```
Dear Ana, 'Dune' is due on Friday.
Regular expressions are a
small language for
describing text.
A very long book...
```

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 9.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 9.1` shows other valid solutions and their trade-offs.

`9.1` build from a spec URL slugs from any title
`9.2` build from a spec Parse a log line with named groups
`9.3` fix the bug The greedy tag stripper
`9.4` build from a spec A text table with format specs
`9.5` build from a spec Redact personal data
`9.6` mini-project Mini-project: a calculator from scratch

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

`str` is immutable Unicode text (code points); normalise with `unicodedata`, compare with `casefold`. Format specs: `{x:>10,.2f}`. Regex via `re` (Perl-style syntax, raw strings, named groups, `sub` with functions, `VERBOSE`).

12. Stdlib and tooling

`re`, `string`, `textwrap`, `unicodedata`, `difflib` cover most text work.

IN OTHER LANGUAGES

Regex syntax is nearly the same everywhere (Perl-compatible): JavaScript `/\d+/g`, Java `Pattern.compile`, `grep -E`, your editor's search box. What changes is the API around it and a few features (named group syntax, lookbehind support).

DEEP DIVE (your vault)

Vault notes: 5.11 Strings, Unicode, and textual value · 5.12 Bytes and the text boundary · 7.9 Strings as immutable sequences with textual semantics.

Chapter 10

The algorithm toolkit

the right container turns a slow program into a fast one

Lists, dicts and sets solve most problems. A handful of standard-library modules cover almost everything else that algorithm courses and interviews ask for. Each one exists because a common operation is **slow** with the obvious structure:

you need to…	obvious way	cost	use instead	cost
count things	dict + `if k in d`	fiddly	`collections.Counter`	O(n)
take from the front of a queue	`list.pop(0)`	**O(n)**	`deque.popleft()`	O(1)
repeatedly get the smallest	`min(xs)` then remove	**O(n)**	`heapq.heappop`	O(log n)
find a position in sorted data	scan the list	**O(n)**	`bisect.bisect_left`	O(log n)
test membership many times	`x in some_list`	**O(n)**	`x in some_set`	O(1)
every pair / combination	nested loops	fiddly	`itertools.combinations`	lazy

The difference is not academic. The same job, 50,000 items, measured on this machine while building the book:

```python
import time
from collections import deque

def drain_list(n):
    q = list(range(n))
    while q:
        q.pop(0)                # shifts every remaining item left

def drain_deque(n):
    q = deque(range(n))
    while q:
        q.popleft()             # just moves a pointer

for fn in (drain_list, drain_deque):
    t = time.perf_counter(); fn(50_000); ms = (time.perf_counter() - t) * 1000
    print(f"{fn.__name__:12} {ms:8.1f} ms")
```

```
drain_list      118.0 ms
drain_deque       1.7 ms
```

KEY IDEA

Big-O (Ch 20) tells you how the cost grows. O(n) per operation inside a loop of n is O(n²) overall: 50,000 items means ~1.25 billion element moves. Choosing the structure is the optimisation; micro-tuning the loop rarely matters.

collections: Counter, defaultdict, deque

```python
from collections import Counter, defaultdict

words = "the cat and the hat and the bat".split()
c = Counter(words)
print(c.most_common(2), c["the"], c["dog"])         # missing keys count as 0
print(c + Counter(["cat", "dog"]))                    # counters add like multisets

by_letter = defaultdict(list)                         # missing key -> list() made for you
for w in words:
    by_letter[w[0]].append(w)
print(dict(by_letter))
```

```
[('the', 3), ('and', 2)] 3 0
Counter({'the': 3, 'cat': 2, 'and': 2, 'hat': 1, 'bat': 1, 'dog': 1})
{'t': ['the', 'the', 'the'], 'c': ['cat'], 'a': ['and', 'and'], 'h': ['hat'], 'b': ['bat']}
```

plain dict

```python
groups = {}
for w in "ab ac bd".split():
    if w[0] not in groups:
        groups[w[0]] = []
    groups[w[0]].append(w)
print(groups)
```

```
{'a': ['ab', 'ac'], 'b': ['bd']}
```

defaultdict

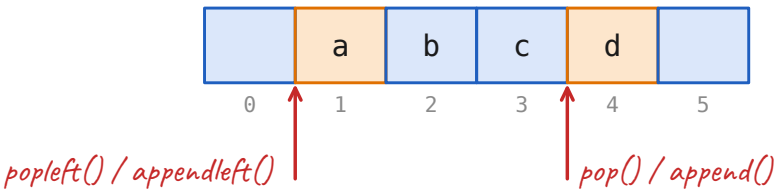
```python
from collections import defaultdict
groups = defaultdict(list)
for w in "ab ac bd".split():
    groups[w[0]].append(w)

print(dict(groups))
```

```
{'a': ['ab', 'ac'], 'b': ['bd']}
```



deque: a queue with two fast ends



```python
from collections import deque

recent = deque(maxlen=3)              # keeps only the last 3: older items fall off the left
for page in ["home", "dune", "emma", "ubik", "kim"]:
    recent.append(page)
print(recent, recent[0])

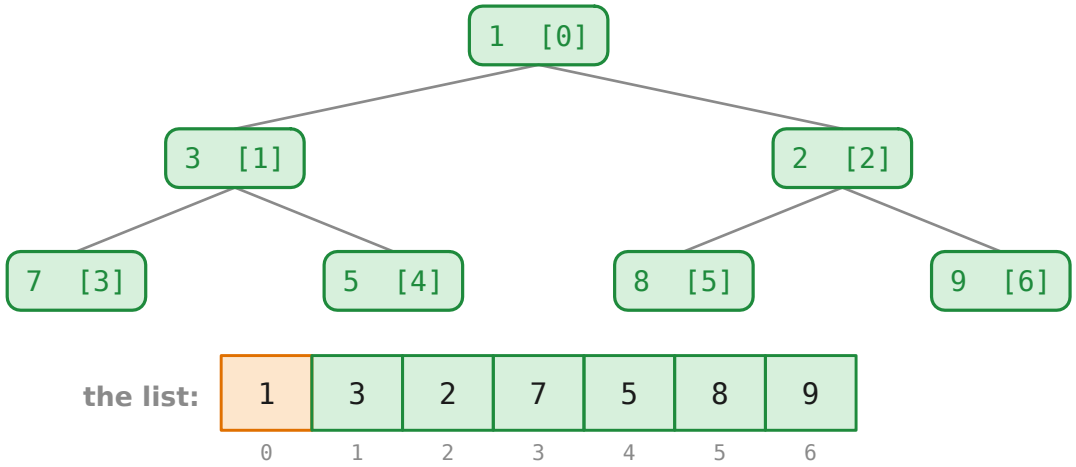
q = deque(["ana", "ben"])
q.append("cy"); first = q.popleft(); q.rotate(1)
print(first, q)
```

```
deque(['emma', 'ubik', 'kim'], maxlen=3) emma
ana deque(['cy', 'ben'])
```

also in collections	for
`OrderedDict`	`move_to_end()`, `popitem(last=False)`: an LRU cache in a few lines (practice 10.6)
`ChainMap(a, b)`	look up in `a`, then `b`: layered config (command line > env > defaults)
`namedtuple("P", "x y")`	a tuple with field names (prefer `typing.NamedTuple` or a dataclass)

heapq: always know the smallest

A **heap** is a list kept in a special order: every parent is ≤ its children, so the smallest item is always at index 0. It's a binary tree stored flat in a list: the children of index `i` are at `2i+1` and `2i+2`. Push and pop are O(log n) because only one root-to-leaf path is fixed up:




```python
import heapq

h = [7, 3, 9, 1, 5, 8, 2]
heapq.heapify(h)                          # O(n), in place
print(h, h[0])
heapq.heappush(h, 0)
print([heapq.heappop(h) for _ in range(3)])

tasks = []                                # (priority, order, item): ties broken by order
for i, (pri, job) in enumerate([(2, "email"), (1, "backup"), (2, "report")]):
    heapq.heappush(tasks, (pri, i, job))
print([heapq.heappop(tasks)[2] for _ in range(len(tasks))])

scores = {"ana": 81, "ben": 67, "cy": 94, "di": 72}
print(heapq.nlargest(2, scores, key=scores.get))     # top k without a full sort
```

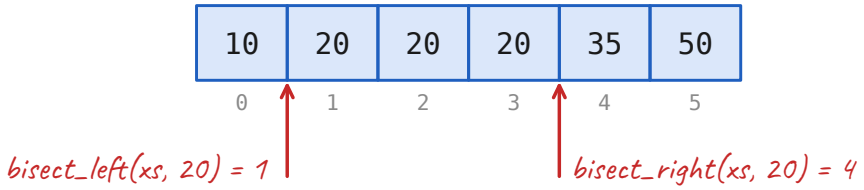
```
[1, 3, 2, 7, 5, 8, 9] 1
[0, 1, 2]
['backup', 'email', 'report']
['cy', 'ana']
```

TRAP

`heapq` is a **min**-heap only. For a max-heap push negated keys (`-score`). And a heap list is not sorted: only `h[0]` is guaranteed to be the minimum.

bisect: binary search in sorted data

`bisect` finds where a value belongs in a sorted list by halving the search range each step: about 17 steps for 100,000 items instead of up to 100,000:




```python
import bisect

xs = [10, 20, 20, 20, 35, 50]
print(bisect.bisect_left(xs, 20), bisect.bisect_right(xs, 20))
print(bisect.bisect_right(xs, 20) - bisect.bisect_left(xs, 20), "copies of 20")

bisect.insort(xs, 27)                       # insert keeping it sorted
print(xs)

def grade(score, cutoffs=(40, 50, 60, 70), letters="FDCBA"):
    return letters[bisect.bisect_right(cutoffs, score)]
print([grade(s) for s in (12, 40, 55, 69, 70, 99)])
```

```
1 4
3 copies of 20
[10, 20, 20, 20, 27, 35, 50]
['F', 'D', 'C', 'B', 'A', 'A']
```

WHY

`bisect_left` gives the first position where the value could go (before equal items), `bisect_right` the last (after them). Their difference counts the equal items; `bisect_right` on cutoffs turns a chain of `if score >= 70 ... elif` into one line.

itertools: loops you don't have to write

```python
from itertools import (accumulate, chain, combinations, count, groupby, islice,
                       pairwise, permutations, product, batched)

print(list(combinations("ABC", 2)), len(list(permutations("ABC"))))
print(list(product("xy", [0, 1])))                      # nested loops, flattened
print(list(accumulate([3, 1, 4, 1, 5])))                # running totals
print(list(pairwise([1, 4, 9, 16])))                    # neighbours
print(list(batched("ABCDEFG", 3)))                      # chunks (3.12+)
print(list(islice(count(10, 5), 4)), list(chain("ab", [1, 2])))
```

```
[('A', 'B'), ('A', 'C'), ('B', 'C')] 6
[('x', 0), ('x', 1), ('y', 0), ('y', 1)]
[3, 4, 8, 9, 14]
[(1, 4), (4, 9), (9, 16)]
[('A', 'B', 'C'), ('D', 'E', 'F'), ('G',)]
[10, 15, 20, 25] ['a', 'b', 1, 2]
```

`groupby` groups **consecutive** equal keys, so it needs sorted input to group everything:

unsorted: surprise

```python
from itertools import groupby
ws = ["ant", "bee", "ape", "bat"]
print([(k, len(list(g)))
       for k, g in groupby(ws, key=lambda w: w[0])])
```

```
[('a', 1), ('b', 1), ('a', 1), ('b', 1)]
```

sorted first

```python
from itertools import groupby
ws = sorted(["ant", "bee", "ape", "bat"])
print([(k, len(list(g)))
       for k, g in groupby(ws, key=lambda w: w[0])])
```

```
[('a', 2), ('b', 2)]
```

function	gives	count for n items
`product(xs, repeat=k)`	every k-tuple, with repeats, order matters	n^k
`permutations(xs, k)`	ordered arrangements, no repeats	n! / (n−k)!
`combinations(xs, k)`	unordered selections, no repeats	n! / (k!(n−k)!)
`combinations_with_replacement`	unordered, repeats allowed	(n+k−1)! / (k!(n−1)!)

TRAP

These grow explosively: `permutations(range(12))` is 479 million tuples. They're lazy, so creating one is free, but `list()` on it will exhaust memory. Loop over them and stop early.

functools and operator

```python
from functools import reduce, cache, partial, cmp_to_key
from operator import mul, itemgetter

print(reduce(mul, range(1, 6)))                          # 1*2*3*4*5

@cache
def ways(n):                                             # staircase: 1 or 2 steps at a time
    return 1 if n <= 1 else ways(n - 1) + ways(n - 2)
print(ways(80))

def by_length_then_reverse_alpha(a, b):                  # old-style comparator: <0, 0, >0
    return (len(a) - len(b)) or ((a < b) - (a > b))
print(sorted(["bb", "a", "ccc", "aa"], key=cmp_to_key(by_length_then_reverse_alpha)))

rows = [("ana", 3), ("ben", 1), ("cy", 2)]
print(sorted(rows, key=itemgetter(1)), max(rows, key=itemgetter(1))[0])
```

```
120
37889062373143906
['a', 'bb', 'aa', 'ccc']
[('ben', 1), ('cy', 2), ('ana', 3)] ana
```

structure	add	remove	find min/max	x in s	notes
`list`	O(1) end	O(n) front	O(n)	O(n)	indexing O(1)
`deque`	O(1) both ends	O(1) both ends	O(n)	O(n)	indexing middle O(n)
`dict` / `set`	O(1)	O(1)	O(n)	**O(1)**	hashable keys; order kept (dict)
`heapq` list	O(log n)	O(log n) min	**O(1)** min	O(n)	min-heap only
sorted list + `bisect`	O(n) insort	O(n)	O(1) ends	O(log n)	great when read-mostly

☐ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 10.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 10.1` shows other valid solutions and their trade-offs.

`10.1` build from a spec The k most frequent words
`10.2` build from a spec Merge sorted streams lazily
`10.3` measure and explain Pairs that sum to a target, fast
`10.4` build from a spec Running median with two heaps
`10.5` build from a spec Binary search questions
`10.6` mini-project Mini-project: an LRU cache

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

Beyond list/dict/set: `Counter`, `defaultdict`, `deque`, `OrderedDict`, `heapq` (min-heap on a list), `bisect` (binary search), lazy `itertools`, `functools.cache`/`reduce`.

12. Stdlib and tooling

Know each structure's costs; pick by the operation you do most.

IN OTHER LANGUAGES

Every language ships the same toolkit under other names: Java `ArrayDeque`, `PriorityQueue`, `Collections.binarySearch`, `HashMap`; C++ `std::deque`, `std::priority_queue` (a max-heap!), `std::lower_bound`; Rust `VecDeque`, `BinaryHeap`, `slice::binary_search`.

DEEP DIVE (your vault)

Vault notes: 7.2 A minimal cost vocabulary · 7.10 Choosing among sequence representations · 7.11 Sorting and ordering contracts · 7.15 A structure-selection case study · 11.11 collections · 11.12 itertools · 11.13 functools.

Chapter 11

Numbers and time

count exactly, measure approximately, keep time carefully

Python has five kinds of number. Picking the wrong one gives answers that are **almost** right, which is worse than an error because nothing tells you:

type	exact?	size	use it for
`int`	**yes**	unlimited: grows as needed	counting, indexes, ids, money stored as pence
`float`	**no**: nearest binary fraction	64 bits, about 15-17 significant digits	measurements, science, graphics
`Decimal`	**yes** for base-10 digits	you choose the precision	money, anything a person types in decimal
`Fraction`	**yes**	unlimited	exact ratios, probability, maths
`complex`	as two floats	two 64-bit floats	signals, geometry, `(-1) ** 0.5`

```
1 from decimal import Decimal
2 from fractions import Fraction
3
4 print(2 ** 100)                                  # ints never overflow
5 print(0.1 + 0.2, 0.1 + 0.2 == 0.3)               # floats are approximations
6 print(Decimal("0.1") + Decimal("0.2"), Fraction(1, 3) + Fraction(1, 6))
7 print(7 / 2, 7 // 2, type(2 ** 0.5).__name__, (-1) ** 0.5)
```

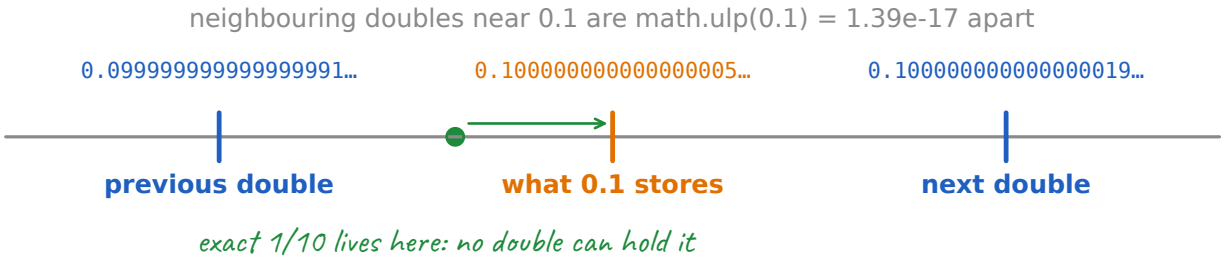
```
1267650600228229401496703205376
0.30000000000000004 False
0.3 1/2
3.5 3 float (6.123233995736766e-17+1j)
```

line 5: the famous one: next page explains it

Time is worse. Clocks get corrected, countries move their clocks twice a year, and "3 pm" means nothing without a place. The second half of the chapter gives you the three rules that avoid nearly every time bug.

Why 0.1 + 0.2 isn't 0.3

A float stores `sign × 1.fraction × 2^exponent` with 52 bits of fraction. Only numbers that are sums of powers of two fit exactly. One tenth in binary is `0.0001100110011001100…` forever, just as 1/3 is `0.333…` in decimal, so Python stores the **nearest** double instead:




```
1 import math
2 from decimal import Decimal
3
4 print(Decimal(0.1))                          # the exact value actually stored
5 print((0.1).as_integer_ratio())              # = that value as a fraction over 2**55
6 print(0.1 + 0.2)                            # two rounding errors, then a third
7 print(math.isclose(0.1 + 0.2, 0.3), abs((0.1 + 0.2) - 0.3) < 1e-9)
8 print(sum([0.1] * 10), math.fsum([0.1] * 10))
9 print(math.ulp(1.0), math.ulp(1e16), 1e16 + 1 == 1e16)
```

```
0.1000000000000000055511151231257827021181583404541015625
(3602879701896397, 36028797018963968)
0.30000000000000004
True True
1.0 1.0
2.220446049250313e-16 2.0 True
```

line 7: compare with a tolerance

line 8: fsum tracks the lost bits

line 9: big floats have big gaps

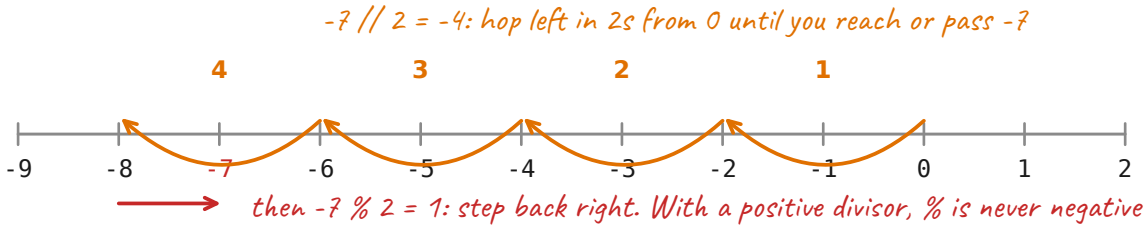
trap	what happens	do this instead
`a == b` on results	tiny differences make equal values unequal	`math.isclose(a, b)`
adding many floats	errors pile up	`math.fsum`, or Decimal / int pence
huge + tiny	the tiny one vanishes (`1e16 + 1`)	rescale, or sort small-first
`float("nan")`	`nan != nan`, and it spreads through maths	`math.isnan(x)`
money in floats	`0.1 + 0.2` pence errors in totals	`Decimal("0.10")` or int pence

IN OTHER LANGUAGES

This is IEEE 754, built into the CPU. C `double`, Java `double`, JavaScript `Number` (its **only** number type), Rust `f64` all print `0.30000000000000004`. It is not a Python bug.

Integers, division and rounding

Python ints grow as large as memory allows, so there is no overflow. `/` always gives a float; `//` gives the **floor** (rounds towards minus infinity), and `%` is whatever is left over:




```
1 import math
2
3 print(2 ** 200 % 1000, (2 ** 200).bit_length())
4 print(7 // 2, -7 // 2, 7 % 3, -7 % 3, divmod(-7, 2))
5 print(int(-3.7), math.floor(-3.7), math.trunc(-3.7), math.ceil(-3.7))
6 print(round(2.5), round(3.5), round(0.125, 2), round(2.675, 2))
7 print(pow(3, 200, 1000), math.gcd(12, 18), math.lcm(4, 6), math.isqrt(99))
8 print(math.comb(5, 2), math.perm(5, 2), math.factorial(20))
```

```
376 201
3 -4 1 2 (-4, 1)
-3 -4 -3 -3
2 4 0.12 2.67
1 6 12 9
10 20 2432902008176640000
```

line 4: floor: -3.5 becomes -4

line 6: half to even; 2.675 is really 2.67499...

line 7: modular power: fast, no huge number

WHY

Flooring keeps `a == (a // b) * b + a % b` true **and** keeps `%` the same sign as the divisor, so `(i - 1) % n` wraps round a list cleanly. C, Java and JavaScript truncate towards zero instead: `-7 / 2` is `-3` and `-7 % 2` is `-1` there. Port maths code carefully.

you want	use	2.5 →	-2.5 →
round to nearest, ties to even (less bias)	`round(x)`	2	-2
round half up (school rounding)	`Decimal.quantize(..., ROUND_HALF_UP)`	3	-3
always down / always up	`math.floor` / `math.ceil`	2 / 3	-3 / -2
chop the fraction off	`int(x)` or `math.trunc(x)`	2	-2

decimal and fractions: exact arithmetic

`Decimal` does arithmetic in base 10, the way you do it on paper, so `0.10` stays exactly `0.10`. It's slower than float, and that never matters for money:

float

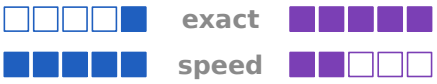
```
prices = [0.10] * 3
total = sum(prices)
print(total, total == 0.30)
print(f"{total:.2f}")
```

```
0.30000000000000004 False
0.30
```

Decimal

```
from decimal import Decimal
prices = [Decimal("0.10")] * 3
total = sum(prices)
print(total, total == Decimal("0.30"))
```

```
0.30 True
```




```
1  from decimal import Decimal, ROUND_HALF_UP, localcontext
2
3  print(Decimal(0.1) == Decimal("0.1"))         # made from a float: inherits its error
4  price = Decimal("19.99")
5  vat = (price * Decimal("0.20")).quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)
6  print(vat, price + vat)
7  print(Decimal("2.675").quantize(Decimal("0.01")), Decimal(1) / Decimal(7))
8  with localcontext() as ctx:
9      ctx.prec = 50                             # digits of precision, for this block only
10     print(Decimal(1) / Decimal(7))
```

```
False
4.00 23.99
2.68 0.14285714285714285714285714286
0.14285714285714285714285714285714285714285714285714
```

line 3: always build from strings

line 5: quantize = round to these places

Fraction: exact ratios

```
1 import math
2 from fractions import Fraction
3
4 x = Fraction(1, 3) + Fraction(1, 6)
5 print(x, x.numerator, x.denominator, float(x))
6 print(Fraction("0.75"), Fraction(0.1) == Fraction(1, 10))
7 print(Fraction(math.pi).limit_denominator(1000))
8 print(sum(Fraction(1, n) for n in range(1, 11)))
```

```
1/2 1 2 0.5
3/4 False
355/113
7381/2520
```

← best fraction with a small denominator

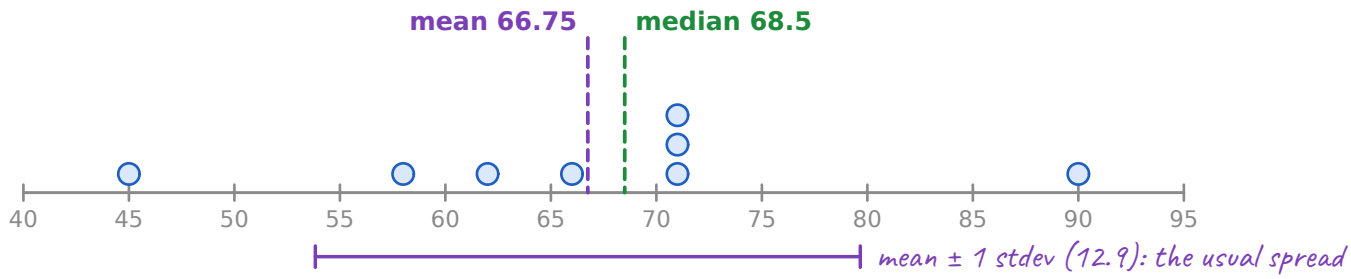
1 + 1/2 + … + 1/10, exactly

RULE OF THUMB

Money: `Decimal` built from strings, or whole pence in an `int`. Measurements: `float` and compare with `math.isclose`. Exact ratios: `Fraction`. Never `Decimal(some_float)`: the error is already inside the float.

math and statistics

`statistics` covers what a first statistics module teaches, with careful numerics. Here are eight exam marks drawn as dots:




```
1 import statistics as st
2
3 marks = [62, 71, 71, 58, 90, 45, 71, 66]
4 print(st.mean(marks), st.median(marks), st.mode(marks))
5 print(round(st.stdev(marks), 2), round(st.pstdev(marks), 2))
6 print(st.quantiles(marks, n=4))                    # the three quartile cut points
7
8 hours, score = [2, 4, 5, 7, 9], [50, 62, 65, 74, 88]
9 fit = st.linear_regression(hours, score)
10 print(round(st.correlation(hours, score), 3), round(fit.slope, 2), round(fit.intercept, 2))
```

```
66.75 68.5 71
12.91 12.08
[59.0, 68.5, 71.0]
0.994 5.22 39.62
```

line 5: sample (n-1) vs population (n)

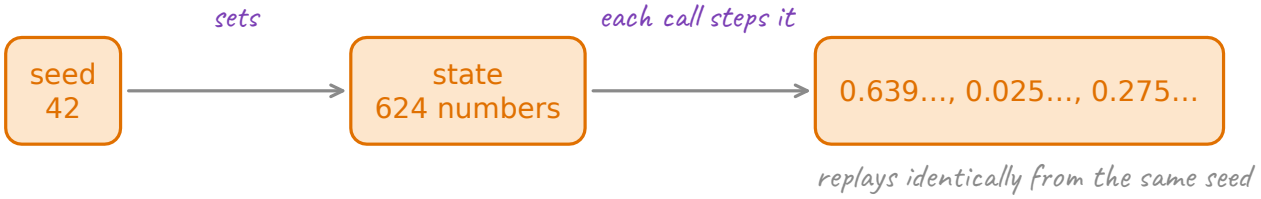
WHY

`stdev` divides by n − 1 because your data is a **sample**: its own mean sits closer to its points than the true mean would, so dividing by n underestimates the spread. Use `pstdev` only when you have the whole population (every student, not some).

math has...	for
`sqrt`, `isqrt`, `cbrt`, `exp`, `log`, `log2`, `log10`	roots and logs (`log(x, base)` too)
`sin`, `cos`, `atan2`, `radians`, `degrees`	trigonometry, in radians
`hypot`, `dist`, `prod`, `sumprod`	lengths, distances, products
`pi`, `tau`, `e`, `inf`, `nan`	constants
`isclose`, `isfinite`, `isnan`, `nextafter`, `ulp`	float housekeeping

random versus secrets

`random` is a **pseudo**-random generator: a deterministic formula (the Mersenne Twister) that turns a seed into a long sequence that only looks random. Same seed, same sequence:




```
1  import random
2
3  rng = random.Random(42)                       # your own generator, reproducible
4  print(rng.randint(1, 6), rng.choice("ABC"), round(rng.random(), 3))
5  again = random.Random(42)
6  print(again.randint(1, 6), again.choice("ABC"), round(again.random(), 3))
7  deck = list(range(10))
8  rng.shuffle(deck)
9  print(deck)
10 print(rng.sample(range(100), 3), rng.choices("HT", weights=[9, 1], k=8))
11 print(round(rng.gauss(0, 1), 3), round(rng.uniform(1.5, 2.5), 3))
```

```
6 A 0.025
6 A 0.025
[7, 6, 2, 9, 0, 5, 1, 8, 3, 4]
[4, 3, 11] ['H', 'H', 'H', 'H', 'H', 'H', 'H', 'H']
0.042 2.306
```

line 6: identical: same seed

line 10: sample: no repeats; choices: repeats, weighted

TRAP

Watch 624 outputs of `random` and you can rebuild its state and predict every later number. Fine for games, simulations and tests; useless against an attacker. Tokens, passwords, keys and anything with money riding on it use `secrets` (Ch 13), which reads the OS's cryptographic generator.

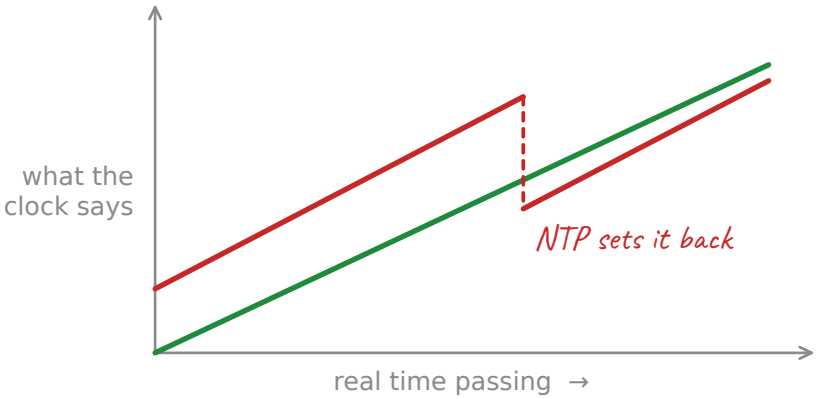
you need	use
a simulation, test data or game you can replay	`random.Random(seed)` (pass the rng in)
shuffle quiz questions	`random.shuffle`
a password-reset token or session id	`secrets.token_urlsafe(32)`
pick the winner of a paid draw	`secrets.choice(entries)`
millions of random numbers for NumPy	`np.random.default_rng(seed)` (Ch 18)

RULE OF THUMB

Pass a `random.Random` object into code that needs randomness instead of calling the global `random.randint`. Tests can then give it a fixed seed, or a fake, and get the same result every run. Practice 11.6 makes you do exactly this.

Three clocks, three jobs

A computer has more than one clock. Using the wrong one is how a 30-second timeout fires instantly, or a benchmark reports negative time:



time.time(): the wall clock. Seconds since 1970 UTC. Can jump either way (NTP sync, a user changing it, a VM resuming).

time.monotonic(), **time.perf_counter()**: never go backwards. Their zero means nothing; only differences do.

call	use it for	can jump?
`time.time()`	timestamps you store: when did this happen?	**yes**
`time.monotonic()`	timeouts, deadlines, retry back-off	**never back**
`time.perf_counter()`	measuring how long code takes (finest resolution)	**never back**
`time.process_time()`	CPU time used by this process (ignores sleeping)	**never back**
`time.sleep(s)`	pause the thread for at least s seconds	

```python
import time

print(time.get_clock_info("time").adjustable, time.get_clock_info("monotonic").adjustable)

deadline = time.monotonic() + 0.05            # 50 ms from now, immune to clock changes
polls = 0
while time.monotonic() < deadline:
    polls += 1
    time.sleep(0.01)
print(polls >= 3)
```

```
True False
True
```

line 3: True: can be set; False: can't

line 5: the deadline pattern

datetime: naive, aware, and the time zone

A `datetime` with no `tzinfo` is **naive**: "14:30 on 28 Sept", but where? One with a zone from `zoneinfo` is **aware**: a single instant everyone agrees on. Only aware times can be converted or compared across places:

```python
from datetime import date, datetime, timedelta, timezone
from zoneinfo import ZoneInfo

naive = datetime(2026, 9, 28, 14, 30)
aware = datetime(2026, 9, 28, 14, 30, tzinfo=ZoneInfo("Europe/London"))
print(naive.tzinfo, aware.utcoffset(), aware.tzname())
print(aware.astimezone(timezone.utc))
print(aware.astimezone(ZoneInfo("Asia/Tokyo")).isoformat())
print(date(2026, 12, 25) - date(2026, 9, 28), date(2026, 9, 28).strftime("%A %d %B"))
print(datetime.fromisoformat("2026-09-28T14:30:00+01:00") == aware)
```

```
None 1:00:00 BST
2026-09-28 13:30:00+00:00
2026-09-28T22:30:00+09:00
88 days, 0:00:00 Monday 28 September
True
```

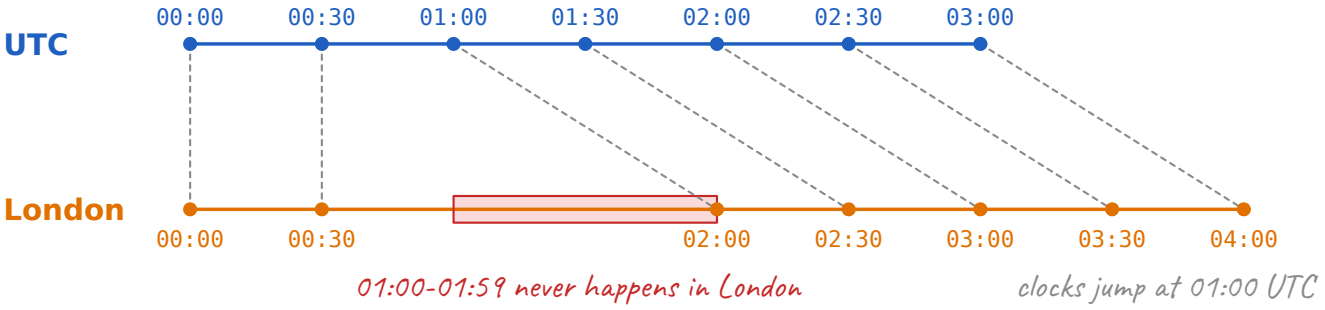
line 7: same instant, other zone

line 10: ISO 8601: the format to store and send

```python
print(naive < aware)
```

```
TypeError: can't compare offset-naive and offset-aware datetimes
```

Clock changes: the gap and the fold




```
1 london = ZoneInfo("Europe/London")
2 before = datetime(2026, 3, 28, 12, 0, tzinfo=london)
3 after = before + timedelta(days=1)            # wall-clock arithmetic within one zone
4 print(after, after - before)
5 print(after.astimezone(timezone.utc) - before.astimezone(timezone.utc))
6 gap = datetime(2026, 3, 29, 1, 30, tzinfo=london)     # never happened
7 print(gap.astimezone(timezone.utc).astimezone(london))
8 twice = datetime(2026, 10, 25, 1, 30, tzinfo=london)  # happened twice
9 print(twice.utcoffset(), twice.replace(fold=1).utcoffset())
```

```
2026-03-29 12:00:00+01:00 1 day, 0:00:00
23:00:00
2026-03-29 02:30:00+01:00
1:00:00 0:00:00
```

line 4: says 1 day…

line 5: …but only 23 hours passed

line 7: shifted out of the gap

line 9: fold picks which 01:30

RULE OF THUMB

1. Store and compute in **UTC**, as aware datetimes: `datetime.now(timezone.utc)` (never `utcnow()`, which is deprecated and naive). 2. Convert to local time only to show a person. 3. Send and store times as ISO 8601 strings with the offset. Repeating events ("every Monday at 9 local") are the one case where you compute in local time, then convert (practice 11.4).

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 11.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 11.1` shows other valid solutions and their trade-offs.

`11.1` build from a spec Split a bill to the penny
`11.2` fix the bug A float range that doesn't drift
`11.3` fix the bug Running mean and variance that stay accurate
`11.4` fix the bug A weekly meeting across clock changes
`11.5` learn an unseen module Calendar questions with an unseen module
`11.6` mini-project A seeded dice roller

PASSPORT STAMP ask the same question of every new language (Appendix B)

3. Typing

Numbers: `int` is unlimited; `float` is an IEEE 754 double (inexact); `Decimal` and `Fraction` are exact library types. `/` gives float, `//` floors. Booleans are ints (`True + True == 2`).

12. Stdlib and tooling

`math`, `statistics`, `random` (reproducible, not secure), `secrets`, `decimal`, `fractions`, `time` (three clocks), `datetime` + `zoneinfo`.

IN OTHER LANGUAGES

C and Java ints have fixed sizes and overflow (silently in C, wrapping in Java); Rust panics on overflow in debug builds. Java has `BigDecimal`, C# `decimal`, JavaScript `BigInt`. Every language has the same naive/aware time split: Java `LocalDateTime` vs `ZonedDateTime`, JS `Temporal.PlainDateTime` vs `Temporal.ZonedDateTime`.

DEEP DIVE (your vault)

Vault notes: 5.9 Numeric objects · 16.3 Descriptive location and dispersion · 16.11 Reductions, axes, and numerical stability · 16.12 Randomness, generators, and reproducibility · 16.21 Time data, ordering, and time zones · 19.22 Cryptographic randomness.

Chapter 12

sqlite3: a database in a file

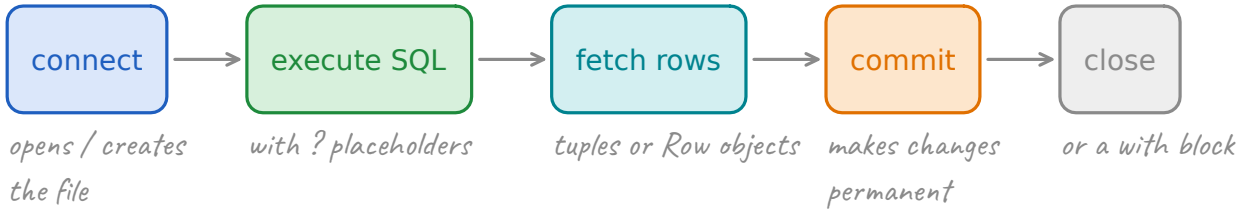
a file you can ask questions, that survives crashes

Sooner or later a program needs to remember things between runs: users, bookings, scores, loans. A JSON file works until you need to change one record without rewriting the lot, find "every loan overdue by a week", or survive the power going off halfway through a save.

SQLite is a full SQL database that lives in **one ordinary file** and ships inside Python as `sqlite3`. No server, no install. It's in every phone and browser, and it's the right first database for almost any project.

need	JSON file	SQLite
change one record	rewrite the whole file	`UPDATE` touches one row
"which loans are overdue?"	load everything, loop in Python	one `SELECT ... WHERE`
crash halfway through a save	file can be left half-written	transaction: all or nothing
two programs at once	last writer wins, silently	locking handles it
rules ("email is unique")	your code must remember to check	the database refuses bad rows

The lifecycle you'll use every time:



Your first database, end to end

```python
import sqlite3

conn = sqlite3.connect("library.db")
conn.execute("""CREATE TABLE IF NOT EXISTS books (
                    id     INTEGER PRIMARY KEY,
                    title  TEXT NOT NULL,
                    year   INTEGER)""")
conn.execute("INSERT INTO books (title, year) VALUES (?, ?)",
             ("Dune", 1965))
conn.commit()
conn.close()

conn = sqlite3.connect("library.db")
print(conn.execute("SELECT * FROM books").fetchall())
conn.close()
```

← 1. connect (creates the file)

← 2. execute SQL

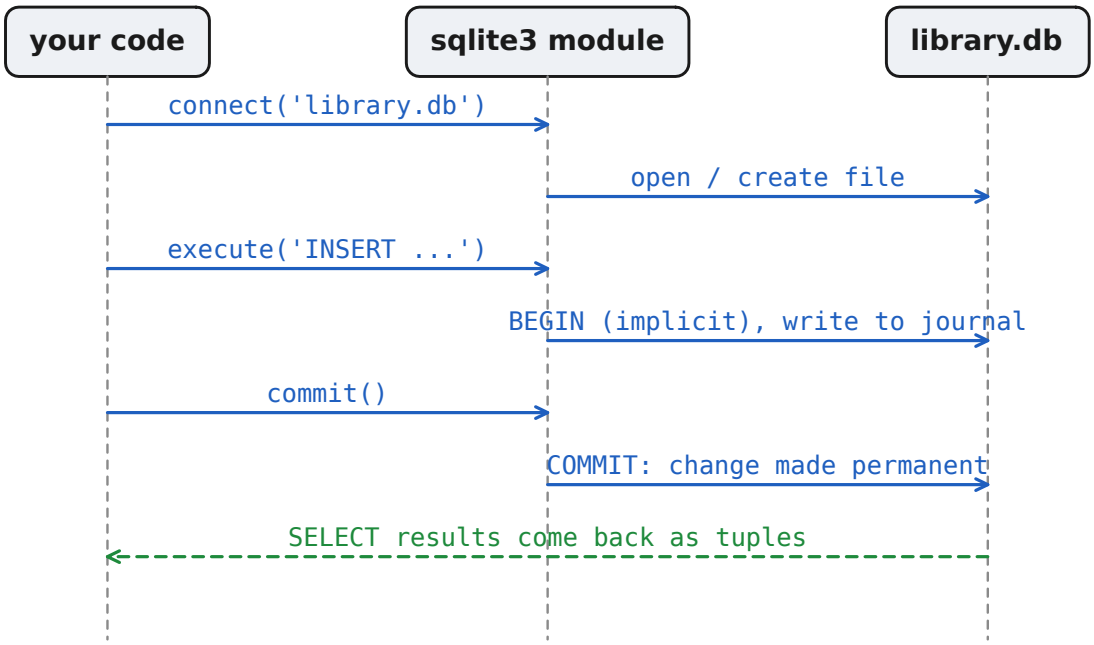
← 3. commit: now on disk

4. close

a later run: still there

```
[(1, 'Dune', 1965)]
```

That's the whole library. Everything else in this chapter is SQL (the language you send through `execute`) and the habits that keep it safe. Here's what happens between your program and the file:

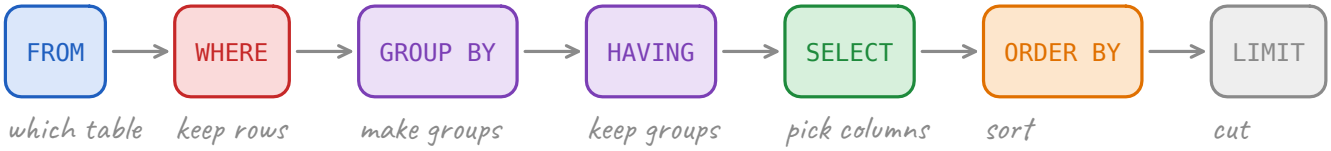


TRAP

Close without `commit()` and your inserts are **silently thrown away**. sqlite3 opens a transaction automatically before the first `INSERT`/`UPDATE`/`DELETE`, and only `commit()` ends it. (The `with conn:` form in a few pages does this for you.)

SQL in one picture: a query is a pipeline

You write a query in one order, but the database **runs** it in another. Read it in run order and SQL stops being mysterious:



Eight books are already in a table. Every grid below is the real result:

```sql
SELECT title, author, year FROM books WHERE year > 1900 ORDER BY year LIMIT 4
```

title	author	year
Dune	Herbert	1965
Children of Dune	Herbert	1976
Neuromancer	Gibson	1984
Count Zero	Gibson	1986

```sql
SELECT author, COUNT(*) AS n_books, SUM(copies) AS copies
FROM books
GROUP BY author
HAVING COUNT(*) > 1
ORDER BY n_books DESC, author
```

author	n_books	copies
Austen	3	6
Gibson	2	1
Herbert	2	3

✎ YOUR GO (pencil, or the REPL, first)

Before reading on, predict the rows: SELECT title FROM books WHERE copies = 0 OR author = 'Simmons'. Then: why can't WHERE use the alias n_books? (Look at the pipeline.)

```sql
SELECT title FROM books WHERE copies = 0 OR author = 'Simmons'
```

title
Count Zero
Hyperion

clause	job	tiny example
SELECT	columns or expressions to return	SELECT title, year + 1 AS next
WHERE	filter rows (before grouping)	WHERE year < 1990
GROUP BY	one output row per group	GROUP BY author
HAVING	filter groups (after grouping)	HAVING COUNT(*) > 1

`ORDER BY`	sort; `DESC` reverses	`ORDER BY year DESC, title`
`LIMIT`	first n rows (`OFFSET` skips)	`LIMIT 10 OFFSET 20` (page 3)

Changing data: INSERT, UPDATE, DELETE

```sql
UPDATE books SET copies = copies + 1 WHERE title = 'Count Zero'
```

```
ok: 1 row changed
```

```sql
DELETE FROM books WHERE year < 1812
```

```
ok: 1 row changed
```

```sql
SELECT title, copies FROM books WHERE author = 'Austen' OR title = 'Count Zero'
```

title	copies
Emma	3
Persuasion	1
Count Zero	1

TRAP

An `UPDATE` or `DELETE` with no `WHERE` changes **every row**. Write the `WHERE` first, and try it as a `SELECT` before turning it into an `UPDATE`.

Placeholders, and the attack they stop

The single most important habit in this chapter. Never build SQL by pasting values into the string. Here's a login check written the wrong way, and what an attacker types as the username:

```
import sqlite3
users = sqlite3.connect(":memory:")
users.execute("CREATE TABLE users (name TEXT, pw TEXT)")
users.executemany("INSERT INTO users VALUES (?, ?)",
                  [("alice", "x1"), ("bob", "x2")])

def find_user_BAD(name):
    q = f"SELECT name FROM users WHERE name = '{name}'"
    print("   runs:", q)
    return users.execute(q).fetchall()

print(find_user_BAD("alice"))
print(find_user_BAD("nobody' OR '1'='1"))
```

← value pasted into SQL

← attacker's 'name'

```
   runs: SELECT name FROM users WHERE name = 'alice'
[('alice',)]
   runs: SELECT name FROM users WHERE name = 'nobody' OR '1'…
[('alice',), ('bob',)]
```

The quote in the input **closed the string early**, and the rest became SQL: `OR '1'='1'` is true for every row, so the "login" matched everyone. With a placeholder, the value travels separately from the SQL and can never become code:

```
def find_user(name):
    q = "SELECT name FROM users WHERE name = ?"
    return users.execute(q, (name,)).fetchall()

print(find_user("alice"))
print(find_user("nobody' OR '1'='1"))
print(find_user("O'Brien"))
```

← (name,) is a 1-tuple

← quotes just work, too

```
[('alice',)]
[]
[]
```

placeholder style	looks like	params
qmark	`name = ? AND year = ?`	a tuple, in order: `(name, year)`
named	`name = :n AND year = :y`	a dict: `{"n": name, "y": 1965}`

TRAP

Placeholders are for **values only**. A table or column name can't be a `?`. If one has to vary, check it against a fixed list (`if col not in {"title", "year"}: raise ValueError`) before putting it in the string. And `(name)` is not a tuple; `(name,)` is.

Getting rows out

`execute` returns a **cursor**: a pointer into the result that you read from. Four ways to read it:

```
cur = conn.execute("SELECT title, year FROM books ORDER BY year")
print(cur.fetchone())            # the next row, or None at the end
print(cur.fetchmany(2))          # the next 2 rows
print(len(cur.fetchall()))       # everything left

for title, year in conn.execute("SELECT title, year FROM books LIMIT 2"):
    print(title, year)           # iterate: one row at a time, low memory
```

```
('Emma', 1815)
[('Persuasion', 1817), ('Dune', 1965)]
4
Dune 1965
Neuromancer 1984
```

Tuples are fine for two columns, but `row[3]` is unreadable. Set `row_factory = sqlite3.Row` and rows work by name as well as position:

```
import sqlite3
conn.row_factory = sqlite3.Row
row = conn.execute("SELECT * FROM books WHERE title = ?", ("Dune",)).fetchone()
print(row["title"], row["year"], row.keys())
print(dict(row))
conn.row_factory = None
```

```
Dune 1965 ['id', 'title', 'author', 'year', 'copies']
{'id': 1, 'title': 'Dune', 'author': 'Herbert', 'year': 1965, 'copies': 2}
```

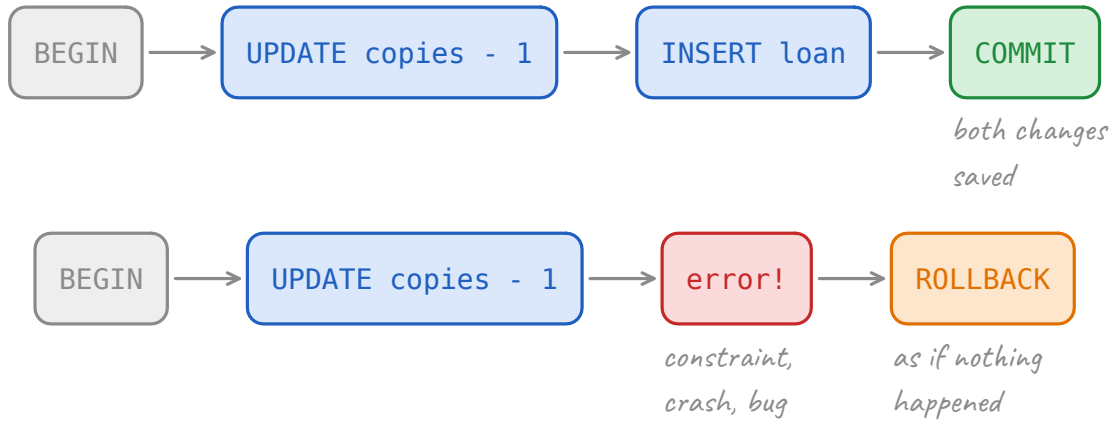
Insert many rows at once with `executemany`, and get the id SQLite gave a new row with `lastrowid`:

```
new = [("Ubik", "Dick", 1969, 1), ("Solaris", "Lem", 1961, 2)]
conn.executemany("INSERT INTO books (title, author, year, copies) "
                 "VALUES (?, ?, ?, ?)", new)
cur = conn.execute("INSERT INTO books (title, author) VALUES (?, ?)",
                   ("Kindred", "Butler"))
print("new id:", cur.lastrowid)
conn.commit()
```

```
new id: 10
```

Transactions: all or nothing

Lending a book is two changes: take a copy off the shelf **and** record the loan. If the program crashes between them, the copy has vanished and nobody has it. A **transaction** groups changes so they all happen or none do.



`with conn:` is the tidy way: it commits if the block finishes and rolls back if an exception escapes. Here the `CHECK (copies >= 0)` rule fires on the second loan of a one-copy book:

```
conn.execute("CREATE TABLE IF NOT EXISTS loans (book_id INTEGER, who TEXT)")
conn.commit()

def lend(book_id, who):
    with conn:                                  # one transaction
        conn.execute("INSERT INTO loans VALUES (?, ?)", (book_id, who))
        conn.execute("UPDATE books SET copies = copies - 1 WHERE id = ?", (book_id,))

nid = conn.execute("SELECT id FROM books WHERE title='Neuromancer'").fetchone()[0]
lend(nid, "ana")
try:
    lend(nid, "ben")                            # no copies left
except sqlite3.IntegrityError as e:
    print("refused:", e)
print(conn.execute("SELECT who FROM loans").fetchall())
print(conn.execute("SELECT copies FROM books WHERE id=?", (nid,)).fetchone())
```

```
refused: CHECK constraint failed: copies >= 0
[('ana',)]
(0,)
```

Ben's loan row was inserted first, then the `UPDATE` failed, and the rollback removed his loan too. The database never shows half a loan.

TRAP

`with conn:` commits or rolls back, but it does **not close** the connection (unlike `with open(...)`). Close it yourself, or use `contextlib.closing(sqlite3.connect(...))`.

HONEST NOTE

Python 3.12 added `sqlite3.connect(..., autocommit=False)`, which gives PEP 249 behaviour (always in a transaction; you commit). The default you've seen here is the older "legacy" mode, and it's what most existing code and tutorials assume.

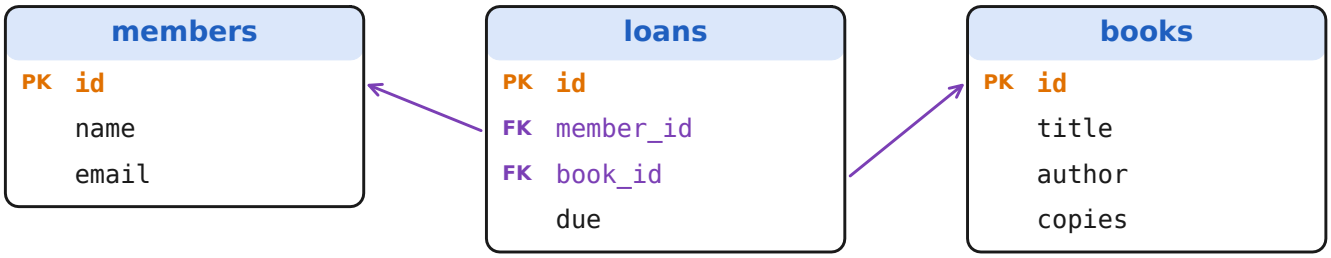
Designing tables: one fact in one place

The first design everyone writes is one wide table. It works until a fact needs to change:

```sql
SELECT * FROM loans_flat
```

member	email	book	due
Ana Silva	ana@x.org	Dune	2026-10-01
Ana Silva	ana@x.org	Emma	2026-10-03
Ben Okafor	ben@x.org	Dune	2026-10-02
Ana Silva	ana@old.org	Ubik	2026-10-09

Ana's email is stored three times, and one copy is already out of date. Which is right? This is an **update anomaly**. The fix is **normalisation**: each fact is stored once, in the table it belongs to, and other tables **refer to it by id**.



constraint	the database refuses...	error
`PRIMARY KEY`	a duplicate or missing id (`INTEGER PRIMARY KEY` auto-numbers)	IntegrityError
`NOT NULL`	a missing value	IntegrityError
`UNIQUE`	a second row with the same value (two members, one email)	IntegrityError
`CHECK (expr)`	a row where expr is false (`copies >= 0`)	IntegrityError
`REFERENCES t(id)`	an id that doesn't exist in `t`, **only if** `PRAGMA foreign_keys = ON`	IntegrityError

```sql
INSERT INTO loans2 (member_id, book_id, due) VALUES (99, 1, '2026-11-01')
```

```
sqlite3.IntegrityError: FOREIGN KEY constraint failed
```

TRAP
SQLite ignores `REFERENCES` unless you run `PRAGMA foreign_keys = ON` on **every new connection**, straight after connecting (it does nothing inside an open transaction). Put it in your `connect()` helper and forget about it.

JOIN: putting the facts back together

Normalised tables are split apart; a **join** glues them back for one query. `JOIN ... ON` pairs every row of one table with the rows of the other where the condition holds:

```sql
SELECT m.name, b.title, l.due
FROM loans2 AS l
JOIN members AS m ON m.id = l.member_id
JOIN books   AS b ON b.id = l.book_id
ORDER BY l.due
```

name	title	due
Ana Silva	Dune	2026-10-01
Ben Okafor	Dune	2026-10-02
Ana Silva	Emma	2026-10-03

Cat has no loans, so she isn't in that result: a plain (inner) join keeps only rows that match. To keep everyone from the left table, use `LEFT JOIN`; the missing side comes back as `NULL`, which `COUNT(column)` doesn't count:

```sql
SELECT m.name, COUNT(l.id) AS loans
FROM members AS m
LEFT JOIN loans2 AS l ON l.member_id = m.id
GROUP BY m.id
ORDER BY loans DESC
```

name	loans
Ana Silva	2
Ben Okafor	1
Cat Moreau	0

Let SQL do it

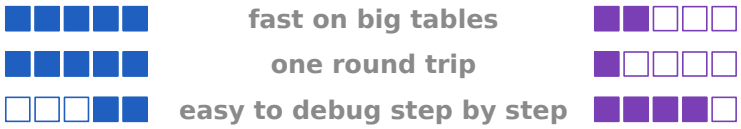
```
rows = conn.execute("""
    SELECT m.name, COUNT(l.id)
    FROM members m
    LEFT JOIN loans2 l
      ON l.member_id = m.id
    GROUP BY m.id
    ORDER BY m.name""").fetchall()
print(rows)
```

```
[('Ana Silva', 2), ('Ben Okafor', 1), ('Cat…
```

Fetch everything, loop in Python

```
counts = {}
for (mid, name) in conn.execute(
        "SELECT id, name FROM members"):
    counts[mid] = [name, 0]
for (mid,) in conn.execute(
        "SELECT member_id FROM loans2"):
    counts[mid][1] += 1
print(sorted(map(tuple, counts.values())))
```

```
[('Ana Silva', 2), ('Ben Okafor', 1), ('Cat…
```



RULE OF THUMB

Filter, join, count and sort **in SQL**, then fetch only what you need. Moving 100,000 rows into Python to count them is the classic beginner slowdown.

Indexes: why some queries are instant

To find rows matching a `WHERE`, SQLite either reads **every row** (a scan, O(n)) or jumps straight to them through an **index**, a sorted B-tree kept beside the table (O(log n)). `EXPLAIN QUERY PLAN` tells you which it chose:

```python
import sqlite3, time
big = sqlite3.connect(":memory:")
big.execute("CREATE TABLE visits (id INTEGER PRIMARY KEY, card TEXT, day TEXT)")
rows = ((f"C{i % 20000:05d}", f"2026-{i % 12 + 1:02d}-01")
        for i in range(200_000))
big.executemany("INSERT INTO visits (card, day) VALUES (?, ?)", rows)

def plan(q):
    return big.execute("EXPLAIN QUERY PLAN " + q, ("?",)).fetchone()[-1]

def timed(q, runs=200):
    t = time.perf_counter()
    for _ in range(runs):
        big.execute(q, ("C01234",)).fetchall()
    return (time.perf_counter() - t) / runs * 1000

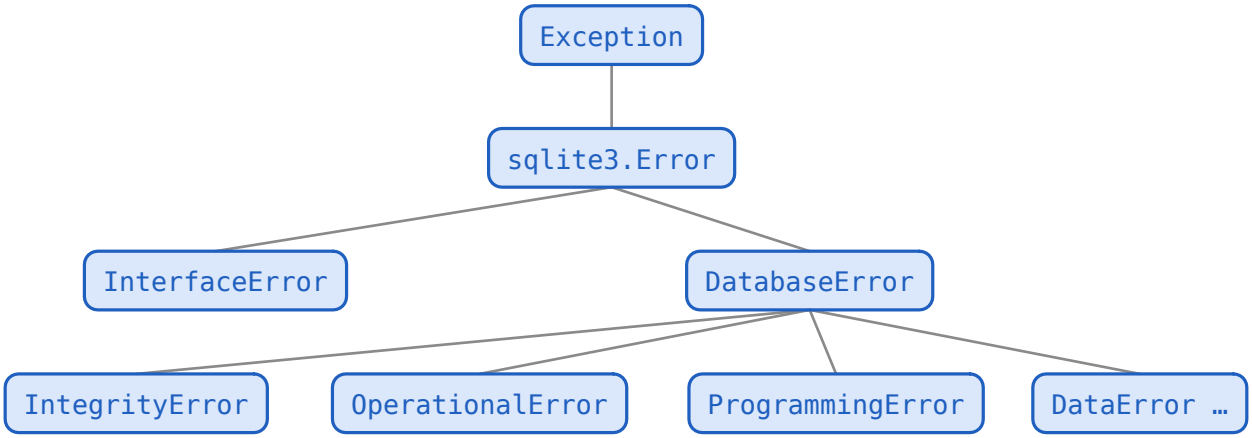
Q = "SELECT day FROM visits WHERE card = ?"
print(plan(Q), f"{timed(Q, 20):.2f} ms")
big.execute("CREATE INDEX idx_visits_card ON visits(card)")
print(plan(Q), f"{timed(Q):.3f} ms")
```

```
SCAN visits 5.52 ms
SEARCH visits USING INDEX idx_visits_card (card=?) 0.005 ms
```

Same query, same answer; the index turned a 200,000-row scan into a lookup that's hundreds to thousands of times faster. Primary keys and `UNIQUE` columns get an index automatically.

index a column when...	don't bother when...
you search or join on it often (`WHERE card = ?`, foreign keys)	the table is small (hundreds of rows)
you sort by it a lot (`ORDER BY due`)	the column has a handful of values (yes/no)
the table will grow large	the table is written far more than read (each index slows writes)

When it goes wrong: the error family



error	usually means	fix
IntegrityError	a constraint said no (UNIQUE, NOT NULL, CHECK, foreign key)	expected: catch it and tell the user
OperationalError	bad SQL, missing table, "database is locked", can't open file	a bug or an environment problem
ProgrammingError	wrong number of ? values, using a closed connection	a bug in your code

```sql
INSERT INTO members (name, email) VALUES ('Ana Two', 'ana@x.org')
```

```
sqlite3.IntegrityError: UNIQUE constraint failed: members.email
```

```sql
SELEC title FROM books
```

```
sqlite3.OperationalError: near "SELEC": syntax error
```

```sql
SELECT * FROM books WHERE id = ? AND year = ?
```

```
sqlite3.ProgrammingError: Incorrect number of bindings supplied. The current statemen…
```

RULE OF THUMB

Catch `IntegrityError` where you can say something useful ("that email is already registered"). Let `OperationalError` and `ProgrammingError` crash loudly during development: they're bugs, and hiding them makes them worse.

Organising database code

Scattering `conn.execute` through a program makes it hard to change. Keep all the SQL in one module with small functions that take a connection, and the rest of the program never sees SQL:

```python
import sqlite3
from dataclasses import dataclass

SCHEMA = """
CREATE TABLE IF NOT EXISTS members (
    id INTEGER PRIMARY KEY, name TEXT NOT NULL, email TEXT UNIQUE NOT NULL);
"""

@dataclass
class Member:
    id: int
    name: str
    email: str

def connect(path=":memory:"):
    conn = sqlite3.connect(path)
    conn.execute("PRAGMA foreign_keys = ON")
    conn.executescript(SCHEMA)
    return conn

def add_member(conn, name, email) -> int:
    with conn:
        return conn.execute("INSERT INTO members (name, email) VALUES (?, ?)",
                            (name, email)).lastrowid

def get_member(conn, member_id) -> Member | None:
    row = conn.execute("SELECT id, name, email FROM members WHERE id = ?",
                       (member_id,)).fetchone()
    return Member(*row) if row else None

conn = connect()
mid = add_member(conn, "Ana", "ana@x.org")
print(get_member(conn, mid), get_member(conn, 99))
```

```
Member(id=1, name='Ana', email='ana@x.org') None
```

habit	why
one `connect()` helper	every connection gets the same PRAGMAs and schema
functions take `conn` as a parameter	tests pass `":memory:"`; no global state
return dataclasses, not raw tuples	`m.email` beats `row[2]`; the rest of the code doesn't know it's SQL
schema version in `PRAGMA user_version`	a tiny migration system: `if version < 2: ALTER TABLE ...`
dates as ISO text `'2026-09-28'`	they sort correctly as strings; the old automatic date converters are deprecated since 3.12

The card, practice and stamp

LIBRARY CARD `sqlite3`

Job	A full SQL database in one file (or `":memory:"`), built into Python.
Core objects	`Connection` (from `sqlite3.connect`): `execute`, `executemany`, `commit`, `rollback`, `close`. `Cursor`: `fetchone`, `fetchall`, iterate, `lastrowid`. `sqlite3.Row` for rows by name.
Lifecycle	connect → `PRAGMA foreign_keys = ON` → execute with `?` → fetch → commit (or `with conn:`) → close.
Errors	`IntegrityError` (constraints), `OperationalError` (bad SQL, locked), `ProgrammingError` (wrong params, closed). All subclass `sqlite3.Error`.
Traps	**f-strings in SQL** (injection) · forgetting `commit` · `with conn` doesn't close · foreign keys off by default · `(x)` isn't a tuple.
Docs	docs.python.org/3/library/sqlite3.html · sqlite.org/lang.html (the SQL itself)

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 12.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 12.1` shows other valid solutions and their trade-offs.

`12.1` build from a spec A schema that refuses bad data
`12.2` fix the bug Close the injection hole
`12.3` build from a spec Top borrowers (JOIN + GROUP BY)
`12.4` build from a spec Lend and return, atomically
`12.5` measure and explain Make the slow queries use an index
`12.6` mini-project Mini-project: a library loans module

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Persistent structured data: `sqlite3` built in (one file, SQL, transactions). Other databases (PostgreSQL, MySQL) use the same DB-API shape: connect, cursor, execute, commit.

8. Errors

Errors from a library form a class tree (`sqlite3.Error` → `IntegrityError`...); catch the most specific one you can handle.

IN OTHER LANGUAGES

Every language has this exact shape. Java: JDBC `Connection`, `PreparedStatement` (the `?` placeholders), `commit()`. Go: `database/sql` with `db.Query(q, args...)`. The SQL and the injection rule are identical everywhere.

DEEP DIVE (your vault)

Vault notes: 11.1 Boundaries turn assumptions into failure modes · 19.12 Injection as a general grammar failure · 11.4 Context managers and deterministic release · 11.6 Buffers, flushing, and durability · 16.19 Relational joins and cardinality.

Chapter 13

Security primitives

pick the right tool; never invent your own

Security code fails silently. A broken sort gives wrong output you'll notice; a broken password check works perfectly until someone steals the database. So this chapter is less about clever code and more about **choosing the right primitive** and using it exactly as intended.

Start every security task with one question: **what am I protecting against?**

I need to...	use	module
detect accidental change (a corrupted download)	a hash	`hashlib`
prove a message came from me and wasn't edited	a MAC (keyed hash)	`hmac`
store passwords so a stolen database doesn't leak them	a slow, salted KDF	`hashlib.scrypt`
make an unguessable token, reset link or key	a CSPRNG	`secrets`
keep data secret (only key holders can read it)	authenticated encryption	**not in stdlib**: `cryptography`
talk to a server without eavesdroppers	TLS	`ssl` (via `urllib`, `http.client`)
put bytes in text (JSON, URLs, email)	an **encoding**, not security	`base64`
name things uniquely	an id, **not a secret**	`uuid`

RULE OF THUMB

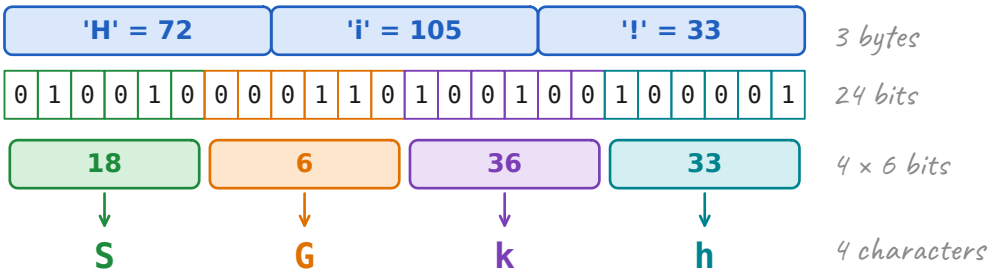
Never design your own crypto ("I'll XOR it with the key", "I'll hash it twice"). Even experts get it wrong. Combine well-known primitives in well-known ways, as this chapter does.

Everything here speaks bytes

Every primitive in this chapter takes and returns `bytes`, not `str`. Text becomes bytes with `.encode()` (UTF-8 by default); digests and keys come out as raw bytes, which you usually show as **hex** or **base64**:

```
>>> import hashlib, base64
>>> raw = hashlib.sha256("hello".encode()).digest()
>>> len(raw), raw[:6]
(32, b',\xf2M\xba_\xb0')
>>> raw.hex()[:24]
'2cf24dba5fb0a30e26e83b2a'
>>> base64.b64encode(raw)[:16]
b'LPJNul+wow4m6Dsq'
```

Base64 turns any bytes into 64 safe characters (A–Z a–z 0–9 + /), so binary data can travel in JSON, URLs or email. Every 3 bytes become 4 characters:




```
>>> base64.b64encode(b"Hi!"), base64.b64encode(b"Hi")
(b'SGkh', b'SGk=')
>>> base64.urlsafe_b64encode(b"\xfb\xff"), base64.b64encode(b"\xfb\xff")
(b'-_8=', b'+/8=')
>>> base64.b64decode("SGkh")
b'Hi!'
```

TRAP

Base64 is **not encryption**. Anyone can decode it in one line, as above. If you see a "secret" stored as base64, it's stored in plain text. Use `urlsafe_b64encode` (- and _ instead of + and /) for anything that goes in a URL; `=` is padding to a multiple of 4.

hashlib: a fingerprint for data

A cryptographic hash turns any amount of data into a fixed-size fingerprint. Four properties make it useful:

property	means	so you can
deterministic	same input → same digest, always, everywhere	compare files without sending them
fixed size	1 byte or 10 GB → 32 bytes (SHA-256)	store or index fingerprints cheaply
one-way	can't get the input back from the digest	publish a digest safely
avalanche	change 1 bit → about half the output bits flip	spot any edit at all

```
1 import hashlib
2
3 a = hashlib.sha256(b"transfer 100 to alice").hexdigest()
4 b = hashlib.sha256(b"transfer 900 to alice").hexdigest()
5 print(a[:32])
6 print(b[:32])
7 flipped = bin(int(a, 16) ^ int(b, 16)).count("1")
8 print(f"{flipped} of 256 bits differ")
```

← one character changed

← count differing bits

```
766e90601b2e01f7c7a633d6368a07a5
eb9535686c306af9c27094fd36a17273
116 of 256 bits differ
```

For big files, feed the hash in chunks with `.update()`, so memory use stays tiny however large the file is:

```
1 with open("big.bin", "wb") as out:
2     out.write(b"x" * 5_000_000)
3
4 def file_sha256(path, chunk=64 * 1024):
5     h = hashlib.sha256()
6     with open(path, "rb") as fh:
7         while block := fh.read(chunk):
8             h.update(block)
9     return h.hexdigest()
10
11 print(file_sha256("big.bin")[:16], len(hashlib.sha256().digest()), "bytes")
```

```
03a7bd518f3e4eca 32 bytes
```

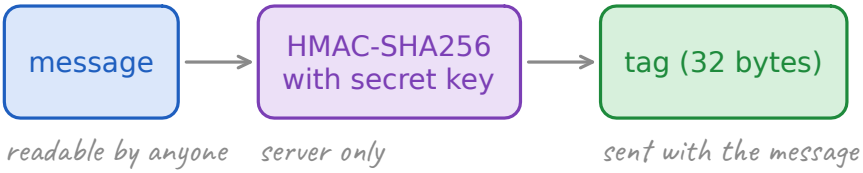
line 7: 64 KB at a time

line 8: same result as one big call

algorithm	status
`sha256`, `sha512`, `sha3_256`, `blake2b`	good for integrity and fingerprints
`md5`, `sha1`	**broken** for security (collisions can be manufactured); still fine as a non-security checksum
any of these for passwords	**wrong tool**: they're designed to be fast (next pages)

hmac: a hash only the key holder can make

A plain hash proves nothing about **who** made it: an attacker who edits a message just recomputes the hash. An **HMAC** mixes a secret key into the hash, so only someone with the key can produce a valid tag.




```python
import hmac, hashlib

KEY = b"k3y-from-an-env-variable"
msg = b"user=42;role=student"
tag = hmac.new(KEY, msg, hashlib.sha256).hexdigest()

def valid(msg, tag):
    expected = hmac.new(KEY, msg, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, tag)

print(valid(msg, tag))
print(valid(b"user=42;role=admin", tag))
print(valid(msg, hashlib.sha256(msg).hexdigest()))
```

← never ==

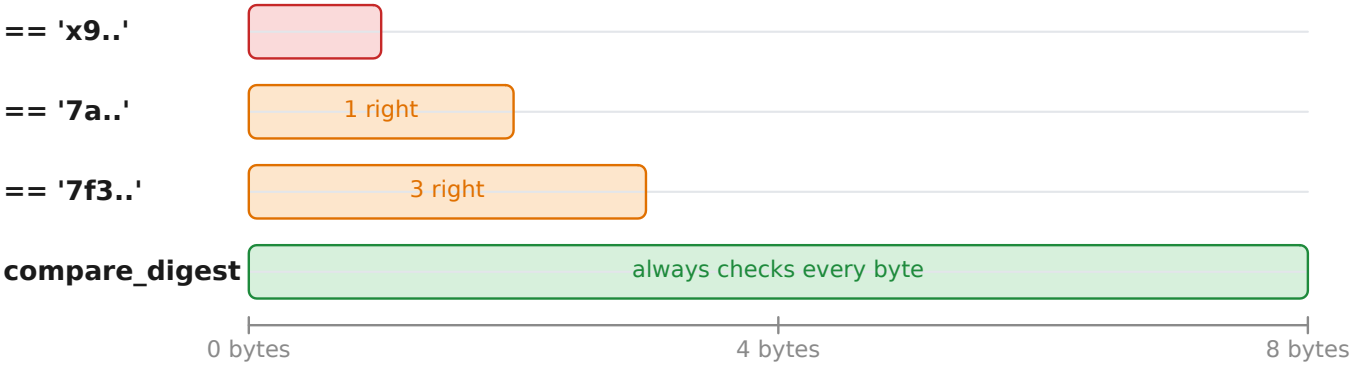
← edited: False

← no key: can't forge

```
True
False
False
```

Why compare_digest and not ==

`==` on strings stops at the **first differing character**. An attacker who can time thousands of requests sees that a guess with the right first character takes slightly longer, then works out the tag one character at a time. `compare_digest` always takes the same time. Say the real tag starts `7f3`:

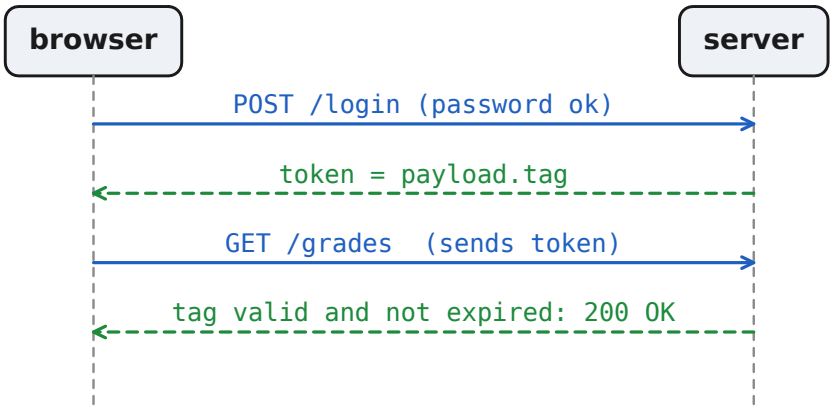


KEY IDEA

A MAC gives **integrity + authenticity**, not secrecy. The message is still readable; it just can't be changed without detection.

Building a signed token

Put the last two pages together and you have the idea behind login cookies, password-reset links and JWTs: **data + expiry, base64url-encoded, with an HMAC tag**. The server gives it out, the client hands it back, and the server can trust it without a database lookup.



```python
import base64, hashlib, hmac, json, time

KEY = b"server-secret"

def b64(b: bytes) -> str:
    return base64.urlsafe_b64encode(b).rstrip(b"=").decode()

def unb64(s: str) -> bytes:
    return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))

def issue(user, ttl, now):
    payload = b64(json.dumps({"u": user, "exp": now + ttl}).encode())
    tag = b64(hmac.digest(KEY, payload.encode(), "sha256"))
    return f"{payload}.{tag}"

def check(token, now):
    payload, _, tag = token.partition(".")
    good = b64(hmac.digest(KEY, payload.encode(), "sha256"))
    if not hmac.compare_digest(good, tag):
        raise ValueError("bad signature")
    data = json.loads(unb64(payload))
    if now >= data["exp"]:
        raise ValueError("expired")
    return data["u"]

t = issue("ana", ttl=3600, now=1_000_000)
print(t)
print(check(t, now=1_000_100))
forged = b64(b'{"u": "bob", "exp": 9999999}') + "." + t.split(".")[1]
for bad, when in [(forged, 1_000_100), (t, 1_004_000)]:
    try:
        check(bad, when)
    except ValueError as e:
        print("refused:", e)
print(unb64(t.split(".")[0]))
```

```
eyJ1IjogImFuYSIsICJleHAiOiAxMDAzNjAwfQ.ZRGnPrzceX2Gc9cMguMbjPTH2q-nlOKI-QIqckDMQx8
ana
refused: bad signature
refused: expired
b'{"u": "ana", "exp": 1003600}'
```

TRAP

The last line shows the payload is **readable by anyone**: signed is not encrypted. Never put secrets in a token's payload. And always check the signature **before** trusting anything inside, including the expiry.

secrets: randomness an attacker can't predict

The `random` module is a Mersenne Twister: fast, great for simulations and games, and completely **predictable**. Anyone who sees 624 outputs can compute every future one. Even easier, a seed reproduces the whole stream:

random: reproducible (bad for secrets)

```
import random
random.seed(2026)
print(random.randint(0, 10**6))
random.seed(2026)
print(random.randint(0, 10**6))
```

```
124906
124906
```

secrets: from the OS (os.urandom)

```
import secrets
a = secrets.randbelow(10**6)
b = secrets.randbelow(10**6)
print(a != b or "astronomically rare")
print(len(secrets.token_bytes(16)))
```

```
True
16
```

need	call	gives
a URL-safe token (reset link, session id)	`secrets.token_urlsafe(32)`	43 chars, 256 bits
a hex token (API key)	`secrets.token_hex(16)`	32 hex chars, 128 bits
raw key bytes	`secrets.token_bytes(32)`	32 bytes
pick from a list (password chars)	`secrets.choice(seq)`	one item
a number in [0, n)	`secrets.randbelow(n)`	an int

```
1  import secrets, string
2
3  def make_password(n=16):
4      alphabet = string.ascii_letters + string.digits + "!@#$%&*-_"
5      while True:
6          pw = "".join(secrets.choice(alphabet) for _ in range(n))
7          if (any(c.islower() for c in pw) and any(c.isupper() for c in pw)
8                  and any(c.isdigit() for c in pw)):
9              return pw
10
11 print(len(make_password()), len(secrets.token_urlsafe(32)))
```

```
16 43
```

line 6: secrets, not random

line 7: retry until every class appears

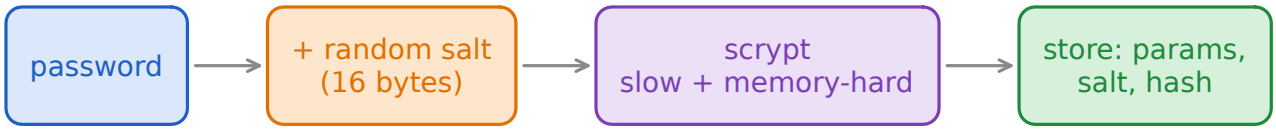
RULE OF THUMB

If a person or attacker must not guess it, use `secrets`. Tokens, keys, salts, one-time codes, temporary passwords. `random` is for everything else.

Storing passwords: salted and deliberately slow

Assume the database **will** be stolen one day. Then the question is how long it takes the attacker to recover passwords from what you stored. Each step below closes one attack:

you store	attacker does	verdict
the password	reads it	**never**
`sha256(password)`	looks it up in a precomputed table, or tries billions per second on a GPU	**still broken**
`sha256(salt + password)`	can't use tables (each user has a random salt), but still billions of guesses per second	**better, not enough**
`scrypt(password, salt, n, r, p)`	each guess costs ~40 ms and 16 MB of memory	**right**




```python
import hashlib, hmac, secrets, time

N, R_, P = 2**14, 8, 1

def hash_password(pw: str) -> str:
    salt = secrets.token_bytes(16)
    h = hashlib.scrypt(pw.encode(), salt=salt, n=N, r=R_, p=P)
    return f"scrypt${N}${R_}${P}${salt.hex()}${h.hex()}"

def verify_password(pw: str, stored: str) -> bool:
    _, n, r, p, salt, h = stored.split("$")
    got = hashlib.scrypt(pw.encode(), salt=bytes.fromhex(salt),
                         n=int(n), r=int(r), p=int(p))
    return hmac.compare_digest(got.hex(), h)

t = time.perf_counter()
s = hash_password("correct horse battery staple")
ms = (time.perf_counter() - t) * 1000
print(s[:44] + "...")
print(verify_password("correct horse battery staple", s),
      verify_password("Tr0ub4dor&3", s))
print(hash_password("same") != hash_password("same"), f"{ms:.0f} ms per hash")
```

```
scrypt$16384$8$1$22338bfc68b892b3bf0f5085d76...
True False
True 35 ms per hash
```

The parameters are stored **with** each hash, so you can raise the cost next year and old hashes still verify (re-hash them when the user next logs in). `hashlib.pbkdf2_hmac('sha256', pw, salt, 600_000)` is the other stdlib option; `argon2` (a pip package) is the current favourite.

TRAP

The slowness is the point. 40 ms is nothing for one login and ruinous for an attacker trying a billion guesses. Don't "optimise" it away, and do rate-limit login attempts too.

Encryption: the one thing the stdlib doesn't do

To keep data **secret** (only key holders can read it) you need encryption, and Python's standard library deliberately ships none: no AES. The standard answer is the `cryptography` package and its **Fernet** recipe, which bundles AES encryption with an HMAC so tampering is detected too ("authenticated encryption"). It's the first pip package in this course; Chapter 19 covers choosing and installing packages safely.

```
$ python3 -m venv .venv && source .venv/bin/activate
$ pip install cryptography
```

```python
from cryptography.fernet import Fernet, InvalidToken

key = Fernet.generate_key()          # 32 random bytes, base64: keep it SECRET
box = Fernet(key)

token = box.encrypt(b"exam room B12, 09:00")
print(token[:28], b"...")
print(box.decrypt(token))

tampered = token[:-2] + (b"A" if token[-2:-1] != b"A" else b"B") + token[-1:]
try:
    box.decrypt(tampered)
except InvalidToken:
    print("tampered: refused")
try:
    Fernet(Fernet.generate_key()).decrypt(token)
except InvalidToken:
    print("wrong key: refused")
```

```
b'gAAAAABqxDQyATNi38yoKziUdQA0' b'...'
b'exam room B12, 09:00'
tampered: refused
wrong key: refused
```

key management question	sensible default
where does the key live?	an environment variable or secrets file, **never in the source or git**
what if it leaks?	generate a new one and re-encrypt (`MultiFernet` helps rotate)
what if it's lost?	the data is gone. Back keys up separately from the data.

HONEST NOTE

Most programs don't need encryption code at all. Data in transit is TLS's job (next page); data at rest is often better handled by disk encryption. Reach for Fernet when one specific field must stay secret even from someone holding the database file.

ssl: TLS, the lock icon

TLS (what `https://` uses) encrypts the connection **and** proves you're talking to the real server, by checking its certificate against trusted authorities. `ssl.create_default_context()` turns all the checks on; `urllib.request` and `http.client` use it automatically:

```
import ssl
ctx = ssl.create_default_context()
print(ctx.verify_mode == ssl.CERT_REQUIRED, ctx.check_hostname)
print(ctx.minimum_version.name, ssl.OPENSSL_VERSION)
```

```
True True
TLSv1_2 OpenSSL 3.5.5 27 Jan 2026
```

TRAP

Tutorials "fix" certificate errors with `ssl._create_unverified_context()`, `CERT_NONE` or `verify=False`. That keeps the encryption but **drops the identity check**, so anyone in the middle (café Wi-Fi) can impersonate the server. Fix the real cause (wrong clock, missing CA bundle, wrong hostname) instead.

uuid: unique, but not secret

```
>>> import uuid
>>> uuid.uuid4().version, len(str(uuid.uuid4()))
(4, 36)
>>> a, b = uuid.uuid7(), uuid.uuid7()
>>> a < b
True
```

`uuid4` is random, `uuid7` (new in 3.14) starts with a timestamp, so ids sort by creation time, which databases like. Use them as **names**. For anything that grants access, use `secrets`.

Checklist, card, practice

- Bytes in, bytes out: `.encode()` text going in; `.hex()` or base64 to display.
- Integrity only: `hashlib.sha256`. Integrity + who: `hmac` with a secret key.
- Compare secrets with `hmac.compare_digest`, never `==`.
- Tokens, salts, keys: `secrets`. Never `random`.
- Passwords: `hashlib.scrypt` with a per-user salt; store the parameters.
- Secrecy: `cryptography`'s Fernet; the key lives outside the code.
- Never disable certificate checks. Signed ≠ encrypted. Base64 ≠ encrypted.

LIBRARY CARD **`hashlib · hmac · secrets`**

Job	Fingerprints (`hashlib`), keyed tags (`hmac`), unguessable randomness (`secrets`).
Core objects	`hashlib.sha256(b)` → hash object: `.update(b)`, `.digest()`, `.hexdigest()`. `hmac.new(key, msg, 'sha256')`, `hmac.digest(...)`, `hmac.compare_digest(a, b)`. `secrets.token_urlsafe(n)`, `choice`, `randbelow`.
Lifecycle	new → update… → digest. No closing. Keys come from outside the code.
Errors	`TypeError` for `str` instead of `bytes`; `ValueError` for unknown algorithms / bad scrypt parameters.
Traps	== on tags (timing) · md5/sha1 · fast hashes for passwords · `random` for tokens · thinking base64 or signing hides data.
Docs	docs.python.org/3/library/{hashlib,hmac,secrets}.html · OWASP Password Storage Cheat Sheet

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 13.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 13.1` shows other valid solutions and their trade-offs.

13.1 build from a spec Fingerprint a file in chunks
13.2 build from a spec Sign and verify messages with HMAC
13.3 build from a spec Store passwords properly
13.4 build from a spec A password generator you'd trust
13.5 learn an unseen module URL-safe base64 without padding
13.6 mini-project Mini-project: expiring signed tokens

PASSPORT STAMP ask the same question of every new language (Appendix B)

12. Stdlib and tooling

Security primitives ship in the stdlib (`hashlib`, `hmac`, `secrets`, `ssl`) except symmetric encryption, which comes from the `cryptography` package.

6. Data structures

`bytes` vs `str` is a hard boundary: crypto, sockets and files in binary mode all speak bytes.

DEEP DIVE (your vault)

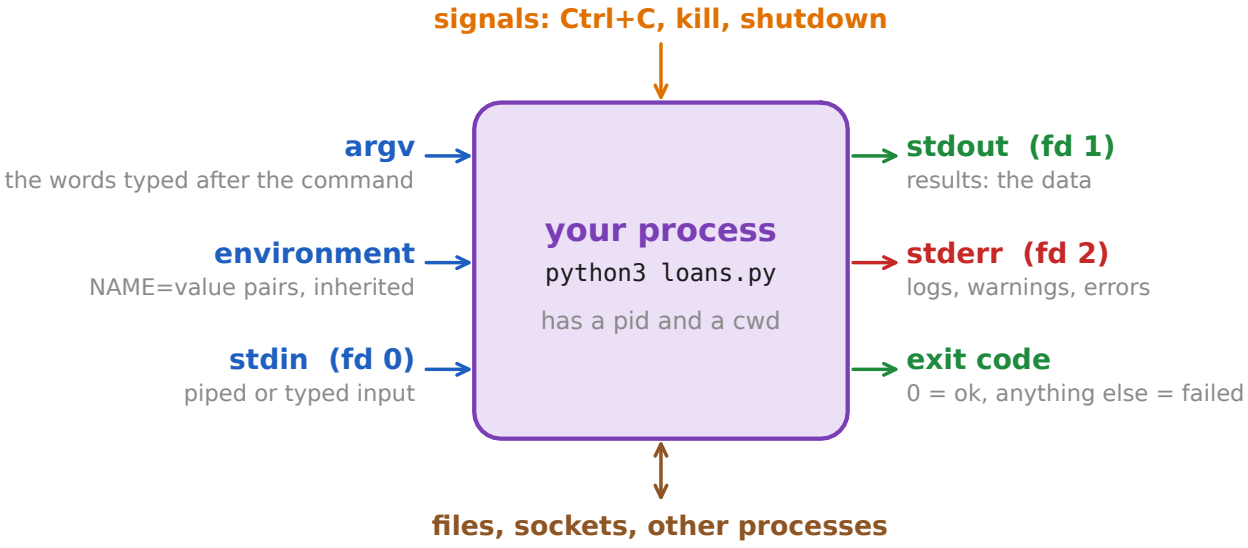
Vault notes: 19.16 Cryptographic objectives and the rule against invention · 19.17 Hash functions · 19.18 Message authentication codes · 19.19 Symmetric authenticated encryption · 19.21 Password storage · 19.22 Cryptographic randomness · 19.23 Key management · 5.12 Bytes and the text boundary.

Chapter 14

The OS and the command line

every program talks to the OS the same way

To the operating system, every program in every language is a **process** with the same few connections. Learn them once and you can drive any program from Python, and write Python tools that behave like real Unix commands:



A shell pipeline like `grep ERROR app.log | sort | uniq -c > counts.txt` works only because every program in it follows these conventions: read stdin, write results to stdout, complain on stderr, report success with the exit code. This chapter makes your Python programs good citizens of that world.

sys and os: the process from the inside

`sys` is about the **interpreter** (its arguments, streams, version, exit); `os` is about the **operating system** (environment, files, process ids). Running a child Python is the easiest way to see argv and exit codes for real:

```
1 import subprocess, sys
2
3 child = "import sys; print(sys.argv[1:]); sys.exit(3)"
4 r = subprocess.run([sys.executable, "-c", child, "a b", "c"], capture_output=True, text=True)
5 print(r.returncode, r.stdout.strip())
6
7 r = subprocess.run([sys.executable, "-c", "raise SystemExit('bad input')"],
8                     capture_output=True, text=True)
9 print(r.returncode, repr(r.stdout), r.stderr.strip())
```

```
3 ['a b', 'c']
1 '' bad input
```

line 4: "a b" stays one argument

line 7: SystemExit(str): message to stderr, exit code 1

exit code	means	who uses it
`0`	success	everything: `&&` in the shell, CI, cron, systemd
`1`	general failure	`sys.exit("message")`, an uncaught exception
`2`	you used it wrong (bad arguments)	argparse, grep, most Unix tools
`126` / `127`	found but not executable / not found	the shell
`128 + n`	killed by signal n (`130` = Ctrl+C, `137` = SIGKILL)	the shell

```
1 import os, subprocess, sys
2 from pathlib import Path
3
4 print(sys.platform, sys.version_info[:2], os.path.basename(sys.executable))
5 print(os.getcwd() == str(Path.cwd()), os.getpid() > 1, os.cpu_count() >= 1)
6 print(os.sep, repr(os.pathsep), sys.stdout.isatty())
7 print(os.getenv("LOANS_DB", "loans.db"))           # environment variable, with a default
8
9 env = {**os.environ, "LOANS_DB": "test.db"}       # the child gets its own copy
10 r = subprocess.run([sys.executable, "-c", "import os; print(os.environ['LOANS_DB'])"],
11                     env=env, capture_output=True, text=True)
12 print(r.stdout.strip(), "LOANS_DB" in os.environ)
```

```
linux (3, 14) python3
True True True
/ ':' False
loans.db
test.db False
```

line 6: False: output is being captured

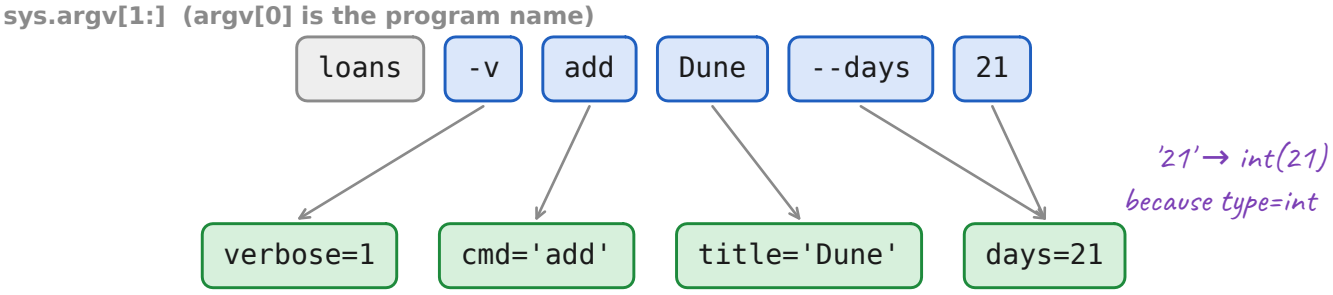
line 12: changes never flow back to the parent

TRAP

Environment variables are inherited by **every** child process, appear in crash reports, and can be read from `/proc` by the same user. They're the standard place for configuration and tolerable for secrets in containers, but never log `os.environ`, and don't pass secrets as command-line arguments: `ps` shows argv to every user on the machine.

argparse: a real command-line interface

`argparse` turns the list of words in `sys.argv` into a tidy object, converts types, checks choices, and writes `--help` and error messages for you:




```
1  import argparse
2
3  def make_parser():
4      p = argparse.ArgumentParser(prog="loans", description="Track library loans.")
5      p.add_argument("-v", "--verbose", action="count", default=0, help="-vv for more")
6      sub = p.add_subparsers(dest="cmd", required=True)
7      add = sub.add_parser("add", help="lend a book")
8      add.add_argument("title")
9      add.add_argument("--days", type=int, default=14, help="default: %(default)s")
10     ls = sub.add_parser("list", help="show loans")
11     ls.add_argument("--overdue", action="store_true")
12     ls.add_argument("--sort", choices=["title", "due"], default="due")
13     return p
14
15 print(make_parser().parse_args(["-v", "add", "Dune", "--days", "21"]))
16 print(make_parser().parse_args(["list", "--overdue"]))
```

```
Namespace(verbose=1, cmd='add', title='Dune', days=21)
Namespace(verbose=0, cmd='list', overdue=True, sort='due')
```

line 5: -v, -vv, -vvv

line 6: subcommands, like git commit

line 8: positional: required

line 9: option: has a default

line 11: a flag: True if present

```python
print(make_parser().format_help())
```

```
usage: loans [-h] [-v] {add,list} ...

Track library loans.

positional arguments:
  {add,list}
    add          lend a book
    list         show loans

options:
  -h, --help     show this help message and exit
  -v, --verbose  -vv for more
```

main(argv=None): the pattern that makes a CLI testable

```python
def main(argv=None):
    args = make_parser().parse_args(argv)        # None means: use sys.argv[1:]
    if args.cmd == "add":
        print(f"lent {args.title} for {args.days} days")
    return 0

print(main(["add", "Emma"]))                     # a test calls main() with a list
# in loans.py:   if __name__ == "__main__": sys.exit(main())
```

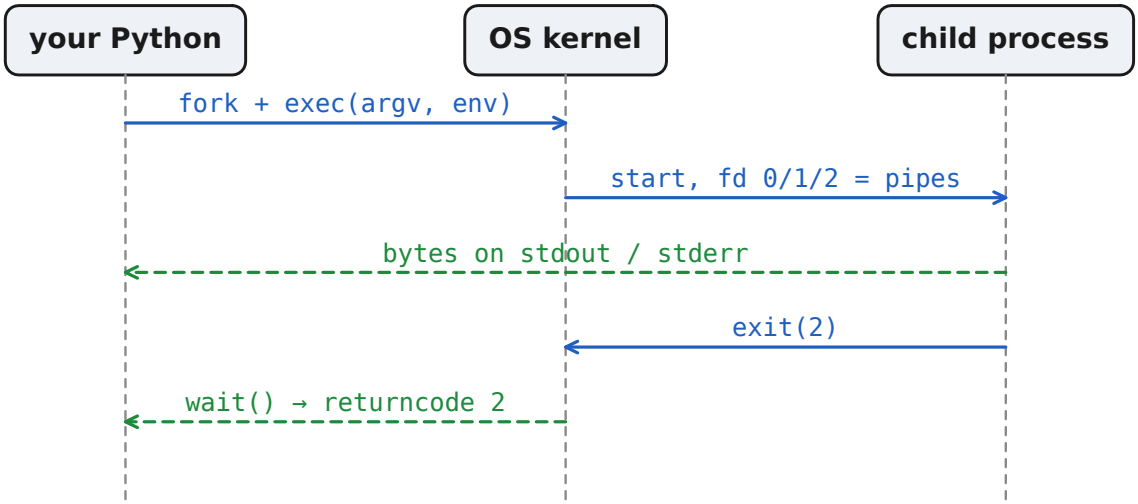
```
lent Emma for 14 days
0
```

the same program from the shell (usage errors go to stderr, exit code 2)

```
$ python3 loans.py add Dune --days 21
lent Dune for 21 days
$ echo $?
0
$ python3 loans.py list --sort author
usage: loans list [-h] [--overdue] [--sort {title,due}]
loans list: error: argument --sort: invalid choice: 'author' (choose from 'title', 'due')
$ echo $?
2
```

subprocess: running other programs

`subprocess.run` starts a program, waits for it, and hands back its exit code and output. Under the hood the OS creates a child process wired to pipes:




```
1  import subprocess, sys
2
3  try:
4      subprocess.run(["ls", "/no/such/dir"], capture_output=True, text=True, check=True)
5  except subprocess.CalledProcessError as e:
6      print("failed with", e.returncode, "|", e.stderr.strip())
7
8  try:
9      subprocess.run([sys.executable, "-c", "import time; time.sleep(5)"], timeout=0.2)
10 except subprocess.TimeoutExpired as e:
11     print("gave up after", e.timeout, "s (the child is killed)")
```

```
failed with 2 | ls: cannot access '/no/such/dir': No such file or directory
gave up after 0.2 s (the child is killed)
```

line 4: check=True: non-zero exit raises

line 9: always set a timeout for things that can hang

Never build a shell command from data

With `shell=True` your string goes to `/bin/sh`, which treats `;`, `|`, `$( )` and spaces as syntax. If any part came from a user or a filename, they can run anything. Here the "filename" smuggles in a second command (a harmless `echo`, but it could be `rm -rf ~`):

shell=True + f-string: injected

```
import subprocess
name = "notes.txt; echo HACKED"
r = subprocess.run(f"echo {name}", shell=True,
                   capture_output=True, text=True)
print(r.stdout)
```

```
notes.txt
HACKED
```

a list of arguments: safe

```
import subprocess
name = "notes.txt; echo HACKED"
r = subprocess.run(["echo", name],
                   capture_output=True, text=True)
print(r.stdout)
```

```
notes.txt; echo HACKED
```

RULE OF THUMB

Pass a **list**: each item reaches the program as exactly one argument and no shell ever parses it. If a value could start with `-`, put `"--"` before it so the program can't mistake it for an option. This is the same bug as SQL injection (Ch 12): data mixed into code.

Signals: being asked to stop

A signal is a tiny message from the OS: "stop", "the terminal closed", "reload". Ctrl+C sends `SIGINT`, which Python turns into `KeyboardInterrupt`. `kill`, `systemctl stop` and `docker stop` send `SIGTERM`, which by default kills Python on the spot: no `finally`, no flush. Handle it to shut down cleanly:

```
1  import subprocess, sys, textwrap
2
3  child = textwrap.dedent('''
4      import signal, sys, time
5      def stop(signum, frame):
6          print("got", signal.Signals(signum).name, "- saving and exiting")
7          sys.exit(0)                          # raises SystemExit: finally blocks run
8      signal.signal(signal.SIGTERM, stop)
9      print("ready", flush=True)
10     time.sleep(30)
11 ''')
12 p = subprocess.Popen([sys.executable, "-c", child], stdout=subprocess.PIPE, text=True)
13 print(p.stdout.readline().strip())
14 p.terminate()                                 # sends SIGTERM
15 out, _ = p.communicate(timeout=5)
16 print(out.strip(), "| exit code", p.returncode)
```

```
ready
got SIGTERM - saving and exiting | exit code 0
```

line 8: install the handler

line 12: Popen: start without waiting

signal	sent by	default	Python
SIGINT (2)	Ctrl+C	stop	raises KeyboardInterrupt
SIGTERM (15)	kill, systemd, Docker, Kubernetes	stop at once	handle it: clean up, exit
SIGKILL (9)	kill -9, the OOM killer	stop at once	**can't be caught**
SIGHUP (1)	terminal closed; daemons: "reload config"	stop	handle if you're a daemon
SIGPIPE (13)	reader of your pipe went away (| head)	stop	ignored: you get BrokenPipeError

HONEST NOTE

Windows has no SIGTERM in this sense: `terminate()` there kills the process outright. Only Ctrl+C (SIGINT) and SIGBREAK are portable. Signal handlers run only in the main thread, between bytecodes, so keep them tiny: set a flag or raise.

Logging, and the good-citizen checklist

A command-line tool's stdout is often piped into another program, so it must contain only results. Progress and diagnostics go to stderr through `logging`, with `-v` turning up the detail:

```
1  import io, logging
2
3  def setup_logging(verbose, stream):                    # pass sys.stderr in real life
4      log = logging.getLogger("loans")
5      log.setLevel([logging.WARNING, logging.INFO, logging.DEBUG][min(verbose, 2)])
6      handler = logging.StreamHandler(stream)
7      handler.setFormatter(logging.Formatter("%(levelname)s %(message)s"))
8      log.handlers[:] = [handler]
9      log.propagate = False
10     return log
11
12 for v in (0, 2):
13     buf = io.StringIO()
14     log = setup_logging(v, buf)
15     log.debug("opening loans.db"); log.info("3 loans found"); log.warning("1 overdue")
16     print(f"-v x{v}:", buf.getvalue().splitlines())
```

```
-v x0: ['WARNING 1 overdue']
-v x2: ['DEBUG opening loans.db', 'INFO 3 loans found', 'WARNING 1 overdue']
```

line 5: -v count picks the level

a good command-line program...	so that...
writes results to stdout, messages to stderr	`prog > out.txt` and pipes get clean data
exits 0 on success, non-zero on failure (2 for usage)	&&, scripts, CI and cron can react
reads stdin when given no file, or `-`	it works in the middle of a pipeline

has `--help` and clear error messages	nobody has to read the source (argparse does it)
takes config as flags > env vars > file > defaults	the same build runs anywhere (practice 14.4)
skips colour and progress bars if `not sys.stdout.isatty()`	logs and pipes stay readable
handles SIGTERM and Ctrl+C: cleans up, then exits	containers and services stop it safely

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 14.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 14.1` shows other valid solutions and their trade-offs.

`14.1` build from a spec An argparse CLI from a spec
`14.2` fix the bug Close the shell-injection hole
`14.3` build from a spec A grep-like filter with proper exit codes
`14.4` build from a spec Config from flags, environment and defaults
`14.5` learn an unseen module Read a .env file with shlex
`14.6` mini-project A tiny task runner

PASSPORT STAMP ask the same question of every new language (Appendix B)

1. How does code run?

To the OS, a Python program is a process: argv and environment in, stdin/stdout/stderr, an exit code out, signals from outside. `python3 file.py` runs the interpreter, which runs your file.

10. I/O

Text streams `sys.stdin`/`stdout`/`stderr`; results to stdout, messages to stderr.

12. Stdlib and tooling

`sys`, `os`, `argparse`, `subprocess` (list args, `check`, `timeout`), `signal`, `logging`, `shlex`.

IN OTHER LANGUAGES

C: `int main(int argc, char **argv)`, the return value is the exit code. Java: `main(String[] args)`, `System.exit`, `ProcessBuilder`. Go: `os.Args`, `flag`, `os/exec`. Rust: `std::env::args`, `std::process::Command`, and the `clap` crate as its argparse. The process model underneath is identical.

DEEP DIVE (your vault)

Vault notes: 20.2 Programs as managed processes · 20.3 Exit status and supervision · 20.4 Configuration · 20.5 Filesystem and runtime conventions · 19.11 Command injection and subprocess boundaries · 19.15 Secrets · 12.12 Logging and diagnostics.

Chapter 15

Networking

a network connection is just a file that someone else writes to

Networking is built in **layers**. Each one uses the layer below and hides its mess from the layer above: IP loses and reorders packets, TCP turns them into a reliable stream, HTTP gives the stream a request/response shape. Python has a module at each level:

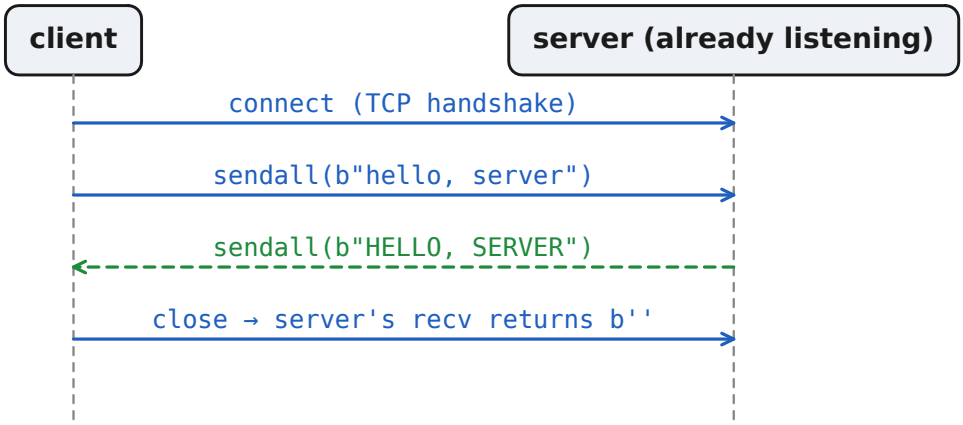
layer	what it gives the layer above	Python
application	HTTP, SMTP, your JSON	`urllib, http.client/server, smtplib`
security	TLS: encryption + identity	`ssl`
transport	TCP: reliable byte stream, ports	`socket (SOCK_STREAM)`
network	IP: addresses, packets, routing	`socket (AF_INET / AF_INET6)`
link	Ethernet, Wi-Fi	`the OS and the network card`

Every example in this chapter really runs: it starts a server on your own machine (`127.0.0.1`, "localhost") in a background thread and talks to it. No internet needed, and the output shown is what happened.

> **KEY IDEA**
> An address is a pair: **(host, port)**. The host finds the machine; the port (0-65535) finds the program on it. Ports below 1024 are reserved for well-known services (80 HTTP, 443 HTTPS, 22 SSH, 25 SMTP) and need admin rights to listen on.

TCP sockets: a phone line for bytes

A server **listens** on a port; a client **connects**; then both sides have a socket they can `sendall` bytes into and `recv` bytes out of, until one side closes:




```python
import socket, threading

def echo_server(listener):
    conn, addr = listener.accept()                  # wait for one client
    with conn:
        while data := conn.recv(1024):              # b"" means the client hung up
            conn.sendall(data.upper())

listener = socket.create_server(("127.0.0.1", 0))   # port 0: the OS picks a free one
port = listener.getsockname()[1]
threading.Thread(target=echo_server, args=(listener,), daemon=True).start()

with socket.create_connection(("127.0.0.1", port), timeout=5) as c:
    c.sendall(b"hello, server")
    print(c.recv(1024))
    print(c.getsockname()[0], "->", c.getpeername()[0], 1023 < port < 65536)
listener.close()
```

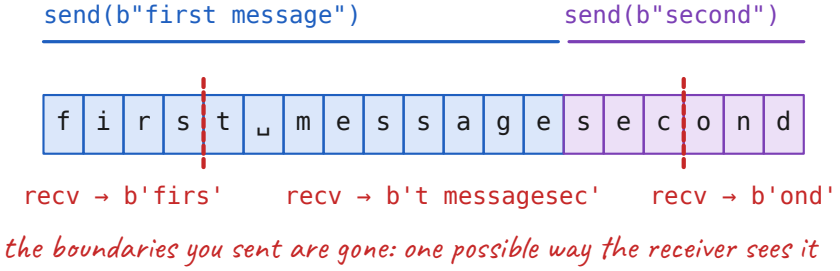
```
b'HELLO, SERVER'
127.0.0.1 -> 127.0.0.1 True
```

line 4: accept blocks

line 13: always a timeout

The big surprise: TCP has no messages

TCP promises the bytes arrive complete and in order, nothing more. Two `send`s can arrive as one `recv`, and one `send` can arrive as three:



the boundaries you sent are gone: one possible way the receiver sees it

```python
a, b = socket.socketpair()              # two connected sockets, handy for experiments
a.sendall(b"first message")
a.sendall(b"second")
print(b.recv(1024))                     # both sends, glued together

reader = b.makefile("rb")               # a file view of the socket
a.sendall(b"first message\n" + b"second\n")
print(reader.readline(), reader.readline())
```

```
b'first messagesecond'
b'first message\n' b'second\n'
```

line 7: framing: newline ends a message

RULE OF THUMB

Every protocol on TCP needs **framing**: a way to know where a message ends. The two classics are a delimiter (a newline, as SMTP, Redis and this chapter's practice 15.6 use) and a length prefix (send `struct.pack("!I", len(data))` first, as HTTP's Content-Length and practice 15.1 do). `recv(n)` returns **up to** n bytes, so loop until you have them all.

HTTP: text over TCP

HTTP is a framing convention on top of TCP: some lines of text, a blank line, then an optional body whose length is in a header. First, a small JSON API server using only the standard library:

```python
import json, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

BOOKS = {1: {"id": 1, "title": "Dune"}, 2: {"id": 2, "title": "Emma"}}

class Api(BaseHTTPRequestHandler):
    def send_json(self, status, obj):
        body = json.dumps(obj).encode()
        self.send_response(status)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()                          # the blank line
        self.wfile.write(body)

    def do_GET(self):                               # one method per HTTP verb
        if self.path == "/books":
            return self.send_json(200, list(BOOKS.values()))
        book_id = self.path.removeprefix("/books/")
        if book_id.isdigit() and int(book_id) in BOOKS:
            return self.send_json(200, BOOKS[int(book_id)])
        self.send_json(404, {"error": f"no such book: {self.path}"})

    def do_POST(self):
        data = json.loads(self.rfile.read(int(self.headers["Content-Length"])))
        book = {"id": max(BOOKS) + 1, "title": data["title"]}
        BOOKS[book["id"]] = book
        self.send_json(201, book)

    def log_message(self, *args):                   # keep the server quiet
        pass

server = ThreadingHTTPServer(("127.0.0.1", 0), Api)
threading.Thread(target=server.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{server.server_port}"
```

line 24: read exactly the body

line 32: a thread per request

This is what travels over the TCP connection. Each part has a job:

client sends

```
POST /books HTTP/1.1                    ← request line: method, path, version
Host: lib.example.org                   ← headers: Name: value
Content-Type: application/json
Content-Length: 17
↵                                       ← blank line (\r\n): headers end
{"title": "Ubik"}                       ← body: exactly Content-Length bytes
```

server answers

```
HTTP/1.1 201 Created                    ← status line: version, code, reason
Content-Type: application/json
Location: /books/3
↵
{"id": 3, "title": "Ubik"}
```

Now speak HTTP to our server by hand over a raw socket (dropping the `Date` header, which changes every run):

```
with socket.create_connection(("127.0.0.1", server.server_port), timeout=5) as c:
    c.sendall(b"GET /books/1 HTTP/1.0\r\nHost: localhost\r\n\r\n")
    reply = b""
    while chunk := c.recv(4096):                    # HTTP/1.0: server closes when done
        reply += chunk
print("\n".join(l for l in reply.decode().splitlines() if not l.startswith("Date")))
```

```
HTTP/1.0 200 OK
Server: BaseHTTP/0.6 Python/3.14.6
Content-Type: application/json
Content-Length: 26

{"id": 1, "title": "Dune"}
```

Calling a JSON API with urllib

`urllib.request` does the HTTP for you: building the request, reading the status and headers, following redirects. Errors with a status of 400 or more raise `HTTPError`, which still carries the response body:

```
import json, urllib.error, urllib.request

def get_json(url):
    with urllib.request.urlopen(url, timeout=5) as resp:
        return resp.status, json.load(resp)

print(get_json(base + "/books/2"))

req = urllib.request.Request(base + "/books", method="POST",
                             data=json.dumps({"title": "Ubik"}).encode(),
                             headers={"Content-Type": "application/json"})
with urllib.request.urlopen(req, timeout=5) as resp:
    print(resp.status, resp.headers["Content-Type"], json.load(resp))

try:
    get_json(base + "/books/99")
except urllib.error.HTTPError as e:
    print(e.code, e.reason, json.load(e))
server.shutdown()                                   # stop the background server
```

```
(200, {'id': 2, 'title': 'Emma'})
201 application/json {'id': 3, 'title': 'Ubik'}
404 Not Found {'error': 'no such book: /books/99'}
```

line 4: no timeout = may hang forever

line 18: the error body is JSON too

status	means	common ones
2xx	it worked	`200 OK`, `201 Created` (+ `Location`), `204 No Content`
3xx	look elsewhere	`301 Moved Permanently`, `304 Not Modified` (use your cache)
4xx	**you** got it wrong: don't retry as is	`400`, `401` who are you?, `403` not allowed, `404`, `409` conflict, `429` slow down
5xx	**the server** broke: retry later	`500`, `502 Bad Gateway`, `503 Unavailable`

method	does	safe?	repeatable (idempotent)?
`GET`	read	**yes**	**yes**
`POST`	create / do something	no	**no**: twice makes two
`PUT`	replace at this URL	no	**yes**
`DELETE`	remove	no	**yes**

WHY

Idempotency decides what is safe to **retry** after a timeout: you don't know whether the request arrived. Retrying a GET or PUT is harmless; retrying a POST might lend the book twice, which is why payment APIs make you send an idempotency key.

URLs, TLS, and email

```python
from urllib.parse import parse_qs, quote, urlencode, urljoin, urlsplit

u = urlsplit("https://lib.example.org:8443/search?q=dune&page=2#results")
print(u.scheme, u.hostname, u.port, u.path, u.query, u.fragment)
print(parse_qs(u.query))
print(urlencode({"q": "science fiction", "lang": "en"}), quote("a/b c&d", safe=""))
print(urljoin("https://lib.example.org/books/", "../authors/7"))
```

```
https lib.example.org 8443 /search q=dune&page=2 results
{'q': ['dune'], 'page': ['2']}
q=science+fiction&lang=en a%2Fb%20c%26d
https://lib.example.org/authors/7
```

line 6: never glue URLs with +

TLS (the S in HTTPS) encrypts the stream and proves the server's identity with a certificate. `urllib` uses it automatically for `https://`. The one rule is to leave verification on:

```python
import ssl

ctx = ssl.create_default_context()          # trusted CAs, hostname checks, modern TLS
print(ctx.verify_mode == ssl.CERT_REQUIRED, ctx.check_hostname, ctx.minimum_version)
```

```
True True 771
```

wrapping a socket yourself (not run here: needs the internet)

```python
with socket.create_connection(("example.org", 443), timeout=10) as raw:
    with ctx.wrap_socket(raw, server_hostname="example.org") as tls:
        print(tls.version(), tls.getpeercert()["subject"])
```

TRAP

`ssl._create_unverified_context()`, `verify=False` in requests, or `CERT_NONE` make the error go away by letting anyone in the middle read and change the traffic. Fix the certificate (or install the right CA) instead. And if your server fetches URLs that users supply, check where they point first: `http://169.254.169.254/` is how cloud credentials get stolen (SSRF).

Email: build with email, send with smtplib

```python
from email.message import EmailMessage

msg = EmailMessage()
msg["From"], msg["To"] = "library@example.org", "ana@example.org"
msg["Subject"] = "Your loan is due"
msg.set_content("Dune is due back on 12 October.\n")
print(msg.as_string())
```

```
From: library@example.org
To: ana@example.org
Subject: Your loan is due
Content-Type: text/plain; charset="utf-8"
Content-Transfer-Encoding: 7bit
MIME-Version: 1.0

Dune is due back on 12 October.
```

sending (not run here: needs a mail server)

```python
import smtplib

with smtplib.SMTP("smtp.example.org", 587, timeout=10) as s:
    s.starttls(context=ssl.create_default_context())    # upgrade to TLS first
    s.login(user, password)                              # from the environment (Ch 14)
    s.send_message(msg)
```

need	use
a quick GET/POST from a script	`urllib.request` (stdlib)
many requests to one host, low-level control	`http.client` (keeps the connection open)
a real application: sessions, retries, nicer API	`requests` or `httpx` from pip (Ch 19)
serve HTTP in production	a framework (Flask, Django, FastAPI) behind a real server, not `http.server`

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 15.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 15.1` shows other valid solutions and their trade-offs.

15.1 build from a spec Length-prefixed messages over TCP
15.2 fix the bug Parse a raw HTTP request properly
15.3 learn an unseen module URL surgery with urllib.parse
15.4 build from a spec A JSON API client that handles failure
15.5 build from a spec Serve a small JSON API
15.6 mini-project A line-based key-value server

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Network I/O is bytes over sockets: `socket` (TCP byte stream, needs framing), `ssl` for TLS, `urllib.request` / `http.client` for HTTP, `http.server` for tiny servers.

12. Stdlib and tooling

`socket`, `socketserver`, `ssl`, `urllib.request`, `urllib.parse`, `http.client`, `http.server`, `http.HTTPStatus`, `json`, `email`, `smtplib`.

IN OTHER LANGUAGES

The socket API is the same everywhere because it's the OS's (BSD sockets, 1983): C `socket()/bind()/listen()/accept()`, Java `ServerSocket`, Go `net.Listen`, Rust `std::net::TcpListener`. HTTP clients differ in style, not substance: Java `HttpClient`, Go `net/http`, JS `fetch`.

DEEP DIVE (your vault)

Vault notes: 18.2 Networks and layered agreements · 18.3 Addresses, ports, and names · 18.4 TCP and the meaning of a byte stream · 18.5 TLS - authenticated protected transport · 18.7 HTTP messages · 18.8 Methods and method properties · 18.9 Status codes · 18.13 From a request to a Python callable · 18.26 Server-side request forgery.

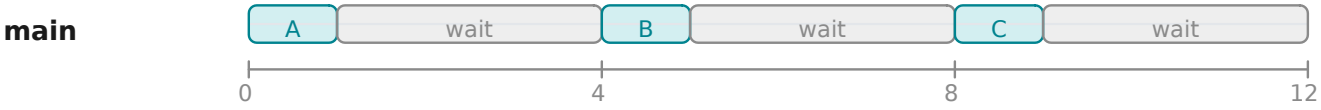
Chapter 16

Concurrency

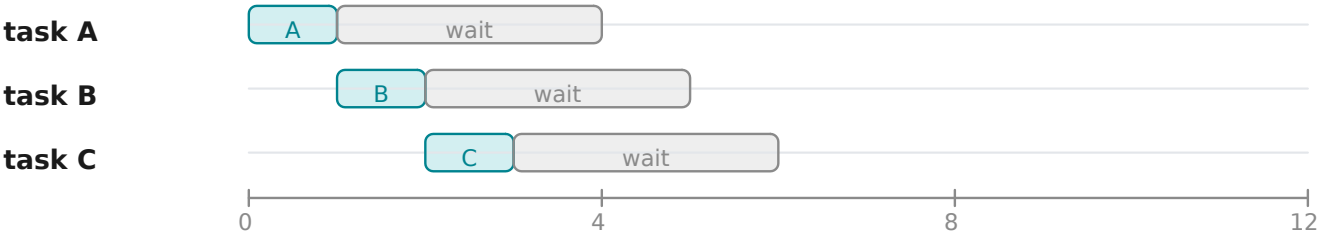
doing several things at once, and the bugs that come with it

Most programs spend their time **waiting**: for the network, the disk, a database. **Concurrency** means making progress on several tasks by working on one while the others wait. **Parallelism** means literally computing on several CPU cores at the same instant. Three jobs, each 1 unit of work plus 3 units of waiting for a server:

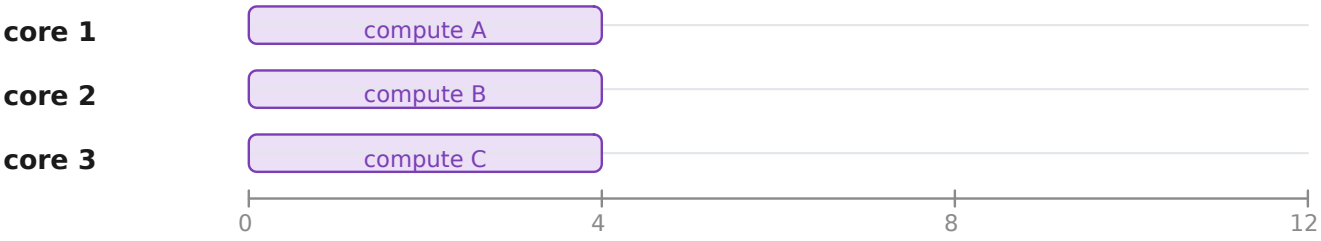
One at a time: 12 units



Concurrent (threads or asyncio, one core is enough): the waits overlap, 6 units



Parallel (processes, several cores): pure computing finishes 3× sooner



your program mostly...	use	why
waits on I/O (network, disk, DB)	threads, or `asyncio`	waiting releases the CPU for others
waits on thousands of connections	`asyncio`	a task costs KB; a thread costs MB
computes (CPU-bound pure Python)	processes (`ProcessPoolExecutor`)	each process has its own GIL
crunches arrays	NumPy (Ch 18)	its C loops already run outside the GIL

Threads: several flows in one program

A thread is a separate flow of execution sharing the program's memory. `start()` launches it, `join()` waits for it to finish. Here five slow "downloads" (a 0.2 s sleep stands in for the network) run one by one, then all at once:

```
1  import threading, time
2
3  def download(name, results):
4      time.sleep(0.2)                          # waiting: the thread gives up the CPU
5      results[name] = f"{name}: 200 OK"
6
7  names = ["dune", "emma", "ubik", "kim", "beloved"]
8
9  results, t0 = {}, time.perf_counter()
10 for n in names:
11     download(n, results)
12 print("one by one:", round(time.perf_counter() - t0, 1), "s")
13
14 results, t0 = {}, time.perf_counter()
15 threads = [threading.Thread(target=download, args=(n, results)) for n in names]
16 for t in threads:
17     t.start()
18 for t in threads:
19     t.join()                                 # wait for every thread
20 print("threaded:  ", round(time.perf_counter() - t0, 1), "s,", len(results), "results")
```

```
one by one: 1.0 s
threaded:   0.2 s, 5 results
```

line 4: I/O-bound: threads help

line 19: forget join() and you read results too early

Shared memory means races

Threads can be paused **between any two steps**. `balance = balance + 1` is three steps: read, add, write. If two threads interleave them, one update is lost:

step	thread 1	thread 2	balance
1	reads 5		5
2		reads 5	5
3	writes 5 + 1		6
4		writes 5 + 1	6 (should be 7)

```
1  balance = 0
2  lock = threading.Lock()
3
4  def deposit(times, use_lock):
5      global balance
6      for _ in range(times):
7          if use_lock:
8              with lock:                        # only one thread inside at a time
9                  balance = balance + 1
10         else:
11             current = balance
12             time.sleep(0)                     # any switch here loses updates
13             balance = current + 1
14
15 for use_lock in (False, True):
16     balance = 0
17     ts = [threading.Thread(target=deposit, args=(1000, use_lock)) for _ in range(4)]
18     for t in ts: t.start()
19     for t in ts: t.join()
20     print("with lock:" if use_lock else "no lock:  ", balance == 4000)
```

```
no lock:   False
with lock: True
```

line 8: the critical section

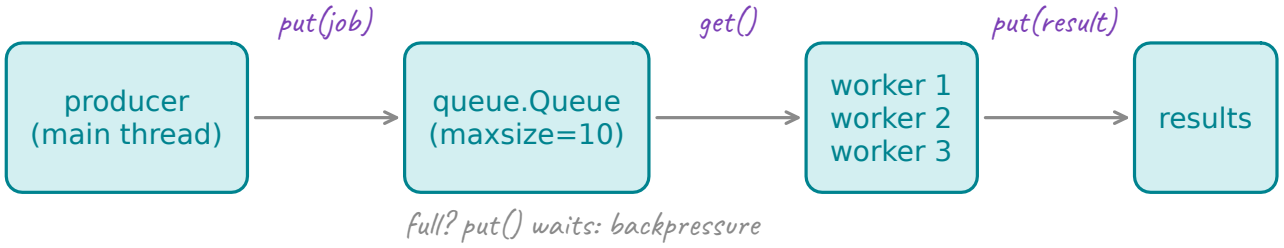
line 12: sleep(0) makes the race reliable to show

TRAP

A race may pass every test and fail once a week in production, because it depends on timing. The GIL does **not** save you: it stops two threads running bytecode at the same instant, but a thread can still be switched out between the read and the write. Protect every shared mutable value with a lock, or don't share it (next page).

Queues: share by passing messages

The safest shared data is none. Instead, threads hand work to each other through a `queue.Queue`, which does its own locking. Producers `put`, consumers `get`; a full or empty queue makes the caller wait:




```python
import queue, threading

jobs, results = queue.Queue(maxsize=10), queue.Queue()

def worker():
    while (n := jobs.get()) is not None:      # None = "no more work" (a sentinel)
        results.put((n, n * n))

workers = [threading.Thread(target=worker) for _ in range(3)]
for w in workers: w.start()
for n in range(8):
    jobs.put(n)
for _ in workers:
    jobs.put(None)                            # one sentinel per worker
for w in workers: w.join()
print(sorted(results.get() for _ in range(results.qsize())))
```

```
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25), (6, 36), (7, 49)]
```

line 6: get() blocks while empty

line 14: how workers know to stop

RULE OF THUMB

Prefer queues to locks. A lock protects data that many threads touch; a queue moves each piece of data so only one thread owns it at a time. Go's motto is the same idea: "don't communicate by sharing memory; share memory by communicating."

TRAP

Deadlock: thread 1 holds lock A and waits for B while thread 2 holds B and waits for A. Both wait forever. Avoid it by always taking locks in the same global order (practice 16.1), by holding only one at a time, or by using `lock.acquire(timeout=...)` and backing off.

Pools, and the GIL

Starting and joining threads by hand gets tedious. `concurrent.futures` gives you a pool of workers and a **future** for each job: a placeholder for a result that will exist later.

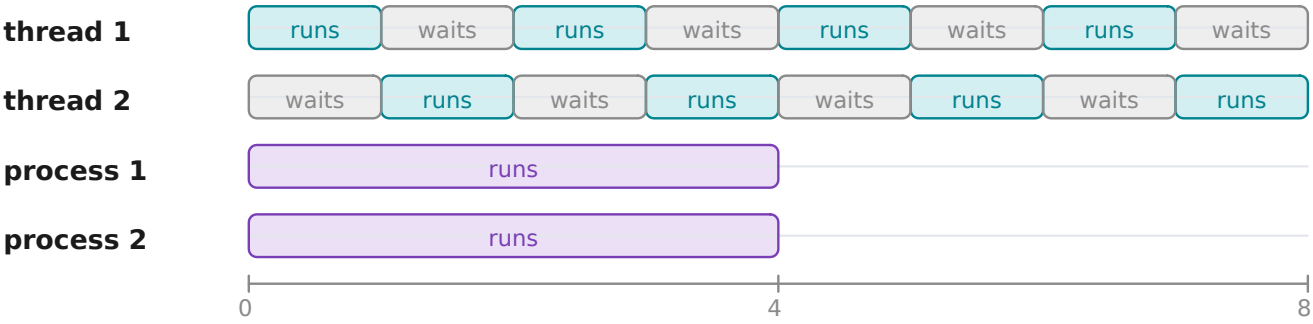
```
1  import time
2  from concurrent.futures import ThreadPoolExecutor, as_completed
3
4  def check(url):
5      time.sleep(0.1 if "slow" not in url else 0.3)
6      if "bad" in url:
7          raise ConnectionError(f"{url} refused")
8      return url, 200
9
10 urls = ["a.org", "slow.org", "bad.org", "b.org"]
11 with ThreadPoolExecutor(max_workers=4) as pool:
12     print(list(pool.map(check, ["a.org", "b.org"])))          # results in input order
13     futures = {pool.submit(check, u): u for u in urls}
14     for fut in as_completed(futures):                         # in finishing order
15         try:
16             print("done:", fut.result())
17         except ConnectionError as e:
18             print("failed:", e)
```

```
[('a.org', 200), ('b.org', 200)]
done: ('a.org', 200)
failed: bad.org refused
done: ('b.org', 200)
done: ('slow.org', 200)
```

line 13: submit returns a Future at once

line 16: result() re-raises the worker's error

So far all the waiting was I/O. For pure computation, CPython's **GIL** (global interpreter lock) lets only one thread run Python bytecode at a time, so threads don't speed it up; separate processes do:



Two CPU-bound threads take turns holding the GIL, so together they're no faster than one. Two processes each have their own interpreter and GIL, and finish the same work in half the time. This script was run while building this page:

primes.py

```python
import os, sys, time
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def count_primes(lo, hi):                       # pure CPU work: no waiting at all
    n = 0
    for k in range(max(lo, 2), hi):
        if all(k % d for d in range(2, int(k ** 0.5) + 1)):
            n += 1
    return n

CHUNKS = [(i * 150_000, (i + 1) * 150_000) for i in range(4)]

def timed(label, run):
    t0 = time.perf_counter()
    total = run()
    print(f"{label:<12} {total} primes  {time.perf_counter() - t0:5.2f} s")

if __name__ == "__main__":                      # required: worker processes import this file
    print("GIL enabled:", sys._is_gil_enabled(), "| cores:", os.cpu_count())
    timed("sequential", lambda: sum(count_primes(lo, hi) for lo, hi in CHUNKS))
    with ThreadPoolExecutor(4) as ex:
        timed("4 threads", lambda: sum(ex.map(count_primes, *zip(*CHUNKS))))
    with ProcessPoolExecutor(4) as ex:
        timed("4 processes", lambda: sum(ex.map(count_primes, *zip(*CHUNKS))))
```

its real output on the machine that built this book

```
$ python3 primes.py
GIL enabled: True | cores: 8
sequential   49098 primes   1.16 s
4 threads    49098 primes   1.12 s
4 processes  49098 primes   0.48 s
```

HONEST NOTE

Processes cost more: each starts a fresh interpreter, and arguments and results are pickled between them, so only send small inputs and get back small outputs. Python 3.13+ also ships an optional **free-threaded** build (`python3.14t`, PEP 703/779) with no GIL, where threads do run CPU work in parallel. It's officially supported from 3.14 but not the default, and C extensions must opt in. Locks matter even more there.

asyncio: thousands of waits on one thread

`asyncio` runs many tasks on **one** thread. An `async def` function is a coroutine; at each `await` it pauses and the **event loop** runs another task that's ready. No thread switches you out by surprise: tasks switch only at `await`, so races are much rarer.

```
1  import asyncio, time
2
3  async def fetch(name, delay):
4      await asyncio.sleep(delay)              # pause here; the loop runs other tasks
5      return f"{name} done"
6
7  async def main():
8      t0 = time.perf_counter()
9      results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.1), fetch("c", 0.15))
10     print(results, round(time.perf_counter() - t0, 1), "s")
11
12 asyncio.run(main())                          # start the event loop, run main, stop
```

```
['a done', 'b done', 'c done'] 0.2 s
```

line 9: run all three concurrently; results in argument order

Structured concurrency: TaskGroup and timeouts

```
1  async def main():
2      try:
3          async with asyncio.timeout(0.12):            # a deadline for the whole block
4              async with asyncio.TaskGroup() as tg:     # every task finishes before we leave
5                  fast = tg.create_task(fetch("fast", 0.05))
6                  slow = tg.create_task(fetch("slow", 1.0))
7      except TimeoutError:
8          print("timed out:", fast.result(), "| slow cancelled:", slow.cancelled())
9
10 asyncio.run(main())
```

```
timed out: fast done | slow cancelled: True
```

line 4: if one task fails, the rest are cancelled

The one rule: never call blocking code inside a coroutine. `time.sleep`, `requests.get` or a slow loop freeze the whole loop, and every other task with it:

time.sleep: blocks the loop

```
import asyncio, time
async def job():
    time.sleep(0.1)
async def main():
    t0 = time.perf_counter()
    await asyncio.gather(job(), job(), job())
    print(round(time.perf_counter() - t0, 1))
asyncio.run(main())
```

```
0.3
```

await asyncio.sleep: yields

```
import asyncio, time
async def job():
    await asyncio.sleep(0.1)
async def main():
    t0 = time.perf_counter()
    await asyncio.gather(job(), job(), job())
    print(round(time.perf_counter() - t0, 1))
asyncio.run(main())
```

```
0.1
```

RULE OF THUMB

Stuck with a blocking library in async code? `await asyncio.to_thread(blocking_fn, args)` runs it in a thread and awaits the result, so the loop keeps going.

An async chat server, end to end

This is where asyncio shines: a server holding many idle connections. Each client gets a coroutine; while it waits for that client's next line, the loop serves everyone else. Two clients connect and one talks:

```
1  import asyncio
2
3  clients = set()
4
5  async def handle(reader, writer):                    # one coroutine per client
6      clients.add(writer)
7      try:
8          while line := await reader.readline():       # b"" when the client leaves
9              for w in clients - {writer}:
10                 w.write(line)
11                 await w.drain()                      # wait if their buffer is full
12     finally:
13         clients.discard(writer)
14         writer.close()
15
16 async def main():
17     server = await asyncio.start_server(handle, "127.0.0.1", 0)
18     port = server.sockets[0].getsockname()[1]
19     async with asyncio.timeout(5):
20         r1, w1 = await asyncio.open_connection("127.0.0.1", port)
21         r2, w2 = await asyncio.open_connection("127.0.0.1", port)
22         await asyncio.sleep(0.05)                    # let the server register both
23         w1.write(b"hi from ana\n")
24         print(await r2.readline())
25         w1.close(); w2.close()
26     server.close()
27
28 asyncio.run(main())
```

```
b'hi from ana\n'
```

line 8: framing by newline (Ch 15)

line 11: backpressure

	threads	asyncio	processes
switches	anywhere (preemptive)	only at `await` (cooperative)	separate programs
shared memory	yes: needs locks	yes, safer: no surprise switches	no: pickle messages
cost each	~MB of stack	~KB	a whole interpreter
CPU-bound speed-up	no (GIL), yes when free-threaded	no	**yes**

works with blocking libraries	**yes**	only via `to_thread`	**yes**

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 16.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 16.1` shows other valid solutions and their trade-offs.

`16.1` fix the bug Make the bank thread-safe (and deadlock-free)
`16.2` build from a spec Run jobs concurrently, at most N at a time
`16.3` build from a spec A concurrent crawler with a work queue
`16.4` build from a spec asyncio: limited, with a timeout each
`16.5` learn an unseen module Debounce with threading.Timer
`16.6` mini-project An asyncio chat server with nicknames

PASSPORT STAMP ask the same question of every new language (Appendix B)

11. Concurrency

Threads share memory (protect it with `Lock`, or pass data through `queue.Queue`). The GIL stops CPU parallelism in threads on the default build; use processes for CPU work. `asyncio`: one thread, cooperative tasks that switch at `await`; `TaskGroup` and `asyncio.timeout` keep them structured.

12. Stdlib and tooling

`threading`, `queue`, `concurrent.futures`, `multiprocessing`, `asyncio`, `contextvars`.

IN OTHER LANGUAGES

Java and C#: real parallel threads, `synchronized`/`lock`, executors and futures. JavaScript: one thread and an event loop, exactly asyncio's model (`async/await` came from C#). Go: goroutines (cheap threads) and channels (queues). Rust: threads with the compiler refusing to build data races, plus async with tokio.

DEEP DIVE (your vault)

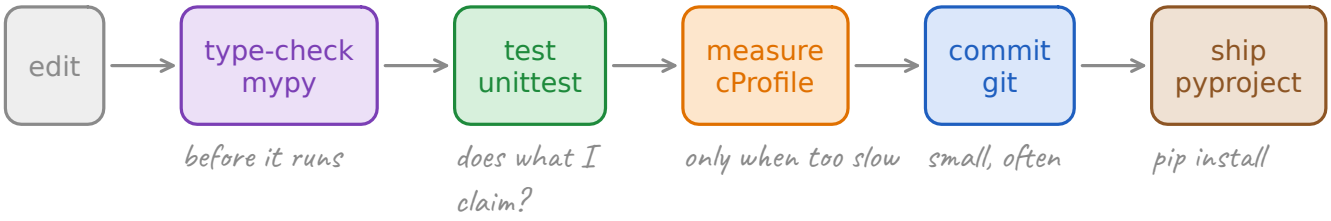
Vault notes: 14.1 Concurrency is a lifetime relation · 14.2 Interleaving and race conditions · 14.4 Threads · 14.5 The CPython global interpreter lock · 14.6 Thread communication with queues · 14.7 Processes · 14.9 Asynchronous I/O · 14.10 Structured concurrency with task groups · 14.11 Cancellation and timeouts · 14.12 Backpressure · 14.13 Choosing a concurrency model · 15.16 The GIL and free-threaded internals.

Chapter 17

Quality: tests, speed, types, shipping

how professionals know their code works, and keep it working

Writing code that works once is the easy half. Quality means being able to **show** it works, find out **why** it's slow, catch mistakes **before** running it, let other people **install** it, and **undo** any change. Each of those has a standard Python tool, and every other language has the same five:

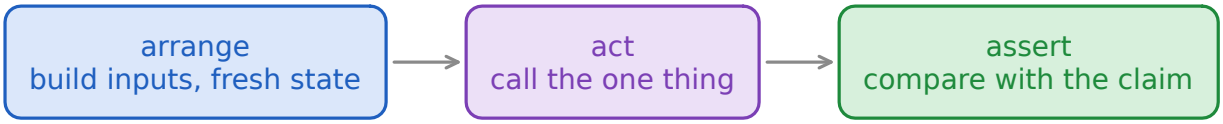


the question	Python tool	in the stdlib?
Does it do what I claim, and still after my change?	`unittest`, `doctest`, `unittest.mock`	yes
Where does the time go?	`timeit`, `cProfile` + `pstats`	yes
Do the types line up, before I run anything?	type hints + `mypy` or `pyright`	hints yes, checker no
Can someone else install and run it?	`pyproject.toml`, `venv`, `pip`, `build`	venv yes, rest pip
What changed, when, why, and can I undo it?	Git	separate program

The habit matters more than the tool: a test written before the fix, a measurement taken before the optimisation, a commit made before the experiment. The practice repo you've been using is built exactly this way: every exercise is a unittest suite.

unittest: claims the computer checks

A test is a **claim about behaviour** written as code: "a book returned 4 days late costs 1.00". Each test follows the same three steps:




```
1  import sys, unittest
2  from datetime import date, timedelta
3
4  def late_fee(due, returned, per_day=0.25, cap=5.0):
5      days = (returned - due).days
6      return 0.0 if days <= 0 else min(days * per_day, cap)
7
8  class LateFeeTest(unittest.TestCase):
9      due = date(2026, 3, 1)
10
11     def test_on_time_is_free(self):
12         self.assertEqual(late_fee(self.due, self.due), 0.0)
13
14     def test_per_day(self):
15         self.assertEqual(late_fee(self.due, date(2026, 3, 5)), 1.0)
16
17     def test_capped(self):
18         self.assertEqual(late_fee(self.due, date(2026, 6, 1)), 5.0)
19
20     def test_rejects_text(self):
21         with self.assertRaises(TypeError):
22             late_fee(self.due, "2026-03-05")
23
24 def run(*cases, v=1):                    # what `python -m unittest` does for you
25     load = unittest.defaultTestLoader.loadTestsFromTestCase
26     suite = unittest.TestSuite(load(c) for c in cases)
27     unittest.TextTestRunner(stream=sys.stdout, verbosity=v).run(suite)
28
29 run(LateFeeTest, v=2)
```

```
test_capped (__main__.LateFeeTest.test_capped) ... ok
test_on_time_is_free (__main__.LateFeeTest.test_on_time_is_free) ... ok
test_per_day (__main__.LateFeeTest.test_per_day) ... ok
test_rejects_text (__main__.LateFeeTest.test_rejects_text) ... ok

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK
```

line 8: subclass TestCase

line 11: every test_* method is one test

line 21: the error is the expected behaviour

A failing test tells you which claim broke and shows both values. Here the claim is wrong, not the code: floats can't hold 0.1 exactly (Ch 11), so compare money with a tolerance or use `Decimal`:

```
class TenPence(unittest.TestCase):
    def test_three_days(self):
        self.assertEqual(late_fee(date(2026, 3, 1), date(2026, 3, 4), per_day=0.1), 0.3)

run(TenPence)
```

```
F
======================================================================
FAIL: test_three_days (__main__.TenPence.test_three_days)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "<chapter 17>", line 3, in test_three_days
AssertionError: 0.30000000000000004 != 0.3

----------------------------------------------------------------------
Ran 1 test in 0.000s

FAILED (failures=1)
```

assert method	checks	use it for
`assertEqual(a, b)`	`a == b`, with a diff on failure	most things; lists and dicts get a diff
`assertAlmostEqual(a, b)`	equal to 7 decimal places	floats
`assertTrue(x)` / `assertIn(a, b)`	truthiness / membership	flags, containers
`assertRaises(E)` as a `with`	the block raises `E`	error paths (they need tests too)
`self.subTest(**info)`	labels one case in a loop	table-driven tests (next)

Which cases? Partitions and boundaries

You can't test every input, so split the inputs into **partitions** that should behave the same, test one from each, and add the **boundaries** between them, where off-by-one bugs live:

partition	days late	expected fee	why this case
returned early	`-3`	`0.0`	negative days must not give a refund
on the due date	`0`	`0.0`	boundary: `<` vs `<=`
first late day	`1`	`0.25`	boundary: the first charge
just under the cap	`19`	`4.75`	boundary below the cap
exactly at the cap	`20`	`5.0`	boundary: `min` kicks in
far over the cap	`90`	`5.0`	the cap holds

```
class Boundaries(unittest.TestCase):
    def test_table(self):
        due = date(2026, 3, 1)
        for days, fee in [(-3, 0.0), (0, 0.0), (1, 0.25), (19, 4.75), (20, 5.0), (90, 5.0)]:
            with self.subTest(days=days):
                self.assertEqual(late_fee(due, due + timedelta(days)), fee)

run(Boundaries)
```

```
.
----------------------------------------------------------------------
Ran 1 test in 0.000s

OK
```

line 5: a failure names the case and keeps going

setUp and addCleanup: a fresh world for every test

```
import sqlite3

class LoanStore(unittest.TestCase):
    def setUp(self):                                # runs before every test
        self.db = sqlite3.connect(":memory:")
        self.addCleanup(self.db.close)              # runs after, even if the test fails
        self.db.execute("CREATE TABLE loans (title TEXT, due TEXT)")

    def test_insert(self):
        self.db.execute("INSERT INTO loans VALUES ('Dune', '2026-03-01')")
        self.assertEqual(self.db.execute("SELECT count(*) FROM loans").fetchone(), (1,))

    def test_starts_empty(self):                    # test_insert's row is gone
        self.assertEqual(self.db.execute("SELECT count(*) FROM loans").fetchone(), (0,))

run(LoanStore, v=2)
```

```
test_insert (__main__.LoanStore.test_insert) ... ok
test_starts_empty (__main__.LoanStore.test_starts_empty) ... ok

----------------------------------------------------------------------
Ran 2 tests in 0.000s

OK
```

RULE OF THUMB

Tests must not depend on each other or on their order. Each one builds what it needs (in `setUp`, or a temporary folder from `tempfile`) and leaves nothing behind. If a test passes alone but fails in the suite, some state is leaking.

doctest: examples that can't go stale

A `>>>` example in a docstring documents the function **and** is a test: `doctest` runs it and compares the printed result. Good for small pure functions; use unittest for anything with setup.

```
1  import doctest
2
3  def fine(days_late, per_day=0.25, cap=5.0):
4      '''Late fee for a book, capped.
5
6      >>> fine(0)
7      0.0
8      >>> fine(4)
9      1.0
10     >>> fine(100)
11     5.0
12     '''
13     return min(max(days_late, 0) * per_day, cap)
14
15 runner = doctest.DocTestRunner()
16 for test in doctest.DocTestFinder().find(fine, globs={"fine": fine}):
17     runner.run(test)
18 print(runner.summarize(verbose=False))
```

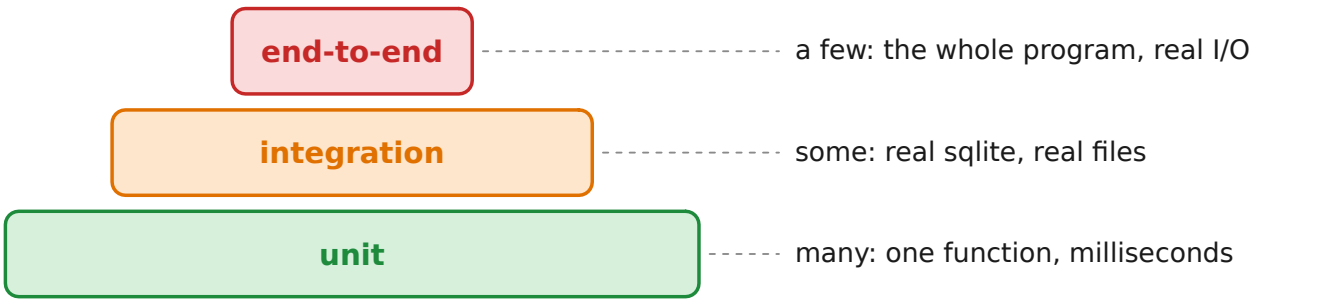
```
TestResults(failed=0, attempted=3)
```

line 6: the REPL session, pasted in

line 16: python -m doctest file.py does this

How big should a test be?

A **unit** test checks one function on its own; an **integration** test checks pieces working together, with a real database or real files; an **end-to-end** test runs the whole program the way a user would. Aim for this shape:



slower, costlier, flakier as you go up

unittest.mock: replacing the outside world

Some code talks to things a test can't use: a mail server, the network, the clock. A **test double** stands in for them. The cleanest seam is a parameter, so the test can pass a `Mock`, which accepts any call and records it:

```
1  from unittest.mock import Mock
2
3  def send_reminders(loans, today, mailer):
4      for who, title, due in loans:
5          if due < today:
6              mailer.send(who, f"'{title}' was due {due:%d %b}")
7
8  mailer = Mock()
9  loans = [("ana@x.org", "Dune", date(2026, 3, 1)), ("ben@x.org", "Emma", date(2026, 4, 1))]
10 send_reminders(loans, date(2026, 3, 10), mailer)
11 mailer.send.assert_called_once_with("ana@x.org", "'Dune' was due 01 Mar")
12 print(mailer.send.call_count, mailer.send.call_args)
```

```
1 call('ana@x.org', "'Dune' was due 01 Mar")
```

line 3: the dependency comes in as an argument

line 8: no SMTP server needed

line 11: raises AssertionError if not

double	what it does	in Python
dummy	fills a parameter, never used	`None`, `object()`
stub	returns canned answers	`Mock(return_value=...)`
fake	a real, simpler implementation	sqlite `:memory:`, a dict, `io.StringIO`
spy / mock	records calls so you can check them	`Mock()` + `assert_called_with`

patch: when you can't pass the dependency in

`patch` swaps a name for a `MagicMock` during a `with` block and puts the original back afterwards. `return_value` decides what the fake returns; `side_effect` makes it raise, so you can test the error path without unplugging the network:

```
1  import io, json, urllib.request
2  from unittest.mock import patch
3
4  def book_title(isbn):
5      url = f"https://openlibrary.org/isbn/{isbn}.json"
6      with urllib.request.urlopen(url, timeout=5) as r:
7          return json.load(r)["title"]
8
9  with patch("urllib.request.urlopen") as urlopen:
10     urlopen.return_value.__enter__.return_value = io.BytesIO(b'{"title": "Dune"}')
11     print(book_title("9780441013593"), "|", urlopen.call_args.args[0])
12
13 with patch("urllib.request.urlopen", side_effect=TimeoutError("too slow")):
14     try:
15         book_title("9780441013593")
16     except TimeoutError as e:
17         print("error path tested:", e)
```

```
Dune | https://openlibrary.org/isbn/9780441013593.json
error path tested: too slow
```

line 10: what `with ... as r` will give

line 13: the fake raises instead

TRAP

Patch the name **where it is looked up**, not where it was defined. If `loans.py` did `from urllib.request import urlopen`, it holds its own reference, so you must patch `"loans.urlopen"`. Patching `urllib.request.urlopen` would leave that copy untouched.

WHY

Mock too much and your tests check **how** the code works (which calls it makes) instead of **what** it achieves, so every refactor breaks them. Prefer fakes (an in-memory database, a temporary folder) and keep mocks for the edges of your program: network, mail, clock, randomness.

Measure, then optimise

Programmers are famously bad at guessing where time goes. The professional loop never skips a step:




```python
import cProfile, pstats

def is_member(members, mid):
    return mid in members

def report(n):
    members = [f"m{i:05}" for i in range(n)]
    loans = [f"m{i * 7 % (2 * n):05}" for i in range(n)]
    return sum(is_member(members, m) for m in loans)

with cProfile.Profile() as prof:
    report(4000)
funcs = pstats.Stats(prof).get_stats_profile().func_profiles
for name, p in sorted(funcs.items(), key=lambda kv: -kv[1].tottime)[:3]:
    print(f"{name[:12]:12} calls={p.ncalls:>5}  own time={p.tottime:.3f} s")
```

```
is_member    calls= 4000  own time=0.065 s
report       calls=    1  own time=0.002 s
<genexpr>    calls= 4001  own time=0.001 s
```

line 4: `in` on a list: a linear scan

line 11: record every call

line 14: sort by time spent inside each function

`is_member` takes nearly all the time, and it's called 4000 times, each scanning up to 4000 names. Change one line, membership in a `set` (Ch 10), then check it's faster **and** still gives the same answer:

```python
import timeit

def report_fast(n):
    members = {f"m{i:05}" for i in range(n)}      # a set: the only change
    loans = [f"m{i * 7 % (2 * n):05}" for i in range(n)]
    return sum(is_member(members, m) for m in loans)

assert report(4000) == report_fast(4000)
for fn in (report, report_fast):
    t = min(timeit.repeat(lambda: fn(4000), number=3, repeat=3)) / 3
    print(f"{fn.__name__:12} {t * 1000:7.1f} ms")
```

```
report         65.5 ms
report_fast     1.7 ms
```

line 8: same result first

line 10: min of repeats: least noise

HONEST NOTE

Timings on this page were measured while building the book, so yours will differ. From the shell: `python3 -m timeit -s "xs = list(range(1000))" "999 in xs"` for one-liners, and `python3 -m cProfile -s tottime script.py` for a whole program. Profilers slow code down, so trust their proportions more than their seconds.

Type hints, and a checker that reads them

Python checks types only at run time, and only when an operation fails. **Type hints** write your intent down; Python itself ignores them:

```
def double(n: int) -> int:
    return n * 2

print(double(21), double("ha"), double.__annotations__)
```

```
42 haha {'n': <class 'int'>, 'return': <class 'int'>}
```

← runs fine: hints aren't enforced

A separate **static checker**, `mypy` or `pyright` (both pip installs; editors run them as you type), reads the hints and finds the mismatch without running anything:

loans.py

```
from dataclasses import dataclass

@dataclass
class Loan:
    title: str
    days: int

def double(n: int) -> int:
    return n * 2
double("ha")

def find(loans: list[Loan], title: str) -> Loan | None:
    return next((x for x in loans if x.title == title), None)
print(find([], "Dune").title)
```

```
$ mypy loans.py
loans.py:10: error: Argument 1 to "double" has incompatible type "str"; expected "int"  [arg-type]
loans.py:14: error: Item "None" of "Loan | None" has no attribute "title"  [union-attr]
Found 2 errors in 1 file (checked 1 source file)
```

The second error is the valuable kind: `find` can return `None`, and the caller forgot. That bug would pass every test that happens to find the book, then crash in production.

write	means
`list[str]`, `dict[str, int]`, `tuple[int, ...]`	containers and what they hold
`Loan | None`	maybe missing: the checker makes you handle `None`
`Callable[[int], str]` (from `collections.abc`)	a function taking an int, returning a str
`Literal["due", "title"]`	only these exact values
`def first[T](xs: list[T]) -> T:` (3.12+)	generic: the output type follows the input
`class Mailer(Protocol): def send(...)`	anything with a matching `send` method fits

```
1  from typing import Protocol
2
3  class Mailer(Protocol):
4      def send(self, to: str, text: str) -> None: ...
5
6  class PrintMailer:                              # never mentions Mailer
7      def send(self, to: str, text: str) -> None:
8          print(f"to {to}: {text}")
9
10 def remind(m: Mailer) -> None:
11     m.send("ana@x.org", "Dune is due")
12
13 def first[T](items: list[T], default: T) -> T:
14     return items[0] if items else default
15
16 remind(PrintMailer())
17 print(first([3, 1], 0), first([], "none"))
```

```
to ana@x.org: Dune is due
3 none
```

line 3: structural: any class with this send fits

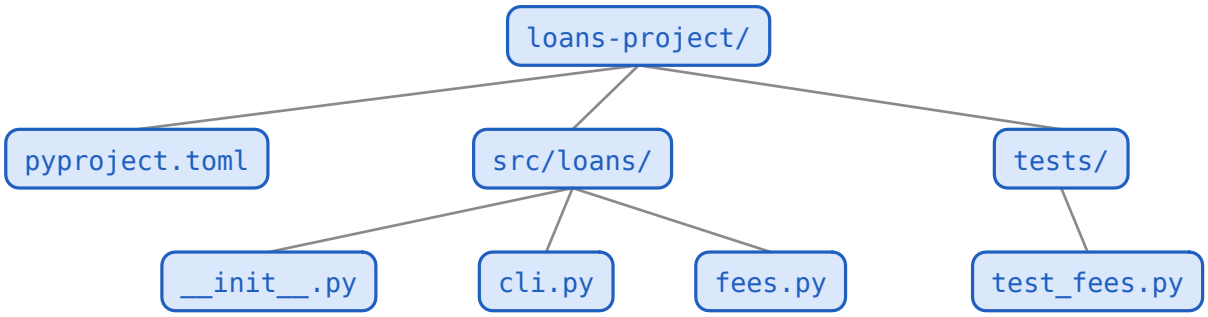
line 11: T: the checker knows first(...) is an int

HONEST NOTE

Hints are **gradual**: unannotated code is simply not checked, so you can add them one module at a time. They don't make Python faster. Since 3.14 annotations are evaluated lazily (PEP 649), so a hint can name a class defined further down the file.

pyproject.toml: making it installable

A project others can install has a standard shape. The **src layout** keeps your package out of the current folder, so tests run against the **installed** code, exactly as users will (Ch 2's sys.path):




```
1  import tomllib
2
3  PYPROJECT = """
4  [build-system]
5  requires = ["setuptools>=77"]
6  build-backend = "setuptools.build_meta"
7
8  [project]
9  name = "loans"
10 version = "0.3.0"
11 requires-python = ">=3.12"
12 dependencies = []
13
14 [project.optional-dependencies]
15 dev = ["mypy"]
16
17 [project.scripts]
18 loans = "loans.cli:main"
19 """
20 cfg = tomllib.loads(PYPROJECT)                # the stdlib reads TOML (it can't write it)
21 print(cfg["project"]["name"], cfg["project"]["version"], cfg["project"]["scripts"])
```

```
loans 0.3.0 {'loans': 'loans.cli:main'}
```

line 5: the tool that builds the package

line 12: runtime needs (Ch 19)

line 15: extras: pip install -e '.[dev]'

line 18: a command → your main(argv=None)

from a fresh clone to a running command and a wheel

```
$ python3 -m venv .venv && source .venv/bin/activate
(.venv) $ pip install -e ".[dev]"          # editable: edits in src/ apply at once
(.venv) $ loans --help                     # the [project.scripts] entry point
usage: loans [-h] [-v] {add,list} ...
(.venv) $ python -m unittest discover -s tests
....
----------------------------------------------------------------------
Ran 4 tests in 0.002s

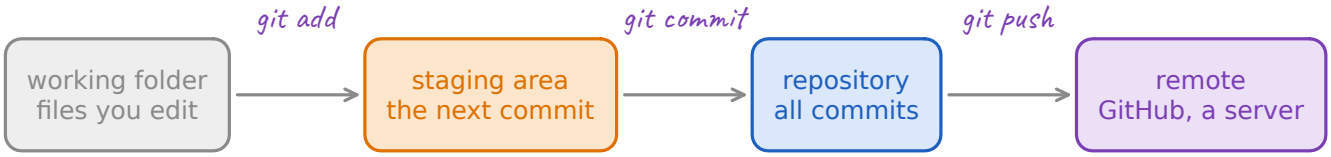
OK
(.venv) $ python -m build                  # pip install build; output goes to dist/
Successfully built loans-0.3.0.tar.gz and loans-0.3.0-py3-none-any.whl
```

WHY

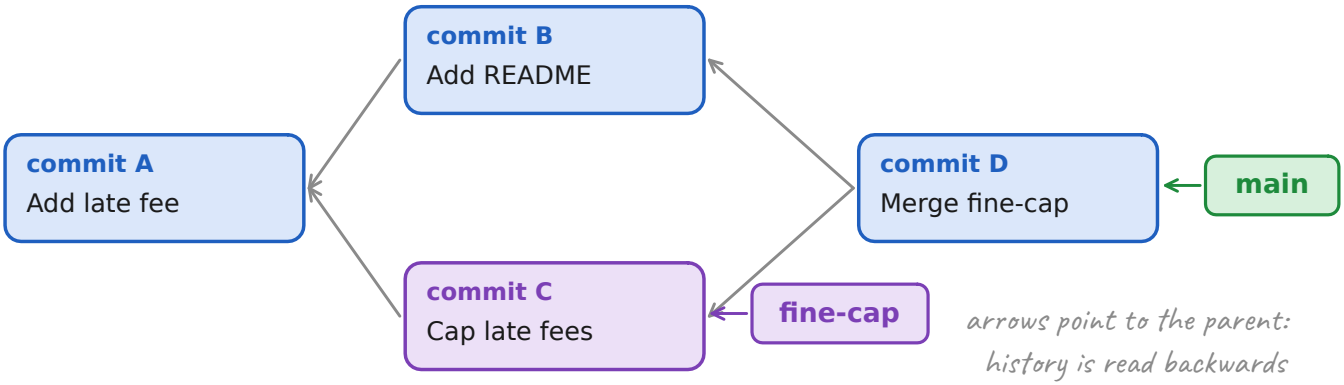
A **wheel** (`.whl`) is a zip of your package plus metadata; `pip install` just unpacks it into `site-packages`. `pyproject.toml` replaced `setup.py` so that any build tool (setuptools, hatchling, flit, uv) can read the same file. Which dependencies to trust is Ch 19.

Git: every change, recorded and undoable

Git stores **snapshots** of your whole project, called commits. Each commit records its parent(s), its author and a message. A change travels from your folder into the repository, then out to a shared copy:



A **branch** is only a movable name pointing at one commit, so branching is instant. Do an experiment on a branch, then merge it: the merge commit has two parents.



a real session, run while building this page

```
# write fees.py
$ git init -q -b main
$ git add fees.py
$ git commit -qm "Add late fee"
$ git switch -qc fine-cap
# edit fees.py: cap the fee at 5.00
$ git commit -qam "Cap late fees at 5.00"
$ git switch -q main
# write README.md
$ git add README.md
$ git commit -qm "Add README"
$ git merge -q --no-edit fine-cap
$ git log --oneline --graph
*   ab8cc0c Merge branch 'fine-cap'
|\
| * 308497a Cap late fees at 5.00
* | f0d8ad4 Add README
|/
* 55c9eb9 Add late fee
```

everyday command	does
`git status`, `git diff`	what changed since the last commit (check before every commit)
`git add -p`	stage piece by piece: one commit = one logical change
`git switch -c name` / `git merge name`	start an experiment / bring it back
`git restore file`	throw away uncommitted edits to a file
`git revert <commit>`	a new commit that undoes an old one (safe on shared history)

TRAP

A secret committed once stays in the history even after you delete the file, and anyone who clones gets it. Add `.env`, keys and `*.db` to `.gitignore` **before** the first commit. If a secret was ever pushed, **rotate it**: rewriting history doesn't remove copies people already have.

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 17.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 17.1` shows other valid solutions and their trade-offs.

17.1 build from a spec Tests that catch bugs (the checker plants them)
17.2 fix the bug Tests that pass alone but fail together
17.3 refactor, tests stay green Make the reminder job testable
17.4 measure and explain Profile it, then make it fast
17.5 learn an unseen module Documentation that tests itself
17.6 learn an unseen module Read type hints at run time
17.7 mini-project Scaffold an installable project

PASSPORT STAMP ask the same question of every new language (Appendix B)

3. Typing

Dynamic and strong at run time. Optional hints (`list[str]`, `X | None`, `Protocol`, generics) are ignored by Python and checked statically by `mypy` or `pyright`.

9. Modules and build

`pyproject.toml` declares the package, dependencies and commands; `venv` + `pip install -e .` for development; wheels for distribution; src layout.

12. Stdlib and tooling

`unittest` (+ `subTest`, `setUp`, `addCleanup`), `doctest`, `unittest.mock`, `timeit`, `cProfile` + `pstats`, `tomllib`, `typing`. Outside: Git, mypy, build.

IN OTHER LANGUAGES

Java: JUnit + Mockito, Maven/Gradle (`pom.xml`), static types checked by the compiler. Rust: `#[test]` and `cargo test`, `cargo bench`, `Cargo.toml`. C: no standard test framework, `make`/CMake, `perf`/`gprof`. TypeScript's types work exactly like Python's hints: checked by `tsc`, erased at run time. Git is the same in every language.

DEEP DIVE (your vault)

Vault notes: 12.3 Testing is executable claim checking · 12.4 Test selection by partitions and boundaries · 12.7 Test doubles and mocking · 12.9 Property-based and metamorphic testing · 12.10 Coverage and mutation testing · 12.18 Type hints · 12.19 Static analysis with mypy · 14.15 Microbenchmarking · 14.16 Profiling · 12.13 Git's object model · 12.14 Branching, merging, and conflicts · 12.21 Continuous integration.

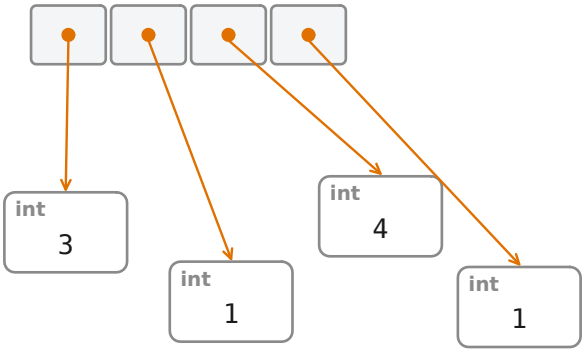
Chapter 18

NumPy done properly

the array is a block of memory with a shape painted on it

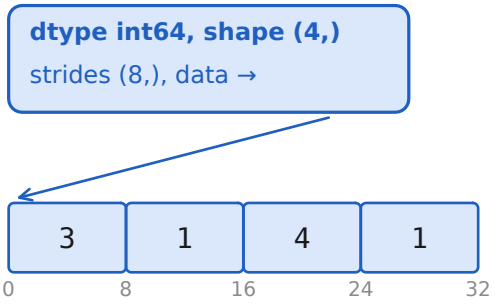
The Computing module showed you NumPy's **what**. This chapter is the **why**: once you can picture an array in memory, views, broadcasting and the speed all stop being magic. Start with the difference that causes everything else:

Python list [3, 1, 4, 1]



4 pointers + 4 full objects, anywhere

np.array([3, 1, 4, 1])



one block: 4 × 8 bytes, no pointers

```
1  import timeit
2  import numpy as np
3
4  days = list(range(1_000_000))
5  arr = np.arange(1_000_000)
6
7  def with_list():
8      return [min(d * 0.25, 5.0) for d in days]      # a million boxed objects, one at a time
9
10 def with_numpy():
11     return np.minimum(arr * 0.25, 5.0)              # C loops over raw doubles
12
13 assert with_list() == with_numpy().tolist()
14 t_list, t_np = (min(timeit.repeat(fn, number=1, repeat=3)) for fn in (with_list, with_numpy))
15 print(f"list {t_list * 1000:.0f} ms, numpy {t_np * 1000:.1f} ms: {t_list / t_np:.0f}x faster")
```

```
list 40 ms, numpy 2.2 ms: 18x faster
```

line 13: same answer first

line 14: best of 3 runs each

Python's loop handles a **boxed object** on every step: look up its type, dispatch the operation, allocate a new object for the result. NumPy checks the type once for the whole block, then runs a tight loop in C over raw machine numbers, often using SIMD instructions to do several at a time.

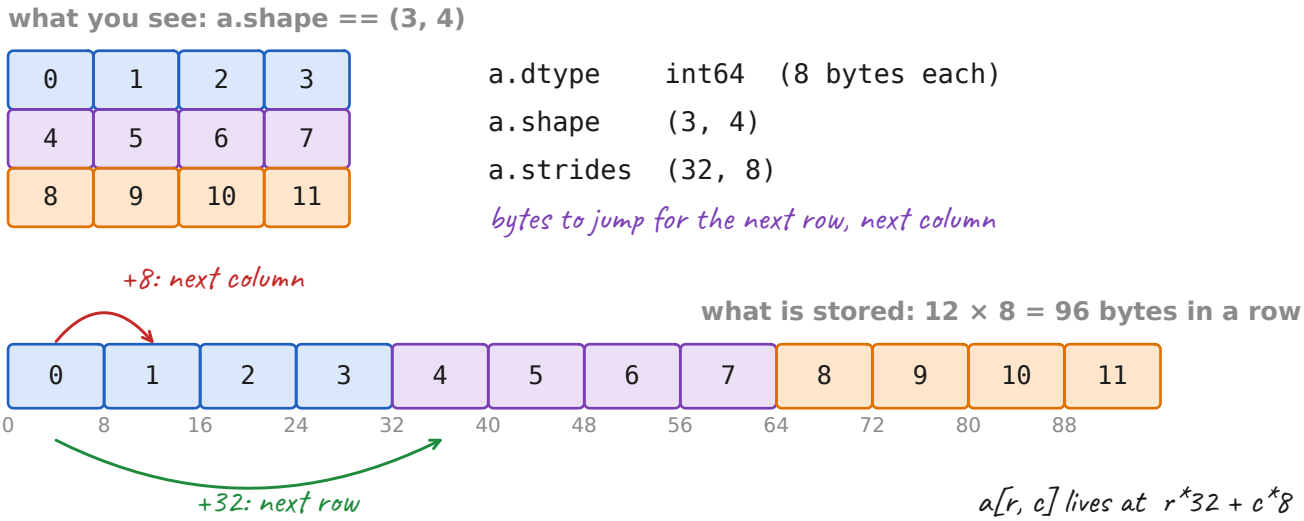
dtype, shape, strides: the three facts

Every array is a **header** plus a **buffer**. The header says how to read the bytes: what each item is (`dtype`), the grid it forms (`shape`), and how many bytes to jump along each axis (`strides`):

```
1 a = np.arange(12).reshape(3, 4)
2 print(a)
3 print(a.dtype, a.shape, a.strides, a.itemsize, a.nbytes)
4 print(a[1, 2], a[1], a[:, 2])
```

← row 1, column 2 / a row / a column

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
int64 (3, 4) (32, 8) 8 96
6 [4 5 6 7] [ 2  6 10]
```



The dtype is a contract: every item has the same type and size, which is what makes the C loop possible. The price is that NumPy numbers behave like C numbers, not Python ones:

```
1 small = np.array([120, 127], dtype=np.int8)
2 print(small + 10)                                  # wraps around, no error
3 print(np.array([1, 2]) / 2, np.array([1, 2]) // 2)
4 print(np.array([1, 2.5, True]).dtype, np.array(["a", 1]).dtype)
5 print(np.array([2**62]) * 4, 2**62 * 4)
```

```
[-126 -119]
[0.5 1. ] [0 1]
float64 <U21
[0] 18446744073709551616
```

line 2: int8 holds -128..127

line 4: everything upcast to one type

line 5: int64 overflow vs Python's big int

dtype	bytes	holds	typical use
`int64` / `int32`	8 / 4	whole numbers, fixed range	counts, indexes (default int)
`uint8`	1	0..255	image pixels
`float64` / `float32`	8 / 4	IEEE floats (Ch 11)	default float / GPUs, big data
`bool`	1	True / False	masks (later this chapter)
`<U10`	40	fixed-width text	labels (pandas usually better)
`object`	8	pointers to Python objects	**avoid**: back to slow boxes

Views and copies: who owns the bytes?

Because the header and buffer are separate, NumPy can make a **new header over the same buffer**. Slicing does exactly that, so it's instant even on a gigabyte array, and writing through the slice writes into the original:

```
1  a = np.arange(12).reshape(3, 4)
2  col = a[:, 1]                          # a view: new header, same bytes
3  col[:] = -1
4  print(a)
5  print(np.shares_memory(a, col), col.strides)
6
7  picked = a[[0, 2]]                     # fancy indexing: always a copy
8  picked[:] = 99
9  print(a[0], np.shares_memory(a, picked))
10
11 t = a.T                                # transpose: just swap the strides
12 print(t.shape, t.strides, np.shares_memory(a, t))
```

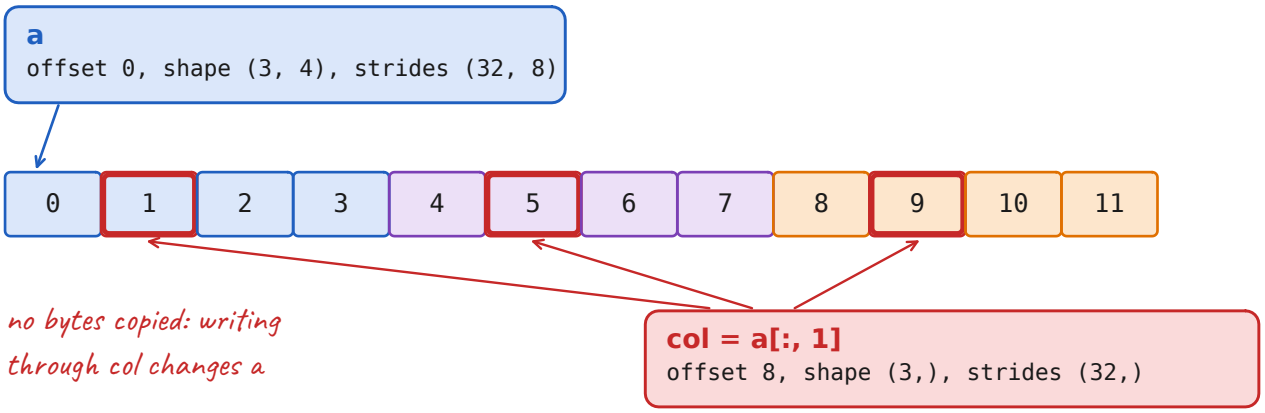
```
[[ 0 -1  2  3]
 [ 4 -1  6  7]
 [ 8 -1 10 11]]
True (32,)
[ 0 -1  2  3] False
(4, 3) (8, 32) True
```

line 3: writes into a

line 11: no data moved at all

operation	result	why
`a[1:3]`, `a[:, 1]`, `a[::2]`	view	a regular step: offset + strides describe it
`a.reshape(...)`, `a.T`, `a.ravel()`	view (usually)	same bytes, new shape or strides
`a[[0, 2]]`, `a[a > 3]`	copy	irregular picks can't be described by strides
`a * 2`, `np.sqrt(a)`, `a.copy()`	new array	its own memory, independent of `a`

Here is the first view in memory: two headers reading the same twelve numbers.



TRAP

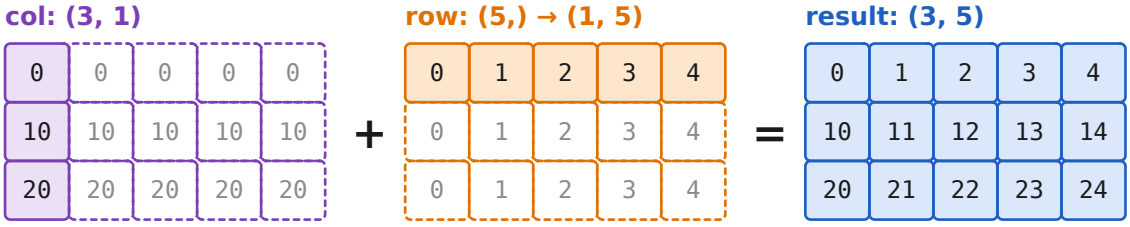
A function that receives a slice and modifies it in place is modifying the caller's array. If a function shouldn't have side effects, `.copy()` its input or build a new result. Strides also explain speed: walking along the last axis reads neighbouring bytes (cache-friendly); walking down a column of a big array jumps through memory.

Broadcasting: shapes that stretch

Arithmetic between arrays of different shapes works if NumPy can stretch them to a common shape. The rule is short enough to memorise:

RULE OF THUMB

Line the shapes up **from the right**. Each pair of sizes must be **equal**, or one of them must be **1** (a missing axis counts as 1). Axes of size 1 are stretched to match.



dashed cells are never stored: NumPy re-reads the same value (a stride of 0)

```python
days = np.arange(0, 25, 6)                      # 0, 6, 12, 18, 24 days late: shape (5,)
rates = np.array([[0.10], [0.25], [0.50]])      # one rate per row: shape (3, 1)
print(np.minimum(days * rates, 5.0))            # (3, 1) with (5,) -> a (3, 5) fee table

due = np.array([3, 10, 4])
print(due[:, None] - due[None, :])              # every pair: (3, 1) - (1, 3) -> (3, 3)
```

```
[[0.  0.6 1.2 1.8 2.4]
 [0.  1.5 3.  4.5 5. ]
 [0.  3.  5.  5.  5. ]]
[[ 0 -7 -1]
 [ 7  0  6]
 [ 1 -6  0]]
```

line 3: no loops, no copies of days or rates

line 6: None adds an axis of size 1

```python
np.ones(3) + np.ones(4)
```

```
ValueError: operands could not be broadcast together with shapes (3,) (4,)
```

shapes	line up from the right	result
`(3, 4)` and `(4,)`	4 = 4; 3 vs missing (1)	`(3, 4)`: the row is added to every row
`(3, 4)` and `(3,)`	4 vs 3: neither is 1	**error**: use `(3, 1)` for one value per row
`(3, 1)` and `(1, 5)`	1 vs 5, 3 vs 1	`(3, 5)`: an outer table
`(2, 3, 4)` and `(3, 1)`	4 vs 1, 3 = 3, 2 vs missing	`(2, 3, 4)`

Vectorise: masks instead of if, whole arrays instead of for

A comparison on an array gives a **boolean array** of the same shape: a mask. Masks replace the `if` inside a loop; `np.where` is the vectorised `x if cond else y`:

```
1 rng = np.random.default_rng(7)
2 days_late = rng.integers(-10, 30, size=12)
3 print(days_late)
4
5 late = days_late > 0                              # bool array, same shape
6 print(late.sum(), days_late[late])                # True counts as 1; a mask selects
7 fees = np.where(late, np.minimum(days_late * 0.25, 5.0), 0.0)
8 print(fees.sum(), fees.max(), fees.argmax())
```

```
[27 15 17 25 13 21 23 -1 -8  2  1 24]
10 [27 15 17 25 13 21 23  2  1 24]
37.0 5.0 0
```

line 5: a mask

line 7: where(cond, if_true, if_false)

line 8: argmax: the index of the max

a Python loop

```
total = 0.0
for d in days_late:
    if d > 0:
        total += min(d * 0.25, 5.0)
print(total)
```

```
37.0
```

vectorised

```
paid = np.clip(days_late, 0, None) * 0.25
total = np.minimum(paid, 5.0).sum()
print(total)
```

```
37.0
```

reads like the formula ■■■■■

speed on big arrays ■■■■■

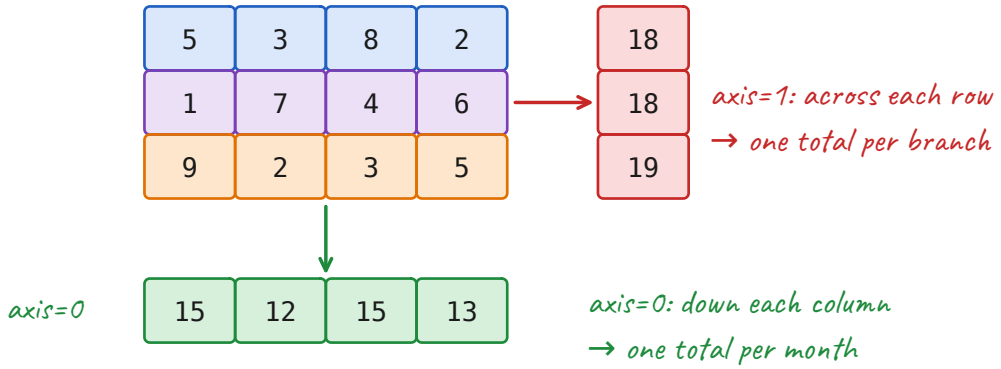
Same answer. The vectorised version states **what** to compute over the whole array and leaves the looping to C, which is where the speed on the chapter's first page came from.

TRAP

`if days_late > 0:` raises `ValueError: The truth value of an array ... is ambiguous`: an array of booleans isn't one boolean. Say which you mean with `.any()` or `.all()`. Likewise combine masks with `&`, `|`, `~` and parentheses, `(a > 0) & (a < 10)`, never `and` / `or`.

Axes and reductions

A **reduction** (`sum`, `mean`, `max`, `argmax`, `std`...) collapses an axis. `axis=0` runs down the rows and leaves one value per column; `axis=1` runs across and leaves one value per row. Here: loans at 3 branches (rows) over 4 months (columns):




```
1 loans = np.array([[5, 3, 8, 2],
2                   [1, 7, 4, 6],
3                   [9, 2, 3, 5]])
4 print(loans.sum(axis=0), loans.sum(axis=1), loans.sum())
5 print(loans.argmax(axis=1))                          # busiest month per branch
6 share = loans / loans.sum(axis=1, keepdims=True)     # (3, 4) / (3, 1)
7 print(share.round(2))
```

```
[15 12 15 13] [18 18 19] 55
[2 1 0]
[[0.28 0.17 0.44 0.11]
 [0.06 0.39 0.22 0.33]
 [0.47 0.11 0.16 0.26]]
```

line 6: keepdims leaves (3, 1), ready to broadcast

HONEST NOTE

Floating-point sums lose precision as they grow (Ch 11). `np.sum` uses pairwise summation, which is far more accurate than a naive loop, and `float64` is the safe default; `float32` accumulates error quickly. For an exact sum of Python floats there's `math.fsum`.

Random numbers, linear algebra, a plot

Make randomness **reproducible** with a seeded `Generator` (not the old `np.random.seed` global). Then let linear algebra fit a straight line through noisy data by least squares:

```
rng = np.random.default_rng(seed=2026)
copies = rng.integers(1, 6, size=40)                # copies of each title on the shelf
loans = 3 * copies + rng.normal(0, 2, size=40)      # the hidden rule, plus noise

A = np.column_stack([copies, np.ones(40)])          # rows: [copies, 1]
(slope, intercept), *_ = np.linalg.lstsq(A, loans)
print(f"loans ≈ {slope:.2f} × copies + {intercept:.2f}")

M, v = np.array([[2, 1], [1, 3]]), np.array([5, 10])
x = np.linalg.solve(M, v)                           # solve M @ x == v exactly
print(x, M @ x)
```

```
loans ≈ 2.77 × copies + 1.49
[1. 3.] [ 5. 10.]
```

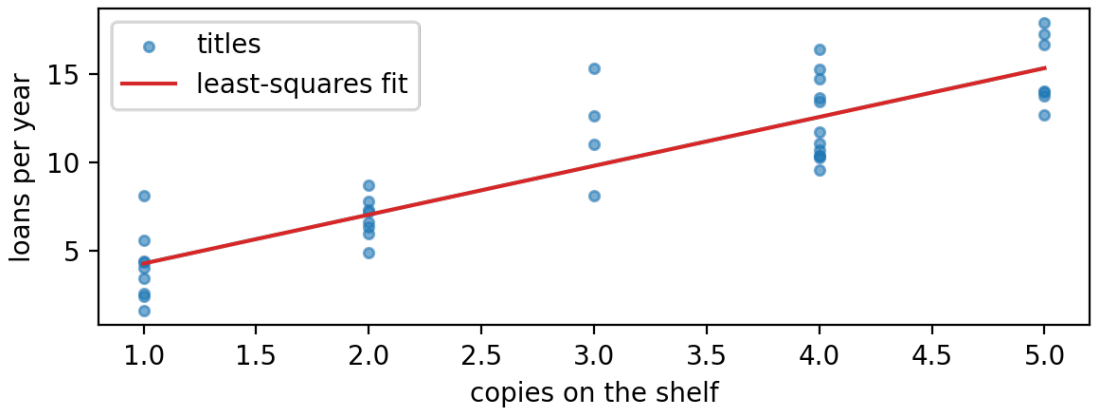
line 1: same seed, same numbers, every run

line 6: best slope and intercept

line 10: @ is matrix multiplication

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 2.4))
ax.scatter(copies, loans, s=14, alpha=0.6, label="titles")
xs = np.array([1, 5])
ax.plot(xs, slope * xs + intercept, color="C3", label="least-squares fit")
ax.set(xlabel="copies on the shelf", ylabel="loans per year")
ax.legend()
fig.tight_layout()                          # then plt.show() or fig.savefig("fit.png")
```



pandas, briefly: labelled columns on top of NumPy

pandas adds names: a DataFrame is a table whose columns are NumPy arrays with labels, missing-value handling and SQL-like operations (Ch 12's GROUP BY in one line):

```
1 import pandas as pd
2
3 df = pd.DataFrame({"branch": ["north", "south", "north", "east", "south", "north"],
4                    "title": ["Dune", "Emma", "Emma", "Dune", "Dune", "Ulysses"],
5                    "days": [14, 21, 7, 14, 28, 14]})
6 print(df.groupby("branch")["days"].agg(["count", "mean"]))
7 print(type(df["days"].to_numpy()).__name__, df["days"].to_numpy().dtype)
```

```
        count       mean
branch
east        1  14.000000
north       3  11.666667
south       2  24.500000
ndarray int64
```

line 6: GROUP BY branch

line 7: underneath: an ndarray

reach for	when
plain Python + csv/sqlite3	small data, mixed types, or logic per record
NumPy	numbers in a grid, the same operation on every element
pandas	tables with named, differently-typed columns: filter, group, join
Matplotlib	any plot; pandas' .plot() calls it for you

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 18.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 18.1` shows other valid solutions and their trade-offs.

18.1 build from a spec A fee table by broadcasting
18.2 fix the bug normalise_rows must not touch its input
18.3 build from a spec A moving average without loops
18.4 learn an unseen module Group without a loop: unique and bincount
18.5 measure and explain Nearest branch for 20,000 readers
18.6 mini-project Game of Life on a torus

PASSPORT STAMP ask the same question of every new language (Appendix B)

2. Memory model

An `ndarray` is one contiguous typed buffer plus a header (dtype, shape, strides). Slices are views that share the buffer; fancy and boolean indexing copy.

6. Data structures

`ndarray` (n-dimensional, one dtype), pandas `Series`/`DataFrame` (labelled columns).

12. Stdlib and tooling

Not stdlib: `numpy`, `matplotlib`, `pandas` from pip (Ch 19). The stdlib's `array` module and `statistics` cover small cases.

IN OTHER LANGUAGES

C: an array **is** this buffer, and a 2-D array is row-major exactly like NumPy's default. Fortran, MATLAB and Julia store columns first. R and MATLAB are vectorised by design. Java has primitive arrays but no broadcasting. Rust: the `ndarray` crate. GPUs (PyTorch, JAX) use the same shape-and-strides model.

DEEP DIVE (your vault)

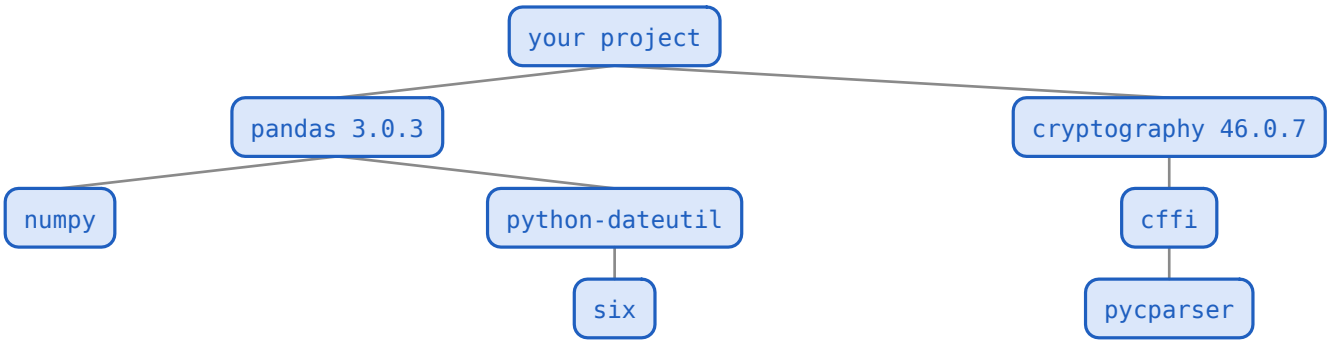
Vault notes: 16.5 Why an array abstraction exists · 16.6 Dtypes are representation contracts · 16.7 Shape, strides, and physical layout · 16.8 Views, copies, and aliasing · 16.9 Broadcasting as shape algebra · 16.10 Vectorisation and native kernels · 16.11 Reductions, axes, and numerical stability · 16.12 Randomness, generators, and reproducibility · 16.13 Linear algebra as array structure · 16.18 Grouping, aggregation, and transformation · 16.25 Visualisation is an encoded argument.

Chapter 19

Choosing pip packages safely

every pip install runs a stranger's code with your permissions

The standard library can't do everything: no friendly HTTP client, no symmetric encryption, no pytest. PyPI has over half a million packages for the rest. Each one you add brings its own dependencies, all written by people you've never met. Here is what two popular packages pull in, read from the metadata on the machine that built this page:




```
1 from importlib.metadata import requires
2 from packaging.requirements import Requirement
3
4 for line in requires("cryptography")[:4]:
5     r = Requirement(line)
6     keep = r.marker is None or r.marker.evaluate({"extra": ""})
7     print(f"{r.name:18} {str(r.specifier):10} {'needed' if keep else 'skipped'}")
```

```
cffi               >=1.14     skipped
cffi               >=2.0.0    needed
typing-extensions  >=4.13.2   skipped
bcrypt             >=3.1.5    skipped
```

line 6: markers: Python version, OS, extras

This chapter gives you a routine for the four decisions every project makes: **whether** to add a package, **which** one, **how** to pin it, and how to notice when the supply chain is attacked.

need	stdlib first	the usual package (and why)
HTTP client	`urllib.request` (Ch 15)	`requests` / `httpx`: sessions, retries
encryption	hashes + HMAC only (Ch 13)	`cryptography`: AES-GCM, Fernet
tests	`unittest` (Ch 17)	`pytest`: `assert`, fixtures, plugins

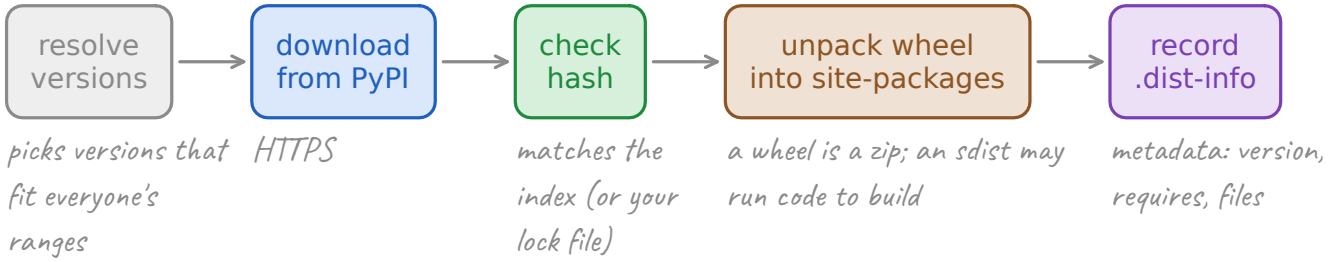
What venv and pip actually do

A **virtual environment** is just a folder with its own `site-packages` and a `python` that uses it, so each project gets its own package versions. Nothing is hidden; look inside one:

a real venv, created while building this page

```
$ python3 -m venv .venv
$ ls .venv
bin
include
lib
lib64
pyvenv.cfg
$ ls .venv/bin
Activate.ps1
activate
activate.csh
activate.fish
python
python3
python3.14
πthon
$ cat .venv/pyvenv.cfg
home = /usr/bin
include-system-site-packages = false
version = 3.14.6
executable = /usr/bin/python3.14
$ .venv/bin/python -c "import sys; print(sys.prefix != sys.base_prefix, sys.prefix)"
True ~/loans/.venv
```

(The `πthon` link is real: a joke alias for version 3.14.)
`pip install requests` inside it then does five things, and each one is a place where trust is needed:




```
1 import sys, sysconfig
2
3 print(sys.prefix == sys.base_prefix)           # True: this book runs outside any venv
4 print(sysconfig.get_paths()["purelib"])       # where pip would put packages
```

```
True
/usr/local/lib/python3.14/site-packages
```

line 3: inside a venv this is False

RULE OF THUMB

One venv per project, never `sudo pip install`, and never install into the system Python (many Linux distributions now refuse: PEP 668, "externally managed environment"). Commit `pyproject.toml` and the lock file, never the `.venv` folder itself (Ch 17's `.gitignore`).

Using a package well: requests

Apply the Ch 7 method to a third-party package exactly as to a stdlib module. A tiny local API to talk to (Ch 15's `http.server`), so this page needs no internet:

```
import json, threading
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

class Api(BaseHTTPRequestHandler):
    def do_GET(self):
        found = self.path == "/books/1"
        body = {"title": "Dune"} if found else {"error": "no such book"}
        self.send_response(200 if found else 404)
        self.send_header("Content-Type", "application/json")
        self.end_headers()
        self.wfile.write(json.dumps(body).encode())
    def log_message(self, *args):
        pass

server = ThreadingHTTPServer(("127.0.0.1", 0), Api)
threading.Thread(target=server.serve_forever, daemon=True).start()
base = f"http://127.0.0.1:{server.server_port}"
```

stdlib: urllib (Ch 15)

```
import json, urllib.request as ur
from urllib.error import HTTPError
try:
    r = ur.urlopen(f"{base}/books/9", timeout=5)
    print(json.load(r))
except HTTPError as e:                # 404 raises
    print(e.code, json.load(e))
```

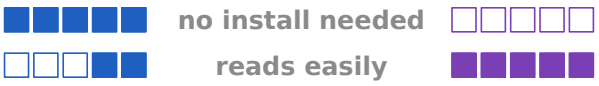
```
404 {'error': 'no such book'}
```

requests

```
import requests

r = requests.get(f"{base}/books/9", timeout=5)
print(r.status_code, r.json())
```

```
404 {'error': 'no such book'}
```



```python
import requests
with requests.Session() as s:                    # keeps connections open between calls
    s.headers["User-Agent"] = "loans/0.3"
    r = s.get(f"{base}/books/1", timeout=5)
    r.raise_for_status()                         # 4xx / 5xx -> requests.HTTPError
    print(r.json(), r.headers["Content-Type"])
    try:
        s.get(f"{base}/books/9", timeout=5).raise_for_status()
    except requests.HTTPError as e:
        print("HTTPError", e.response.status_code)
server.shutdown()
```

```
{'title': 'Dune'} application/json
HTTPError 404
```

line 3: always pass a timeout: the default is to wait forever

LIBRARY CARD **`requests`**

Job	HTTP client: send requests, get `Response` objects. Sync only (`httpx` adds async).
Core objects	`requests.get/post(url, params=, json=, headers=, timeout=)` → `Response`: `.status_code`, `.json()`, `.text`, `.content`, `.headers`, `.raise_for_status()`. `Session()` for many calls to one host.
Lifecycle	request → response; a `Session` is a context manager (`with`) that pools connections.
Errors	`requests.RequestException` ← `ConnectionError`, `Timeout`, `HTTPError` (only if you call `raise_for_status()`).
Traps	no default timeout · a 404 is not an exception · `verify=False` disables TLS checks · `.json()` on a non-JSON body raises.
Docs	requests.readthedocs.io · source: github.com/psf/requests

pytest and cryptography: two more cards

`pytest` runs the same `unittest` suites you already have, and also lets tests be plain functions using plain `assert`. Fixtures replace `setUp`; `parametrize` replaces `subTest` loops:

test_fees.py

```python
import sqlite3
import pytest
from fees import late_fee, DUE, back

@pytest.mark.parametrize("days, fee", [(-3, 0.0), (0, 0.0), (1, 0.25), (20, 5.0), (90, 5.0)])
def test_fee(days, fee):
    assert late_fee(DUE, back(days)) == fee        # failures show both values

@pytest.fixture
def db():
    conn = sqlite3.connect(":memory:")
    yield conn                                      # the test runs here
    conn.close()                                    # cleanup, even after a failure

def test_starts_empty(db):                          # asks for the fixture by name
    assert db.execute("SELECT 1").fetchone() == (1,)
```

```
$ python3 -m pip install pytest
$ python3 -m pytest -q
......                                                                   [100%]
6 passed in 0.04s
```

LIBRARY CARD — **pytest**

Job	Find and run tests (`test_*.py`, `test_*` functions, `unittest` classes too); report clearly.
Core objects	plain `assert` (rewritten to show values) · `@pytest.fixture` · `@pytest.mark.parametrize` · `pytest.raises(E)` · built-ins: `tmp_path`, `monkeypatch`, `capsys`.
Lifecycle	collect → set up fixtures → run → tear down (code after `yield`) → report.
Errors	a failed `assert` = test failure; an exception anywhere else = error; `-x` stops at the first.
Traps	a file not named `test_*.py` is silently skipped · module-scoped fixtures share state · too many plugins make runs mysterious.
Docs	docs.pytest.org · `pytest --fixtures` lists every fixture available

LIBRARY CARD	`cryptography`
Job	Vetted cryptography: Fernet and AES-GCM encryption, signatures, key exchange, X.509 certificates.
Core objects	`fernet.Fernet(key)`: `.encrypt`, `.decrypt` (Ch 13). Lower level: `hazmat.primitives.ciphers.aead.AESGCM`, `hazmat.primitives.asymmetric.ed25519`.
Lifecycle	generate or load a key → make the object → encrypt/sign. Keys live outside the code.
Errors	`InvalidToken` (Fernet), `InvalidTag` (AES-GCM), `InvalidSignature`: tampering or wrong key.
Traps	`hazmat` means hazardous: wrong nonce reuse breaks AES-GCM completely · never invent your own scheme · ships compiled Rust code, so install wheels, not source builds.
Docs	cryptography.io · the 'recipes' layer first, `hazmat` only when a protocol demands it

Is this package worth trusting?

Before `pip install`, spend five minutes on the project page and repository. None of these checks is proof; together they separate a well-run project from a risky one:

check	good sign	warning sign
Does the stdlib already do it?	no, or only painfully	yes: one dependency less to trust
Maintenance	releases this year, issues answered	last release years ago, piles of open PRs
People	several maintainers, an organisation	one anonymous account
Users	widely used, depended on by big projects	brand new, few downloads, name like a famous one
Dependencies	few, well-known	dozens for a small job
Security	advisories handled quickly, a SECURITY.md	no way to report problems
Licence	MIT, BSD, Apache-2.0 (permissive)	none at all (you may not use it), or GPL in closed code
Install	wheels for your platform	only an sdist that runs a build script

Version ranges, pins, hashes, lock files

A dependency line says **which versions** you accept. Wide ranges get bug fixes automatically but can change under you; exact pins are repeatable but go stale. The `packaging` library (what pip itself uses) shows what each specifier accepts:

```
from packaging.specifiers import SpecifierSet
from packaging.version import Version

releases = ["2.28.2", "2.31.0", "2.32.3", "2.32.5", "3.0.0"]
for spec in ["", ">=2.31", "~=2.32.0", ">=2.31,<3", "==2.32.5"]:
    ok = [v for v in releases if Version(v) in SpecifierSet(spec)]
    print(f"{spec or '(any)':10} accepts {ok}")
```

← ~=2.32.0 means >=2.32.0 and ==2.32.*

```
(any)      accepts ['2.28.2', '2.31.0', '2.32.3', '2.32.5', '3.0.0']
>=2.31     accepts ['2.31.0', '2.32.3', '2.32.5', '3.0.0']
~=2.32.0   accepts ['2.32.3', '2.32.5']
>=2.31,<3  accepts ['2.31.0', '2.32.3', '2.32.5']
==2.32.5   accepts ['2.32.5']
```

RULE OF THUMB

Libraries declare ranges in `pyproject.toml` (`requests>=2.31,<3`) so they combine with other libraries. **Applications** also commit a **lock file**: every package, direct and indirect, pinned to an exact version **and hash**, so every install is byte-for-byte the same.

A hash pins the exact file, not just the version number: if anyone replaces the file on the server, the install refuses. The idea in five lines of `hashlib` (Ch 13):

```
import hashlib

wheel = b"PK\x03\x04 ... the real wheel's bytes ..."
pinned = hashlib.sha256(wheel).hexdigest()           # recorded in the lock file
for blob in (wheel, wheel.replace(b"real", b"evil")):
    print("install" if hashlib.sha256(blob).hexdigest() == pinned else "REFUSE: hash mismatch")
```

```
install
REFUSE: hash mismatch
```

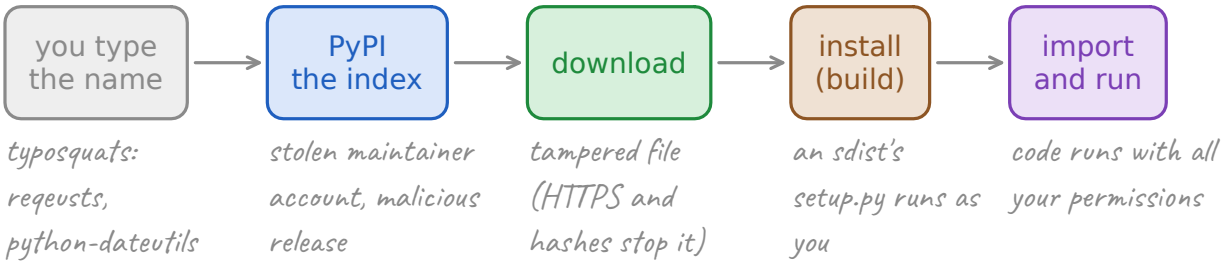
what a hashed lock file looks like (hashes illustrative)

```
$ cat requirements.txt
requests==2.32.5 \
    --hash=sha256:2462f94637a34fd532264295e186976db0f5d453d1cdd31473c85a6a161affb6
urllib3==2.6.3 \
    --hash=sha256:bf272323e553dfb2e87d9bfd225ca7b0f467b919d7bbd355436d3fd37cb0acd4
$ python3 -m pip install --require-hashes -r requirements.txt
```

tool	writes	notes
`pip-tools` (`pip-compile`)	`requirements.txt` with pins and hashes	small, classic, plain pip underneath
`uv`	`uv.lock`	very fast; manages venvs and Python versions too
Poetry / PDM / Hatch	their own lock files	whole project managers
the standard: PEP 751	`pylock.toml`	one lock format tools are converging on

Supply-chain attacks, and the checks that catch them

Attackers aim at the path between the author and your program. Every stage of an install has a known attack:




```
1  import difflib
2
3  popular = ["requests", "numpy", "pandas", "cryptography", "urllib3", "flask", "pytest"]
4  for name in ["reqeusts", "nunpy", "cryptograpy", "requests", "leftpad"]:
5      near = difflib.get_close_matches(name, popular, n=1, cutoff=0.8)
6      if name in popular:
7          verdict = "ok"
8      elif near:
9          verdict = f"did you mean {near[0]}?"
10     else:
11         verdict = "unknown: check it"
12     print(f"{name:12} {verdict}")
```

```
reqeusts     did you mean requests?
nunpy        did you mean numpy?
cryptograpy  did you mean cryptography?
requests     ok
leftpad      unknown: check it
```

line 5: a cheap typosquat alarm (practice 19.5)

a 2024 lock file checked against the PyPI advisory database (OSV)

```
$ python3 -m pip install pip-audit
$ pip-audit -r old-requirements.txt
Found 1 known vulnerability in 1 package
Name    Version ID                  Fix Versions
------- ------- ------------------ ------------
urllib3 2.2.1   GHSA-34jh-p97f-mpxf 1.26.19,2.2.2
```

defence	stops
copy names from the official docs; check the PyPI page	typosquats, look-alike names
lock file with hashes; `--require-hashes`	swapped files, surprise new versions
`pip install --only-binary :all:` where possible	build scripts running at install time
`pip-audit` in CI, then update	known vulnerabilities (only the known ones)
few dependencies; review new ones in code review	everything above, by shrinking the surface
prefer releases with trusted publishing / attestations	uploads from a stolen password

HONEST NOTE

No tool proves a package is safe. Audits only know about **reported** problems, and a clean-looking package can turn malicious in its next release. The strongest defence is the boring one: depend on fewer things, and know why each one is there.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 19.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 19.1` shows other valid solutions and their trade-offs.

`19.1` build from a spec Read a requirement line
`19.2` fix the bug Does this version satisfy the specifier?
`19.3` build from a spec Check downloads against a hashed lock file
`19.4` learn an unseen module Explore importlib.metadata: what is installed, and what does it need?
`19.5` fix the bug A typosquat alarm that knows its names
`19.6` mini-project Mini pip-audit

PASSPORT STAMP ask the same question of every new language (Appendix B)

9. Modules and build

`venv` per project; `pip` resolves, downloads, checks hashes and unpacks wheels into `site-packages`; ranges in `pyproject.toml`, exact pins and hashes in a lock file.

12. Stdlib and tooling

Stdlib: `venv`, `importlib.metadata`, `hashlib`, `difflib`. Third-party staples: `requests`, `pytest`, `cryptography`, `packaging`, `pip-audit`.

IN OTHER LANGUAGES

Every ecosystem has the same parts. JavaScript: npm, `package.json` + `package-lock.json`, and the most famous typosquats. Rust: crates.io, `Cargo.toml` + `Cargo.lock`, `cargo audit`. Java: Maven Central, `pom.xml`. Go: `go.mod` + `go.sum` (hashes built in). C/C++ often vendor code by hand, which is a supply chain too.

DEEP DIVE (your vault)

Vault notes: 19.14 Dependency and supply-chain risk · 19.28 Vulnerability management · 19.17 Cryptographic hash functions · 19.4 Trust and least privilege · 16.28 Reproducible computational environments · 12.5 pytest fundamentals · 12.6 Fixtures and lifetime scopes · 12.21 Continuous integration.

Chapter 20

Cost, recursion, links and trees

count the steps, and the seconds take care of themselves

A faster computer buys you a constant factor. A better algorithm changes the **shape** of the cost. Here is one question, "does this list contain a duplicate?", answered three ways, with the steps each one takes counted rather than timed:

```
import random
from functools import cmp_to_key

def by_pairs(xs):                   # compare every pair
    steps = 0
    for i, a in enumerate(xs):
        for b in xs[i + 1:]:
            steps += 1
            if a == b:
                return steps
    return steps

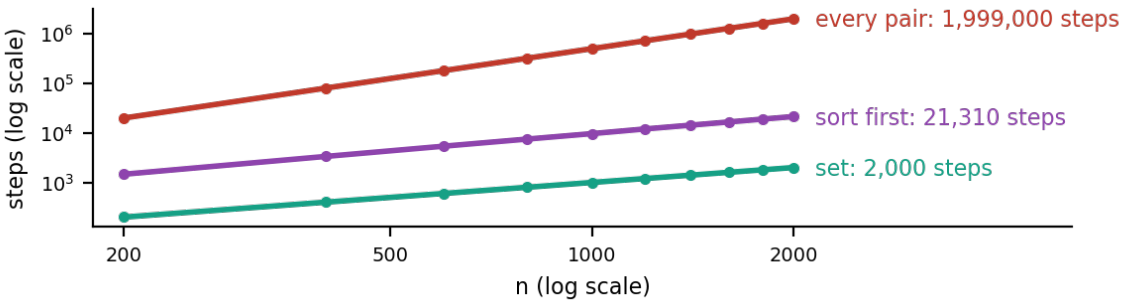
def by_sorting(xs):                 # sort, then compare neighbours
    steps = 0
    def cmp(a, b):
        nonlocal steps
        steps += 1
        return (a > b) - (a < b)
    sorted(xs, key=cmp_to_key(cmp))
    return steps + len(xs) - 1

def by_set(xs):                     # one hash lookup per item
    return len(xs)

rng = random.Random(1)
for n in (250, 500, 1000, 2000):
    xs = rng.sample(range(10 * n), n)          # no duplicates: the worst case
    print(f"{n:>5} {by_pairs(xs):>10,} {by_sorting(xs):>7,} {by_set(xs):>6,}")
```

```
  250     31,125   1,911    250
  500    124,750   4,339    500
 1000    499,500   9,653  1,000
 2000  1,999,000  21,307  2,000
```

line 17: counts each comparison Timsort makes



Double n and the pair-checker does **four times** the work, sorting a bit more than twice, the set exactly twice. On log scales that shows up as the **slope**: 2 for n^2, 1 for n. That ratio, not the seconds on one machine, is what Big-O describes.

Big-O: keep the term that wins

Big-O names how the steps grow as n grows, ignoring constant factors and smaller terms. `3n² + 50n + 7` is O(n^2) because for large n the n^2 term is all that matters:

class	example	n = 10	n = 1,000	n = 1,000,000
`O(1)`	dict lookup	`1`	`1`	`1`
`O(log n)`	binary search	`3`	`10`	`20`
`O(n)`	one loop	`10`	`1,000`	`1,000,000`
`O(n log n)`	sorting	`33`	`9,966`	`≈ 10^7`
`O(n²)`	every pair	`100`	`1,000,000`	`≈ 10^12`
`O(2ⁿ)`	every subset	`1,024`	`≈ 10^301`	`≈ 10^301,030`

RULE OF THUMB

Steps in sequence **add** (keep the biggest). Loops **multiply**: a loop of n around an O(n) body is O(n^2). A **call counts with its own cost**: `x in some_list` inside a loop is a hidden inner loop. Halving the problem each step gives a log n.

What Python's own operations cost

structure	O(1)	O(n) or worse
`list`	`xs[i]`, `xs[i] = v`, `append`, `pop()`, `len`	`insert(0, v)`, `pop(0)`, `x in xs`, `remove`, `xs[a:b]` copies, `sort` O(n log n)
`dict` / `set`	`d[k]`, `k in d`, `d[k] = v`, `del d[k]` (on average)	iterating; a worst case of O(n) when many keys collide
`deque`	`append`, `appendleft`, `pop`, `popleft`	`dq[i]` near the middle
`str`	`s[i]`, `len(s)`	`s + t` copies both; `+=` in a loop can be O(n^2): use `"".join`

Measure to check a claim: double n and look at the ratio of the times. About 2 means linear, about 4 means quadratic:

```
import timeit

for fn in (by_pairs, by_set):
    times = []
    for n in (500, 1000, 2000):
        xs = list(range(n))
        times.append(min(timeit.repeat(lambda: fn(xs), number=1, repeat=3)))
    ratios = [f"x{b / a:.1f}" for a, b in zip(times, times[1:])]
    print(f"{fn.__name__:9} each doubling: {' '.join(ratios)}")
```

```
by_pairs  each doubling: x4.2 x4.0
by_set    each doubling: x0.7 x0.9
```

line 7: best of 3 runs

HONEST NOTE

Big-O hides constants, and constants matter at small n: `sorted()` on 20 items beats any clever structure. It also hides the machine: memory locality (Ch 18) and C-level loops can make an O(n) built-in faster than an O(log n) pure-Python one for every n you'll actually meet. Use Big-O to rule out designs, then measure (Ch 17).

Amortised: why append is O(1) on average

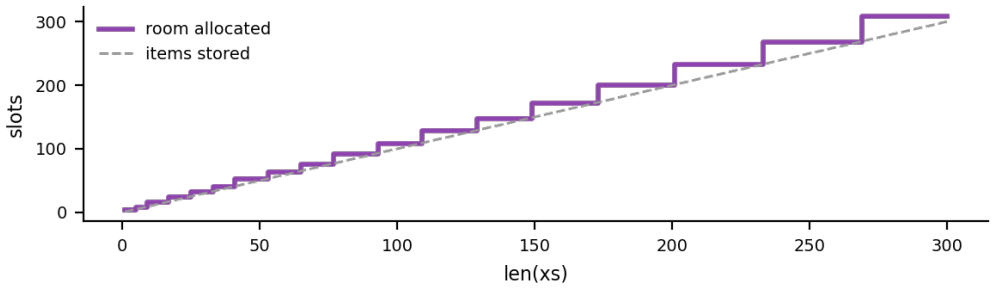
A list is an array of pointers with spare room at the end. When the room runs out, CPython allocates a bigger block and copies everything across: an O(n) step. It doesn't happen often, because each new block is about 1/8 larger than needed plus a little:

```
import sys

xs, last = [], sys.getsizeof([])
for i in range(40):
    xs.append(i)
    size = sys.getsizeof(xs)
    if size != last:                        # a new, bigger block was allocated
        room = (size - sys.getsizeof([])) // 8
        print(f"len {len(xs):>2}: room for {room} items")
        last = size
```

```
len  1: room for 4 items
len  5: room for 8 items
len  9: room for 16 items
len 17: room for 24 items
len 25: room for 32 items
len 33: room for 40 items
```

line 8: 8 bytes per pointer



KEY IDEA

Amortised cost spreads the rare expensive step over the many cheap ones. Because the block grows by a fixed **fraction**, the copying adds up to a constant number of moves per append, however long the list gets. Growing by a fixed **amount** instead (say +10 each time) would make append O(n) on average.

Recursion: a function that trusts itself

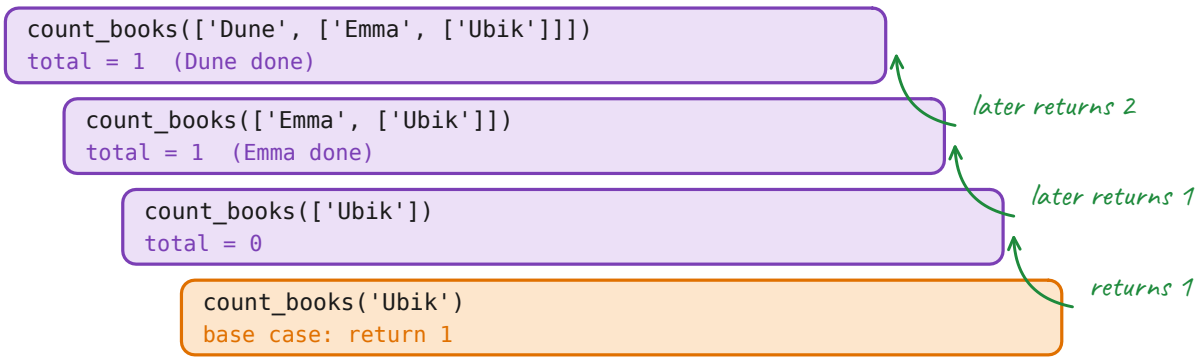
A recursive function solves a problem by solving **smaller** copies of the same problem, and stops at a **base case** small enough to answer directly. Nested data is the natural fit: a shelf holds books and boxes, and a box is a smaller shelf.

```
def count_books(shelf):
    if isinstance(shelf, str):          # base case: a single book
        return 1
    total = 0
    for item in shelf:                  # each item is a smaller shelf
        total += count_books(item)
    return total

print(count_books(["Dune", ["Emma", ["Ubik"]]]))
```

```
3
```

the call stack at its deepest (each call has its own frame and its own total)



RULE OF THUMB

Write recursion in three checks: (1) the base case returns without recursing, (2) every recursive call gets a **strictly smaller** input, (3) assuming the smaller calls are right, this call combines them correctly. That last step is proof by induction (13.5 Correctness through invariants and induction).

Each call is a new **frame** on the call stack (Ch 2), holding its own local variables, so the four `total`s never interfere. Python limits the stack's depth, and has no tail-call optimisation, so deep data needs a loop with an explicit stack instead:

```python
import sys

def count_books_loop(shelf):
    stack, total = [shelf], 0
    while stack:
        item = stack.pop()
        if isinstance(item, str):
            total += 1
        else:
            stack.extend(item)          # look inside this box later
    return total

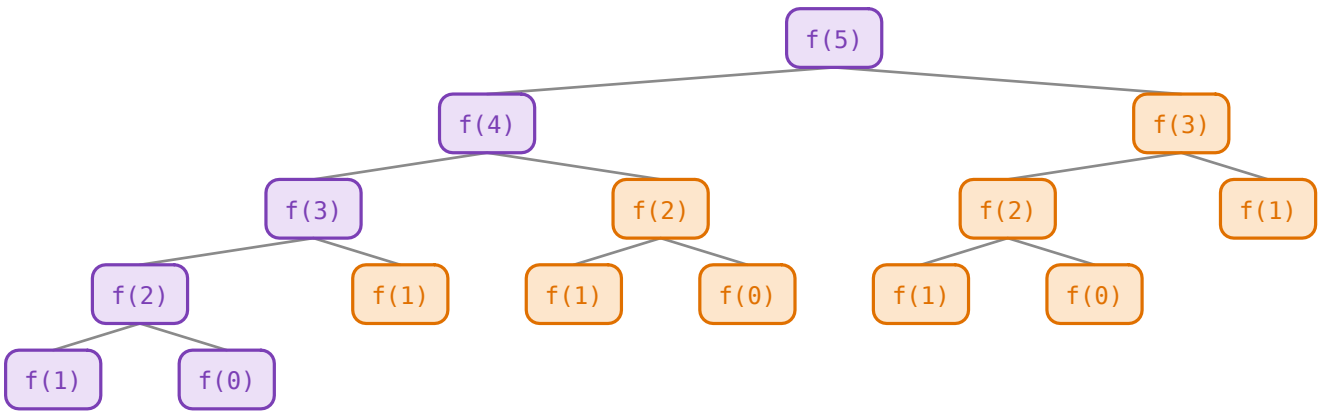
deep = "Dune"
for _ in range(5000):
    deep = [deep]                       # a book inside 5000 boxes
print(sys.getrecursionlimit(), count_books_loop(deep))
try:
    count_books(deep)
except RecursionError as e:
    print("RecursionError:", e)
```

← *the stack is now a list we own*

```
1000 1
RecursionError: maximum recursion depth exceeded
```

Memoisation: never solve the same problem twice

The textbook Fibonacci definition calls itself twice, and the two calls overlap. Drawn out, `fib(5)` recomputes the same values again and again (orange):



```python
from functools import cache

calls = 0
def fib(n):
    global calls
    calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)

print(fib(20), calls)                    # exponential: about 1.6^n calls

@cache                                   # remember every answer by its argument
def fast(n):
    return n if n < 2 else fast(n - 1) + fast(n - 2)

print(fast(20), fast.cache_info())
print(len(str(fast(500))), "digits, instantly")
```

```
6765 21891
6765 CacheInfo(hits=18, misses=21, maxsize=None, currsize=21)
105 digits, instantly
```

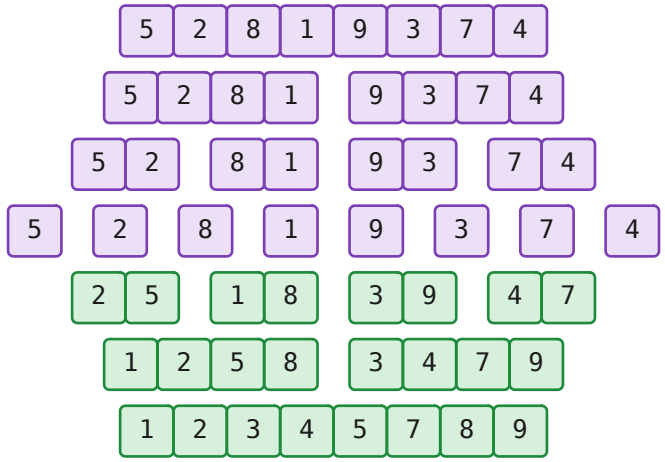
line 7: 9 of fib(5)'s 15 calls are repeats

KEY IDEA

`@cache` (or `@lru_cache(maxsize=N)` to bound memory) stores results in a dict keyed by the arguments, so the arguments must be hashable. Overlapping subproblems plus caching is the core of **dynamic programming**, Ch 21.

Divide and conquer: merge sort

Split the list in half, sort each half (recursively), then **merge** two sorted lists by repeatedly taking the smaller front item. Splitting takes $\log_2 n$ levels, and every level merges all n items:



split: halve until each piece has one item ($\log_2 8 = 3$ levels)

merge: each level walks all 8 items once, so n × log n

```python
def merge_sort(xs):
    if len(xs) <= 1:
        return xs
    mid = len(xs) // 2
    left, right = merge_sort(xs[:mid]), merge_sort(xs[mid:])
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            out.append(left[i]); i += 1
        else:
            out.append(right[j]); j += 1
    return out + left[i:] + right[j:]

print(merge_sort([5, 2, 8, 1, 9, 3, 7, 4]))
loans = [("ana", 3), ("ben", 1), ("cy", 3), ("di", 1)]
print(sorted(loans, key=lambda p: p[1]))       # stable: ana stays before cy
```

```
[1, 2, 3, 4, 5, 7, 8, 9]
[('ben', 1), ('di', 1), ('ana', 3), ('cy', 3)]
```

line 8: <= keeps equal items in order

line 12: one side may have leftovers

WHY

Python's `sorted` is **Timsort**: merge sort that first finds runs already in order, so nearly-sorted data sorts in close to O(n). Like this merge sort it is **stable**: equal keys keep their order, which lets you sort by one key and then another. No comparison sort can beat O(n log n) in the worst case (13.8 Comparison sorting).

Invariants: binary search you can trust

Binary search is famously easy to get almost right. The cure is to write down the **invariant**, a fact that is true before every loop step, and check that each line keeps it true:

```
1  def first_at_least(xs, target):
2      lo, hi = 0, len(xs)          # invariant: the answer is in range(lo, hi + 1)
3      while lo < hi:
4          mid = (lo + hi) // 2
5          if xs[mid] < target:
6              lo = mid + 1         # xs[mid] and everything left of it are too small
7          else:
8              hi = mid             # xs[mid] might be the answer: keep it
9      return lo                    # lo == hi: the range has one candidate left
10
11 ids = [3, 8, 15, 21, 27, 34, 40, 52, 66, 71]
12 print(first_at_least(ids, 40), first_at_least(ids, 41), first_at_least(ids, 99))
```

```
6 7 10
```

line 2: hi = len(xs): 'past the end' is a valid answer

line 4: the range shrinks every step

lo	hi	mid	xs[mid]	step
0	10	5	34	34 < 40: lo = 6
6	10	8	66	66 ≥ 40: hi = 8
6	8	7	52	52 ≥ 40: hi = 7
6	7	6	40	40 ≥ 40: hi = 6
6	6			stop: answer 6

TRAP

Mixing conventions breaks it: `hi = len(xs) - 1` with `while lo < hi` misses the last item; `hi = mid - 1` throws away a possible answer; `lo = mid` (without + 1) loops forever when `hi == lo + 1`. Pick one invariant and derive every line from it. Or just use `bisect_left`, which is exactly this function (Ch 10).

Linked lists from scratch

A linked list stores each item in its own small object, a **node**, that holds the value and a reference to the next node. There's no block to grow and nothing to shift: adding at the front is always O(1). Finding the 500th item means following 500 references.

```python
class Node:
    __slots__ = ("value", "next")            # no per-node __dict__: smaller nodes
    def __init__(self, value, next=None):
        self.value, self.next = value, next

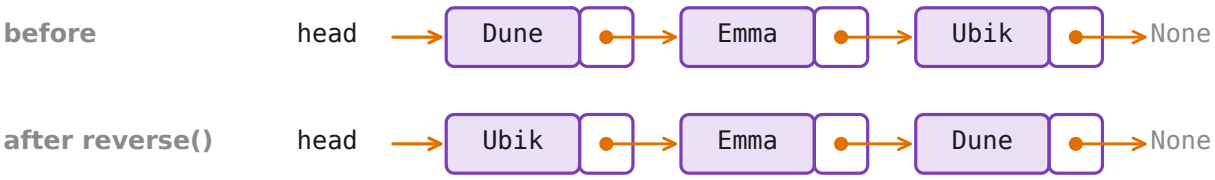
def from_items(items):
    head = None
    for v in reversed(items):
        head = Node(v, head)                 # O(1) push at the front
    return head

def items(node):
    while node is not None:
        yield node.value
        node = node.next

def reverse(node):
    prev = None
    while node is not None:
        nxt = node.next                      # remember the rest
        node.next = prev                     # turn this arrow around
        prev, node = node, nxt
    return prev

head = from_items(["Dune", "Emma", "Ubik"])
print(list(items(head)))
head = reverse(head)
print(list(items(head)))
```

```
['Dune', 'Emma', 'Ubik']
['Ubik', 'Emma', 'Dune']
```



operation	Python list (array)	linked list
get item i	O(1): address arithmetic	**O(n)**: follow i references
add/remove at the front	**O(n)**: shift everything	O(1): one new node
insert after a node you hold	**O(n)**	O(1): relink two references
memory per item	8 bytes (one pointer)	a whole node object (48 bytes, `sys.getsizeof`)

HONEST NOTE

In Python you'll rarely write one: `list` and `deque` (which is itself a doubly linked list of 64-item blocks) are faster in practice. Linked structures matter because **trees and graphs are made of the same nodes and references**, and because in C and Rust you build them by hand. An LRU cache (practice 20.7) is the classic case where they win.

Binary search trees

Give each node **two** references and keep one rule: everything in the left subtree is smaller, everything in the right subtree is larger. Then searching is binary search on a structure you can insert into, and an in-order walk visits the keys sorted:

```python
from dataclasses import dataclass

@dataclass(slots=True)
class TreeNode:
    key: int
    left: "TreeNode | None" = None
    right: "TreeNode | None" = None

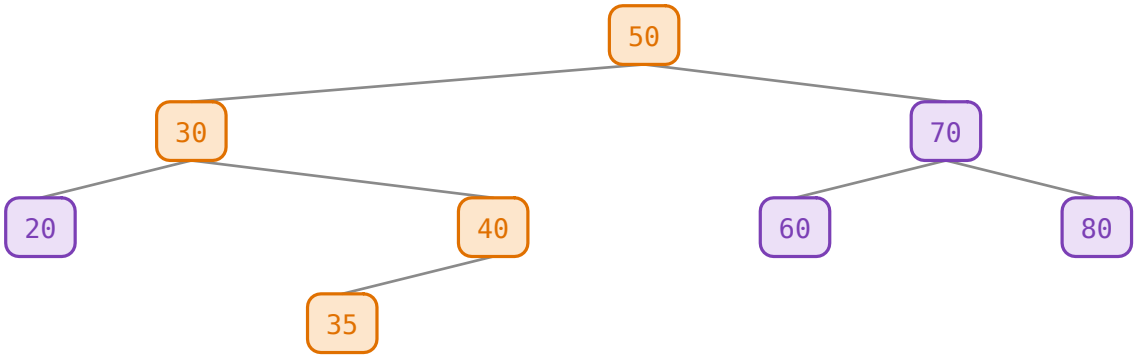
def insert(node, key):
    if node is None:
        return TreeNode(key)
    if key < node.key:
        node.left = insert(node.left, key)
    elif key > node.key:
        node.right = insert(node.right, key)
    return node                                # duplicates are ignored

def contains(node, key):
    while node is not None:                    # searching needs no recursion
        if key == node.key:
            return True
        node = node.left if key < node.key else node.right
    return False

def inorder(node):
    if node is not None:
        yield from inorder(node.left)
        yield node.key
        yield from inorder(node.right)

root = None
for k in [50, 30, 70, 20, 40, 60, 80, 35]:
    root = insert(root, k)
print(list(inorder(root)), contains(root, 35), contains(root, 36))
```

```
[20, 30, 35, 40, 50, 60, 70, 80] True False
```



Searching for 35 visits only the orange path: 50, 30, 40, 35. Each step discards a whole subtree, so a **balanced** tree of n keys needs about $\log_2 n$ steps. The four classic walks visit the same nodes in different orders:

```python
from collections import deque

def preorder(n):
    if n:
        yield n.key; yield from preorder(n.left); yield from preorder(n.right)

def postorder(n):
    if n:
        yield from postorder(n.left); yield from postorder(n.right); yield n.key

def level_order(root):
    queue = deque([root])
    while queue:
        n = queue.popleft()
        if n:
            yield n.key
            queue.extend((n.left, n.right))

print(list(preorder(root)), list(postorder(root)), list(level_order(root)), sep="\n")
```

```
[50, 30, 20, 40, 35, 70, 60, 80]
[20, 35, 40, 30, 60, 80, 70, 50]
[50, 30, 70, 20, 40, 60, 80, 35]
```

walk	order	typical use
in-order	left, node, right	keys in sorted order
pre-order	node, then children	copy or print a tree (parents before children)
post-order	children, then node	delete or free a tree; sizes and heights from the bottom up
level order	row by row (a queue)	shortest path in steps: BFS, Ch 21

The catch: insertion order decides the shape

```python
import random

def height(node):
    return 0 if node is None else 1 + max(height(node.left), height(node.right))

keys = list(range(500))
shuffled = random.Random(20).sample(keys, len(keys))
for name, order in [("random order", shuffled), ("sorted order", keys)]:
    root = None
    for k in order:
        root = insert(root, k)
    print(f"{name}: height {height(root)} for {len(keys)} keys")
```

```
random order: height 24 for 500 keys
sorted order: height 500 for 500 keys
```

line 8: same keys, two insertion orders

HONEST NOTE

Sorted input turns the tree into a linked list: O(n) per search. Real libraries use **self-balancing** trees (red-black, AVL) or **B-trees**, which is what SQLite indexes are (Ch 12). Python's standard library has none: use a `dict`/`set` for lookups, `bisect` on a sorted list for ranges, `heapq` for priorities, or the `sortedcontainers` package (Ch 19).

▢ Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 20.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 20.1` shows other valid solutions and their trade-offs.

20.1 measure and explain Members on both lists, fast
20.2 build from a spec Flatten anything, however deep
20.3 fix the bug Binary search, by the invariant
20.4 build from a spec A linked list with a tail
20.5 build from a spec A binary search tree that survives sorted input
20.6 refactor, tests stay green Count inversions in O(n log n)
20.7 mini-project An O(1) LRU cache

PASSPORT STAMP ask the same question of every new language (Appendix B)

2. Memory model

Linked structures are objects holding references to other objects; `__slots__` shrinks each node. A list is a growable array of pointers with spare room (amortised O(1) append).

5. Functions and scope

Recursion uses one stack frame per call; the default limit is 1000 and there is no tail-call optimisation, so deep data needs an explicit stack. `functools.cache` memoises.

6. Data structures

Know the costs: list O(1) at the end, O(n) at the front; dict/set O(1) on average; deque O(1) at both ends; heap O(log n); no balanced tree in the stdlib.

IN OTHER LANGUAGES

C: a node is a `struct` with a pointer field, and you `malloc`/`free` every node yourself. Java: references like Python, and `TreeMap` is a red-black tree in the standard library. C++: `std::map` is a balanced tree, `std::list` a linked list. Rust: `Option<Box<Node>>`, and ownership makes doubly linked lists famously hard; use `BTreeMap`. Scheme guarantees tail calls; Python and Java never eliminate them, and C compilers only sometimes do.

DEEP DIVE (your vault)

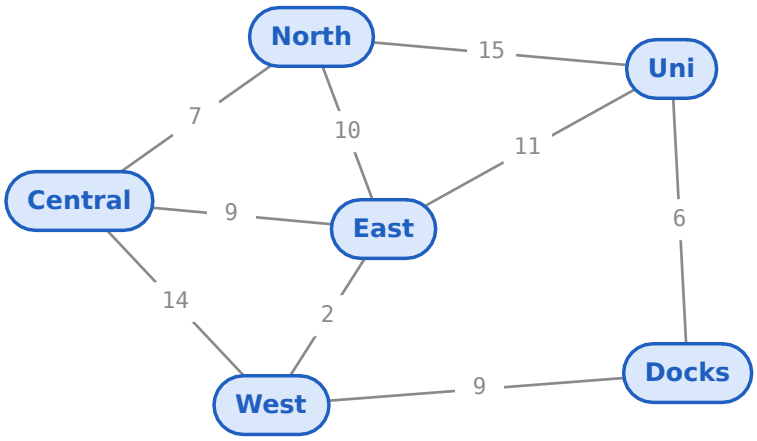
Vault notes: 13.1 A cost claim begins with a model · 13.2 Asymptotic notation · 13.3 Common growth classes · 13.4 Amortised analysis · 13.5 Correctness through invariants and induction · 13.7 Binary search · 13.8 Comparison sorting · 13.9 Python sorting and Timsort · 13.10 Linked lists built from scratch · 13.12 Binary search trees.

Chapter 21

Graphs and dynamic programming

most hard problems are a graph or a table in disguise

A **graph** is things (nodes) and connections (edges): branches and courier routes, web pages and links, modules and prerequisites, people and friendships. Once a problem is drawn as a graph, a handful of algorithms answer most questions about it. Our example: six library branches and the minutes a courier needs between them.



undirected: routes work both ways

weighted: minutes on every edge

6 nodes, 9 edges

```python
from collections import defaultdict

ROUTES = '''Central North 7   Central East 9   Central West 14
            North East 10     North Uni 15     East West 2
            East Uni 11       West Docks 9     Uni Docks 6'''
words = ROUTES.split()
routes = [(a, b, int(m)) for a, b, m in zip(words[::3], words[1::3], words[2::3])]

graph = defaultdict(list)                # adjacency list: node -> [(neighbour, minutes)]
for a, b, minutes in routes:
    graph[a].append((b, minutes))
    graph[b].append((a, minutes))        # undirected: store both directions
print(graph["East"])
```

```
[('Central', 9), ('North', 10), ('West', 2), ('Uni', 11)]
```

store it as	space	neighbours of u	is u–v an edge?	best for
adjacency list (dict of lists)	V + E	O(degree)	O(degree)	almost everything
adjacency matrix (V×V grid)	V^2	O(V)	O(1)	dense graphs, NumPy maths
edge list	E	O(E)	O(E)	input files, sorting edges

Breadth-first search: fewest steps

BFS explores in rings: first every neighbour of the start, then their neighbours, and so on. A **queue** (first in, first out) keeps the rings in order, so the first time BFS reaches a node is by the **fewest edges**:

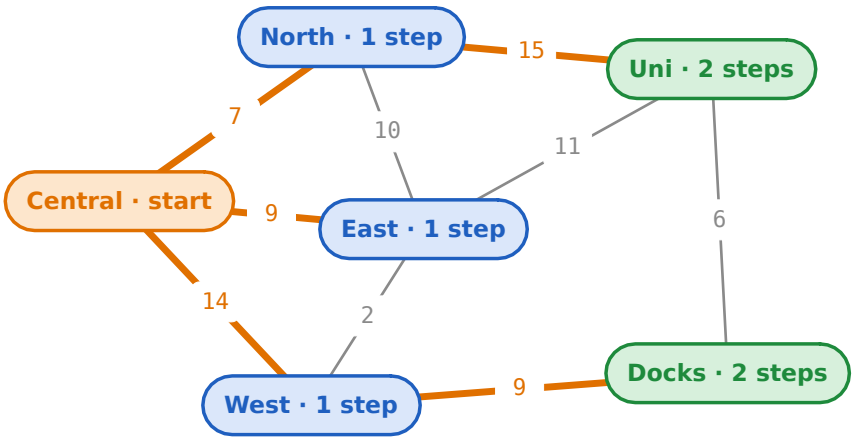
```python
from collections import deque

def bfs(graph, start):
    dist, prev = {start: 0}, {}
    queue = deque([start])
    while queue:
        u = queue.popleft()
        for v, _ in graph[u]:
            if v not in dist:               # first visit = fewest steps
                dist[v], prev[v] = dist[u] + 1, u
                queue.append(v)
    return dist, prev

def path(prev, end):
    out = [end]
    while out[-1] in prev:                  # walk the 'came from' links back
        out.append(prev[out[-1]])
    return out[::-1]

dist, prev = bfs(graph, "Central")
print(dist)
print(path(prev, "Docks"))
```

```
{'Central': 0, 'North': 1, 'East': 1, 'West': 1, 'Uni': 2, 'Docks': 2}
['Central', 'West', 'Docks']
```



thick: how BFS first reached each node

colour: steps from Central

KEY IDEA

BFS ignores the weights: Central → West → Docks is 2 steps but 23 minutes, while the quickest route has 3 steps. Each node and edge is handled once: O(V + E). BFS is the right tool whenever every step costs the same: mazes, word ladders, degrees of separation, the level-order walk of Ch 20.

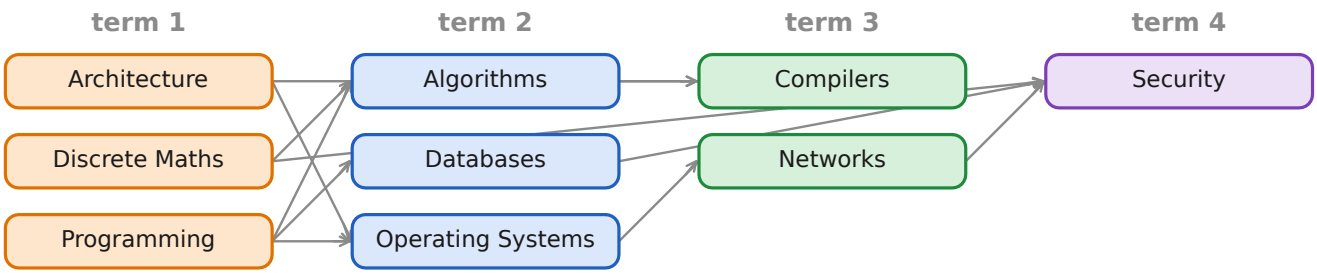
Depth-first search and a safe order of work

Swap the queue for a **stack** and the search goes as deep as it can before backing up. That's DFS: the tool for reachability, cycles and ordering. Module prerequisites are a **directed** graph (an edge means 'needs'), and a valid study plan is a **topological order**: every module after the ones it needs. The stdlib's `graphlib` does it, and can even hand out modules term by term:

```
1  from graphlib import TopologicalSorter, CycleError
2
3  needs = {                                          # module: the modules it needs first
4      "Algorithms": ["Programming", "Discrete Maths"],
5      "Databases": ["Programming"],
6      "Operating Systems": ["Architecture", "Programming"],
7      "Networks": ["Operating Systems"],
8      "Security": ["Networks", "Databases", "Discrete Maths"],
9      "Compilers": ["Algorithms", "Architecture"],
10 }
11 ts = TopologicalSorter(needs)
12 ts.prepare()
13 terms = []
14 while ts.is_active():
15     ready = sorted(ts.get_ready())                 # everything whose needs are all done
16     terms.append(ready)
17     ts.done(*ready)
18 for k, ready in enumerate(terms, 1):
19     print(f"term {k}: {', '.join(ready)}")
```

```
term 1: Architecture, Discrete Maths, Programming
term 2: Algorithms, Databases, Operating Systems
term 3: Compilers, Networks
term 4: Security
```

line 12: raises CycleError if there's a loop



```python
def depends_on(graph, start):
    seen, stack = set(), [start]
    while stack:
        u = stack.pop()                          # a stack: go deep first
        for v in graph.get(u, []):
            if v not in seen:
                seen.add(v)
                stack.append(v)
    return seen

print(sorted(depends_on(needs, "Networks")))
needs["Programming"] = ["Security"]              # a circular requirement
try:
    list(TopologicalSorter(needs).static_order())
except CycleError as e:
    print("cycle:", " -> ".join(e.args[1]))
```

```
['Architecture', 'Operating Systems', 'Programming']
cycle: Programming -> Databases -> Security -> Programming
```

WHY

Topological order exists only when there is no cycle (a DAG: directed acyclic graph). The same idea orders build steps (make), package installs (pip's resolver, Ch 19), spreadsheet cells and database migrations. A DFS-based version is practice 21.2.

Dijkstra: cheapest paths with a heap

With weights, the fewest steps isn't the quickest route. Dijkstra's algorithm grows a set of **settled** nodes whose best time is final, always settling the unsettled node with the smallest known time next. A heap (Ch 10) finds that node in O(log n):

```python
import heapq

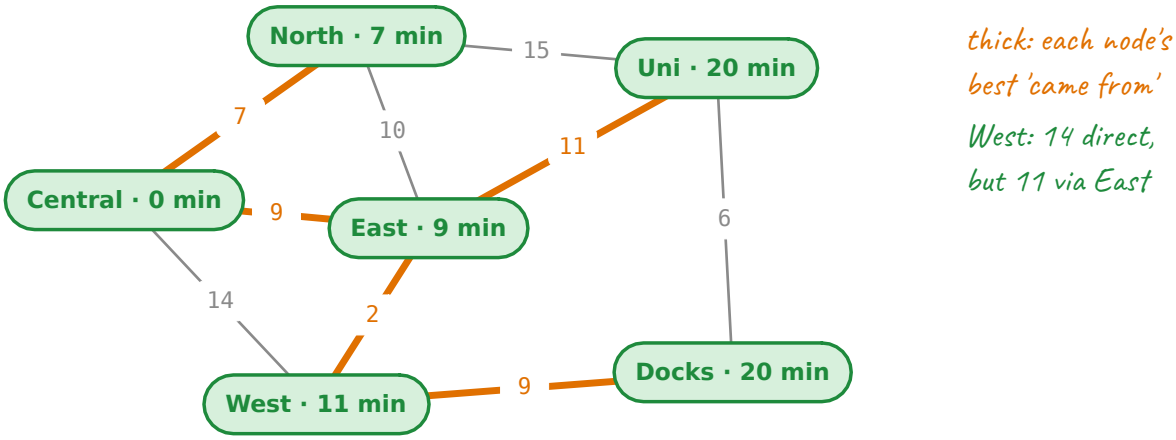
def dijkstra(graph, start):
    dist, prev = {start: 0}, {}
    heap = [(0, start)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist[u]:
            continue                     # stale: u was already settled more cheaply
        for v, minutes in graph[u]:
            new = d + minutes
            if new < dist.get(v, float("inf")):
                dist[v], prev[v] = new, u
                heapq.heappush(heap, (new, v))
    return dist, prev

best, came_from = dijkstra(graph, "Central")
print(path(came_from, "Docks"), best["Docks"], "minutes")
```

```
['Central', 'East', 'West', 'Docks'] 20 minutes
```

line 12: found a quicker way to v

popped	action	improved (pushed)
(0, Central)	settle Central at 0	North 7, East 9, West 14
(7, North)	settle North at 7	Uni 22
(9, East)	settle East at 9	West 11, Uni 20
(11, West)	settle West at 11	Docks 20
(14, West)	stale: skip it	(West was settled at 11)
(20, Docks)	settle Docks at 20	nothing better
(20, Uni)	settle Uni at 20	nothing better
(22, Uni)	stale: skip it	(Uni was settled at 20)



TRAP

Dijkstra is only correct with **non-negative** weights: it assumes a settled node can't get cheaper later. With negative edges use Bellman-Ford (O(V·E)). The `if d > dist[u]: continue` line is the price of not having a decrease-key operation on `heapq`: old entries are skipped when they surface. Cost: O((V + E) log V).

Dynamic programming: from recursion to a table

Dynamic programming is Ch 20's memoisation, applied on purpose: describe the answer in terms of answers to **smaller versions** of the same question, and make sure each smaller answer is computed once. How many ways can you pay 100p with 1p, 2p and 5p coins?

top-down: recursion + cache

```python
from functools import cache

COINS = (1, 2, 5)

@cache
def count(amount, i=0):
    if amount == 0:
        return 1
    if amount < 0 or i == len(COINS):
        return 0
    use = count(amount - COINS[i], i)
    skip = count(amount, i + 1)
    return use + skip

print(count(100))
```

```
541
```

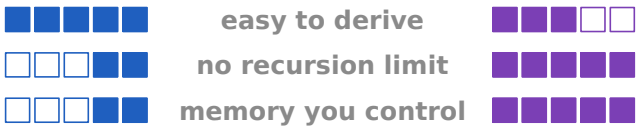
bottom-up: fill a table

```python
COINS = (1, 2, 5)

ways = [1] + [0] * 100     # index = amount
for c in COINS:
    for a in range(c, 101):
        ways[a] += ways[a - c]

print(ways[100])
```

```
541
```



step	question to answer	coins example
1. state	what does a subproblem need to know?	amount left, which coins are still allowed
2. recurrence	how do smaller answers combine?	use coin i again + skip coin i
3. base cases	which answers are known outright?	amount 0: 1 way; no coins left: 0 ways
4. order	what must be computed before what?	smaller amounts first (the table's loop order)
5. answer	which cell holds the result?	`count(100)` / `ways[100]`

A DP table you can see: edit distance

How many single-letter edits turn one word into another? It's the typosquat alarm of Ch 19 and every spell checker. Each cell answers a smaller question, and the table fills row by row:

```python
def edit_table(a, b):
    d = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(len(a) + 1):
        d[i][0] = i                                  # delete all i letters
    for j in range(len(b) + 1):
        d[0][j] = j                                  # insert all j letters
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            d[i][j] = min(d[i - 1][j] + 1,                              # delete a[i-1]
                          d[i][j - 1] + 1,                              # insert b[j-1]
                          d[i - 1][j - 1] + (a[i - 1] != b[j - 1]))  # keep/replace
    return d

d = edit_table("kitten", "sitting")
print(d[-1][-1])
```

```
3
```

		s	i	t	t	i	n	g
	0	1	2	3	4	5	6	7
k	1	1	2	3	4	5	6	7
i	2	2	1	2	3	4	5	6
t	3	3	2	1	2	3	4	5
t	4	4	3	2	1	2	3	4
e	5	5	4	3	2	2	3	4
n	6	6	5	4	3	3	2	3

cell [i][j] = fewest edits to turn the first i letters of a into the first j letters of b

↘ *keep (free) or replace (+1)*
↓ *delete from a (+1)*
→ *insert from b (+1)*

answer: bottom-right = 3

Following the orange cells back from the corner recovers the edits themselves: replace k→s, keep i, t, t, replace e→i, keep n, insert g. The table costs O(len(a) × len(b)) time and memory; keeping only the previous row cuts the memory to O(len(b)).

When greedy is enough, and when it isn't

```python
COINS = [4, 3, 1]

def greedy(amount):                          # always take the biggest coin that fits
    used = []
    for c in COINS:
        while amount >= c:
            amount -= c
            used.append(c)
    return used

def fewest(amount):                          # DP: best[a] = fewest coins that make a
    best = [[]] + [None] * amount
    for a in range(1, amount + 1):
        best[a] = min((best[a - c] + [c] for c in COINS if c <= a), key=len)
    return best[amount]

print(greedy(6), fewest(6))
```

```
[4, 1, 1] [3, 3]
```

KEY IDEA

A **greedy** algorithm commits to the best-looking choice and never reconsiders. It's right only when you can prove the local choice is safe: Dijkstra is greedy (safe because weights are non-negative), and so is paying with UK coins. When choices interact, as with coins 4, 3, 1, DP tries every option for every subproblem, once each.

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 21.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 21.1` shows other valid solutions and their trade-offs.

21.1 build from a spec Fewest steps through a maze
21.2 build from a spec Your own topological sort
21.3 fix the bug Dijkstra that settles too early
21.4 learn an unseen module Islands in a random network
21.5 build from a spec Longest common subsequence
21.6 refactor, tests stay green Packing the book bag: knapsack by DP
21.7 mini-project A courier route planner

PASSPORT STAMP ask the same question of every new language (Appendix B)

6. Data structures

Graphs as `dict` of neighbour lists (or `defaultdict(list)`); `deque` for BFS, a list as a stack for DFS, `heapq` for Dijkstra. DP tables are lists of lists, or `@cache` on a recursive function.

12. Stdlib and tooling

`graphlib.TopologicalSorter` (ordering, cycle detection), `heapq`, `collections.deque`, `functools.cache`. For serious graph work: `networkx` from pip (Ch 19).

IN OTHER LANGUAGES

Every language has the parts: C++ `std::priority_queue` and `std::queue`, Java `PriorityQueue` and `ArrayDeque`, Rust `BinaryHeap` (a max-heap: wrap keys in `Reverse`) and `VecDeque`. C has none built in, so a graph is arrays of indices. The algorithms are identical; only the containers' names change.

DEEP DIVE (your vault)

Vault notes: 13.13 Graphs · 13.14 Topological ordering · 13.15 Weighted shortest paths and Dijkstra's boundary · 13.17 Greedy algorithms · 13.18 Dynamic programming · 13.19 Algorithm selection as evidence.

Appendix A

The standard library atlas

know the whole map, learn the roads you use

Python ships 195 public modules. Nobody knows them all, and nobody should: the skill is knowing **which one exists for a job** and learning it on demand (Ch 7). This map is generated from `sys.stdlib_module_names`, so it is complete for the Python that built this book; the build fails if a module is missing.

tier 1: use it weekly (48) **tier 2: know it exists (68)** **tier 3: map only (79)**

Text
string **re** **difflib** **textwrap** **unicodedata** **stringprep** **readline** **rlcompleter**

Functional tools
itertools **functools** **operator**

Compression and archives
compression **zlib** **gzip** **bz2** **lzma** **zipfile** **tarfile**

Operating system
os **io** **time** **logging** **platform** **errno** **ctypes** **argparse** **optparse** **getopt** **getpass** **shlex** **curses**

Internet data
email **json** **mailbox** **mimetypes** **base64** **binascii** **quopri**

Multimedia
wave **colorsys**

Graphical interfaces
tkinter **turtle** **turtledemo** **idlelib**

Python runtime
sys **sysconfig** **builtins** **__future__** **warnings** **dataclasses** **contextlib** **abc** **atexit** **traceback** **gc** **inspect** **site** **this** **antigravity**

Unix only
posix **pwd** **grp** **termios** **tty** **pty** **fcntl** **resource** **syslog**

Binary data
struct **codecs** **encodings**

Numbers and maths
numbers **math** **cmath** **decimal** **fractions** **random** **statistics**

Persistence
pickle **copyreg** **shelve** **marshal** **dbm** **sqlite3**

File formats
csv **configparser** **tomllib** **netrc** **plistlib**

Networking and signals
socket **ssl** **select** **selectors** **signal** **mmap**

Markup
html **xml** **pyexpat**

Internationalisation
gettext **locale**

Debugging and profiling
bdb **faulthandler** **pdb** **timeit** **trace** **tracemalloc** **cProfile** **profile** **pstats**

Language services
ast **symtable** **token** **keyword** **tokenize** **tabnanny** **pyclbr** **py_compile** **compileall** **dis** **pickletools** **opcode** **sre_compile** **sre_constants** **sre_parse**

Data types
datetime **zoneinfo** **calendar** **collections** **heapq** **bisect** **array** **weakref** **types** **copy** **pprint** **reprlib** **enum** **graphlib**

Files and directories
pathlib **stat** **filecmp** **tempfile** **glob** **fnmatch** **linecache** **shutil** **fileinput** **genericpath** **posixpath** **ntpath**

Cryptographic services
hashlib **hmac** **secrets**

Concurrency
threading **multiprocessing** **concurrent** **subprocess** **sched** **queue** **contextvars** **asyncio**

Internet protocols
webbrowser **wsgiref** **urllib** **http** **ftplib** **poplib** **imaplib** **smtplib** **uuid** **socketserver** **xmlrpc** **ipaddress**

Development tools
typing **annotationlib** **pydoc** **pydoc_data** **doctest** **unittest** **venv** **ensurepip** **zipapp**

Interpreters and imports
code **codeop** **cmd** **zipimport** **pkgutil** **modulefinder** **runpy** **importlib**

Windows only
msvcrt **winreg** **winsound** **nt** **nturl2path**

How to read the lists

Each job group lists its modules with a tier dot and the chapters of this book that use them (found automatically from the snippets). Tier 1 is the working set of a Python programmer: learn its API until you stop looking it up. Tier 2: know what it's for, so you reach for it instead of writing it yourself. Tier 3: recognise the name.

```python
import sys

names = sorted(sys.stdlib_module_names)
public = [n for n in names if not n.startswith("_") or n == "__future__"]
print(sys.version.split()[0], len(names), "names,", len(public), "public")
print([n for n in public if n.startswith("con")])
```

```
3.14.6 297 names, 195 public
['concurrent', 'configparser', 'contextlib', 'contextvars']
```

RULE OF THUMB

A leading underscore means private. `_json` is the C speed-up that `json` uses when it can, `_pydecimal` the pure-Python fallback for `decimal`, `_thread` the layer under `threading`. Import the public module; its internals can change in any release. The grey box lists them only so the map is complete.

Underscore modules: private internals and C accelerators (102)

`_abc` `_aix_support` `_android_support` `_apple_support` `_ast` `_ast_unparse` `_asyncio` `_bisect` `_blake2` `_bz2` `_codecs` `_codecs_cn` `_codecs_hk` `_codecs_iso2022` `_codecs_jp` `_codecs_kr` `_codecs_tw` `_collections` `_collections_abc` `_colorize` `_compat_pickle` `_contextvars` `_csv` `_ctypes` `_curses` `_curses_panel` `_datetime` `_dbm` `_decimal` `_elementtree` `_frozen_importlib` `_frozen_importlib_external` `_functools` `_gdbm` `_hashlib` `_heapq` `_hmac` `_imp` `_interpchannels` `_interpqueues` `_interpreters` `_io` `_ios_support` `_json` `_locale` `_lsprof` `_lzma` `_markupbase` `_md5` `_multibytecodec` `_multiprocessing` `_opcode` `_opcode_metadata` `_operator` `_osx_support` `_overlapped` `_pickle` `_posixshmem` `_posixsubprocess` `_py_abc` `_py_warnings` `_pydatetime` `_pydecimal` `_pyio` `_pylong` `_pyrepl` `_queue` `_random` `_remote_debugging` `_scproxy` `_sha1` `_sha2` `_sha3` `_signal` `_sitebuiltins` `_socket` `_sqlite3` `_sre` `_ssl` `_stat` `_statistics` `_string` `_strptime` `_struct` `_suggestions` `_symtable` `_sysconfig` `_thread` `_threading_local` `_tkinter` `_tokenize` `_tracemalloc` `_types` `_typing` `_uuid` `_warnings` `_weakref` `_weakrefset` `_winapi` `_wmi` `_zoneinfo` `_zstd`

IN OTHER LANGUAGES

Every language has an atlas like this: the C standard library (small: `stdio.h`, `stdlib.h`, `string.h`...), Java's `java.base` packages, Rust's `std` modules. Python's is unusually large ("batteries included"), which is why Ch 19's question 'do I need a package at all?' so often ends with no.

Text		chapters
`string`	constants (`ascii_letters`, `digits`), `Template` substitution	9 · 13
`re`	regular expressions: search, match, split, substitute	9
`difflib`	diffs between sequences; close-match suggestions	7 · 9 · 19
`textwrap`	wrap, fill, indent and dedent paragraphs	9 · 14
`unicodedata`	character names, categories, normalisation (NFC/NFKC)	9
`stringprep`	internet string preparation tables (RFC 3454)	
`readline`	line editing and history for `input()` on Unix	
`rlcompleter`	tab completion of Python names for readline	

Binary data

		chapters
● `struct`	pack and unpack C structs and binary file headers	8 · 15
● `codecs`	the encoder/decoder registry behind `str.encode`	
● `encodings`	the codec implementations (utf_8, latin_1, idna...)	

Data types

		chapters
● **`datetime`**	dates, times, durations; aware vs naive	8 · 11 · 17
● **`zoneinfo`**	IANA time zones for aware datetimes	11
● `calendar`	calendars, leap years, weekday arithmetic	
● **`collections`**	`deque`, `Counter`, `defaultdict`, `namedtuple`, `ChainMap`	1 · 4 · 10 ...
● **`heapq`**	a min-heap on a plain list; priority queues, top-k	10 · 20 · 21
● **`bisect`**	binary search and insertion into sorted lists	10 · 20
● `array`	compact typed arrays of numbers	3
● `weakref`	references that don't keep an object alive; caches	
● `types`	names for built-in types, `SimpleNamespace`, `MappingProxyType`	
● **`copy`**	shallow and deep copies	3
● `pprint`	pretty-print nested data structures	
● `reprlib`	size-limited `repr()` for huge or recursive objects	
● **`enum`**	named constants: `Enum`, `IntEnum`, `Flag`, `StrEnum`	5
● `graphlib`	`TopologicalSorter`: dependency order and cycle detection	21

Numbers and maths

		chapters
● `numbers`	abstract number classes (`Integral`, `Real`...)	
● **`math`**	floats and integers: `sqrt`, `isclose`, `comb`, `gcd`, `fsum`	1 · 7 · 11 ...
● `cmath`	the same functions for complex numbers	
● `decimal`	exact decimal arithmetic with chosen rounding (money)	5 · 11
● `fractions`	exact rational numbers	11
● **`random`**	pseudo-random numbers for simulations (never secrets)	0 · 2 · 11 ...
● **`statistics`**	mean, median, stdev, quantiles, correlation	7 · 11 · 18

Functional tools

		chapters
● **`itertools`**	lazy iterator building blocks: chain, groupby, product...	10
● **`functools`**	`cache`, `partial`, `reduce`, `wraps`, `total_ordering`	4 · 5 · 10 ...
● **`operator`**	operators as functions: `itemgetter`, `attrgetter`	4 · 10

Files and directories

		chapters
● **`pathlib`**	paths as objects: join, glob, read, write	0 · 2 · 8 ...
● `stat`	decode `os.stat()` mode bits	
● `filecmp`	compare files and directory trees	
● **`tempfile`**	temporary files and directories, cleaned up for you	6 · 8 · 17
● `glob`	shell-style wildcard file matching	8
● `fnmatch`	match one name against a wildcard pattern	
● `linecache`	random access to lines of source files (tracebacks)	
● **`shutil`**	copy, move, delete trees; archives; `which`	8
● `fileinput`	loop over lines of many files or stdin	
● `genericpath`	shared code behind `os.path` (internal)	
● `posixpath`	`os.path` on Unix	
● `ntpath`	`os.path` on Windows	

Persistence

		chapters
●	`pickle`	serialise Python objects (never load untrusted data) — 8
●	`copyreg`	register how pickle and copy handle a type
●	`shelve`	a persistent dict of pickled values
●	`marshal`	the internal format of .pyc files
●	`dbm`	simple key-value database files
●	**`sqlite3`**	a full SQL database in one file — 0 · 6 · 7 …

Compression and archives

		chapters
●	`compression`	new home of the codecs, including `compression.zstd` (3.14) — 8
●	`zlib`	deflate compression and CRC-32 — 8
●	`gzip`	read and write .gz files — 8
●	`bz2`	bzip2 compression — 8
●	`lzma`	xz/LZMA compression — 8
●	`zipfile`	read and write ZIP archives — 7 · 8
●	`tarfile`	read and write tar archives (use extraction filters) — 8

File formats

		chapters
●	**`csv`**	comma-separated files, including quoting rules — 8 · 18
●	`configparser`	INI-style configuration files
●	`tomllib`	read TOML (pyproject.toml); read-only — 8 · 17
●	`netrc`	parse ~/.netrc login files
●	`plistlib`	Apple property lists

Cryptographic services

		chapters
●	**`hashlib`**	SHA-2/SHA-3/BLAKE2 digests; `scrypt`, `pbkdf2_hmac` — 7 · 13 · 19
●	**`hmac`**	keyed message authentication; `compare_digest` — 0 · 7 · 13
●	**`secrets`**	unguessable tokens and choices for security — 2 · 11 · 13

Operating system

		chapters
●	**`os`**	processes, environment, files, directories — 6 · 8 · 14 …
●	**`io`**	the stream classes behind `open()`; `StringIO`, `BytesIO` — 2 · 7 · 14 …
●	**`time`**	clocks (`perf_counter`, `monotonic`), sleep, timestamps — 4 · 6 · 10 …
●	**`logging`**	levelled, configurable logging for libraries and apps — 6 · 14
●	`platform`	which OS, machine and Python this is
●	`errno`	symbolic names for OS error codes
●	`ctypes`	call C functions in shared libraries
●	**`argparse`**	command-line parsing with help and subcommands — 7 · 14
●	`optparse`	the older command-line parser
●	`getopt`	C-style command-line parsing
●	`getpass`	read a password without echoing it
●	`shlex`	split and quote shell-style command lines safely — 14
●	`curses`	full-screen terminal interfaces

Concurrency

		chapters
●	**`threading`**	threads, locks, events, conditions — 6 · 15 · 16 …
●	`multiprocessing`	processes with queues, pipes, shared memory, pools — 16
●	**`concurrent`**	`concurrent.futures`: thread and process pools — 16
●	**`subprocess`**	run other programs; pipes and exit codes — 7 · 14
●	`sched`	a simple event scheduler
●	**`queue`**	thread-safe FIFO, LIFO and priority queues — 16

● `contextvars`	context-local state (per task in asyncio)	16
● **`asyncio`**	async/await: event loop, tasks, streams	0 · 7 · 16

Networking and signals

chapters

● **`socket`**	TCP and UDP sockets	15
● `ssl`	TLS for sockets: certificates, contexts	13 · 15
● `select`	wait for I/O on many file descriptors	
● `selectors`	high-level I/O multiplexing (epoll/kqueue)	
● `signal`	handle Unix signals (SIGINT, SIGTERM...)	14
● `mmap`	memory-mapped files	

Internet data

chapters

● `email`	build and parse email messages (MIME)	15
● **`json`**	JSON encode and decode	0 · 2 · 7 ...
● `mailbox`	mbox and Maildir mailboxes	
● `mimetypes`	guess a MIME type from a file name	
● **`base64`**	base64/base32/base85 encodings (not encryption)	7 · 13
● `binascii`	low-level binary-to-ASCII conversions	
● `quopri`	quoted-printable encoding	

Markup

chapters

● `html`	`escape`, `unescape`, and a simple HTML parser	9
● `xml`	ElementTree, DOM, SAX (use defusedxml for untrusted input)	
● `pyexpat`	the Expat parser behind `xml`	

Internet protocols

chapters

● `webbrowser`	open a URL in the user's browser	
● `wsgiref`	reference WSGI server and utilities	
● **`urllib`**	URLs: parse, quote, fetch (`urllib.request`)	7 · 13 · 15 ...
● **`http`**	`http.client`, `http.server`, status codes, cookies	7 · 13 · 15 ...
● `ftplib`	FTP client	
● `poplib`	POP3 client	
● `imaplib`	IMAP4 client	
● `smtplib`	send mail over SMTP	15
● `uuid`	universally unique identifiers (uuid4, uuid7)	13
● `socketserver`	a framework for TCP/UDP servers	15
● `xmlrpc`	XML-RPC client and server	
● `ipaddress`	IPv4/IPv6 addresses and networks	

Multimedia

chapters

● `wave`	read and write WAV files	
● `colorsys`	convert between RGB, HLS, HSV, YIQ	

Internationalisation

chapters

● `gettext`	translations of messages	
● `locale`	locale-aware formatting and sorting	

Graphical interfaces

chapters

● `tkinter`	the Tk GUI toolkit	7
● `turtle`	turtle graphics for teaching	
● `turtledemo`	demos for turtle	

idlelib	the IDLE editor's code	

Development tools

		chapters
typing	type hints: generics, `Protocol`, `TypedDict`...	5 · 10 · 17
annotationlib	inspect annotations, which are lazy since 3.14	
pydoc	documentation from docstrings (`help()`)	
pydoc_data	data files for pydoc	
doctest	run the examples in docstrings as tests	17
unittest	the test framework; `unittest.mock`	0 · 7 · 17 ...
venv	virtual environments	17 · 19
ensurepip	bootstrap pip into an environment	
zipapp	bundle a program into a runnable .pyz	

Debugging and profiling

		chapters
bdb	the debugger framework under pdb	
faulthandler	dump tracebacks on crashes and hangs	
pdb	the interactive debugger (`breakpoint()`)	0 · 2
timeit	time small snippets reliably	0 · 17 · 18 ...
trace	trace execution and line coverage	
tracemalloc	find where memory is allocated	
cProfile	deterministic profiler (C)	0 · 17
profile	the same profiler in pure Python	
pstats	sort and print profiler results	17

Python runtime

		chapters
sys	interpreter state: argv, path, modules, exit, stdout	2 · 3 · 4 ...
sysconfig	install paths and build configuration	19
builtins	the module holding `len`, `print`, `open`...	
__future__	opt in to future language behaviour	
warnings	issue and filter warnings (deprecations)	
dataclasses	classes from field declarations	5 · 8 · 12 ...
contextlib	`contextmanager`, `suppress`, `ExitStack`	6 · 12
abc	abstract base classes	5
atexit	run functions at interpreter exit	
traceback	format and print exceptions and stacks	6
gc	the cycle collector: collect, inspect, disable	3
inspect	look inside live objects: signatures, source, frames	7
site	start-up path configuration (site-packages)	
this	the Zen of Python (`import this`)	
antigravity	an xkcd joke (opens a browser)	

Interpreters and imports

		chapters
code	build your own interactive console	
codeop	compile possibly incomplete input (REPLs)	
cmd	line-oriented command interpreters	
zipimport	import modules from ZIP files	
pkgutil	iterate over and load package data	
modulefinder	find the modules a script uses	
runpy	run modules as scripts (`python -m`)	
importlib	the import system; `importlib.metadata`, `.resources`	2 · 19

Language services

		chapters
●	`ast`	parse source into a tree; analyse or rewrite code 2 · 9
●	`symtable`	the compiler's symbol tables
●	`token`	token type constants
●	`keyword`	is this string a keyword?
●	`tokenize`	split source into tokens (highlighters, linters) 2
●	`tabnanny`	detect ambiguous indentation
●	`pyclbr`	browse classes and functions without importing
●	`py_compile`	compile one file to .pyc
●	`compileall`	compile a whole tree to .pyc
●	`dis`	disassemble bytecode 2
●	`pickletools`	inspect and optimise pickles
●	`opcode`	bytecode opcode tables
●	`sre_compile`	deprecated alias of `re` internals
●	`sre_constants`	deprecated alias of `re` internals
●	`sre_parse`	deprecated alias of `re` internals

Unix only

		chapters
●	`posix`	the raw OS calls behind `os` on Unix
●	`pwd`	the password database (users)
●	`grp`	the group database
●	`termios`	terminal control
●	`tty`	raw and cbreak terminal modes
●	`pty`	pseudo-terminals
●	`fcntl`	file and I/O control, file locks
●	`resource`	process resource limits and usage
●	`syslog`	the Unix system log

Windows only

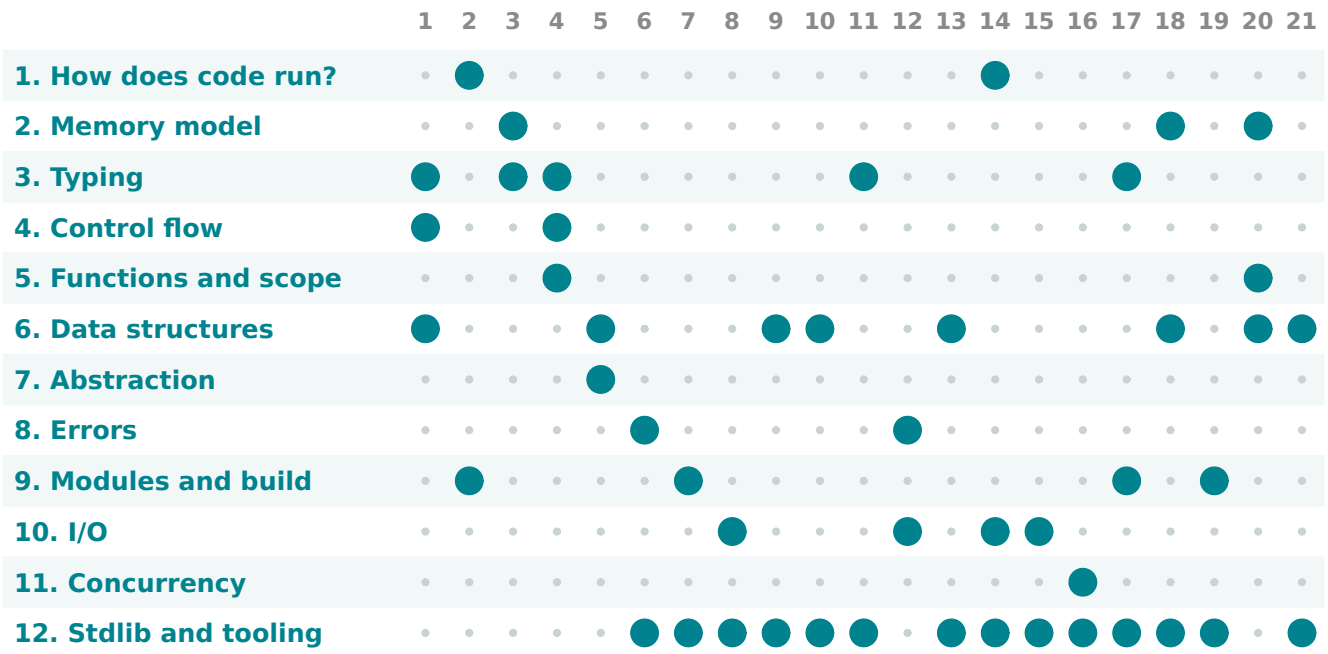
		chapters
●	`msvcrt`	Windows C runtime: console keys, file locking
●	`winreg`	the Windows registry
●	`winsound`	play sounds on Windows
●	`nt`	the raw OS calls behind `os` on Windows
●	`nturl2path`	file URL to path conversion (internal)

Appendix B

The language passport

the same twelve questions, for every language you learn

Each chapter ended with a stamp: Python's answer to the passport questions it covered. Here they are collected, first as a map of **where each answer was built**, then as one page. The last pages ask the same questions of C, Java and Rust, and give a ten-minute experiment for answering each one in any language.



Each dot is a chapter whose stamp answered that question. Data structures (question 6) and the stdlib (12) are stamped most because almost every chapter adds to them; abstraction and concurrency are answered in one place, so they are worth rereading before an exam.

> **KEY IDEA**
> The passport is a reading guide, not a summary. When a new language's tutorial says something, ask which of the twelve questions it answers. The parts it skips are where your Python habits will mislead you.

Python's passport

LANGUAGE PASSPORT: Python

same 12 questions for C, Java, Rust later

1. How does code run?

CPython compiles each module to bytecode (cached in `__pycache__`) and a stack VM runs it. Syntax errors at compile time, the rest at run time. To the OS: a process with argv, environment, three streams and an exit code (Ch 2, 14).

2. Memory model

Every value is a heap object; names, list slots and attributes hold references, and assignment never copies. Reference counting plus a cycle collector free memory. NumPy: one typed buffer, views share it (Ch 3, 18, 20).

3. Typing

Dynamic and strong: values have types, names don't; `'5' + 5` is a TypeError. Mutable vs immutable; only hashable objects are dict keys. Hints are checked by mypy or pyright, never by Python (Ch 3, 11, 17).

4. Control flow

Indentation makes blocks. `if`/`elif`, `for` over any iterable, `while`, `match`, loop `else`. Generators pause at `yield`; exceptions are non-local exits (Ch 1, 4, 6).

5. Functions and scope

First-class functions made when `def` runs; positional-only, keyword-only, `*args`, `**kwargs`; defaults evaluated once. LEGB scope, closures, decorators. No tail calls; recursion limit 1000 (Ch 4, 20).

6. Data structures

`list`, `tuple`, `dict` (ordered), `set`, `str`, `bytes` built in; `deque`, `Counter`, `heapq`, `bisect` in the stdlib. List end O(1), front O(n); dict and set O(1) on average (Ch 10, 20, 21).

7. Abstraction

Classes with multiple inheritance (C3 MRO, `super()`); duck typing through dunder protocols; ABCs and `typing.Protocol` for explicit interfaces; dataclasses, properties, enums (Ch 5).

8. Errors

Exceptions unwind to the first matching `except`; `else`, `finally`, `raise ... from`, `ExceptionGroup`. EAFP style; `with` guarantees clean-up; library errors form class trees (Ch 6, 12).

9. Modules and build

A module runs once, on first import, and is cached in `sys.modules`; found via `sys.path`. Packages are folders. A `venv` per project; pip installs wheels; `pyproject.toml` plus a lock file (Ch 2, 17, 19).

10. I/O

Files are bytes; text is decoded with an explicit encoding. `open` in a `with`, `pathlib`, many formats in the stdlib, `sqlite3` for structured data; sockets speak bytes (Ch 8, 12, 15).

11. Concurrency

Threads share memory (locks, queues), but the GIL blocks CPU parallelism on the default build: processes for CPU work, `asyncio` for many waits in one thread. Free-threaded builds since 3.13 (Ch 16).

12. Stdlib and tooling

Batteries included: 195 public modules (Appendix A). `unittest`, `doctest`, `pdb`, `timeit`, `cProfile` built in; pip, mypy and Git outside (Ch 7, 17, 19).

Four passports side by side

The same questions for the languages a CS degree usually meets next. One line each is a simplification; it tells you where to look, not everything you'll find.

	Python	C	Java	Rust
1. How does code run?	bytecode on the CPython VM	compiled ahead of time to machine code, then linked	compiled to JVM bytecode, JIT-compiled while running	compiled ahead of time to machine code by rustc
2. Memory model	all values are heap objects; refcount + cycle GC	you manage it: stack values, `malloc`/`free`, pointers	primitives by value, objects by reference; tracing GC	ownership and borrowing, checked at compile time; no GC
3. Typing	dynamic, strong; optional hints	static, weak (implicit conversions, casts)	static, strong, nominal; generics by erasure	static, strong, inferred; traits; no implicit conversions
4. Control flow	indentation; `for` over iterables; `match`	braces; counting `for` loops; `switch`; `goto`	braces; enhanced `for`; `switch` expressions	everything is an expression; exhaustive `match`
5. Functions and scope	first-class, closures, LEGB, `*args`	function pointers, no closures; pass by value	methods in classes; lambdas capture effectively-final	functions and closures (`Fn`, `FnMut`, `FnOnce`); move or borrow
6. Data structures	`list`, `dict`, `set` built in; `deque`, `heapq`	arrays and structs; build the rest yourself	Collections: `ArrayList`, `HashMap`, `TreeMap`, `ArrayDeque`	`Vec`, `HashMap`, `BTreeMap`, `VecDeque`, `BinaryHeap`
7. Abstraction	classes, multiple inheritance, duck typing	structs + function pointers; headers as interfaces	classes, single inheritance, interfaces	structs and enums + traits; no inheritance
8. Errors	exceptions; `with` for clean-up	return codes and `errno`	exceptions (checked and unchecked); try-with-resources	`Result<T, E>`, `Option`, the `?` operator; panic for bugs
9. Modules and build	`import`, venv + pip, `pyproject.toml`	`#include`, separate compilation, make/CMake	packages and JARs; Maven or Gradle	crates and modules; Cargo and crates.io
10. I/O	`open()` file objects; text vs bytes	`FILE*` from stdio, or POSIX `read`/`write`	`java.io` streams; `Files` and `Path` (java.nio)	`std::fs`, `Read`/`Write` traits returning `Result`
11. Concurrency	threads (GIL), processes, `asyncio`	pthreads, `fork`, C11 atomics	threads, executors and concurrent collections, virtual threads	threads with `Send`/`Sync` checked by the compiler; async runtimes
12. Stdlib and tooling	huge stdlib; pip, unittest, pdb	small libc; gdb, valgrind, sanitizers	large JDK; jshell, JUnit, Flight Recorder	lean std; cargo test, fmt, clippy, doc

Ten-minute probes

Run these in any new language, in order. Each one forces an answer to one passport question, and the surprises are what to write in that language's passport.

question	the experiment
1. How does code run?	Write hello world and run it. Which files appear? Add a type error: is it reported before running or only when that line runs?
2. Memory model	Make two names for one list, change it through one, print the other. Then pass it to a function that changes it.
3. Typing	Try `"5" + 5`, `5 / 2` and storing text in a number variable. Which fail, and when?
4. Control flow	Loop over a list, a map and a string. Is there a `match`/`switch`, and must it cover every case?
5. Functions and scope	Return a function that counts its calls (a closure). Can it change the captured variable?
6. Data structures	Find the growable array, hash map, set, deque and heap. Look up the cost of inserting at the front.
7. Abstraction	Define a Shape with `area()` and two implementations; call `area()` on a list holding both.
8. Errors	Open a missing file. Exception, error value, or crash? How is clean-up guaranteed?
9. Modules and build	Split a program into two files, import one from the other, and add one third-party package.
10. I/O	Copy a UTF-8 file line by line and read stdin. What is the default encoding?
11. Concurrency	Add 1 to a shared counter from 4 threads, 100,000 times each. Is the total right?
12. Stdlib and tooling	Write and run one unit test, step through it in the debugger, and time a loop.

RULE OF THUMB

Pair the passport with the library card (Ch 7) for every library and the spike for every unknown: twelve questions for the language, six for each library, twenty lines of code for anything you're unsure of.

Appendix C

Glossary

the words, in one line each, and where to read more

114 terms, each with the chapters that teach it (grey numbers on the right). One line is a reminder, not an explanation: if a definition doesn't click, read the chapter, then the vault notes listed in its deep-dive box.

A

abstract base class	a class that declares methods its subclasses must provide; it can't be instantiated itself	5
adjacency list	a graph stored as a dict from each node to the list of its neighbours	21
aliasing	two names referring to one object, so a change made through one is seen through the other	3
amortised cost	the average cost per operation over a long run: `list.append` is O(1) amortised despite occasional resizes	20
argv	the list of command-line arguments; `sys.argv[0]` is the program itself	14
atomic write	write a temporary file, then `os.replace` it, so readers see the old file or the new one, never half	8
aware datetime	a datetime that carries a time zone; a naive one doesn't, and the two can't be compared	11

B

base case	the input a recursive function answers directly, without recursing	20
base64	an encoding of bytes as ASCII text; anyone can decode it, so it is not encryption	13
big-O	how cost grows with input size, ignoring constant factors: O(1), O(log n), O(n), O(n log n), O(n²)	20
binary search	find a position in sorted data by halving the range each step: O(log n)	10 · 20
breadth-first search	explore a graph in rings of distance using a queue; finds the fewest edges	21
broadcasting	NumPy's rule for combining arrays of different shapes by stretching size-1 axes	18
bytecode	the instructions CPython compiles source to and its virtual machine runs (cached in .pyc files)	2
bytes	an immutable sequence of integers 0-255: what files, sockets and hashes really hold	8 · 13

C

call stack	the stack of frames for the calls in progress; each call pushes a frame, each return pops it	6 · 20
closure	a function that remembers variables from the scope where it was defined	4

code point	the number Unicode assigns to a character; a `str` is a sequence of code points	9
comprehension	an expression building a list, dict or set from a loop: `[x * x for x in xs if x]`	1
context manager	an object used with `with` whose `__exit__` runs however the block ends	6
coroutine	an `async def` function's object; it runs until an `await`, letting the event loop run others	16
cursor	the object that executes SQL and walks through the result rows	12
cycle collector	the part of the garbage collector that frees groups of objects referring only to each other	3

D

dataclass	a class whose `__init__`, `__repr__` and `__eq__` are generated from its field declarations	5
decorator	a function that takes a function and returns a replacement, applied with `@name`	4
deep copy	a copy of an object and, recursively, everything it refers to (`copy.deepcopy`)	3
depth-first search	explore a graph as far as possible along each branch before backing up, using a stack	21
dict	a hash table from keys to values; O(1) lookup on average, remembers insertion order	1 · 10
Dijkstra's algorithm	cheapest paths from one node when weights are non-negative, using a heap	21
duck typing	using an object for what it can do (its methods), not for what class it is	5
dunder method	a `__double_underscore__` method Python calls for syntax: `__len__`, `__eq__`, `__iter__`	5
dynamic programming	solve each overlapping subproblem once and reuse the answer: memoised recursion or a table	20 · 21

E

EAFP	'easier to ask forgiveness than permission': try the operation and handle the exception	6
edit distance	the fewest single-character inserts, deletes and replacements turning one string into another	21
encoding	the rule turning text into bytes and back; UTF-8 unless you have a reason	8 · 13
environment variable	a name-value string a process inherits from its parent (`os.environ`)	14
event loop	asyncio's scheduler: runs ready tasks, and switches at every `await` that has to wait	16
exception	an object raised to signal an error; it unwinds the stack until an `except` matches	6
exit code	the number a process returns to the OS: 0 for success, anything else for failure	14

F

f-string	a string literal with `{expressions}` and format specs: `f"{total:,.2f}"`	9
foreign key	a column referring to another table's primary key, so rows link by id	12

framing	marking where each message ends in a byte stream such as TCP (lines, length prefixes)	15
free-threaded build	a CPython build without the GIL (3.13 onwards), so threads can run Python in parallel	16

G

generator	a function with `yield`; calling it returns an iterator that runs the body lazily	4
GIL	the global interpreter lock: on the default build, one thread runs Python bytecode at a time	16
graph	nodes joined by edges, directed or not, possibly weighted	21
greedy algorithm	always take the best-looking choice now; correct only when that choice can be proved safe	21

H

hash function	a function from data to a fixed-size digest; cryptographic ones make collisions infeasible	13
hashable	has a hash that never changes, so it can be a dict key or set member	3 · 5
heap	a tree stored in a list where every parent is <= its children; `heapq` pops the smallest in O(log n)	10 · 21
HMAC	a tag made from a message and a secret key; proves the message is intact and from a key holder	13
HTTP	the text request-response protocol of the web: method, path, headers, body, status code	15

I

immutable	can't be changed after creation (int, str, tuple, frozenset); 'changes' make new objects	3
index (database)	a sorted structure (a B-tree) that lets a query find rows without scanning the table	12
interpreter	the program that runs Python code: `python3` is CPython's interpreter	14 · 16
invariant	a statement that is true before and after every step of a loop or method	20
iterator	an object with `__next__` that hands out values one at a time; iterables make iterators	4

J

join	an SQL query combining rows of two tables where a condition (usually key = foreign key) holds	12
JSON	a text format for nested lists, dicts, strings, numbers, booleans and null	8

L

LEGB	the order Python looks names up: Local, Enclosing, Global, Built-in	4
library card	this book's six questions for learning any library	7
linked list	nodes each holding a value and a reference to the next; O(1) insert at a known node, O(n) indexing	20
lock	lets one thread at a time into a critical section (`threading.Lock`)	16
lock file	exact versions and hashes of every dependency, so installs are reproducible	19

M

memoisation	caching a function's results by argument (`functools.cache`)	20 · 21
merge sort	split in half, sort each half, merge: O(n log n), stable	20
method resolution order	the order classes are searched for an attribute (C3 linearisation; `Class.__mro__`)	5
mock	a stand-in object that records calls and returns canned answers in a test	17
module	a `.py` file (or compiled extension) loaded by `import`; a package is a folder of modules	2 · 7
mutable	can be changed in place (list, dict, set, most instances)	3

N

ndarray	NumPy's n-dimensional array: one typed buffer plus a header of dtype, shape and strides	18
normalisation	Unicode: one canonical form for equal-looking text (NFC). Databases: each fact stored once	9 · 12

P

parameter binding	passing values to SQL through `?` placeholders, never by building strings: stops injection	12
passport	this book's twelve questions for understanding any programming language	0
pickle	Python's object serialiser; loading a pickle can run arbitrary code, so never load untrusted ones	8
pip	the installer that resolves, downloads, verifies and unpacks packages into an environment	19
process	a running program with its own memory; processes don't share objects	14 · 16
protocol	a set of methods a type must have to be used somewhere (iteration, `typing.Protocol`)	5 · 17
pseudo-random	numbers from a deterministic generator: reproducible with a seed, predictable, so not for secrets	11

R

race condition	a bug where the result depends on how threads' steps interleave	16
recursion	a function solving a problem by calling itself on smaller inputs	20
reference count	how many references point at an object; at zero CPython frees it at once	3
regular expression	a pattern language for matching text (`re`)	9

S

salt	random bytes stored with a password hash so equal passwords hash differently	13
scope	the region of code where a name is visible	4
secrets	the module for unguessable random values (tokens, passwords); `random` is not safe for these	11 · 13
shallow copy	a new container holding the same element objects (`list(xs)`, `xs.copy()`, `copy.copy`)	3
signal	an asynchronous notification from the OS to a process (SIGINT on Ctrl-C, SIGTERM)	14
socket	an endpoint for network communication; TCP sockets carry a byte stream	15

spike	a throwaway 20-line program written to answer one question about a library	7
SQL injection	attacker text ending a string inside an SQL statement and adding its own commands	12
stable sort	a sort that keeps equal items in their original order (Python's always is)	20
stack	last in, first out; a Python list with `append` and `pop`	21
stderr	the stream for messages and errors; stdout is for results, so pipes stay clean	14
stride	how many bytes to step in memory to move one place along an array axis	18
subprocess	running another program from Python and collecting its output and exit code	14
supply-chain attack	attacking users through a dependency: typosquatted names, hijacked or malicious packages	19

T

TCP	the reliable, ordered byte-stream protocol under HTTP, SMTP and most of the internet	15
thread	a line of execution inside a process; threads share memory	16
timing attack	learning a secret from how long a comparison takes; prevented by `hmac.compare_digest`	13
Timsort	Python's sort: a merge sort that exploits runs already in order	20
TLS	encryption and server authentication for a TCP connection (the S in HTTPS)	15
topological order	an order of a directed graph's nodes where every edge points forward; exists only without cycles	21
traceback	the printed stack of calls that led to an exception, oldest first	6
transaction	a group of changes that commit together or not at all	12
tree	nodes with children and no cycles; in a binary search tree, left < node < right	20
type hint	an annotation like `def f(x: int) -> str`; checked by tools, ignored when running	17

U

Unicode	the standard numbering every character in every script	9
unit test	a test of one small piece of code in isolation	17
UTC	Coordinated Universal Time; store and compute in UTC, convert to local time for display	11

V

vectorisation	replacing Python loops with whole-array operations that run in compiled code	18
view	an array sharing another array's buffer: changing one changes the other	18
virtual environment	a folder with its own site-packages so each project has its own dependencies	2 · 19

W

wheel	a built package file (.whl) that pip unpacks without running any setup code	17 · 19